

软件详细设计规范					
项目编号	C1	文件号	C1_MCAL_MPECAN_DRV_SDD	文件版本号	1.0

SW Detail Design Specification

For MCAL MPECAN Driver

MCAL MPECAN Driver 软件详细设计规范

ROLES 角色	Name 姓名	Department 部门	Date 日期
AUTHOR(S) 作者：			
REVIEWER(S) 审查：			
APPROVER (S) 批准：			

SW Detail Design Specification
For IPCS Driver

			IPCS Driver 软件详细设计规范

ROLES 角色
Name 姓名
Department 部门
Date 日期
AUTHOR(S)作者：
 尚亚朋
 软件研发部
 2026.5.22
REVIEWER(S)审查：
 安然/赵笃/喻明睿/张帅/伊焕利/肖敏元/尚亚朋/宋晓婷/栗瑞江/Wenke
 软件研发部
 2026.5.22
APPROVER (S)批准：
 安然
 软件研发部
 2026.5.22

Version / 版本
Date / 日期
Editor / 编辑人
Status / 文档状态
Change description / 变更简述

V0.1

2026.5.7

Cursor Agent

Draft

Initial version for review, 基于软件需求、ipcs-architecture.pdf、reference.md 与 ASPICE SWE.3

生成

V0.2

2026.5.19

Cursor Agent

Draft

按 ASPICE SWE.3 重构为第 1–6 章；新增 章节 2.4–2.6、Linux Refinement、第 5 章与第 6 章追溯

V0.7

2026.5.19

Cursor Agent

Draft

完善 Linux 部署变体函数设计、关键场景 SVG 与分层静态图

V0.8

2026.5.19

Cursor Agent

Draft

章节 5.7/章节 6.7 跨单元场景改为 UML 序列图 (PlantUML→SVG)

V0.9

2026.5.20

Cursor Agent

Draft

新增第 7 章双向追溯矩阵 (架构 章节 2.4 × 章节 2 单元 × 物理实现)

V1.0

2026.5.20

Cursor Agent

Draft

章节序号连贯 (5.x 重排、6.5/6.7 子节)；第 7 章追溯矩阵产品化 (剔除过程类伪追溯行)

V1.1

2026.5.20

Cursor Agent

Draft

章节 5.1/5.2、章节 6.1/6.2 对齐 章节 4.1/4.2；头文件组件 UML→SVG；实现文件列表核对 PASS

V0.6

2026.5.19

Cursor Agent

Draft

补强第 2、3 章；明确 UIO/CDEV 与全内核实现的用户侧/内核侧职责

V0.5

2026.5.19

Cursor Agent

Draft

精简第 2 章为组件—单元映射；重写第 3 章分层与部署变体；清理目录重复项

V0.4

2026.5.19

Cursor Agent

Draft

拆分第 2 章；新增第 3 章三层架构与 Linux 适配；勘误 UIO/CDEV 代理与全内核形态

V0.3

2026.5.19

Cursor Agent

Draft

第 2 章改为架构符合性与软件单元划分；增加 SW-Unit-ID 与组件映射；插图全部 SVG；对照详细设计与实现一致性修订

1. 1 CONTENTS 目录

INTRODUCTION 简介

Confidentiality 保密性

Purpose of the document 文档目的

Scope 范围

References 参考文件

Abbreviations 缩略语

SOFTWARE UNIT IDENTIFICATION 软件单元划分

Software Unit List 软件单元清单

Architecture Component to Software Unit Mapping 架构组件与软件单元映射

LAYERED ARCHITECTURE AND DEPLOYMENT VARIANTS 分层架构与部署变体设计

SHM / OSAL / HAL Interface Contract 三层接口契约

RTOS Deployment Variant RTOS 部署变体

Linux Deployment Variants Linux 部署变体

UIO Implementation UIO 实现

CDEV Implementation CDEV 实现

In-Kernel Implementation 全内核实现

OSAL/HAL Implementation Mapping OSAL/HAL 实现对照

COMMON SOFTWARE UNIT DETAILED DESIGN 公共软件单元详细设计

Definition and Scope 定义与范围

File Structure 文件结构

SWU_IPCS_CORE_SHM Software Unit Design — External Interfaces 软件单元设计 — 外部接口

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE、SWU_IPCS_CORE_UTIL Software Unit Design — Internal Functions 软件单元设计 — 内部函数

Global Variables 全局变量

Data Types 类型定义

Dynamic Detailed Design 动态详细设计

RTOS DEPLOYMENT VARIANT DETAILED DESIGN RTOS 部署变体详细设计

Definition and Scope 定义与范围

File Structure 文件结构

SWU_IPCS_OSAL_AUTOSAR Software Unit Design 软件单元设计

SWU_IPCS_OSAL_FREERTOS Software Unit Design 软件单元设计

SWU_IPCS_OSAL_THREADX Software Unit Design 软件单元设计

SWU_IPCS_HAL_MCU Software Unit Design 软件单元设计

Global Variables 全局变量

Data Types 类型定义

Dynamic Detailed Design 动态详细设计

LINUX DEPLOYMENT VARIANT DETAILED DESIGN Linux 部署变体详细设计

Definition and Scope 定义与范围

File Structure 文件结构

SWU_IPCS_LINUX_OS_KERN Software Unit Design 软件单元设计

SWU_IPCS_LINUX_OS_UIO Software Unit Design 软件单元设计

SWU_IPCS_LINUX_UIO_KO Software Unit Design 软件单元设计

SWU_IPCS_LINUX_OS_CDEV Software Unit Design 软件单元设计

SWU_IPCS_LINUX_CDEV_KO Software Unit Design 软件单元设计

SWU_IPCS_HAL_LINUX Software Unit Design 软件单元设计

Dynamic Detailed Design 动态详细设计

Global Variables 全局变量

Data Types 类型定义

BIDIRECTIONAL TRACEABILITY AND CONSISTENCY 双向追溯与一致性

Traceability Statement 追溯性策略与声明

Bidirectional Traceability Matrix 双向追溯矩阵

2 INTRODUCTION 简介

2.1 CONFIDENTIALITY 保密性

任何披露必须与负责的流程经理协调。

本文件过程说明仅限直接参与项目的人员查看。转让给其他方，尤其是 Star Gather 以外的合作伙伴，必须由项目负责人协调，并受开发合同中有关保密规定的约束。

2. 2 DOCUMENT PURPOSE 文档目的

本文档按照 SWE.3 Software Detailed Design and Unit Construction 的要求，为 IPCS Driver (RTOS 与 Linux 部署变体) 建立软件详细设计。文档内容描述软件单元的静态结构、接口、数据结构、关键动态行为，并与 IPCS 软件架构中定义的组件和接口保持一致。

2. 3 SCOPE 范围

本文档规定 IPC Shared Memory Driver (IPCS Driver) 的软件详细设计 (SWE.3) 范围。设计输入为：已分配的软件需求、软件架构设计、ASPICE SWE.3 过程要求及约束。设计输出为下文各章所述的软件单元结构、接口、数据类型与动态行为。

设计范围包括：

跨部署变体共享的通信核心与队列（架构组件 Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Conf_Cmp）；

RTOS 部署变体 OSAL/HAL 实现 (FreeRTOS、ThreadX、AUTOSAR OS) ；

Linux 部署变体适配与 HAL 实现（全内核、UIO、CDEV）。

2. 4	2. 5	2. 6	2. 7	2. 8	2. 9
2. 10	2. 11	2. 12	2. 13	2. 14	2. 15
2. 16	2. 17	2. 18	2. 19	2. 20	2. 21
2. 22	2. 23	2. 24	2. 25	2. 26	2. 27 REFERE NCES 参考文献

Reference ID / 编号

Document Name / 文档名称

Version / 版本

Date / 日期

Author / 作者

Status / 状态

1

Automotive SPICE® Process Assessment Model, SWE.3 Software Detailed Design and Unit Construction

4.0

2023

VDA

Release

2

IPCS Driver 软件架构设计 ipcs-architecture.pdf

1.0

2026.4.16

尚亚朋

待评审

3

reference.md 详细设计文档模板

N/A

N/A

N/A

模板输入

<i>2. 28</i>	<i>2. 29</i>
<i>2. 30</i>	<i>2. 31</i>
<i>2. 32</i>	<i>2. 33</i>
<i>2. 34</i>	<i>2. 35</i>
<i>2. 36</i>	<i>2. 37</i>
<i>2. 38</i>	<i>2. 39</i>
<i>2. 40</i>	<i>2. 41</i>
<i>2. 42</i>	<i>2. 43</i>
<i>2. 44</i>	<i>2. 45</i>
<i>2. 46</i>	<i>2. 47</i>
<i>2. 48</i>	<i>2. 49</i>
<i>2. 50</i>	<i>2. 51</i>
<i>2. 52</i>	<i>2. 53</i>
<i>2. 54</i>	<i>2. 55</i>
<i>2. 56</i>	<i>2. 57</i>
<i>2. 58</i>	<i>2. 59</i>
<i>2. 60</i>	<i>2. 61</i>
<i>2. 62</i>	<i>2. 63</i>
<i>2. 64</i>	<i>2. 65</i>
<i>2. 66</i>	<i>2. 67 ABBREVIATIONS 缩略语</i>

Abbreviation / 缩写

Meaning/Explanation / 解释

API

Application Programming Interface

AUTOSAR

AUTomotive Open System ARchitecture

BD
Buffer Descriptor
CDD
Complex Device Driver
HAL
Hardware Abstraction Layer
HW
Hardware
IPCS
Inter-Processor Communication System
IRQ
Interrupt Request
ISR
Interrupt Service Routine
OS
Operating System
OSAL
OS Abstraction Layer
RTOS
Real-Time Operating System
SHM
Shared Memory
UIO
Userspace I/O
Deployment Variant
部署变体 (RTOS 部署变体 / Linux 部署变体)
Implementation
实现 (如 FreeRTOS 实现、UIO 实现、全内核实现)
User-Side Proxy
用户侧代理 (Linux UIO/CDEV 用户库: 满足 P4/P5 契约, 对 OS/HW 操作为转发实现)
In-Kernel
全内核实现
CDEV
Character Device

3 SOFTWARE UNIT IDENTIFICATION 软件单元划分

<i>3. 1</i>	<i>3. 2</i>	<i>3. 3</i>
<i>3. 4</i>	<i>3. 5</i>	<i>3. 6</i>
<i>3. 7</i>	<i>3. 8</i>	<i>3. 9</i>
<i>3. 10</i>	<i>3. 11</i>	<i>3. 12</i>
<i>3. 13</i>	<i>3. 14</i>	<i>3. 15</i>
<i>3. 16</i>	<i>3. 17</i>	<i>3. 18</i>
<i>3. 19</i>	<i>3. 20</i>	<i>3. 21</i>
<i>3. 22</i>	<i>3. 23</i>	<i>3. 24</i>
<i>3. 25</i>	<i>3. 26</i>	<i>3. 27</i>
<i>3. 28</i>	<i>3. 29</i>	<i>3. 30</i>
<i>3. 31</i>	<i>3. 32</i>	<i>3. 33</i>
<i>3. 34</i>	<i>3. 35</i>	<i>3. 36</i>
<i>3. 37</i>	<i>3. 38</i>	<i>3. 39</i>
<i>3. 40</i>	<i>3. 41</i>	<i>3. 42 SOFTWARE UNIT LIST</i> <i>软件单元清单</i>

软件单元按可编译实现文件 (.c) 划分。头文件作为单元接口或类型规格，在 章节 4.2 Files 及函数设计表「函数声明文件」中描述，不单独编号为 SWU。

SW-Unit-ID

实现文件

说明

SWU_IPCS_CORE_SHM

ipcs/ipcs_cores/ipc-shm.c

SHM Core, 实现实例、通道、发送接收主流程

SWU_IPCS_CORE_QUEUE

ipcs/ipcs_cores/ipc-queue.c

环形队列实现

SWU_IPCS_CORE_UTIL

ipcs/ipcs_cores/ipc-util.c

Core 工具函数

SWU_IPCS_HAL_MCU
ipcs/mcu/hw/ipc-hw.c
RTOS 部署变体 HAL 实现
SWU_IPCS_HAL_LINUX
ipcs/mpu/hw/c1/ipc-hw.c
Linux 内核侧 HAL 实现
SWU_IPCS_OSAL_AUTOSAR
ipcs/mcu/os/autosar/ipc-os-autosar.c
AUTOSAR OS 实现
SWU_IPCS_OSAL_FREERTOS
ipcs/mcu/os/freertos/ipc-os-freertos.c
FreeRTOS 实现
SWU_IPCS_OSAL_THREADX
ipcs/mcu/os/threadx/ipc-os-threadx.c
ThreadX 实现
SWU_IPCS_LINUX_OS_UIO
ipcs/mpu/os_uio/ipc-os.c
UIO 用户侧 OSAL/HAL 契约代理
SWU_IPCS_LINUX_OS_CDEV
ipcs/mpu/os_cdev/ipc-os.c
CDEV 用户侧 OSAL/HAL 契约代理
SWU_IPCS_LINUX_OS_KERN
ipcs/mpu/os_kernel/ipc-os.c
Linux 全内核 OSAL 实现
SWU_IPCS_LINUX_UIO_KO
ipcs/mpu/os_kernel/ipc-uio.c
UIO 内核 Backend
SWU_IPCS_LINUX_CDEV_KO
ipcs/mpu/os_kernel/ipc-cdev.c
CDEV 内核 Backend

3. 43	3. 44	3. 45
3. 46	3. 47	3. 48
3. 49	3. 50	3. 51
3. 52	3. 53	3. 54
3. 55	3. 56	3. 57
3. 58	3. 59	3. 60 COMPONENT-SWU MAPPING 组件单元映射

架构组件 ID

SW-Unit-ID

适用范围

Drv_Ipcs_Core_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_UTIL

全部部署变体

Drv_Ipcs_Queue_Cmp

SWU_IPCS_CORE_QUEUE

全部部署变体

Drv_Ipcs_Osal_Cmp

SWU_IPCS_OSAL_AUTOSAR、SWU_IPCS_OSAL_FREERTOS、SWU_IPCS_OSAL_THREADX、

SWU_IPCS_LINUX_OS_KERN

RTOS 各 OS 实现、Linux 全内核实现

Drv_Ipcs_Hal_Cmp

SWU_IPCS_HAL_MCU、SWU_IPCS_HAL_LINUX

RTOS HAL、Linux 内核 HAL

Drv_Ipcs_Linux_Adapt_Cmp

SWU_IPCS_LINUX_OS_UIO、SWU_IPCS_LINUX_OS_CDEV、SWU_IPCS_LINUX_OS_KERN、

SWU_IPCS_LINUX_UIO_KO、SWU_IPCS_LINUX_CDEV_KO、SWU_IPCS_HAL_LINUX

Linux 部署变体

第 4 章及以下函数说明表中的「软件单元 ID」引用本章；「对应软件架构 ID」引用 架构设计规范中的架构组件 ID。

4 LAYERED ARCHITECTURE 分层部署变体

4.1	4.2	4.3	4.4	4.5	4.6
4.7	4.8	4.9	4.10	4.11	4.12
4.13	4.14	4.15	4.16	4.17	4.18
4.19	4.20	4.21	4.22	4.23	4.24 SHM/ OSAL/ HAL CONTRACT 三层接口 契约

IPCS 采用 SHM、OSAL、HAL 三层结构。SHM 为应用层提供固定函数原型接口，只通过固定函数原型调用 OSAL 与 HAL；不同部署变体实现不得改变这些接口契约。详细的接口设计参考 IPCS Driver 软件架构设计文档。

层

架构组件

接口符号

接口文件

关键接口

设计约束

SHM

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp

IF_AppSvc

ipc-shm.h

ipcsShmInit、ipcsShmTx、ipcsShmPollChannels

对上提供应用接口，对下只依赖 OSAL/HAL 契约

OSAL

Drv_Ipcs_Osal_Cmp

IF_OSAbst

ipc-os.h 或同名 Linux 用户侧符号

ipcsOsInit、ipcsOsGetLocalShm、ipcsOsPollChannels

提供共享内存映射、收包调度与中断上下文联结

HAL

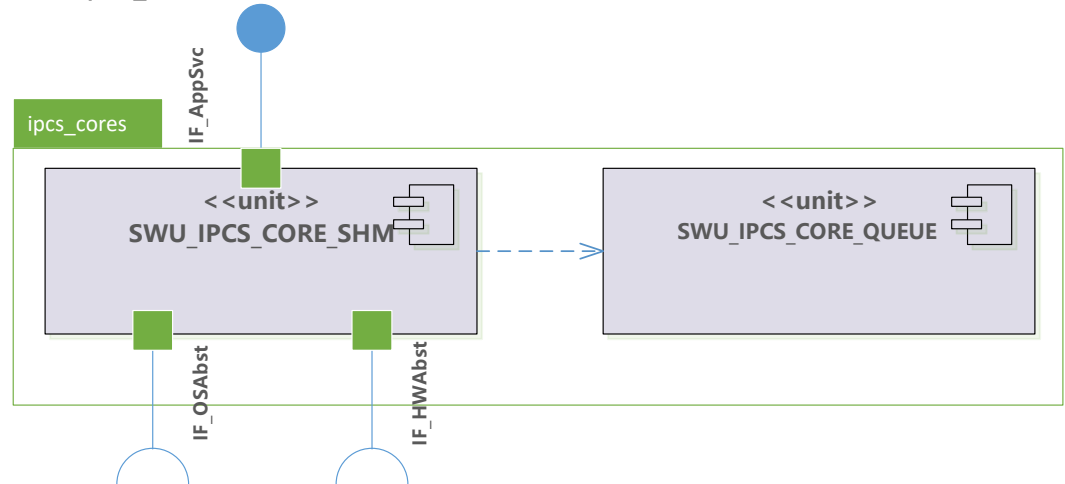
Drv_Ipcs_Hal_Cmp

IF_HWAbst

ipc-hw.h

ipcsHwInit、ipcsHwIrqNotify、ipcsHwIrqEnable

提供 核间中断控制器/IRQ、缓存与平台核索引操作
 该契约保证 ipcs_cores 可在 RTOS、Linux UIO、Linux CDEV、Linux 全内核实现间复用。



4. 25	4. 26	4. 27	4. 28
4. 29	4. 30	4. 31	4. 32
4. 33	4. 34	4. 35	4. 36
4. 37	4. 38	4. 39	4. 40 RTOS DEPLOYMENT VARIANT RTOS 部署变体

RTOS 部署变体包括 FreeRTOS、ThreadX、AUTOSAR OS 三种实现。

实现

OSAL 实现单元

HAL 实现单元

结构说明

FreeRTOS 实现

SWU_IPCS_OSAL_FREERTOS

SWU_IPCS_HAL_MCU

SHM、OSAL、HAL 位于同一地址空间

ThreadX 实现

SWU_IPCS_OSAL_THREADX

SWU_IPCS_HAL_MCU

SHM、OSAL、HAL 位于同一地址空间

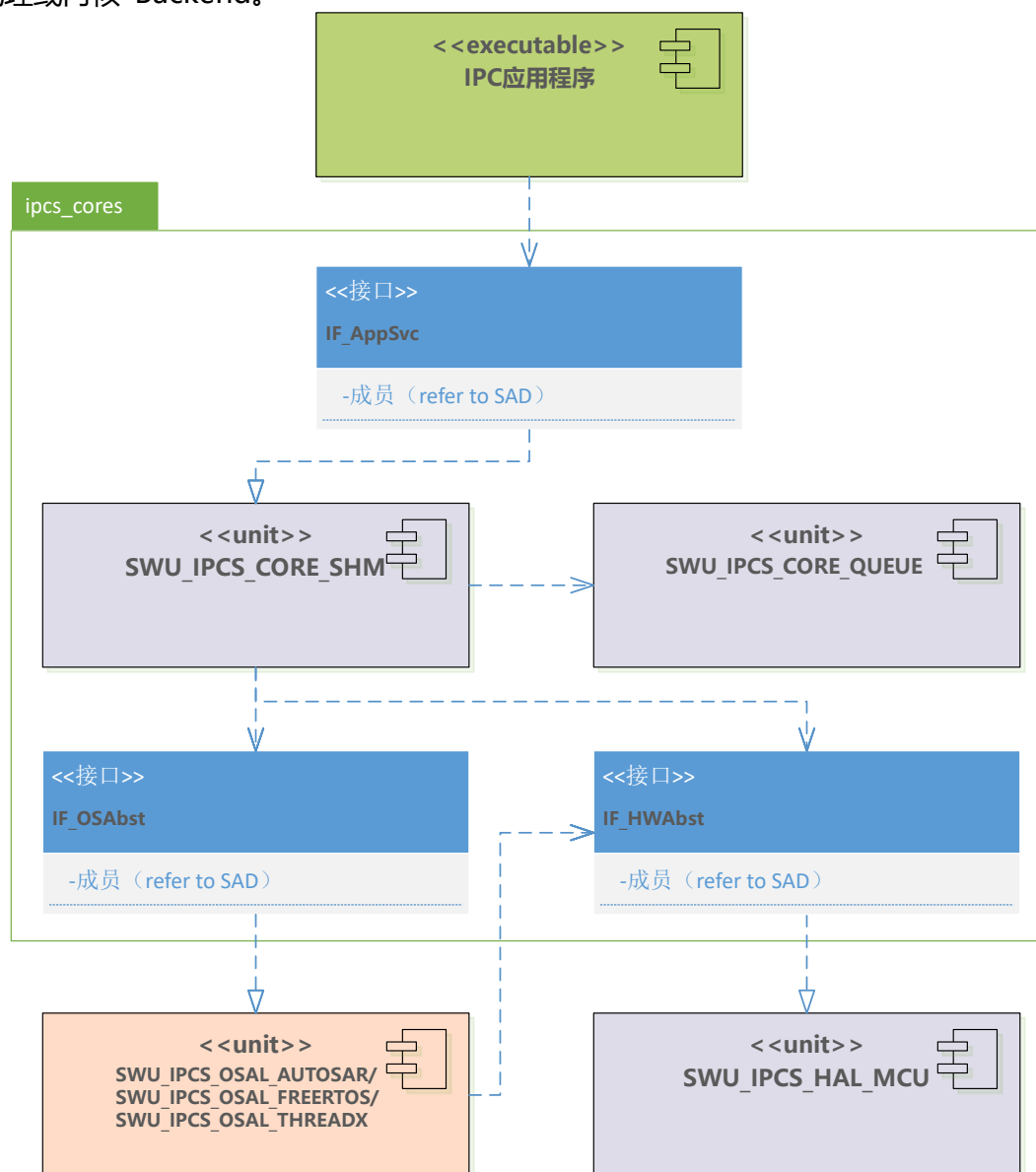
AUTOSAR OS 实现

SWU_IPCS_OSAL_AUTOSAR

SWU_IPCS_HAL_MCU

SHM、OSAL、HAL 位于同一地址空间

RTOS 部署变体中，Core 调用 ipcsOs* 与 ipcsHw* 时直接进入 OSAL/HAL 实现，不存在用户侧代理或内核 Backend。



4. 41 LINUX DEPLOY VARIANTS LINUX 部署变体

Linux 部署变体包括 UIO、CDEV、全内核三种实现。ipcs-architecture.pdf 中的 Drv_Ipcs_Linux_Adapt_Cmp 是逻辑组件；本 SDD 按架构 Refinement 进一步分解其用户侧与内核侧职责。

4. 41. 1	4. 41. 2	4. 41. 3
4. 41. 4	4. 41. 5	4. 41. 6
4. 41. 7	4. 41. 8	4. 41. 9
4. 41. 10	4. 41. 11	4. 41. 12 UIO Implementation UIO 实现

UIO 实现由用户库代理、UIO 内核 Backend、Linux HAL 三部分组成。

部分

实现单元

设计职责

用户侧代理

SWU_IPCS_LINUX_OS_UIO

导出 ipcsOs*、ipcsHw* 同名符号，满足 SHM 对 OSAL/HAL 的接口契约；完成 /dev/mem 映射、

UIO 设备打开、RX 线程创建

内核 Backend

SWU_IPCS_LINUX_UIO_KO

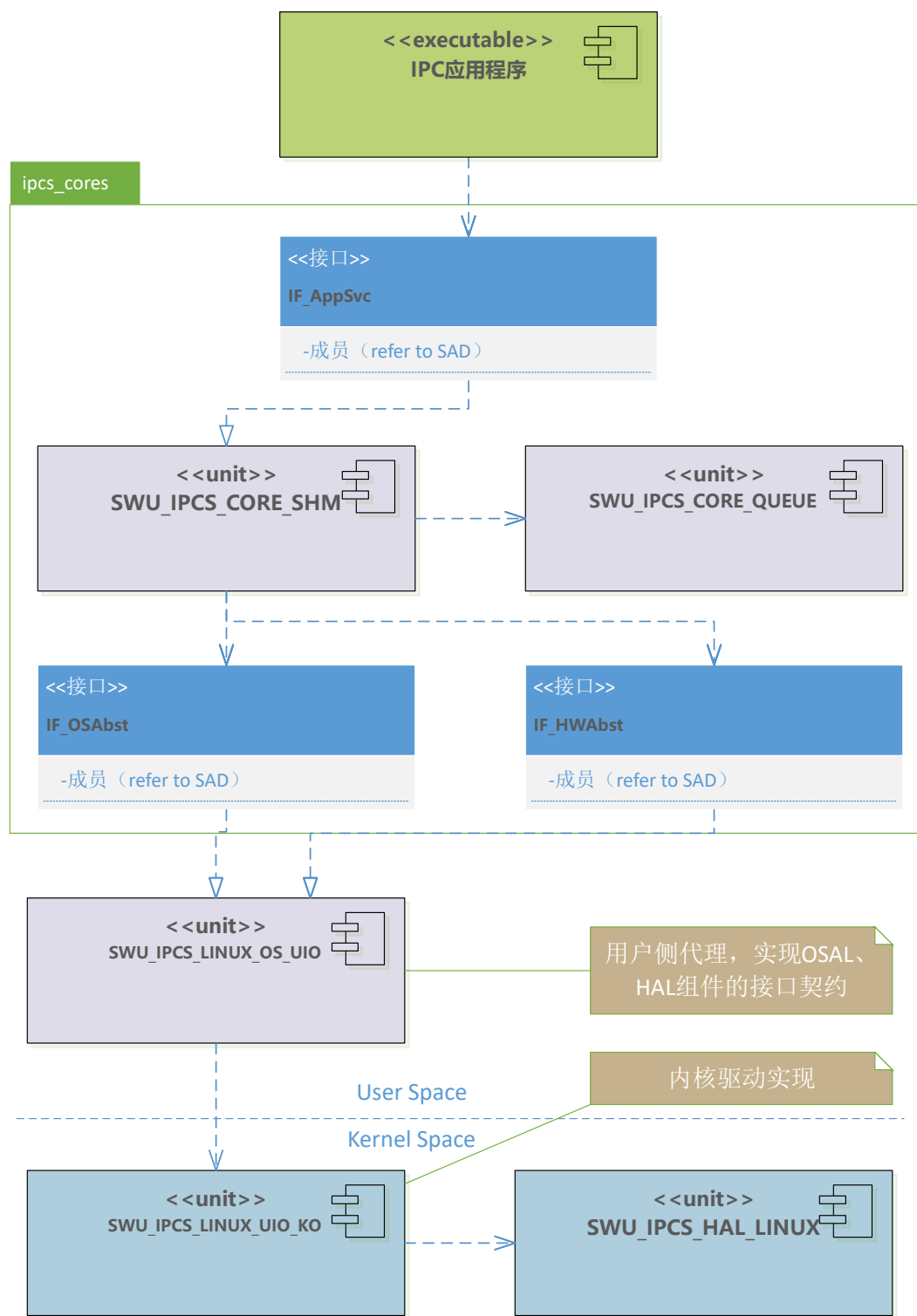
注册 UIO 设备、处理中断、向用户侧传递事件

内核 HAL

SWU_IPCS_HAL_LINUX

执行 核间中断控制器 与 IRQ 相关真实硬件操作

用户侧 ipcsHwIrqEnable、ipcsHwIrqDisable、ipcsHwIrqNotify 通过 ipcsSendUioCmd 写 UIO fd 转发命令；ipcsHwInit、ipcsHwFree 是空实现，设计规定初始化和释放由内核 UIO 模块处理。



4.41.13 User-Kernel Adaptation Interface (IF_LinuxAdapt_UIO) User-Kernel 适配接口

Core 仍只依赖 章节 3.1 的 IF_OSAbst / IF_HWAbst。UIO 变体中，用户侧 SWU_IPCS_LINUX_OS_UIO 对上层呈现这两套接口；跨地址空间实现则经 IF_LinuxAdapt_UIO 由 SWU_IPCS_LINUX_UIO_KO 完成。该接口属于 Drv_Ipcs_Linux_Adapt_Cmp 内部分配接口，不是新的架构层。

共享内存映射 (ipcsOsGetLocalShm / ipcsOsGetRemoteShm) 经 /dev/mem 完成, 不经过本适配接口。本文档不展开 VFS、系统调用、UIO 子系统内部实现; 仅定义 User Adapt SWU 与 Kernel Adapt SWU 之间的操作契约 (定义见 ipcs/mpu/os_kernel/ipc-uio.h) 。

			UIO 变体采用 Init 通道 + Runtime 通道 双通道模型:

通道

用户节点

内核处理

用途

Init

/dev/ipc-cdev-uio (write)

ipc-uio.c → ipcsUiolnit()

传递 {instance, IPCS_SHM_CFG_TYPE}, 触发 ipcsHwlnit 与 UIO 设备注册

Runtime

/dev/uioN (write / read)

ipcsShmUiolrqcontrol() / ipcsShmUioHandler()

HW 中断控制与 RX 事件通知

					适配操作与架构接口映射如下:

适配操作

方向

载荷 / 命令

用户侧入口

内核侧处理

映射架构接口

UIO_INIT_INSTANCE

User → Kernel

struct IPCS_UIO_CDEV_DATA_TYPE

ipcsOslnit 内 write Init 通道

ipcsCdevWrite → ipcsUiolnit

IF_OSAbst 初始化 (内核段) ; IF_HWAbst 初始化
UIO_HW_DISABLE
User → Kernel
IPC_UIO_DISABLE_CMD (0x0001)
ipcsHwlrqDisable
ipcsShmUioIrqcontrol → ipcsHwlrqDisable
IF_HWAbst
UIO_HW_ENABLE
User → Kernel
IPC_UIO_ENABLE_CMD (0x0002)
ipcsHwlrqEnable
ipcsShmUioIrqcontrol → ipcsHwlrqEnable
IF_HWAbst
UIO_HW_TRIGGER
User → Kernel
IPC_UIO_TRIGGER_CMD (0x0003)
ipcsHwlrqNotify
ipcsShmUioIrqcontrol → ipcsHwlrqNotify
IF_HWAbst
UIO_RX_EVENT
Kernel → User
UIO 事件计数 (read 返回)
ipcsShmSoftirq 线程 read Runtime 通道
hardirq → ipcsShmUioHandler → UIO 框架唤醒
IF_OSAbst 收包调度 (rx_cb)
收包路径: 硬件 IRQ → 内核 ipcsShmUioHandler (清中断) → UIO 事件 → 用户线程 read
返回 → 调用 rx_cb → 再次 ipcsHwlrqEnable。动态交互详见 章节 6.7 UIO 序列图。

4. 41. 14	4. 41. 15	4. 41. 16
4. 41. 17	4. 41. 18	4. 41. 19
4. 41. 20	4. 41. 21	4. 41. 22
4. 41. 23	4. 41. 24	4. 41. 25 CDEV Implementation CDEV 实现

CDEV 实现由用户库代理、字符设备 Backend、Linux HAL 三部分组成。

实现单元

设计职责

用户侧代理

SWU_IPCS_LINUX_OS_CDEV

导出 ipcsOs*、ipcsHw* 同名符号，满足 SHM 对 OSAL/HAL 的接口契约；通过 /dev/ipc-shm-cdev 与内核通信

内核 Backend

SWU_IPCS_LINUX_CDEV_KO

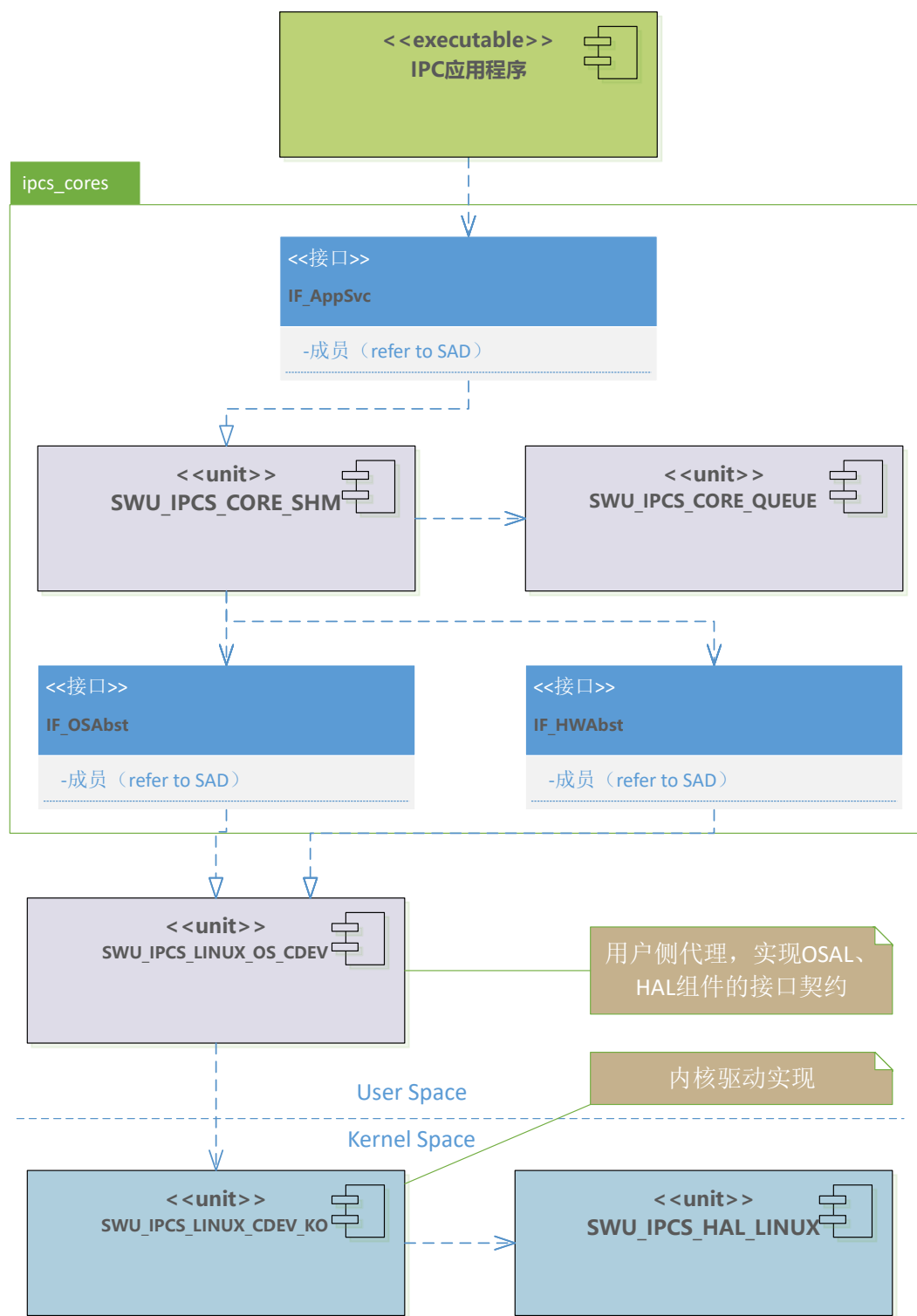
提供字符设备、ioctl、wait queue、ISR 处理

内核 HAL

SWU_IPCS_HAL_LINUX

执行 核间中断控制器 与 IRQ 相关真实硬件操作

用户侧 ipcsHwIrqEnable、ipcsHwIrqDisable、ipcsHwIrqNotify 使用 IPC_CDEV_CMD_* ioctl 转发到内核；ipcsHwInit、ipcsHwFree 是空实现，设计规定由内核模块处理。



4.41.26 User-Kernel Adaptation Interface (IF_LinuxAdapt_CDEV) User-Kernel 适配接口

与 UIO 变体相同，Core 只依赖 IF_OSAbst / IF_HWAbst；用户侧 SWU_IPCS_LINUX_OS_CDEV 对外呈现上述契约，跨域实现经 IF_LinuxAdapt_CDEV 由 SWU_IPCS_LINUX_CDEV_KO 完成。接口定义见 ipcs/mpu/os_kernel/ipc-cdev.h（用户态与内核态共用）。

IF_OSAbst 初始化（内核段）； IF_HWAbst 初始化
CDEV_HW_DISABLE
User → Kernel
IPC_CDEV_CMD_DISABLE_RX_IRQ (instance)
ipcsHwlrqDisable
ipcsCdevloctl → ipcsHwlrqDisable
IF_HWAbst
CDEV_HW_ENABLE
User → Kernel
IPC_CDEV_CMD_ENABLE_RX_IRQ (instance)
ipcsHwlrqEnable
ipcsCdevloctl → ipcsHwlrqEnable
IF_HWAbst
CDEV_HW_TRIGGER
User → Kernel
IPC_CDEV_CMD_TRIGGER_TX_IRQ (instance)
ipcsHwlrqNotify
ipcsCdevloctl → ipcsHwlrqNotify
IF_HWAbst
CDEV_RX_EVENT
Kernel → User
read 阻塞返回 (wait queue 唤醒)
ipcsShmSoftirq 线程 read
hardirq → wake_up_interruptible
IF_OSAbst 收包调度 (rx_cb)

与 UIO 变体对比：CDEV 不依赖 Linux UIO 框架；Init 与 HW 控制均通过自定义 ioctl 完成；RX 通知经内核 wait queue + 用户 read，而非 UIO event read。动态交互详见 章节 6.7 CDEV 序列图。

4. 41. 27	4. 41. 28	4. 41. 29
4. 41. 30	4. 41. 31	4. 41. 32
4. 41. 33	4. 41. 34	4. 41. 35 In-Kernel Implementation 全 内核实现

全内核实现不使用用户侧代理。Core、OSAL、HAL 均在内核模块中运行，形态与 RTOS 部署变体一致。
部分

实现单元

设计职责

OSAL

SWU_IPCS_LINUX_OS_KERN

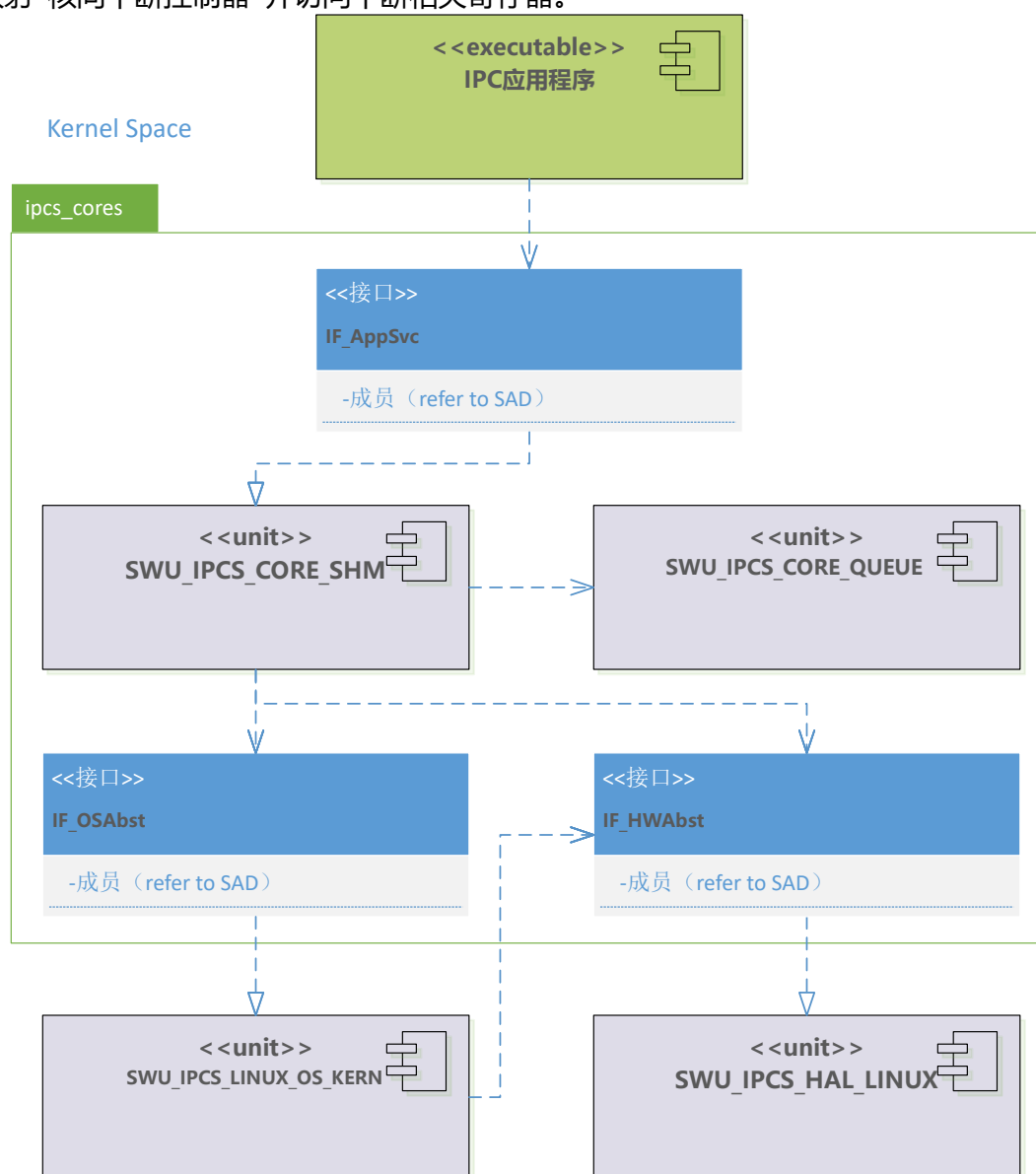
实现 ipcsOsInit、ipcsOsFree、ipcsOsGetLocalShm、ipcsOsGetRemoteShm、ipcsOsPollChannels

HAL

SWU_IPCS_HAL_LINUX

实现 ipcsHwInit、ipcsHwIrqEnable、ipcsHwIrqDisable、ipcsHwIrqNotify、ipcsHwIrqClear 等硬件操作

ipcs/mpu/os_kernel/ipc-os.c 使用 tasklet 进行延迟收包处理，ipcs/mpu/hw/c1/ipc-hw.c 负责映射 核间中断控制器 并访问中断相关寄存器。



5 COMMON SWU DESIGN 公共单元设计

5.1 DEFINITION AND SCOPE 定义与范围

IPCS Driver 是面向同一 SoC 内不同处理核心的 shared memory 通信驱动。公共部分 (ipcs/ipcs_cores) 实现 IPCF Shared Memory 协议核心，支持 managed/unmanaged channel、中断通知与 polling、多 instance/channel。

本章描述跨部署变体共享的 Core、Queue、配置类型 (SHM 层)。分层与部署变体见第 3 章；RTOS 实现见第 5 章；Linux 实现见第 6 章。

5.2 FILE STRUCTURE 文件结构

5.2.1	5.2.2
5.2.3	5.2.4
5.2.5	5.2.6
5.2.7	5.2.8
5.2.9	5.2.10
5.2.11	5.2.12
5.2.13	5.2.14
5.2.15	5.2.16 File List 文件列表

组件
文件
Drv_Ipcs_Queue_Cmp
ipcs/ipcs_cores/ipc-queue.c
Drv_Ipcs_Queue_Cmp
ipcs/ipcs_cores/ipc-queue.h
Drv_Ipcs_Core_Cmp
ipcs/ipcs_cores/ipc-shm.c
Drv_Ipcs_Core_Cmp
ipcs/ipcs_cores/ipc-shm.h
Drv_Ipcs_Conf_Cmp

ipcs/ipcs_cores/ipc-types.h
Drv_Ipcs_Core_Cmp
ipcs/ipcs_cores/ipc-util.c
Drv_Ipcs_Core_Cmp
ipcs/ipcs_cores/ipc-util.h

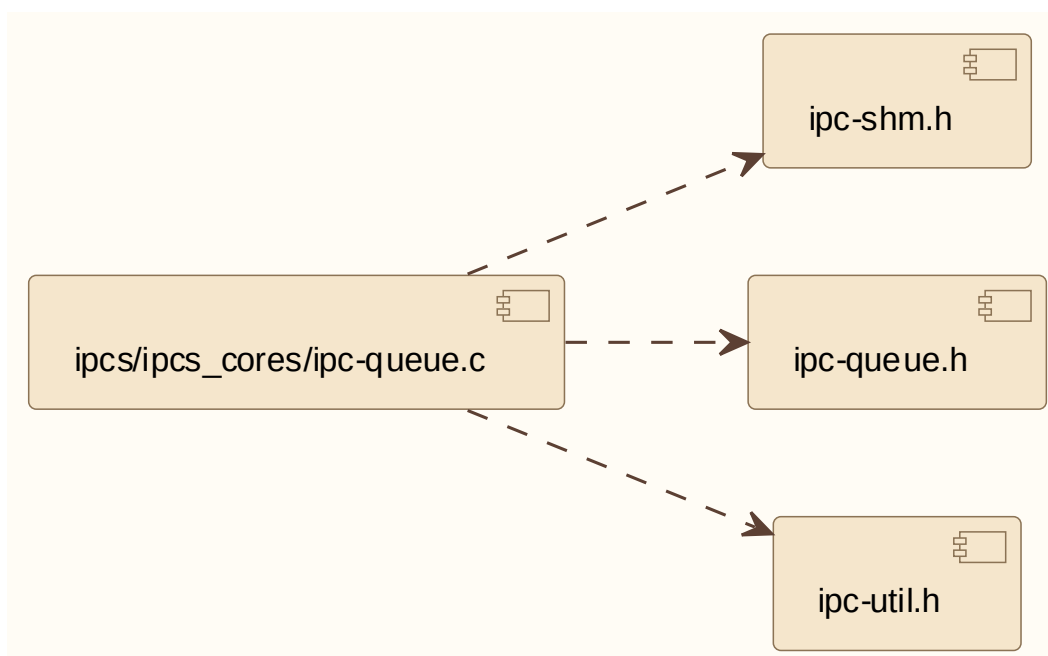
5. 2. 17 ipc-queue. c

描述:

ipcs/ipcs_cores/ipc-queue.c 属于 Drv_Ipcs_Queue_Cmp / IPCS-SHM Queue。

依赖关系:

ipc-shm.h, ipc-queue.h, ipc-util.h (与 ipcs/ipcs_cores/ipc-queue.c 中 #include 顺序一致)



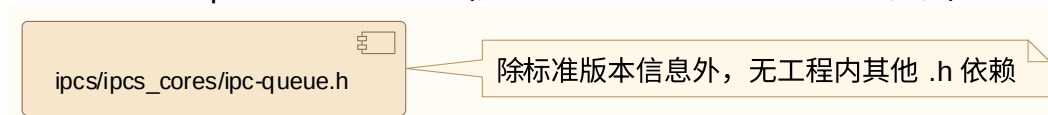
5. 2. 18 ipc-queue. h

描述:

ipcs/ipcs_cores/ipc-queue.h 属于 Drv_Ipcs_Queue_Cmp / IPCS-SHM Queue。

依赖关系:

本头文件未 #include ipcs 内其他头文件 (除标准版本宏定义外无外部文件依赖)。



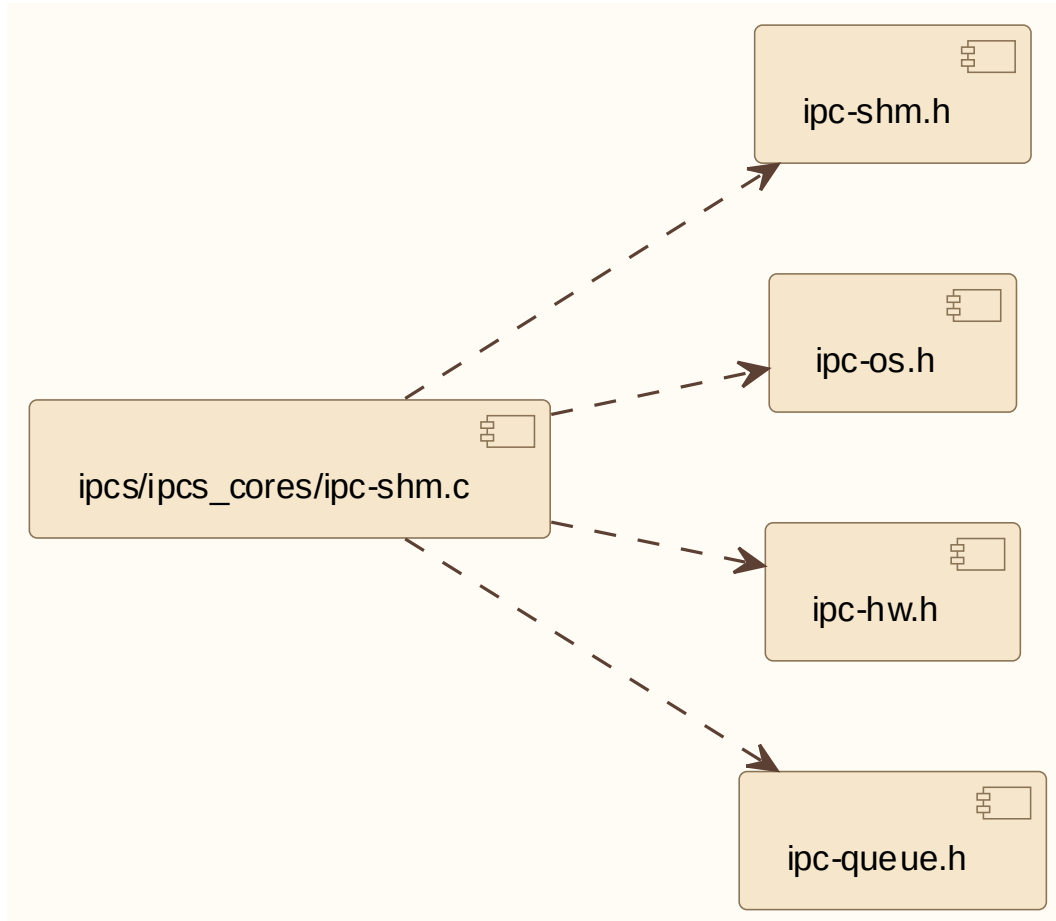
5. 2. 19 ipc-shm. c

描述:

ipcs/ipcs_cores/ipc-shm.c 属于 Drv_Ipcs_Core_Cmp / IPCS-SHM。

依赖关系：

ipc-shm.h, ipc-os.h, ipc-hw.h, ipc-queue.h (与 ipcs/ipcs_cores/ipc-shm.c 中 #include 顺序一致)



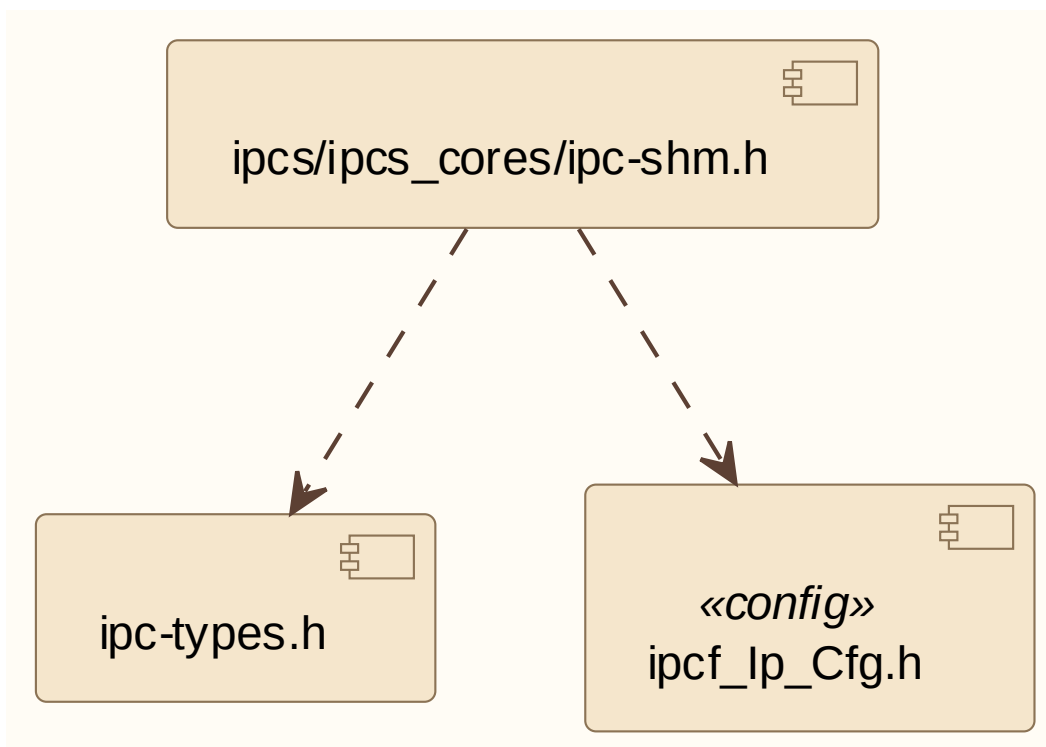
5. 2. 20 ipc-shm.h

描述：

ipcs/ipcs_cores/ipc-shm.h 属于 Drv_Ipcs_Core_Cmp / IPCS-SHM。

依赖关系：

ipc-types.h, ipcf_lp_cfg.h (与 ipcs/ipcs_cores/ipc-shm.h 中 #include 一致;
ipcf_lp_cfg.h 为工程配置头)



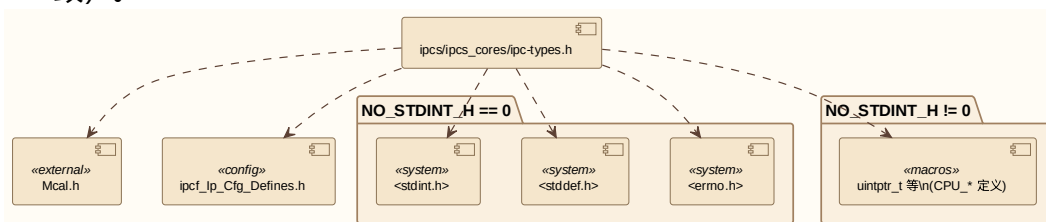
5. 2. 21 ipc-types.h

描述:

ipcs/ipcs_cores/ipc-types.h 属于 Drv_Ipcs_Conf_Cmp / 配置数据类型。

依赖关系:

条件编译: NO_STDINT_H==0 时包含 <stdint.h>、<stddef.h>、<errno.h>; 否则由 CPU 宏定义 uintptr_t 等; 均包含 Mcal.h、ipcf_lp_cfg_Defines.h (与 ipcs/ipcs_cores/ipc-types.h 一致)。



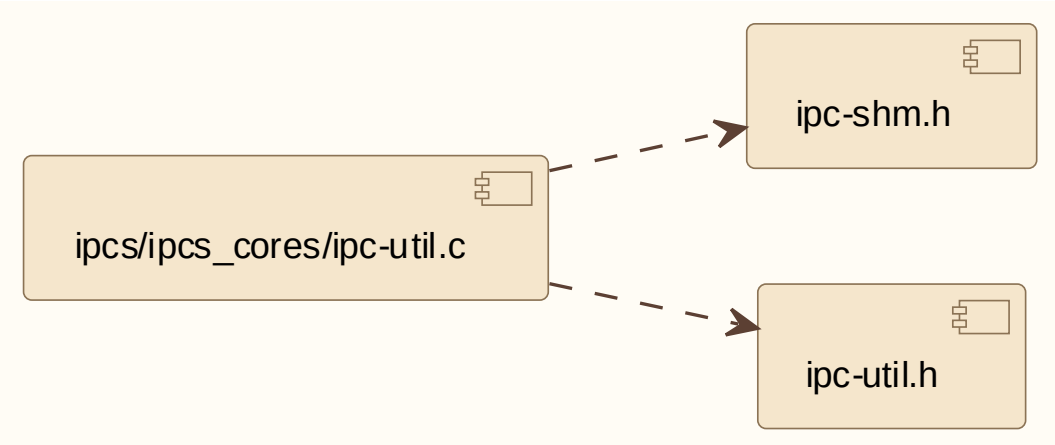
5. 2. 22 ipc-util.c

描述:

ipcs/ipcs_cores/ipc-util.c 属于 Drv_Ipcs_Core_Cmp 公共工具。

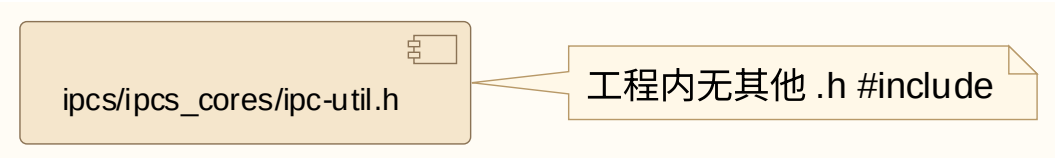
依赖关系:

ipc-shm.h, ipc-util.h (与 ipcs/ipcs_cores/ipc-util.c 中 #include 一致)



5. 2. 23 ipc-util.h

描述：
ipcs/ipcs_cores/ipc-util.h 属于 Drv_Ipcs_Core_Cmp 公共工具。
依赖关系：
本头文件未 #include ipcs 内其他头文件。



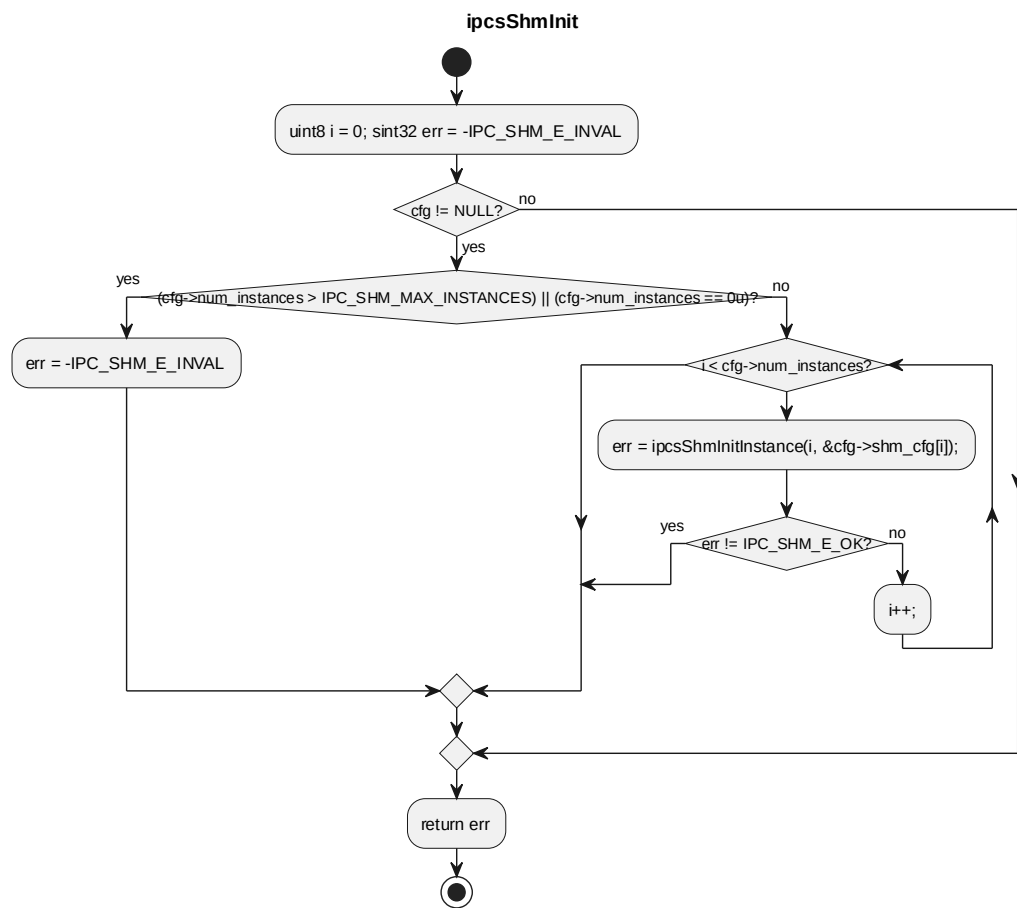
5. 3 SWU_CORE_SHM_EXT INTERFACES 外部接口

5. 3. 1 ipcsShmInit

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数初始化配置中声明的所有 shared memory 通信实例，依次初始化 IPCS-HAL、IPCS-OSAL 和 IPCS-SHM channel 资源。			
函数原型	sint32 ipcsShmInit(const struct IPCS_SHM_INSTANCES_CFG_TYPE *cfg)			
制约条件	Config 指针非空；num_instances 大于 0 且不超过 IPC_SHM_MAX_INSTANCES；每个 instance 的 shared memory 地址、channel 数量等配置有效。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	cfg	const struct IPCS_SHM_INSTANCES_CFG_TYPE *	指向 IPCS shared memory instance 配置的指针
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK：初始化成功； - IPC_SHM_E_INVALID 或下层错误码：配置或初始化失败	

函数定义文件	ipcs/ipcs_cores/ipc-shm.c
函数声明文件	ipcs/ipcs_cores/ipc-shm.h
满足需求	IPCS_001, IPCS_014, IPCS_016, IPCS_017, IPCS_020, IPCS_025, IPCS_028, IPCS_029, IPCS_031, IPCS_034

processing flow

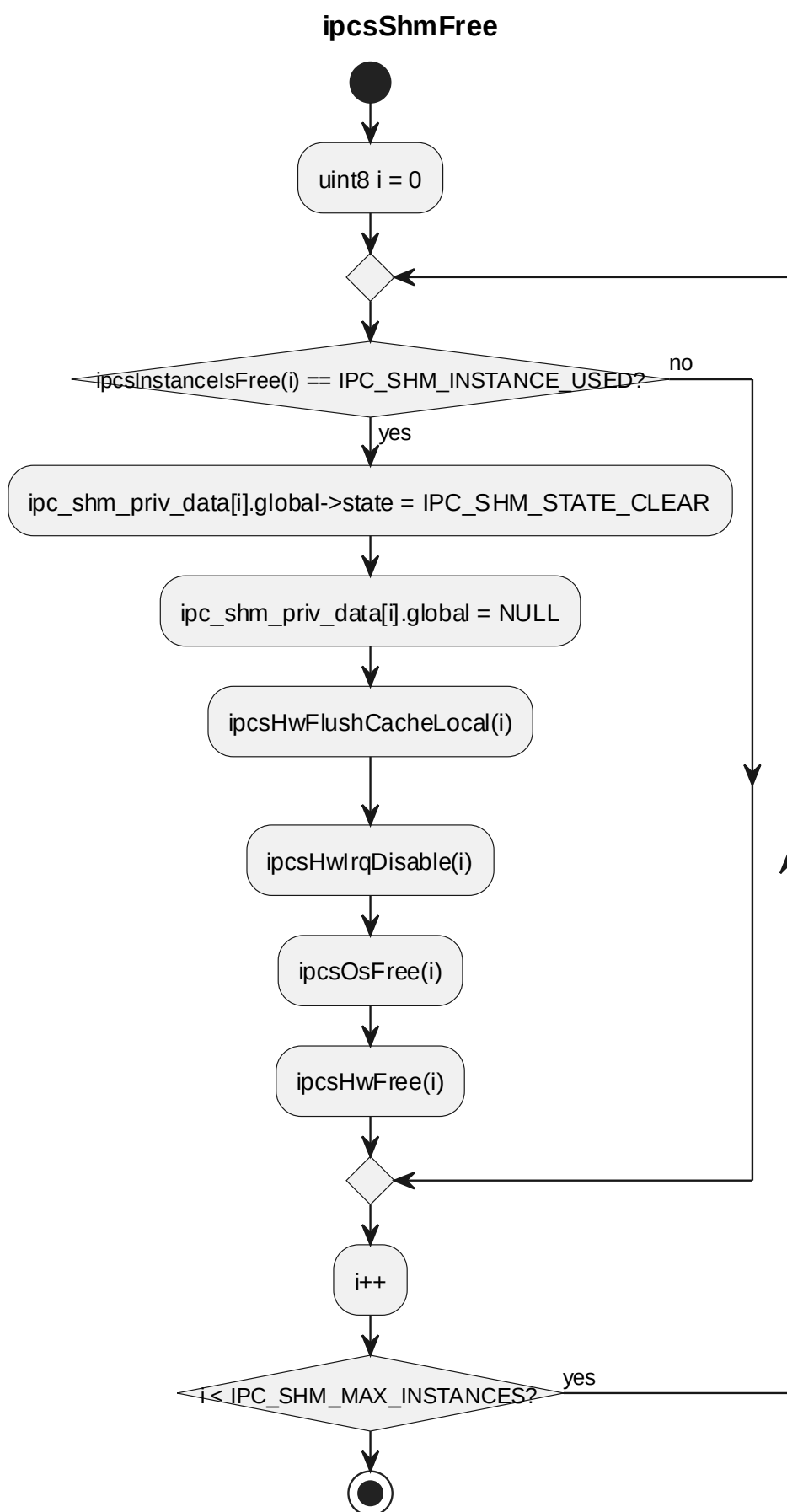


5.3.2 ipcsShmFree

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数释放所有已使用的 shared memory 通信实例，清除本端 ready 状态并释放 OS/HW 资源。			
函数原型	void ipcsShmFree(void)			
制约条件	无输入参数；释放当前处于 IPC_SHM_INSTANCE_USED 状态的 instance。			
输入/输出参数	I/O	参数名	数据类型	说明
	-	-	-	-
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	ipcs/ipcs_cores/ipc-shm.h			

满足需求	IPCS_001, IPCS_028, IPCS_029
------	------------------------------

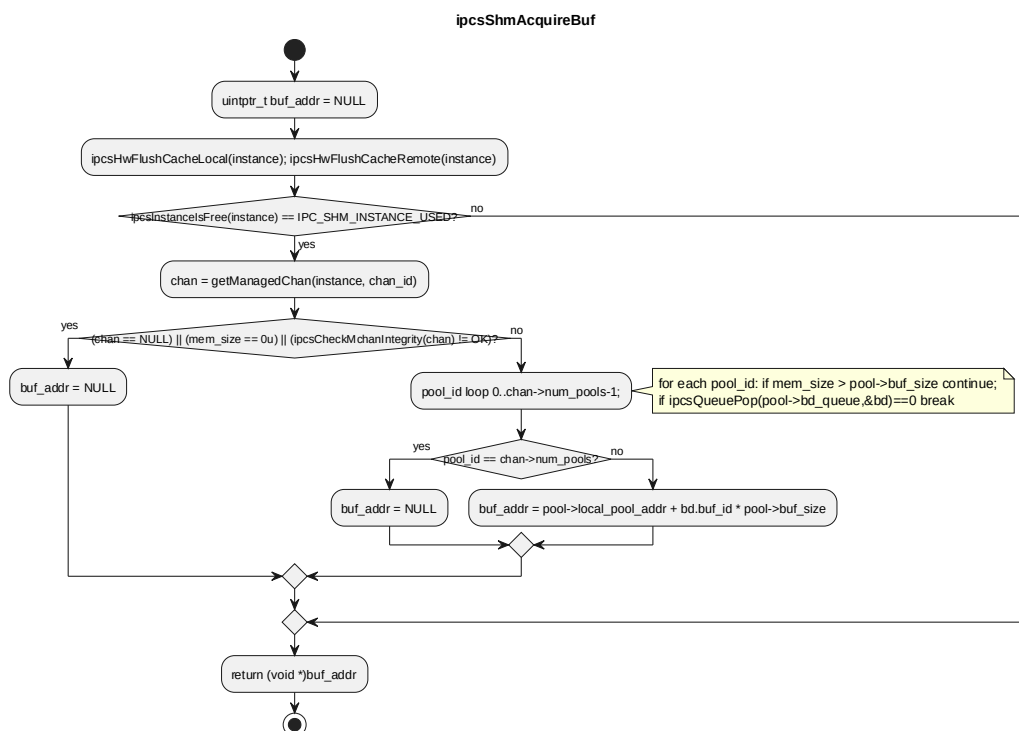
processing flow



5.3.3 ipcsShmAcquireBuf

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数在 managed channel 上按照请求长度获取本端发送 buffer。			
函数原型	void * ipcsShmAcquireBuf(const uint8 instance, sint32 chan_id, uint32 mem_size)			
制约条件	instance 已初始化; chan_id 对应 managed channel; mem_size 非 0; managed channel 与 pool queue integrity 校验通过。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared memory channel 索引
	I	mem_size	uint32	请求获取的 buffer 大小, 单位为 byte
返回值	数据类型		说明	
	void *		非 NULL: 可写发送 buffer 地址; NULL: instance/channel/size/integrity 无效或无可用的 buffer	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	ipcs/ipcs_cores/ipc-shm.h			
满足需求	IPCS 016, IPCS 018, IPCS 025, IPCS 034, IPCS 035			

processing flow

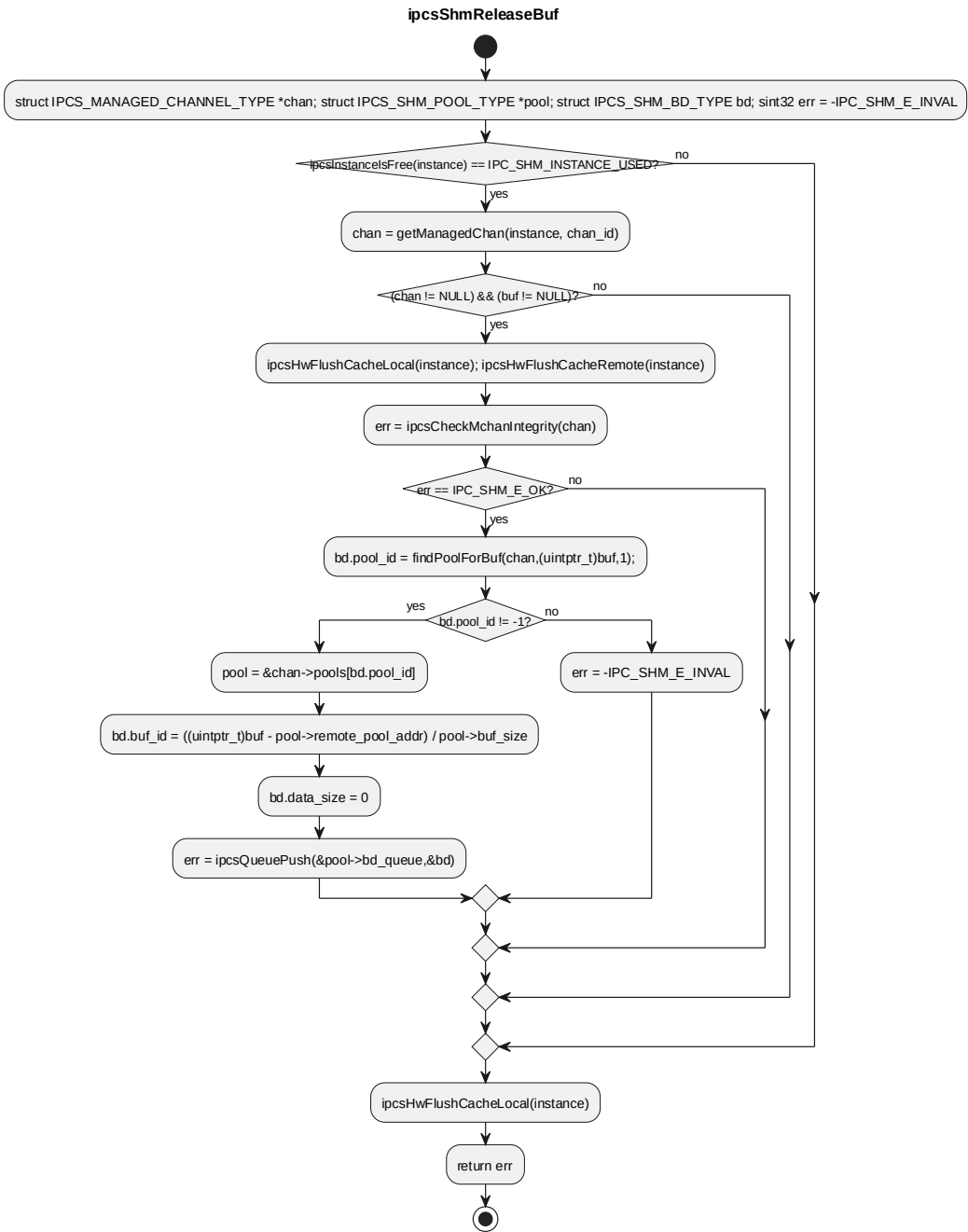


5.3.4 ipcsShmReleaseBuf

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数在 managed channel 上释放接收完成的远端 buffer，并将 buffer descriptor 归还到对应 buffer pool 队列。			
函数原型	sint32 ipcsShmReleaseBuf(const uint8 instance, sint32 chan_id, const void *buf)			
制约条件	instance 已初始化; chan_id 对应 managed channel; buf 非空; managed channel integrity 校验通过; buf 属于某个远端 pool。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared memory channel 索引
	I	buf	const void *	managed channel buffer 指针
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK: 释放成功; 负错误码: instance/channel/buf/integrity 无效或 queue push 失败	

函数定义文件	ipcs/ipcs_cores/ipc-shm.c
函数声明文件	ipcs/ipcs_cores/ipc-shm.h
满足需求	IPCS_016, IPCS_018, IPCS_023, IPCS_034, IPCS_035

processing flow

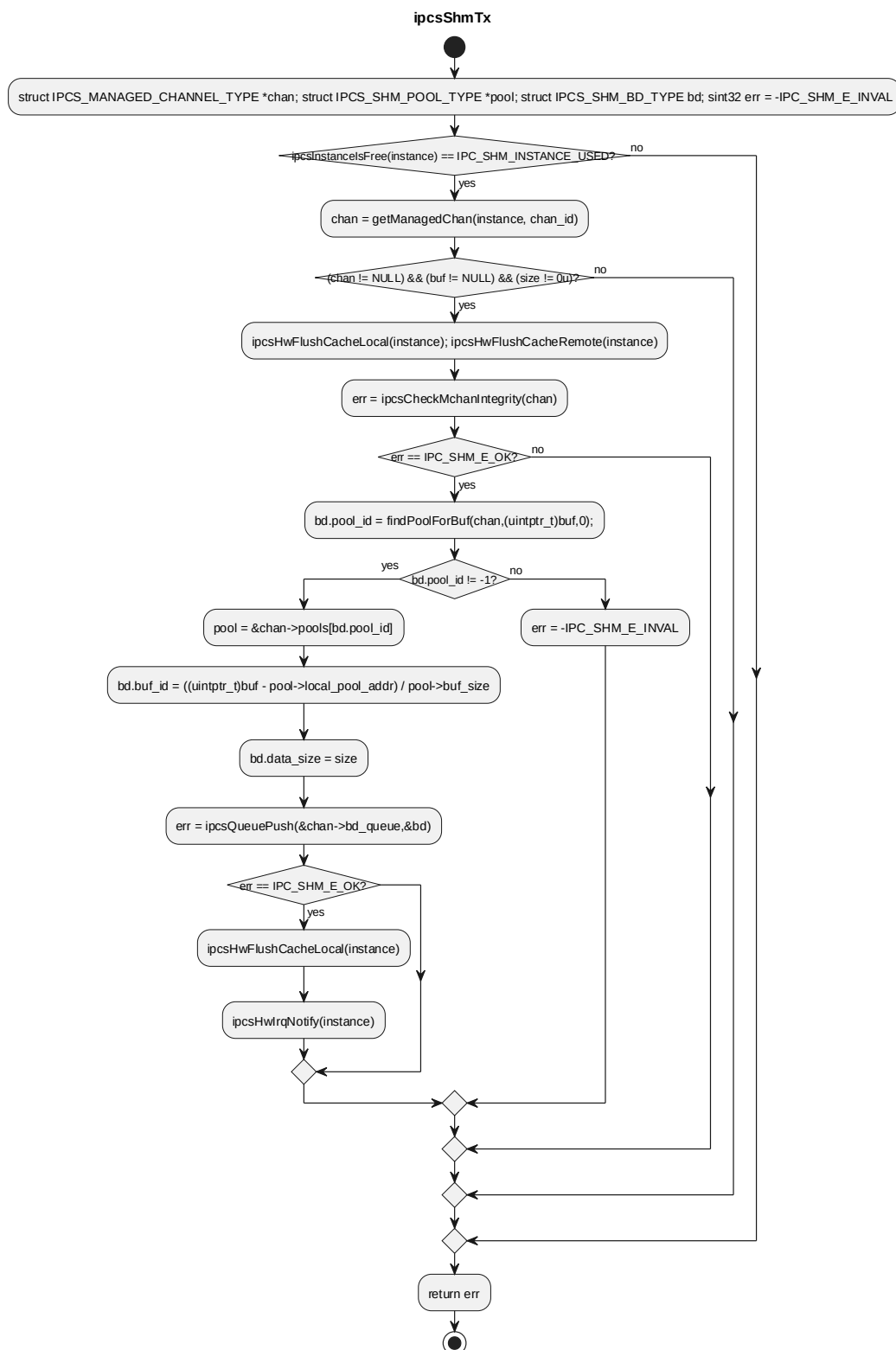


5.3.5 ipcsShmTx

对应软件架构 ID	Drv_Ipcs_Core_Cmp
软件单元 ID	SWU_IPCS_CORE_SHM
函数说明	该函数在 managed channel 上提交已写入的 buffer descriptor，并通过 IPCS-HAL 通知远端。

函数原型	sint32 ipcsShmTx(const uint8 instance, sint32 chan_id, void *buf, uint32 size)			
制约条件	instance 已初始化; chan_id 对应 managed channel; buf 非空; size 非 0; managed channel integrity 校验通过; buf 属于某个本端 pool。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared memory channel 索引
	I	buf	void *	managed channel buffer 指针
	I	size	uint32	已写入 buffer 的有效数据大小, 单位为 byte
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK: 发送提交并通知成功; 负错误码: instance/channel/buf/size/integrity 无效或 queue push 失败	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	ipcs/ipcs_cores/ipc-shm.h			
满足需求	IPCS_001, IPCS_015, IPCS_018, IPCS_022, IPCS_023, IPCS_028, IPCS_034, IPCS_035			

processing flow

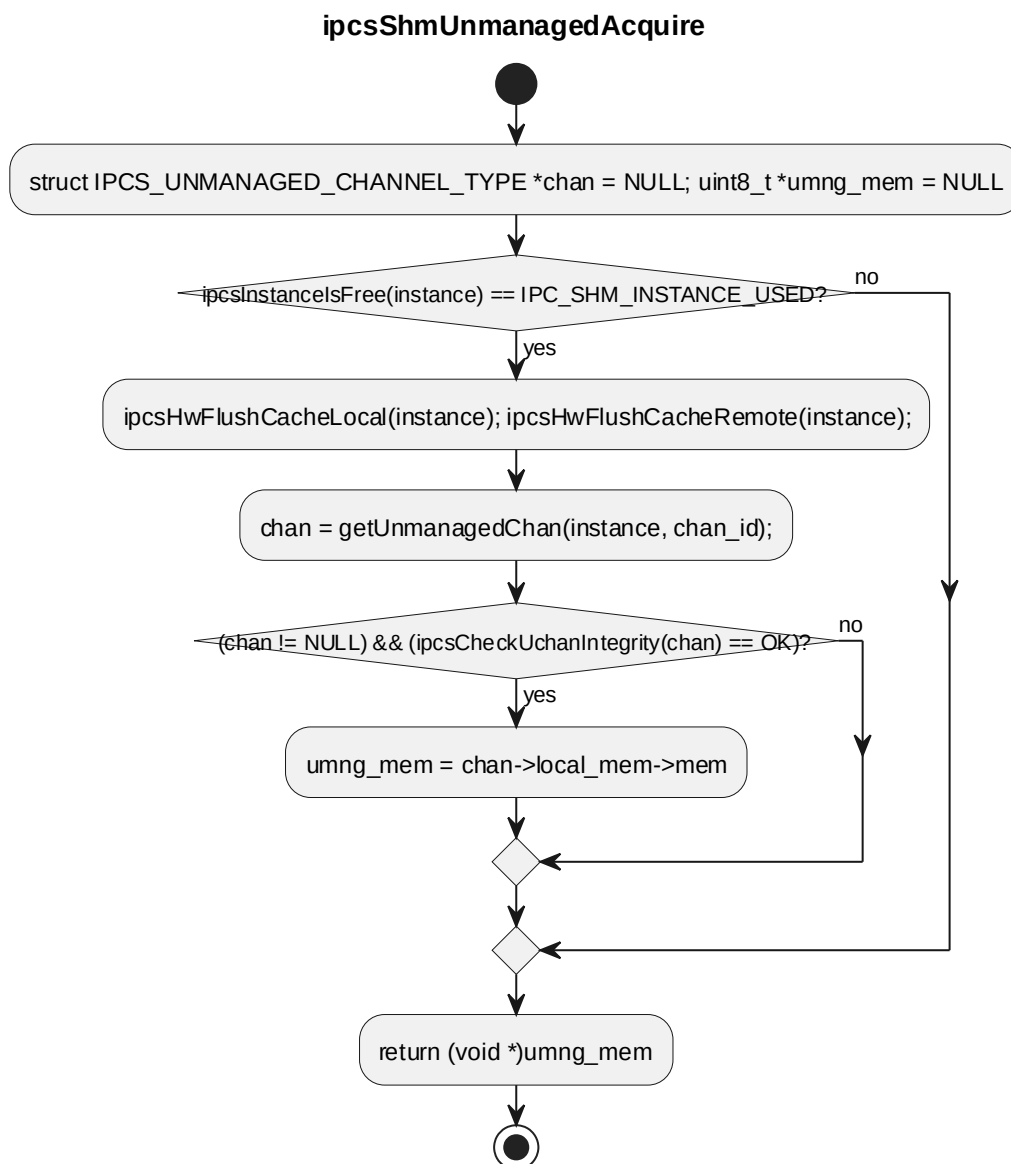


5.3.6 ipcsShmUnmanagedAcquire

对应软件架构 ID	Drv_Ipcs_Core_Cmp
软件单元 ID	SWU_IPCS_CORE_SHM
函数说明	该函数获取 unmanaged channel 的本端通道内存入口。

函数原型	void * ipcsShmUnmanagedAcquire(const uint8 instance, sint32 chan_id)			
制约条件	instance 已初始化; chan_id 对应 unmanaged channel; unmanaged channel integrity 校验通过。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared memory channel 索引
返回值	数据类型		说明	
	void *		非 NULL: unmanaged channel 本端内存地址; NULL: instance/channel/integrity 无效	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	ipcs/ipcs_cores/ipc-shm.h			
满足需求	IPCS 021, IPCS 034, IPCS 035			

processing flow

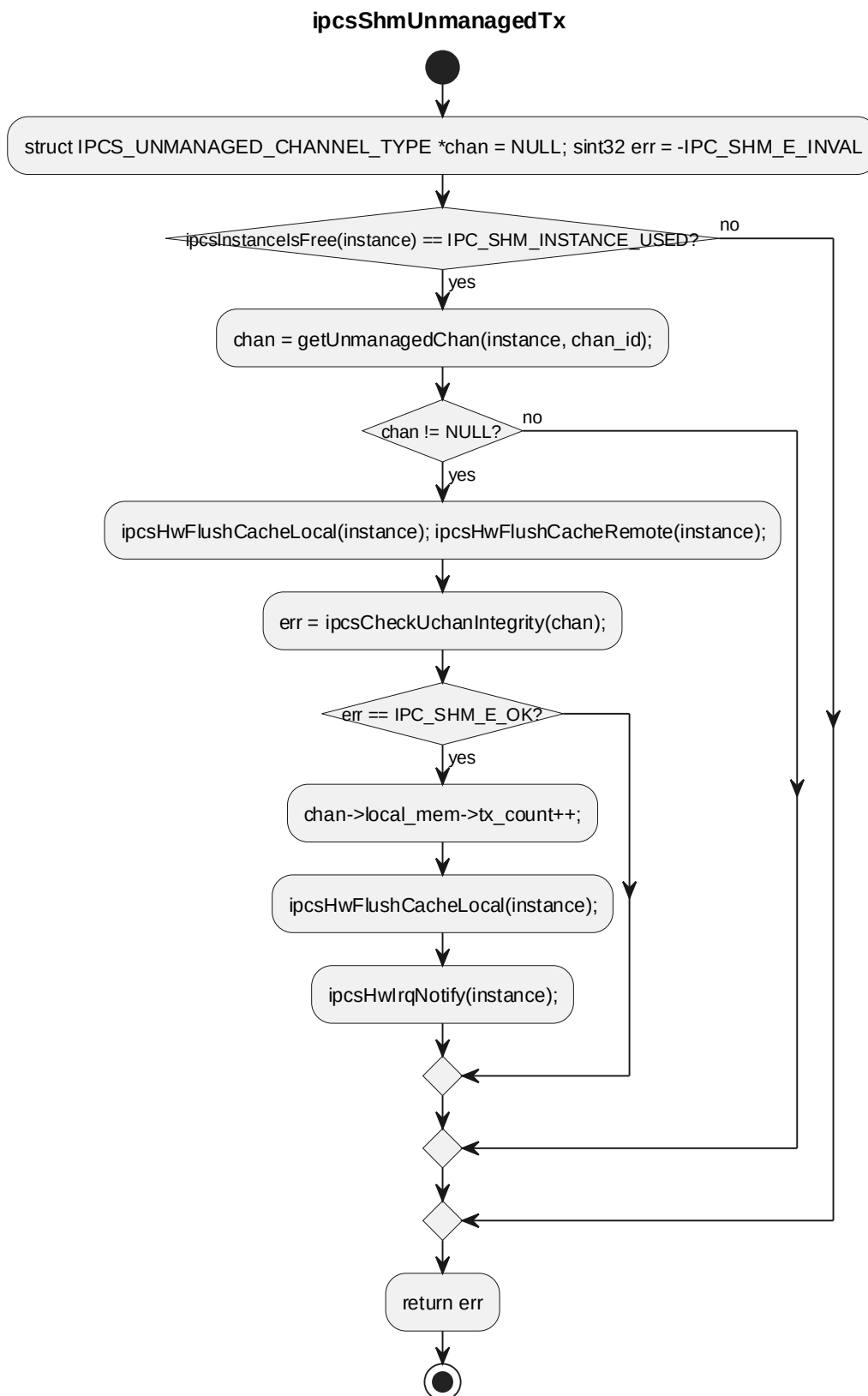


5.3.7 ipcsShmUnmanagedTx

对应软件架构ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数在 unmanaged channel 上递增本端发送计数, 并通过 IPCS-HAL 通知远端。			
函数原型	sint32 ipcsShmUnmanagedTx(const uint8 instance, sint32 chan_id)			
制约条件	instance 已初始化; chan_id 对应 unmanaged channel; unmanaged channel integrity 校验通过。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引

	l	chan_id	sint32	IPCS shared memory channel 索引
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK: 发送计数更新并通知成功; 负错误码: instance/channel/integrity 无效	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	ipcs/ipcs_cores/ipc-shm.h			
满足需求	IPCS_001, IPCS_021, IPCS_022, IPCS_028, IPCS_034, IPCS_035			

processing flow

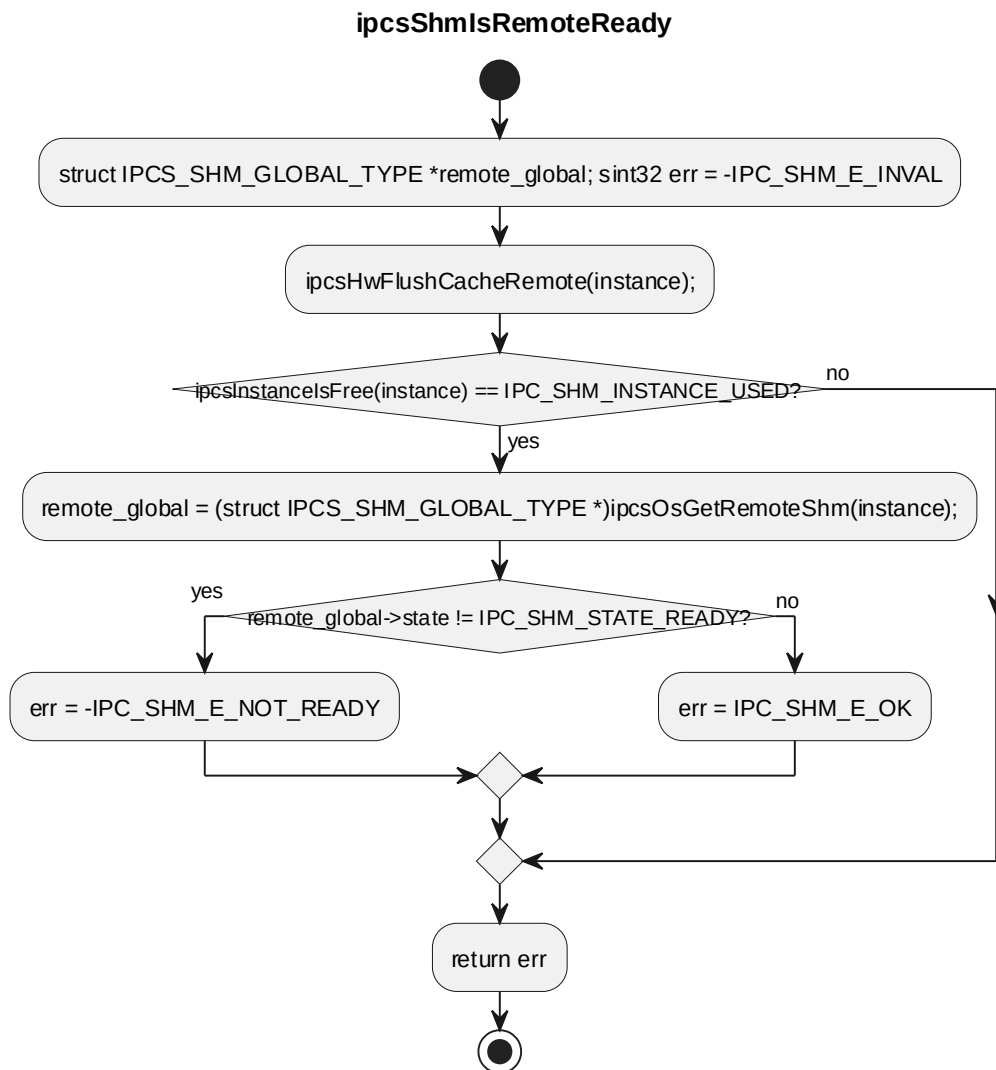


5.3.8 ipcsShmIsRemoteReady

对应软件架构 ID	Drv_Ipcs_Core_Cmp
-----------	-------------------

软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数读取远端 shared memory 起始处的 ready 状态，判断对端是否初始化完成。			
函数原型	sint32 ipcsShmIsRemoteReady(const uint8 instance)			
制约条件	instance 已初始化；远端 shared memory 起始位置包含 IPCS_SHM_GLOBAL_TYPE。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK: 远端 ready; -IPC_SHM_E_NOT_READY: 远端未 ready; -IPC_SHM_E_INVALID: instance 无效	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	ipcs/ipcs_cores/ipc-shm.h			
满足需求	IPCS_001, IPCS_022, IPCS_034			

processing flow

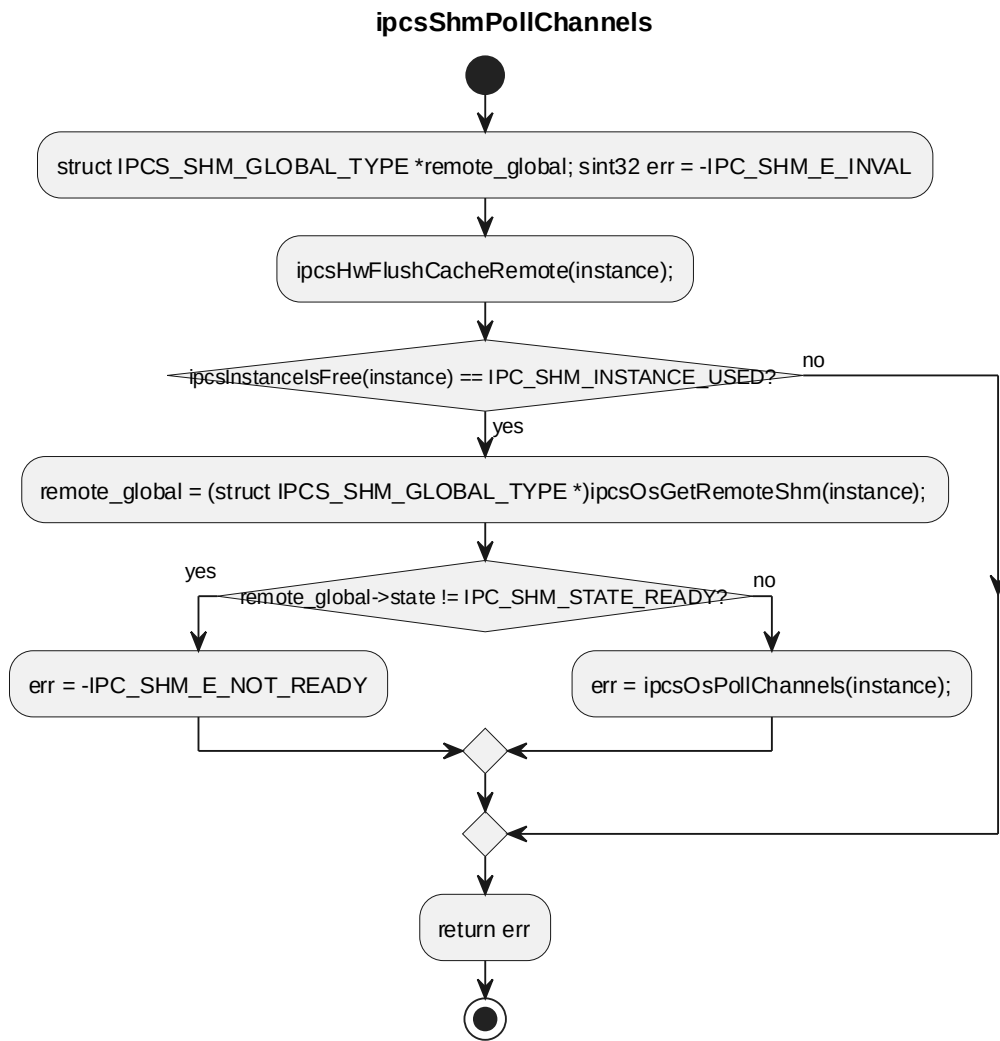


5.3.9 ipcsShmPollChannels

对应软件架构 ID	Drv_lpcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数在 polling 场景下推进当前 instance 的接收处理。			
函数原型	sint32 ipcsShmPollChannels(const uint8 instance)			
制约条件	instance 已初始化; 远端 ready; OSAL 变体支持在 IPC_IRQ_NONE 场景下调用 polling 接收。			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	sint32		非负值: 处理的消息数量; 负错误码: instance 无效、远端未 ready 或	

	OSAL polling 不支持/无效
函数定义文件	ipcs/ipcs_cores/ipc-shm.c
函数声明文件	ipcs/ipcs_cores/ipc-shm.h
满足需求	IPCS_019, IPCS_022, IPCS_023, IPCS_034, IPCS_039

processing flow



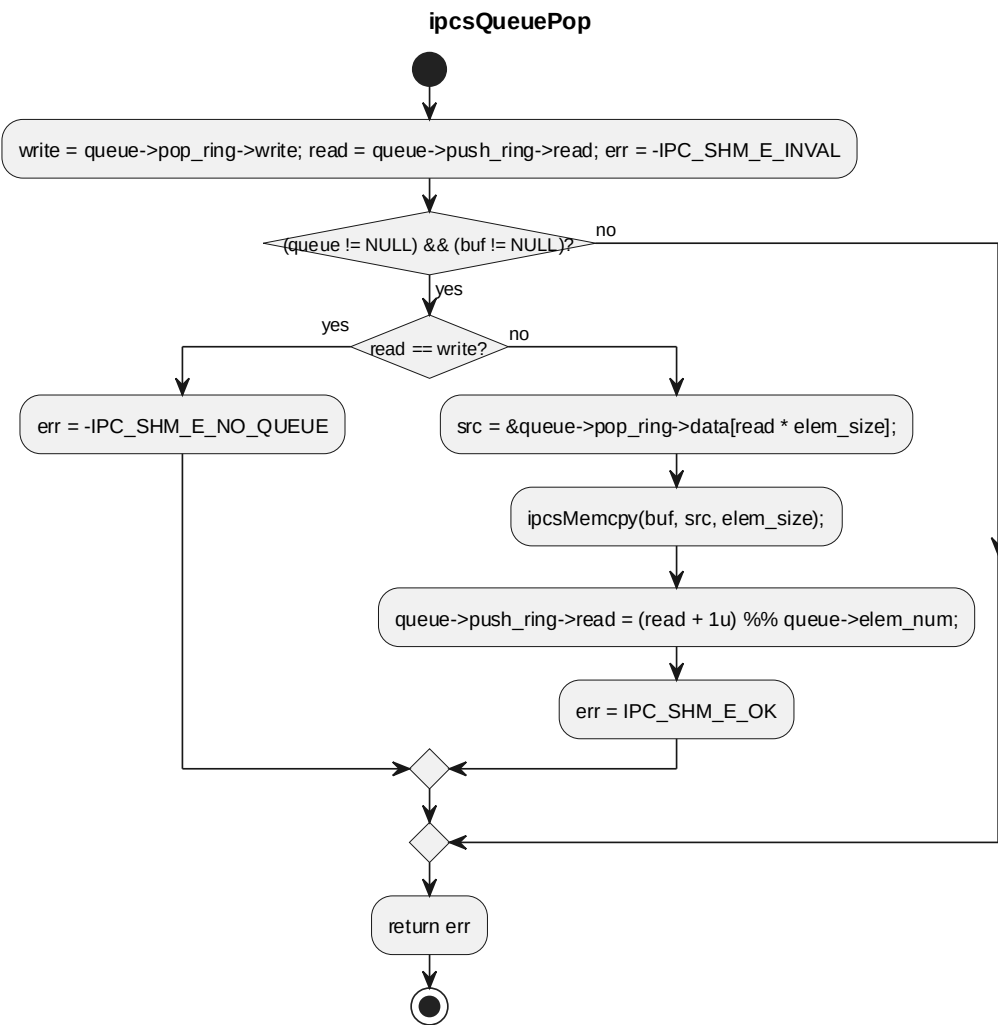
5. 4 SWU CORE/QUEUE/UTIL INTERNAL 内部函数

5. 4. 1 ipcsQueuePop

对应软件架构 ID	Drv_Ipcs_Queue_Cmp			
软件单元 ID	SWU_IPCS_CORE_QUEUE			
函数说明	removes element from queue			
函数原型	sint32 ipcsQueuePop(struct IPCS_QUEUE_TYPE *queue, void *buf)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	queue	struct IPCS_QUEUE_TYP	[IN] queue pointer

			E *	
	I	buf	void *	[OUT] pointer where to copy the removed element
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK on success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-queue.c			
函数声明文件	ipcs/ipcs_cores/ipc-queue.h			

processing flow

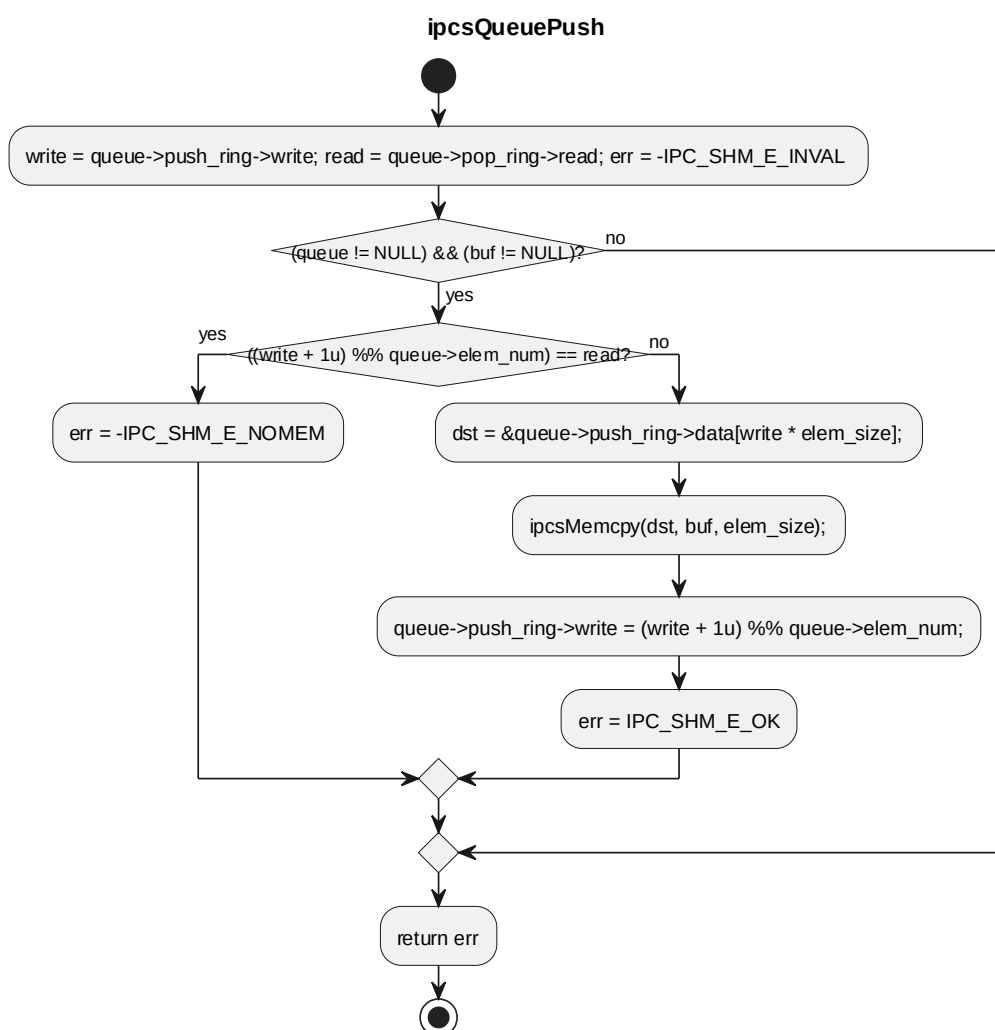


5.4.2 ipcsQueuePush

对应软件架构 ID	Drv_Ipcs_Queue_Cmp
软件单元 ID	SWU_IPCS_CORE_QUEUE
函数说明	pushes element into the queue

函数原型	sint32 ipcsQueuePush(struct IPCS_QUEUE_TYPE *queue, const void *buf)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	queue	struct IPCS_QUEUE_TYP E *	[IN] queue pointer
	I	buf	const void *	[IN] pointer to element to be pushed into the queue
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK on success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-queue.c			
函数声明文件	ipcs/ipcs_cores/ipc-queue.h			

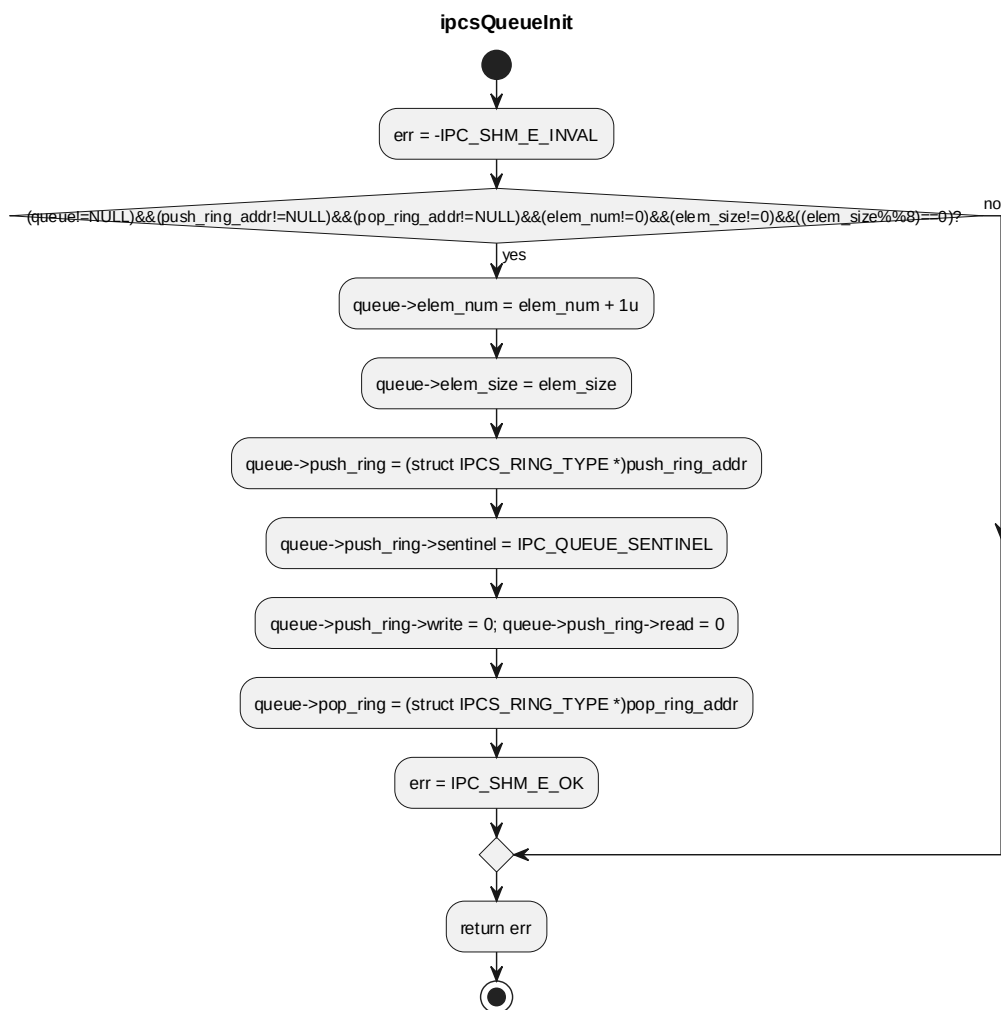
processing flow



5.4.3 ipcsQueueInit

对应软件架构 ID	Drv_Ipcs_Queue_Cmp			
软件单元 ID	SWU_IPCS_CORE_QUEUE			
函数说明	initializes queue and maps push/pop rings in memory			
函数原型	sint32 ipcsQueueInit(struct IPCS_QUEUE_TYPE *queue, uint16 elem_num, uint8 elem_size, uintptr_t push_ring_addr, uintptr_t pop_ring_addr)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	queue	struct IPCS_QUEUE_TYPE *	[IN] queue pointer
	I	elem_num	uint16	[IN] number of elements in queue
	I	elem_size	uint8	[IN] element size in bytes (8-byte multiple)
	I	push_ring_addr	uintptr_t	[IN] local addr where to map the push buffer ring
	I	pop_ring_addr	uintptr_t	[IN] remote addr where to map the pop buffer ring
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK on success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-queue.c			
函数声明文件	ipcs/ipcs_cores/ipc-queue.h			

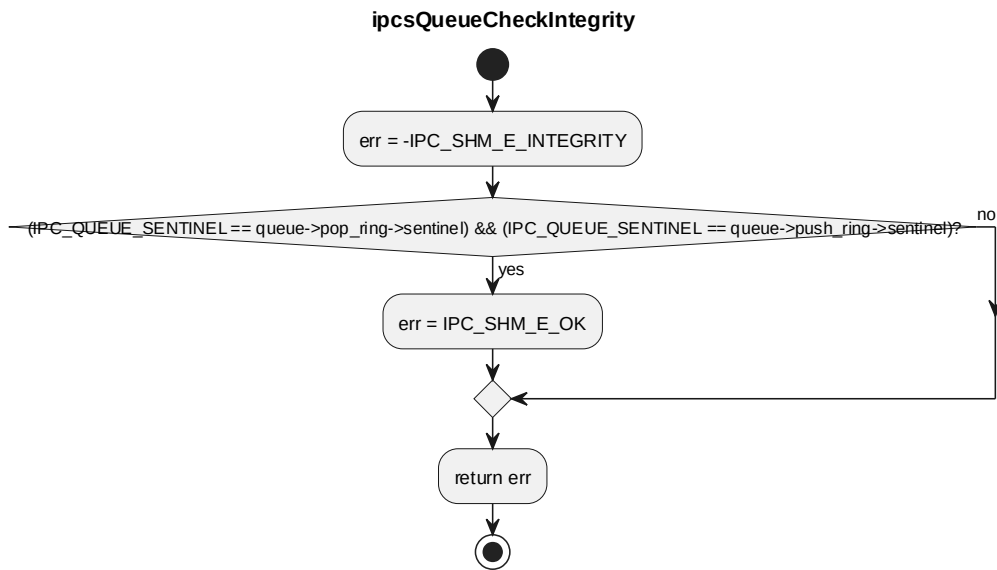
processing flow



5.4.4 ipcsQueueCheckIntegrity

对应软件架构 ID	Drv_Ipcs_Queue_Cmp			
软件单元 ID	SWU_IPCS_CORE_QUEUE			
函数说明	check if the sentinel was not overwritten			
函数原型	sint32 ipcsQueueCheckIntegrity(struct IPCS_QUEUE_TYPE *queue)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	queue	struct IPCS_QUEUE_TYP E *	[IN] queue pointer
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK on success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-queue.c			
函数声明文件	ipcs/ipcs_cores/ipc-queue.h			

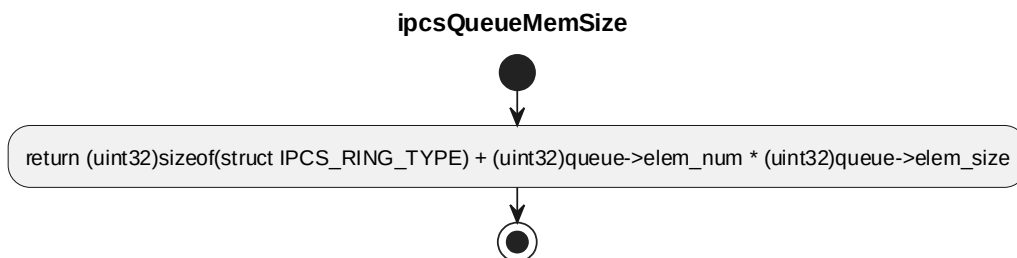
processing flow



5.4.5 ipcsQueueMemSize

对应软件架构 ID	Drv_Ipcs_Queue_Cmp			
软件单元 ID	SWU_IPCS_CORE_QUEUE			
函数说明	return queue footprint in local mapped memory			
函数原型	static inline uint32 ipcsQueueMemSize(struct IPCS_QUEUE_TYPE *queue)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	queue	struct IPCS_QUEUE_TYP E *	[IN] queue pointer
返回值	数据类型		说明	
	uint32		size of local mapped memory occupied by queue	
函数定义文件	ipcs/ipcs_cores/ipc-queue.h			
函数声明文件	-			

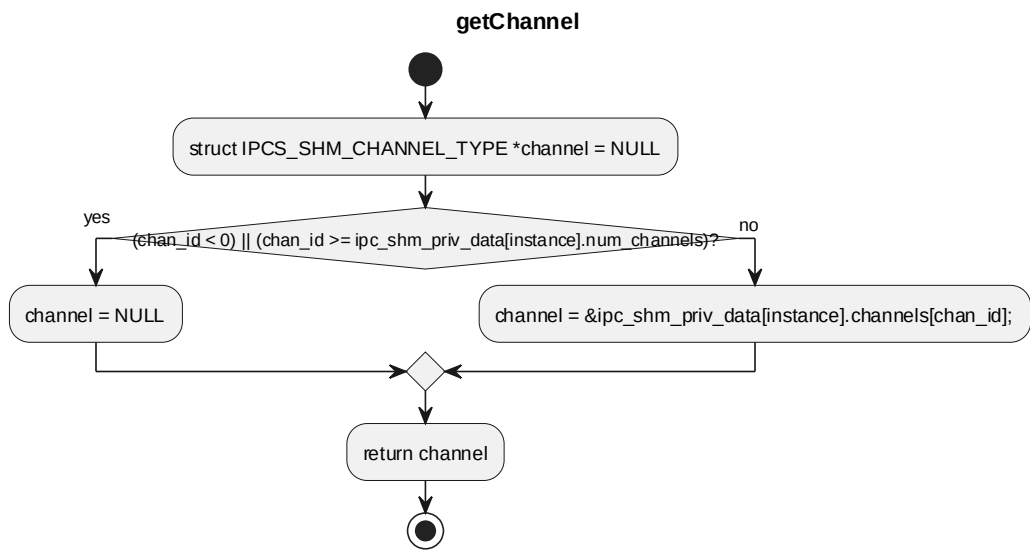
processing flow



5. 4. 6 getChannel

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数校验 channel id 并返回 IPCS shared memory channel 私有数据指针。			
函数原型	static inline struct IPCS_SHM_CHANNEL_TYPE * getChannel(const uint8 instance, sint32 chan_id)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared memory channel 索引
返回值	数据类型		说明	
	struct IPCS_SHM_CHANNEL_TYPE *		源码返回值	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow

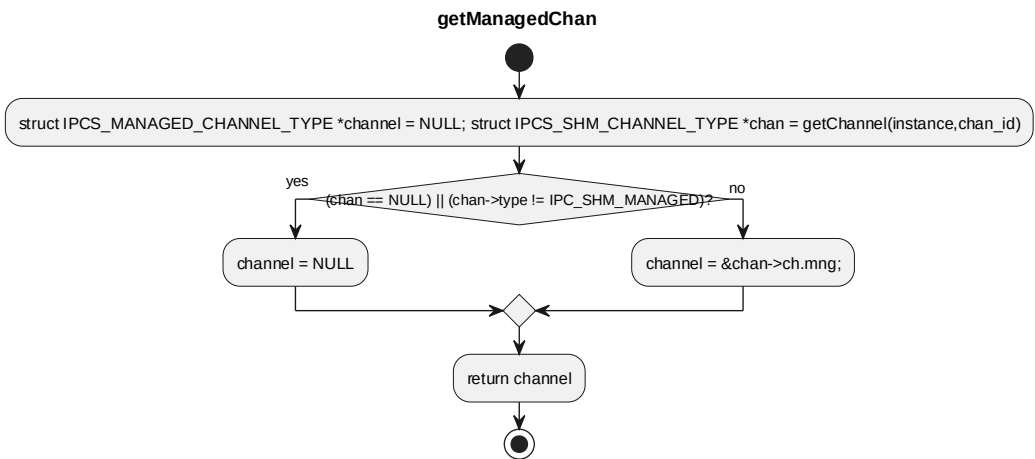


5. 4. 7 getManagedChan

对应软件架构 ID	Drv_Ipcs_Core_Cmp
软件单元 ID	SWU_IPCS_CORE_SHM
函数说明	该函数校验 channel 是否为 managed 类型, 并返回 managed channel 私有数据指针。
函数原型	static inline struct IPCS_MANAGED_CHANNEL_TYPE *

	getManagedChan(const uint8 instance, sint32 chan_id)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared memory channel 索引
返回值	数据类型		说明	
	struct IPCS_MANAGED_CHANNEL_TYPE *		源码返回值	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow

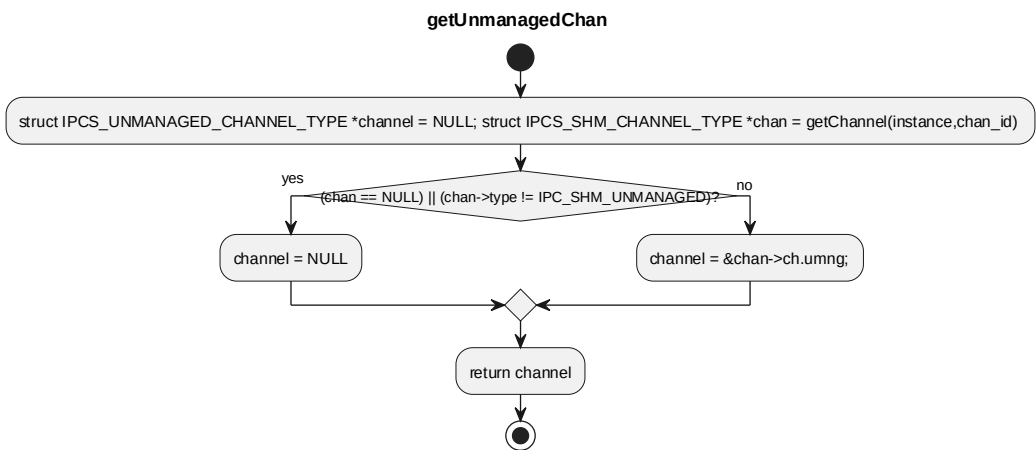


5. 4. 8 getUnmanagedChan

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数校验 channel 是否为 unmanaged 类型，并返回 unmanaged channel 私有数据指针。			
函数原型	static inline struct IPCS_UNMANAGED_CHANNEL_TYPE * getUnmanagedChan(const uint8 instance, sint32 chan_id)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	chan_id	sint32	IPCS shared

				memory channel 索引
返回值	数据类型		说明	
	struct IPCS_UNMANAGED_CHANNEL_TYPE *		源码返回值	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

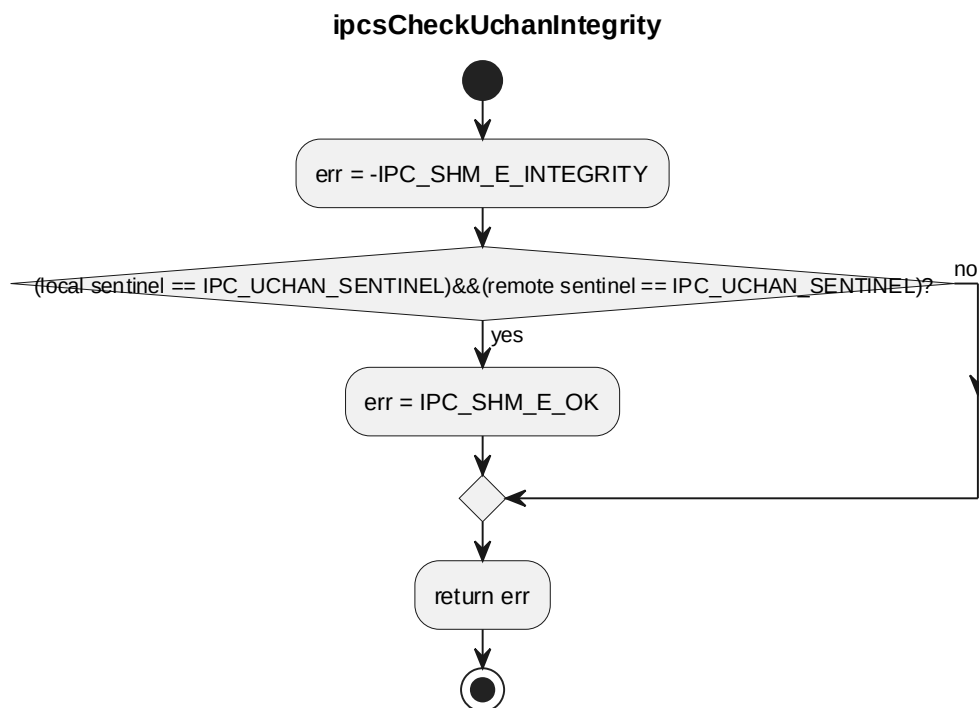
processing flow



5.4.9 ipcsCheckUchanIntegrity

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数检查 unmanaged channel 本端和远端 sentinel 是否保持完整。			
函数原型	static sint32 ipcsCheckUchanIntegrity(const struct IPCS_UNMANAGED_CHANNEL_TYPE *uchan)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	uchan	const struct IPCS_UNMANAG ED_CHANNEL_TY PE *	源码参数
返回值	数据类型		说明	
	sint32		源码返回值	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

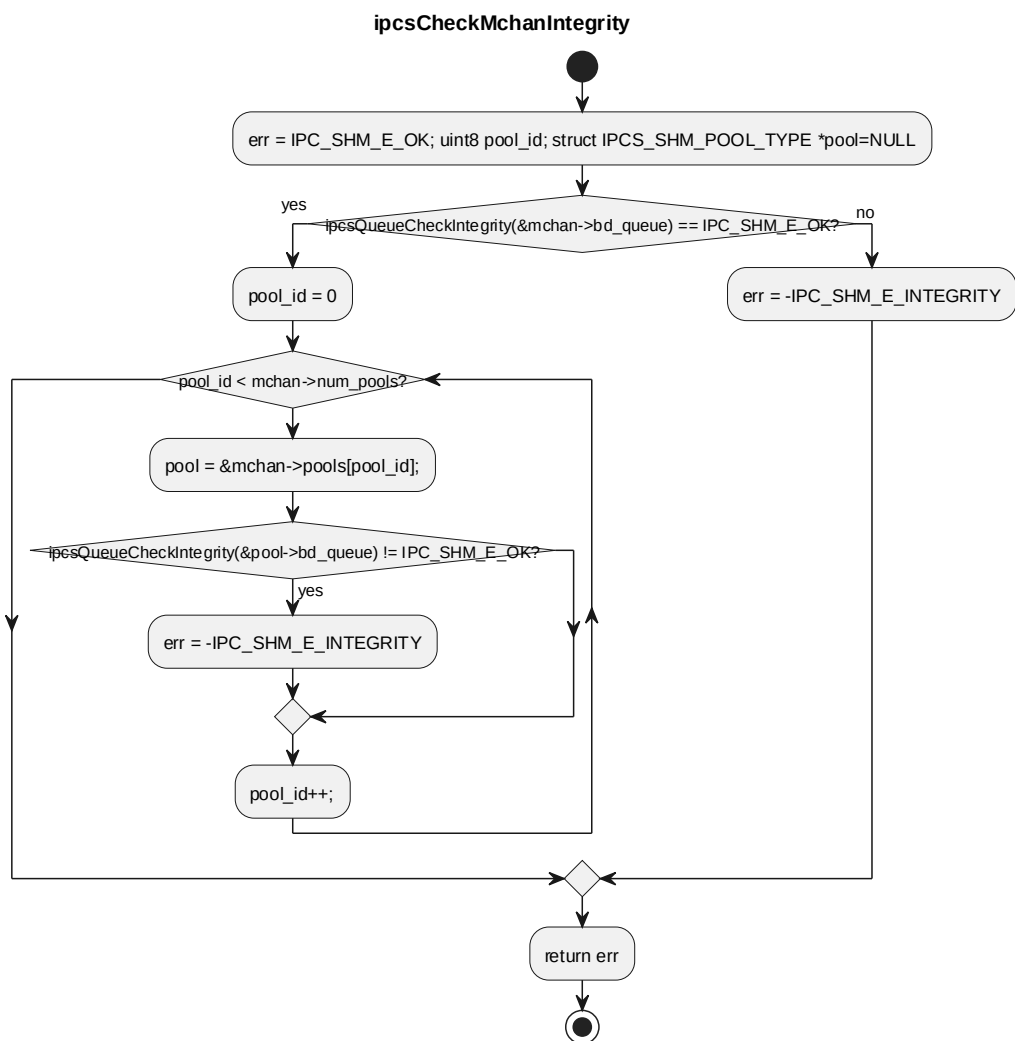
processing flow



5. 4. 10 ipcsCheckMchanIntegrity

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	该函数检查 managed channel 的 channel queue 及所有 pool queue sentinel 是否保持完整。			
函数原型	static sint32 ipcsCheckMchanIntegrity(struct IPCS_MANAGED_CHANNEL_TYPE *mchan)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	mchan	struct IPCS_MANAGED_CHANNEL_TYPE *	源码参数
返回值	数据类型		说明	
	sint32		源码返回值	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow



5. 4. 11 ipcsChannelRx

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	handle Rx for a single channel			
函数原型	static sint32 ipcsChannelRx(const uint8 instance, sint32 chan_id, sint32 budget)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
	I	chan_id	sint32	channel id
	I	budget	sint32	available work budget (number of messages to be processed)
返回值	数据类型		说明	

	sint32	work done
函数定义文件	ipcs/ipcs_cores/ipc-shm.c	
函数声明文件	-	

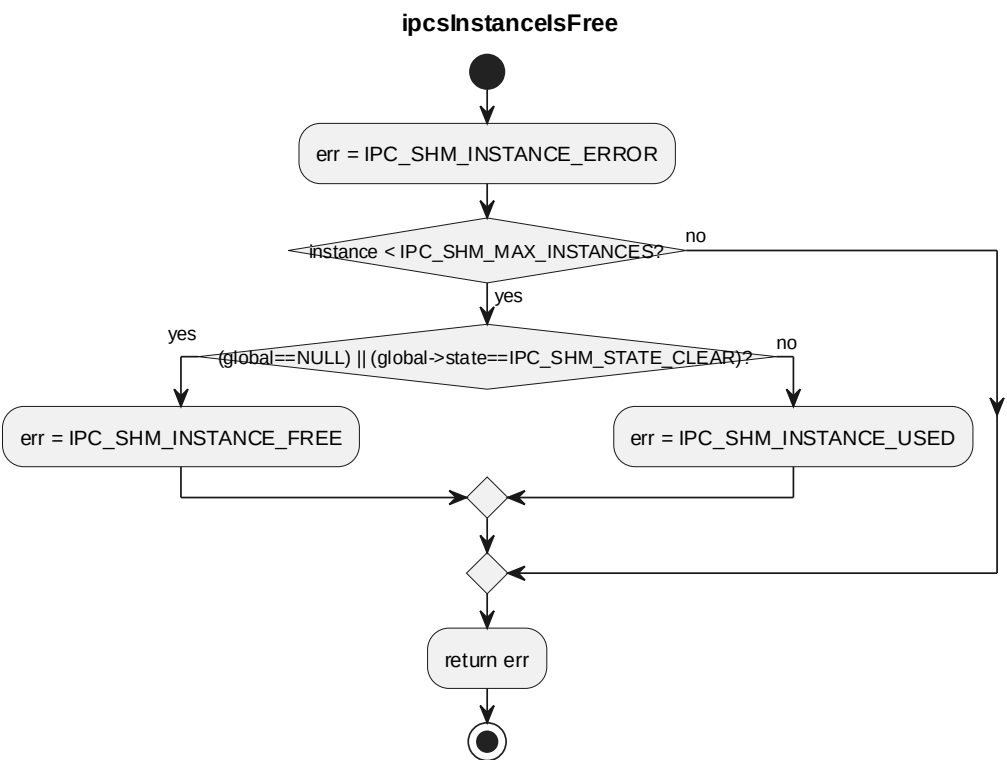
processing flow



5. 4. 12 ipcsInstanceIsFree

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	determine if the instance is used or not			
函数原型	static uint8 ipcsInstancelFree(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
返回值	数据类型		说明	
	uint8		IPC_SHM_INSTANCE_FREE if instance is free,	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

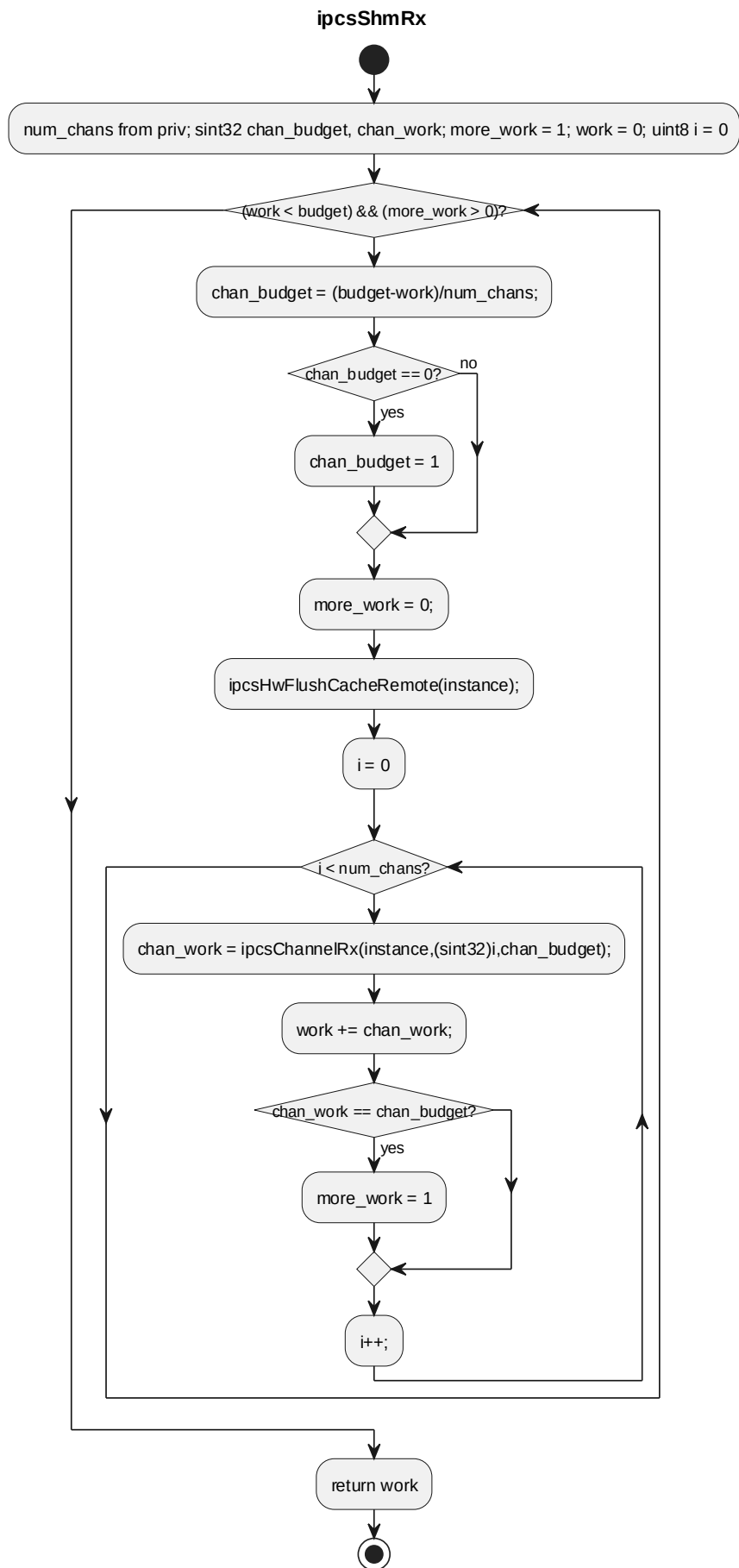
processing flow



5. 4. 13 ipcsShmRx

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	shm Rx handler, called from softirq			
函数原型	static sint32 ipcsShmRx(const uint8 instance, sint32 budget)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
	I	budget	sint32	available work budget (number of messages to be processed)
返回值	数据类型		说明	
	sint32		work done	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

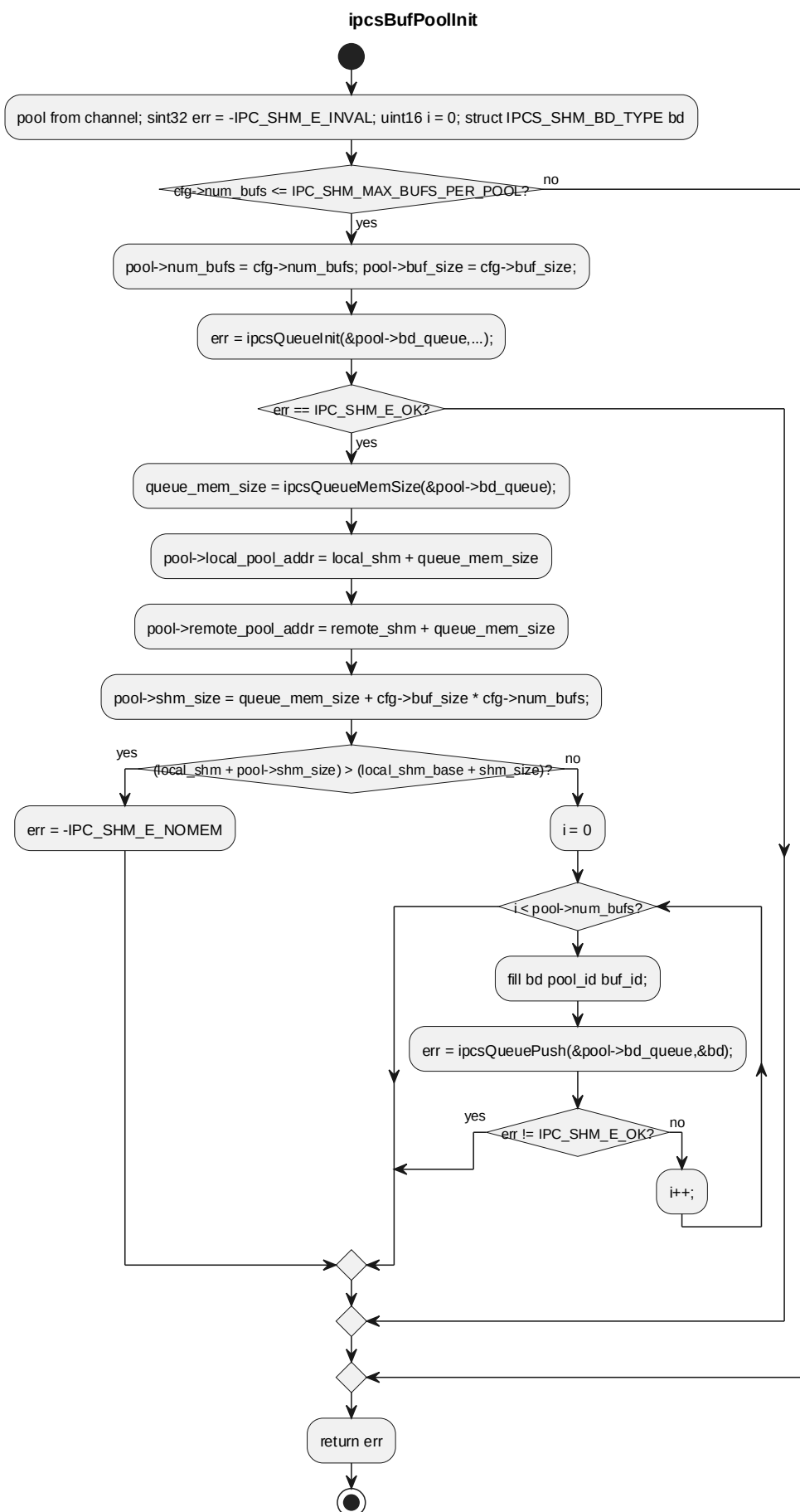
processing flow



5.4.14 ipcsBufPoolInit

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	init buffer pool			
函数原型	static sint32 ipcsBufPoolInit(const uint8 instance, sint32 chan_id, sint32 pool_id, struct IPCS_SHM_POOL_ADDR_TYPE mng_pool, const struct IPCS_SHM_POOL_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
	I	chan_id	sint32	channel index
	I	pool_id	sint32	pool index in channel
	I	mng_pool	struct IPCS_SHM_POOL_ADDR_TYPE	源码参数
	I	cfg	const struct IPCS_SHM_POOL_CFG_TYPE *	channel configuration parameters
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK for success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

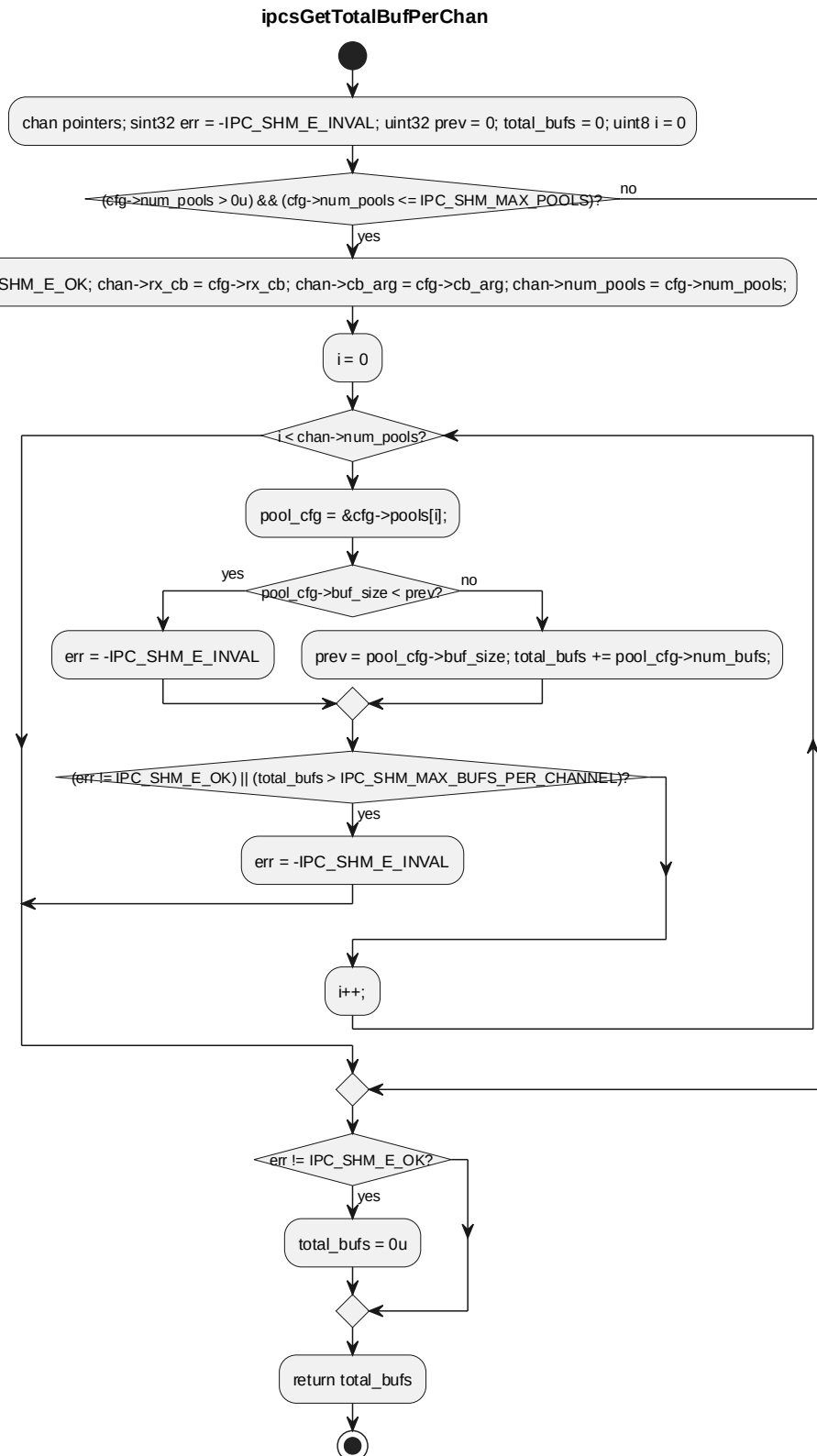
processing flow



5. 4. 15 ipcsGetTotalBufPerChan

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	get total buffers of an managed channel			
函数原型	static uint32 ipcsGetTotalBufPerChan(const uint8 instance, sint32 chan_id, const struct IPCS_SHM_MANAGED_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
	I	chan_id	sint32	channel id
	I	cfg	const struct IPCS_SHM_MANAGED_CFG_TYPE *	managed channel configuration
返回值	数据类型		说明	
	uint32		total buffers, 0 if error	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow

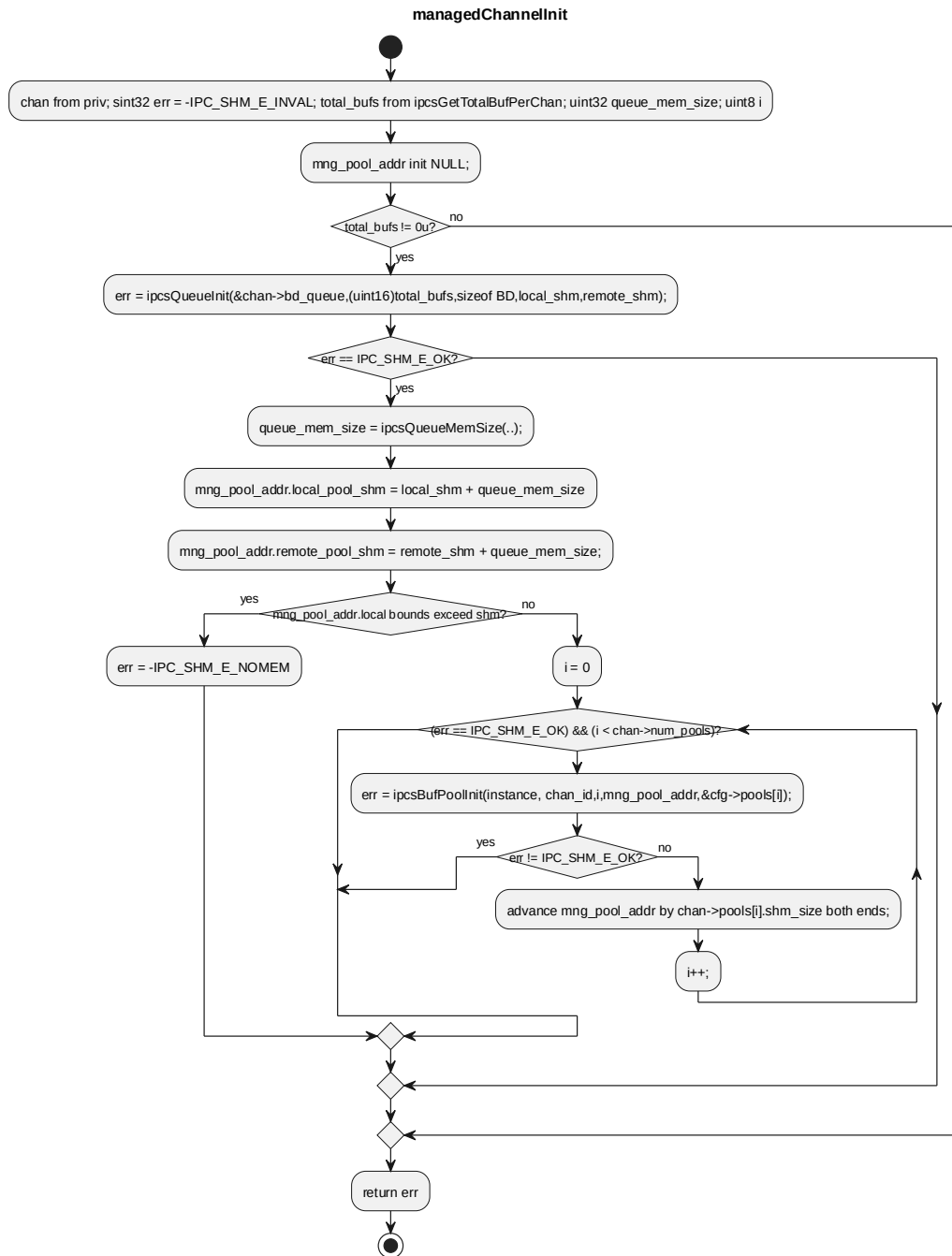


5.4.16 managedChannelInit

对应软件架构 ID	Drv_Ipcs_Core_Cmp
软件单元 ID	SWU_IPCS_CORE_SHM

函数说明	initialize managed channel			
函数原型	static sint32 managedChannelInit(const uint8 instance, sint32 chan_id, uintptr_t local_shm, uintptr_t remote_shm, const struct IPCS_SHM_MANAGED_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
	I	chan_id	sint32	channel id
	I	local_shm	uintptr_t	local shared memory
	I	remote_shm	uintptr_t	remote shared memort
	I	cfg	const struct IPCS_SHM_MANAGED_CFG_TYPE *	managed channel configuration
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK for success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow

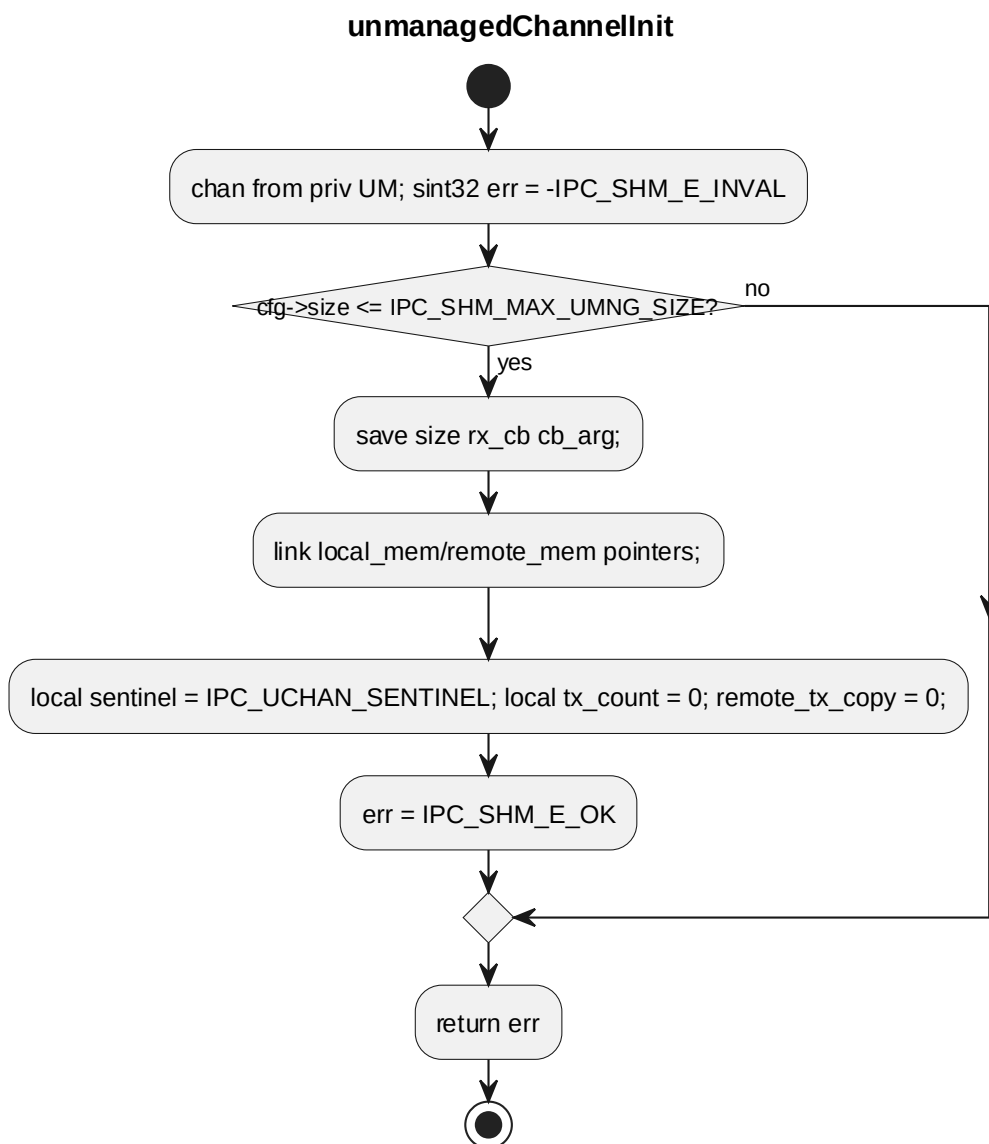


5. 4. 17 unmanagedChannelInit

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	initialize unmanaged channel			
函数原型	static sint32 unmanagedChannellnit(const uint8 instance, sint32 chan_id, uintptr_t local_shm, uintptr_t remote_shm, const struct IPCS_SHM_UNMANAGED_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明

	l	instance	const uint8	instance id
	l	chan_id	sint32	channel id
	l	local_shm	uintptr_t	local shared memory
	l	remote_shm	uintptr_t	remote shared memort
	l	cfg	const struct IPCS_SHM_UNMANAGED_CFG_T YPE *	unmanaged channel configuration
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK for success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow

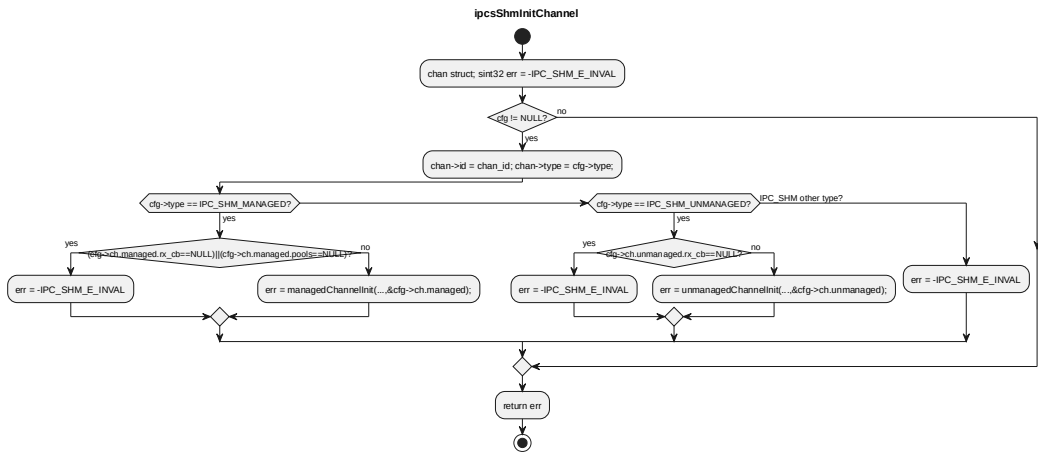


5.4.18 ipcsShmInitChannel

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	initialize a shared memory IPC channel			
函数原型	static sint32 ipcsShmInitChannel(const uint8 instance, sint32 chan_id, uintptr_t local_shm, uintptr_t remote_shm, const struct IPCS_SHM_CHANNEL_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
	I	chan_id	sint32	channel index
	I	local_shm	uintptr_t	local channel

				shared memory address
	l	remote_shm	uintptr_t	remote channel shared memory address
	l	cfg	const struct IPCS_SHM_CHANNEL_CFG_TYPE *	channel configuration parameters
返回值	数据类型		说明	
	sint32		0 for success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

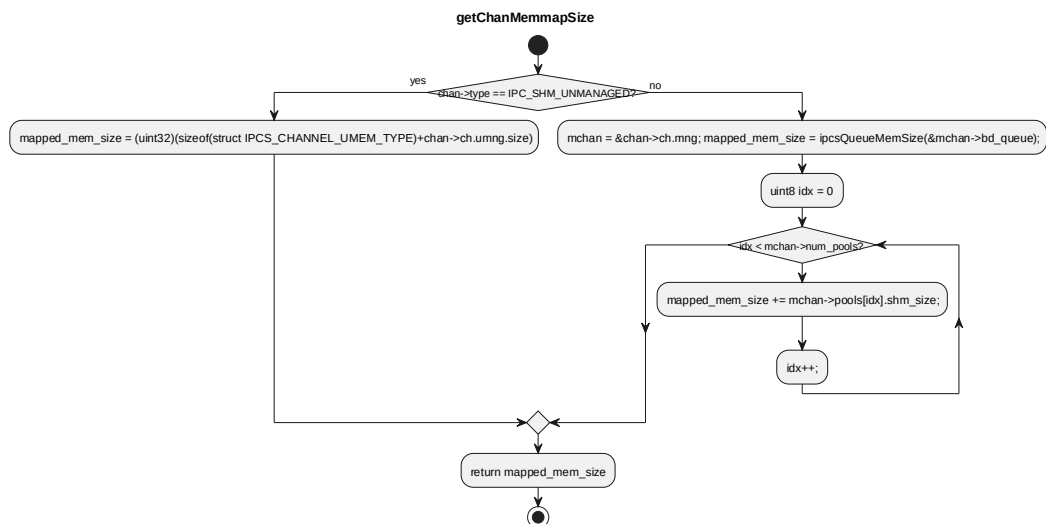
processing flow



5. 4. 19 getChanMemmapSize

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	Get channel local mapped memory size			
函数原型	static uint32 getChanMemmapSize(const uint8 instance, sint32 chan_id)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	l	instance	const uint8	instance id
	l	chan_id	sint32	channel id
返回值	数据类型		说明	
	uint32		Channel memory size	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

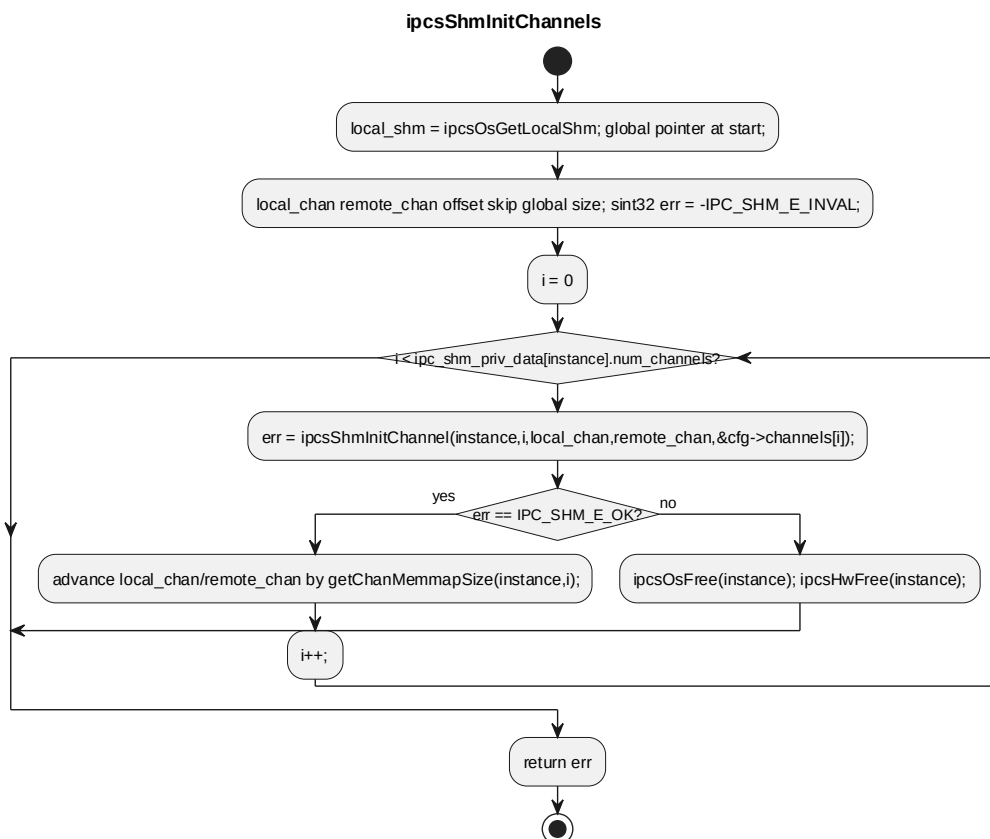
processing flow



5. 4. 20 ipcsShmInitChannels

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	initialize all shared memory IPC channel			
函数原型	static sint32 ipcsShmInitChannels(uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	uint8	instance id
	I	cfg	const struct IPCS_SHM_CFG_TYPE *	ipc-shm instance configuration
返回值	数据类型		说明	
	sint32		0 for success, error code otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

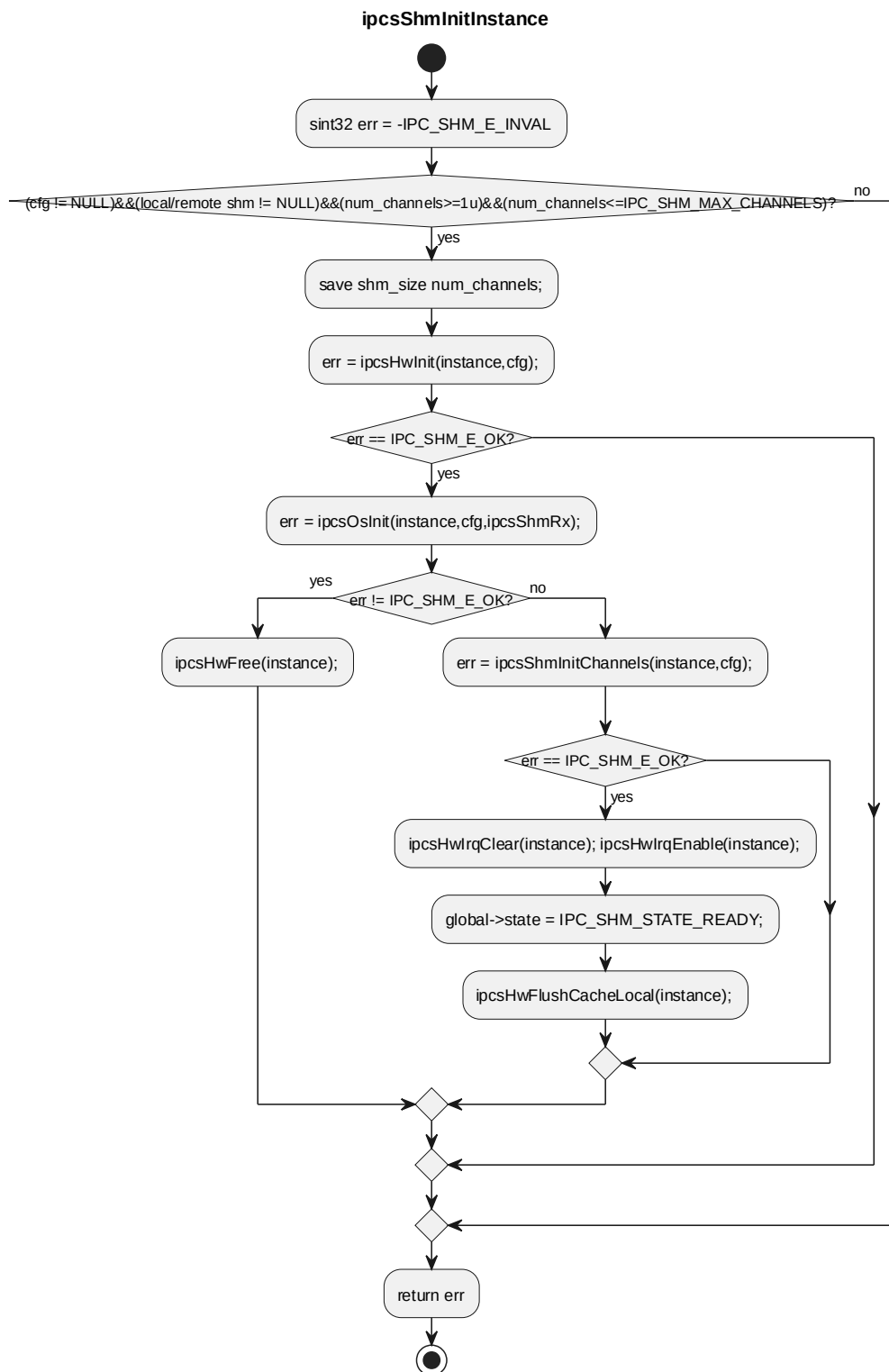
processing flow



5. 4. 21 ipcsShmInitInstance

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_SHM			
函数说明	Initialize only one instance shared memory device			
函数原型	static sint32 ipcsShmInitInstance(uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	uint8	instance id
	I	cfg	const struct IPCS_SHM_CFG_TYPE *	ipc-shm instance configuration
返回值	数据类型		说明	
	sint32		源码返回值	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow



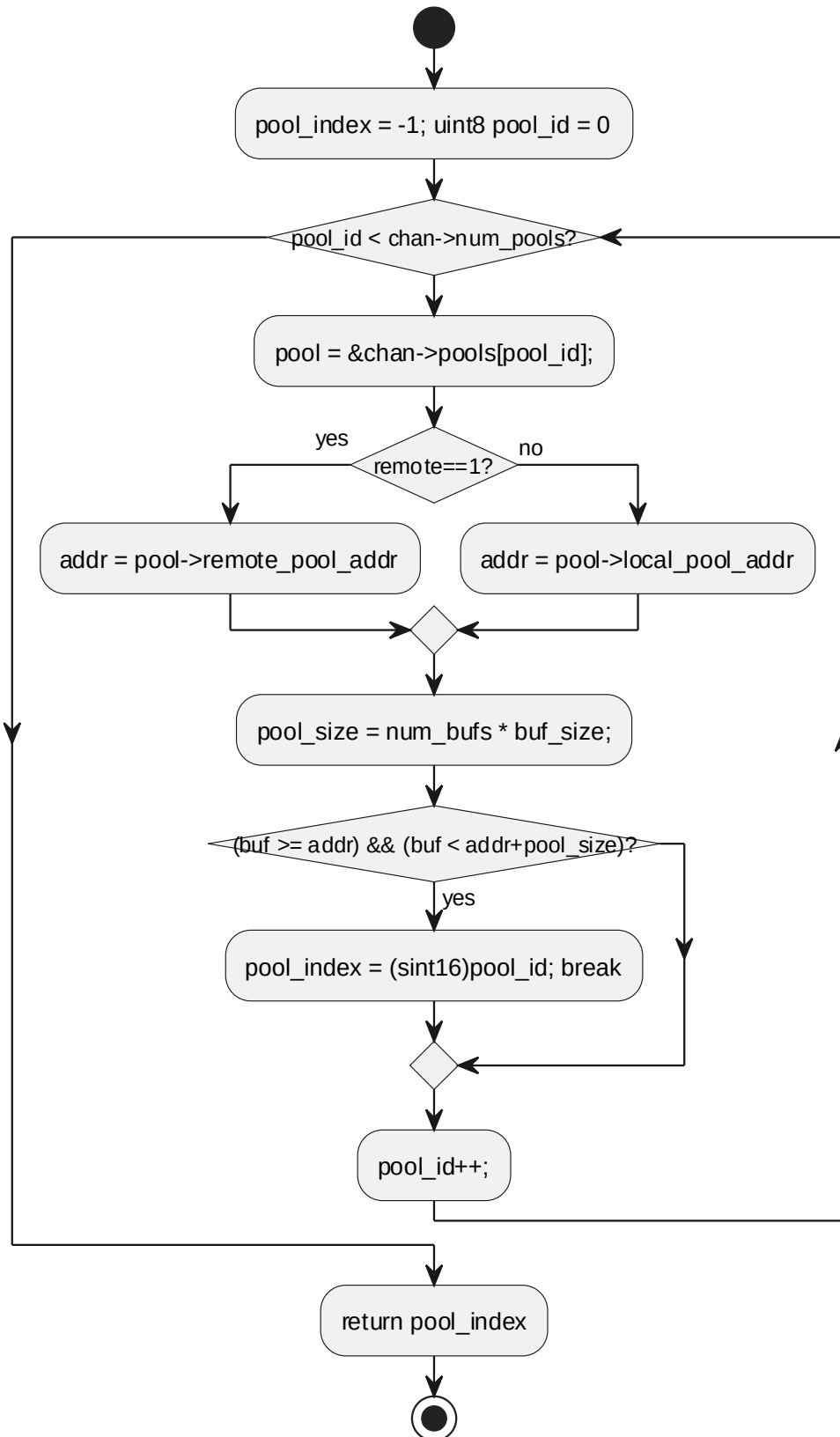
5. 4. 22 findPoolForBuf

对应软件架构 ID	Drv_Ipcs_Core_Cmp
软件单元 ID	SWU_IPCS_CORE_SHM
函数说明	Find the pool that owns the specified buffer.
函数原型	static sint16 findPoolForBuf(struct IPCS_MANAGED_CHANNEL_TYPE *chan,

	uintptr_t buf, sint32 remote)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	chan	struct IPCS_MANAGED _CHANNEL_TYPE *	managed channel pointer
	I	buf	uintptr_t	buffer pointer
	I	remote	sint32	flag telling if buffer is from remote OS
返回值	数据类型		说明	
	sint16		pool index on success, -1 otherwise	
函数定义文件	ipcs/ipcs_cores/ipc-shm.c			
函数声明文件	-			

processing flow

findPoolForBuf

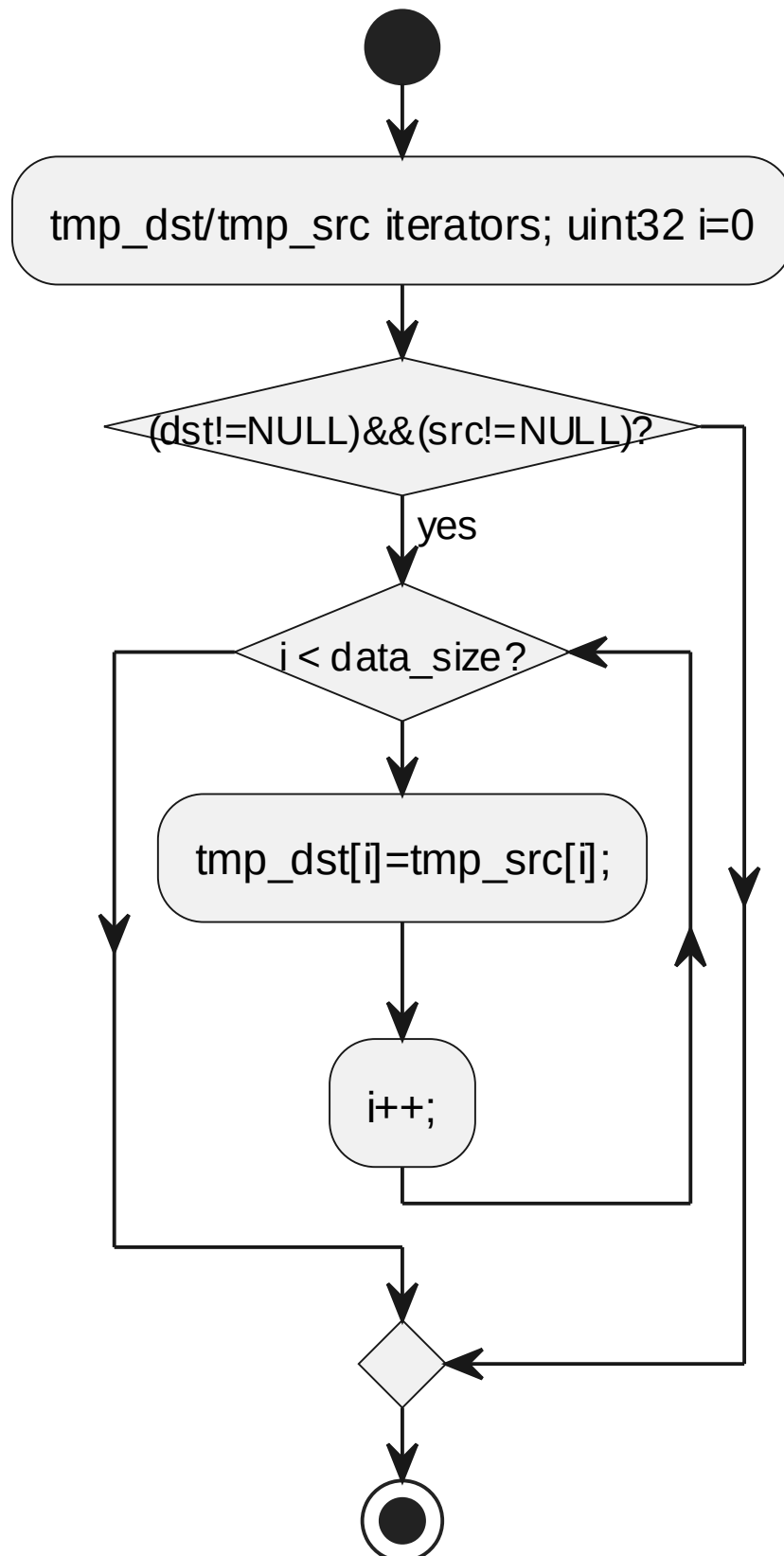


5. 4. 23 ipcsMemcpy

对应软件架构 ID	Drv_Ipcs_Core_Cmp			
软件单元 ID	SWU_IPCS_CORE_UTIL			
函数说明	-			
函数原型	void ipcsMemcpy(void *dst, const void *src, uint32 data_size)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	dst	void *	目的地址指针
	I	src	const void *	源地址指针
	I	data_size	uint32	复制数据大小，单位为 byte
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/ipcs_cores/ipc-util.c			
函数声明文件	ipcs/ipcs_cores/ipc-util.h			

processing flow

ipcsMemcpy



<i>5. 5</i>	<i>5. 6</i>	<i>5. 7</i>	<i>5. 8</i>	<i>5. 9</i>
<i>5. 10</i>	<i>5. 11</i>	<i>5. 12</i>	<i>5. 13</i>	<i>5. 14 GLOBAL VARIABLE S 全局变 量</i>

全局变量名称
全局变量类型
全局变量范围
全局变量描述
全局变量的存储 RAM 区
ipc_shm_priv_data
static struct IPCS_SHM_PRIV_TYPE [IPC_SHM_MAX_INSTANCES]
ipcs/ipcs_cores/ipc-shm.c
IPCS shm private data
编译器/链接器默认 RAM (.bss, 未指定 section)
5. 15 DATA TYPES 类型定义

5. 15. 1	5. 15. 2	5. 15. 3
5. 15. 4	5. 15. 5	5. 15. 6
5. 15. 7	5. 15. 8	5. 15. 9
5. 15. 10	5. 15. 11	5. 15. 12
5. 15. 13	5. 15. 14	5. 15. 15 struct IPCS_RING_TYPE

Type
Name
Description
uint64
sentinel
a magic word to ensure ring integrity
volatile uint32
write
write index, position used to store next byte in the buffer
volatile uint32

read
 read index, read next byte from this position
 uint8
 data[]
 circular buffer

5. 15. 16	5. 15. 17	5. 15. 18
5. 15. 19	5. 15. 20	5. 15. 21
5. 15. 22	5. 15. 23	5. 15. 24
5. 15. 25	5. 15. 26	5. 15. 27
5. 15. 28	5. 15. 29	5. 15. 30 struct IPCS_QUEUE_TYPE

Type
 Name
 Description
 uint16
 elem_num
 number of elements in queue
 uint8
 elem_size
 element size in bytes (8-byte multiple)
 struct IPCS_RING_TYPE
 *push_ring
 push buffer ring mapped in local shared memory
 struct IPCS_RING_TYPE
 *pop_ring
 pop buffer ring mapped in remote shared memory

5. 15. 31	5. 15. 32
5. 15. 33	5. 15. 34
5. 15. 35	5. 15. 36
5. 15. 37	5. 15. 38 enum IPCS_SHM_INSTANCE_STATE_E

Name

Description

IPC_SHM_INSTANCE_USED

instance is used

IPC_SHM_INSTANCE_FREE

instance is free and can be used

IPC_SHM_INSTANCE_ERROR

there are some errors

5. 15. 39	5. 15. 40	5. 15. 41
5. 15. 42	5. 15. 43	5. 15. 44
5. 15. 45	5. 15. 46	5. 15. 47 struct IPCS_SHM_POOL_ ADDR_TYPE

Type

Name

Description

uintptr_t

local_pool_shm

address of local buffer pool

uintptr_t

remote_pool_shm

address of remote buffer pool

5. 15. 48	5. 15. 49	5. 15. 50
5. 15. 51	5. 15. 52	5. 15. 53
5. 15. 54	5. 15. 55	5. 15. 56
5. 15. 57	5. 15. 58	5. 15. 59 struct IPCS_SHM_BD_TY PE

Type

Name

Description

sint16

pool_id

index of buffer pool

uint16

buf_id

index of buffer from buffer pool

uint32

data_size

size of data written in buffer

5. 15. 60	5. 15. 61	5. 15. 62
5. 15. 63	5. 15. 64	5. 15. 65
5. 15. 66	5. 15. 67	5. 15. 68
5. 15. 69	5. 15. 70	5. 15. 71
5. 15. 72	5. 15. 73	5. 15. 74
5. 15. 75	5. 15. 76	5. 15. 77
5. 15. 78	5. 15. 79	5. 15. 80 struct IPCS_SHM_POOL_TYPE

Type

Name

Description

uint16

num_bufs

number of buffers in pool

uint32

buf_size

size of buffers

uint32

shm_size

size of shared memory mapped by this pool (queue + bufs)

uintptr_t

local_pool_addr

address of local buffer pool

uintptr_t

remote_pool_addr

address of remote buffer pool

struct IPCS_QUEUE_TYPE

bd_queue

queue containing BDs of free buffers

5. 15. 81	5. 15. 82	5. 15. 83
5. 15. 84	5. 15. 85	5. 15. 86
5. 15. 87	5. 15. 88	5. 15. 89
5. 15. 90	5. 15. 91	5. 15. 92
5. 15. 93	5. 15. 94	5. 15. 95
5. 15. 96	5. 15. 97	5. 15. 98 struct IPCS_MANAGED_CHANN EL_TYPE

Type

Name

Description

struct IPCS_QUEUE_TYPE

bd_queue

queue containing BDs of sent/received buffers

uint8

num_pools

number of buffer pools

struct IPCS_SHM_POOL_TYPE

pools[IPC_SHM_MAX_POOLS]

buffer pools private data

void (rx_cb)(void cb_arg, const uint8 instance, sint32 chan_id, void *buf, uint32 size)

rx_cb

receive callback

void

*cb_arg

optional receive callback argument

5. 15. 99	5. 15. 100	5. 15. 101
5. 15. 102	5. 15. 103	5. 15. 104
5. 15. 105	5. 15. 106	5. 15. 107
5. 15. 108	5. 15. 109	5. 15. 110 struct IPCS_CHANNEL_UMEM_ TYPE

Type

Name

Description

uint32

sentinel

magic word to ensure unmanaged channel integrity

volatile uint32

tx_count

local channel Tx counter (it wraps around at max uint32)

uint8

mem[]

local channel unmanaged memory buffer

5. 15. 111	5. 15. 112	5. 15. 113
5. 15. 114	5. 15. 115	5. 15. 116
5. 15. 117	5. 15. 118	5. 15. 119
5. 15. 120	5. 15. 121	5. 15. 122
5. 15. 123	5. 15. 124	5. 15. 125
5. 15. 126	5. 15. 127	5. 15. 128
5. 15. 129	5. 15. 130	5. 15. 131 struct IPCS_UNMANAGED_CHA NNEL_TYPE

Type

Name

Description

uint32

size

unmanaged channel memory size requested by app

struct IPCS_CHANNEL_UMEM_TYPE

*local_mem

源码未提供描述

struct IPCS_CHANNEL_UMEM_TYPE

*remote_mem

源码未提供描述

uint32

remote_tx_count

copy of remote Tx counter

void (rx_cb)(void cb_arg, const uint8 instance, sint32 chan_id, void *buf)

rx_cb

receive callback

void

*cb_arg

optional receive callback argument

5. 15. 132	5. 15. 133	5. 15. 134
5. 15. 135	5. 15. 136	5. 15. 137
5. 15. 138	5. 15. 139	5. 15. 140
5. 15. 141	5. 15. 142	5. 15. 143
5. 15. 144	5. 15. 145	5. 15. 146 struct IPCS_SHM_CHANNEL_T YPE

Type

Name

Description

sint32

id

channel id

enum IPCS_SHM_CHANNEL_TYPE_E

type

channel type (see IPCS_SHM_CHANNEL_TYPE_E)

struct IPCS_MANAGED_CHANNEL_TYPE

ch.mng

源码未提供描述

struct IPCS_UNMANAGED_CHANNEL_TYPE

ch.umng

源码未提供描述

5. 15. 147	5. 15. 148	5. 15. 149
5. 15. 150	5. 15. 151	5. 15. 152 struct IPCS_SHM_GLOBA L_TYPE

Type

Name

Description

uint64

state

state to indicate whether local is initialized

5. 15. 153	5. 15. 154	5. 15. 155
5. 15. 156	5. 15. 157	5. 15. 158
5. 15. 159	5. 15. 160	5. 15. 161
5. 15. 162	5. 15. 163	5. 15. 164
5. 15. 165	5. 15. 166	5. 15. 167 struct IPCS_SHM_PRIV_TYPE

Type

Name

Description

uint32

shm_size

local/remote shared memory size

uint8

num_channels

number of shared memory channels

struct IPCS_SHM_CHANNEL_TYPE

channels[IPC_SHM_MAX_CHANNELS]

ipc channels private data

struct IPCS_SHM_GLOBAL_TYPE

*global

local global data shared with remote

5. 15. 168	5. 15. 169
5. 15. 170	5. 15. 171
5. 15. 172	5. 15. 173 enum IPCS_SHM_CHANNEL_TYPE_E

Name

Description

IPC_SHM_MANAGED

channel with buffer management enabled

IPC_SHM_UNMANAGED

buf mgmt disabled, app owns entire channel memory

5. 15. 174	5. 15. 175
5. 15. 176	5. 15. 177
5. 15. 178	5. 15. 179
5. 15. 180	5. 15. 181
5. 15. 182	5. 15. 183
5. 15. 184	5. 15. 185
5. 15. 186	5. 15. 187
5. 15. 188	5. 15. 189
5. 15. 190	5. 15. 191
5. 15. 192	5. 15. 193
5. 15. 194	5. 15. 195 enum IPCS_SHM_CORE_TYPE_E

Name

Description

IPC_CORE_DEFAULT

used for letting driver auto-select remote core type

IPC_CORE_A53

ARM Cortex-A53 core

IPC_CORE_M7

ARM Cortex-M7 core

IPC_CORE_M4

ARM Cortex-M4 core

IPC_CORE_Z7

PowerPC e200z7 core

IPC_CORE_Z4

PowerPC e200z4 core

IPC_CORE_Z2

PowerPC e200z2 core

IPC_CORE_R52

ARM Cortex-R52 core

IPC_CORE_M33

ARM Cortex-M33 core

IPC_CORE_BBE32

Tensilica ConnX BBE32EP core

5. 15. 196	5. 15. 197
5. 15. 198	5. 15. 199
5. 15. 200	5. 15. 201
5. 15. 202	5. 15. 203
5. 15. 204	5. 15. 205
5. 15. 206	5. 15. 207
5. 15. 208	5. 15. 209
5. 15. 210	5. 15. 211
5. 15. 212	5. 15. 213 enum IPCS_SHM_CORE_INDEX_E

Name

Description

IPC_CORE_INDEX_0

Processor index 0

IPC_CORE_INDEX_1

Processor index 1

IPC_CORE_INDEX_2

Processor index 2
IPC_CORE_INDEX_3
Processor index 3
IPC_CORE_INDEX_4
Processor index 4
IPC_CORE_INDEX_5
Processor index 5
IPC_CORE_INDEX_6
Processor index 6
IPC_CORE_INDEX_7
Processor index 7

5. 15. 214	5. 15. 215	5. 15. 216
5. 15. 217	5. 15. 218	5. 15. 219
5. 15. 220	5. 15. 221	5. 15. 222 struct IPCS_SHM_POOL_ CFG_TYPE

Type
Name
Description
uint16
num_bufs
number of buffers
uint32
buf_size
buffer size

5. 15. 223	5. 15. 224	5. 15. 225
5. 15. 226	5. 15. 227	5. 15. 228
5. 15. 229	5. 15. 230	5. 15. 231
5. 15. 232	5. 15. 233	5. 15. 234
5. 15. 235	5. 15. 236	5. 15. 237 struct IPCS_SHM_MANAGED_C FG_TYPE

Type

Name

Description

uint8

num_pools

number of buffer pools

struct IPCS_SHM_POOL_CFG_TYPE

*pools

memory buffer pools parameters

void (rx_cb)(void cb_arg, const uint8 instance, sint32 chan_id, void *buf, uint32 size)

rx_cb

receive callback

void

*cb_arg

optional receive callback argument

5. 15. 238	5. 15. 239	5. 15. 240
5. 15. 241	5. 15. 242	5. 15. 243
5. 15. 244	5. 15. 245	5. 15. 246
5. 15. 247	5. 15. 248	5. 15. 249 struct IPCS_SHM_UNMANAGED _CFG_TYPE

Type

Name

Description

uint32

size

unmanaged channel memory size

void (rx_cb)(void cb_arg, const uint8 instance, sint32 chan_id, void *mem)

rx_cb

receive callback

void

*cb_arg

optional receive callback argument

5. 15. 250	5. 15. 251	5. 15. 252
5. 15. 253	5. 15. 254	5. 15. 255
5. 15. 256	5. 15. 257	5. 15. 258
5. 15. 259	5. 15. 260	5. 15. 261 struct IPCS_SHM_CHANNEL_C FG_TYPE

Type

Name

Description

enum IPCS_SHM_CHANNEL_TYPE_E

type

channel type from &enum IPCS_SHM_CHANNEL_TYPE_E

struct IPCS_SHM_MANAGED_CFG_TYPE

ch.managed

源码未提供描述

struct IPCS_SHM_UNMANAGED_CFG_TYPE

ch.unmanaged

源码未提供描述

5. 15. 262	5. 15. 263	5. 15. 264
5. 15. 265	5. 15. 266	5. 15. 267
5. 15. 268	5. 15. 269	5. 15. 270 struct IPCS_SHM_REMOTE_CO RE_TYPE

Type

Name

Description

enum IPCS_SHM_CORE_TYPE_E

type

core type from &enum IPCS_SHM_CORE_TYPE_E

enum IPCS_SHM_CORE_INDEX_E

index

core number

5. 15. 271	5. 15. 272	5. 15. 273
5. 15. 274	5. 15. 275	5. 15. 276
5. 15. 277	5. 15. 278	5. 15. 279
5. 15. 280	5. 15. 281	5. 15. 282 struct IPCS_SHM_LOCAL_COR E_TYPE

Type

Name

Description

enum IPCS_SHM_CORE_TYPE_E

type

core type from &enum IPCS_SHM_CORE_TYPE_E

enum IPCS_SHM_CORE_INDEX_E

index

core number targeted by remote core interrupt

uint32

trusted

trusted cores mask

5. 15. 283	5. 15. 284	5. 15. 285
5. 15. 286	5. 15. 287	5. 15. 288
5. 15. 289	5. 15. 290	5. 15. 291
5. 15. 292	5. 15. 293	5. 15. 294
5. 15. 295	5. 15. 296	5. 15. 297
5. 15. 298	5. 15. 299	5. 15. 300
5. 15. 301	5. 15. 302	5. 15. 303
5. 15. 304	5. 15. 305	5. 15. 306
5. 15. 307	5. 15. 308	5. 15. 309
5. 15. 310	5. 15. 311	5. 15. 312
5. 15. 313	5. 15. 314	5. 15. 315
5. 15. 316	5. 15. 317	5. 15. 318
5. 15. 319	5. 15. 320	5. 15. 321 struct IPCS_SHM_CFG_TYPE

Type

Name

Description

uintptr_t

local_shm_addr

local shared memory physical address

uintptr_t

remote_shm_addr
remote shared memory physical address
uint32
shm_size
local/remote shared memory size
sint32
inter_core_tx_irq
inter-core interrupt reserved for shm driver Tx
sint32
inter_core_rx_irq
inter-core interrupt reserved for shm driver Rx
uint8
mru_tx_channel_id
mru channel index for shm driver Tx
uint8
mru_rx_channel_id
mru channel index for shm driver Rx
struct IPCS_SHM_LOCAL_CORE_TYPE
local_core
local core targeted by remote core interrupt
struct IPCS_SHM_REMOTE_CORE_TYPE
remote_core
remote core to trigger the interrupt on
uint8
num_channels
number of shared memory channels
struct IPCS_SHM_CHANNEL_CFG_TYPE *
channels
IPC channels parameters array
ISRTType
isr_id_handler
the name of OsIsr defined to handle the interrupt

5. 15. 322	5. 15. 323	5. 15. 324
5. 15. 325	5. 15. 326	5. 15. 327
5. 15. 328	5. 15. 329	5. 15. 330 struct IPCS_SHM_INSTA NCES_CFG_TYPE

Type
 Name
 Description
 uint8
 num_instances
 number of shared memory instances
 struct IPCS_SHM_CFG_TYPE
 *shm_cfg
 IPC shm parameters array

5. 16	5. 17	5. 18	5. 19
5. 20	5. 21	5. 22	5. 23
5. 24	5. 25	5. 26	5. 27
5. 28	5. 29	5. 30	5. 31
5. 32	5. 33	5. 34	5. 35
5. 36	5. 37	5. 38	5. 39 DYNAMIC DETAILED DESIGN 动态详 细设计

本节描述与部署变体无关的 逻辑 数据路径 (Core + Queue) 。

场景 ID

场景名称

涉及软件单元

设计依据

CORE-S01

初始化

CORE_SHM、HAL、OSAL、CORE_QUEUE

ipcsShmInit → ipcsHwInit → ipcsOsInit → ipcsShmInitChannels

CORE-S02

Managed 发送

CORE_SHM、CORE_QUEUE、HAL

ipcsShmAcquireBuf → ipcsShmTx → ipcsQueuePush → ipcsHwIrqNotify

CORE-S03

Managed 接收与释放

HAL、OSAL、CORE_SHM、CORE_QUEUE

ipcsShmRx → ipcsChannelRx → rx_cb → ipcsShmReleaseBuf

CORE-S04

Unmanaged 收发

CORE_SHM、HAL

ipcsShmUnmanagedTx / 对端 tx_count 比对

CORE-S05

中断与轮询

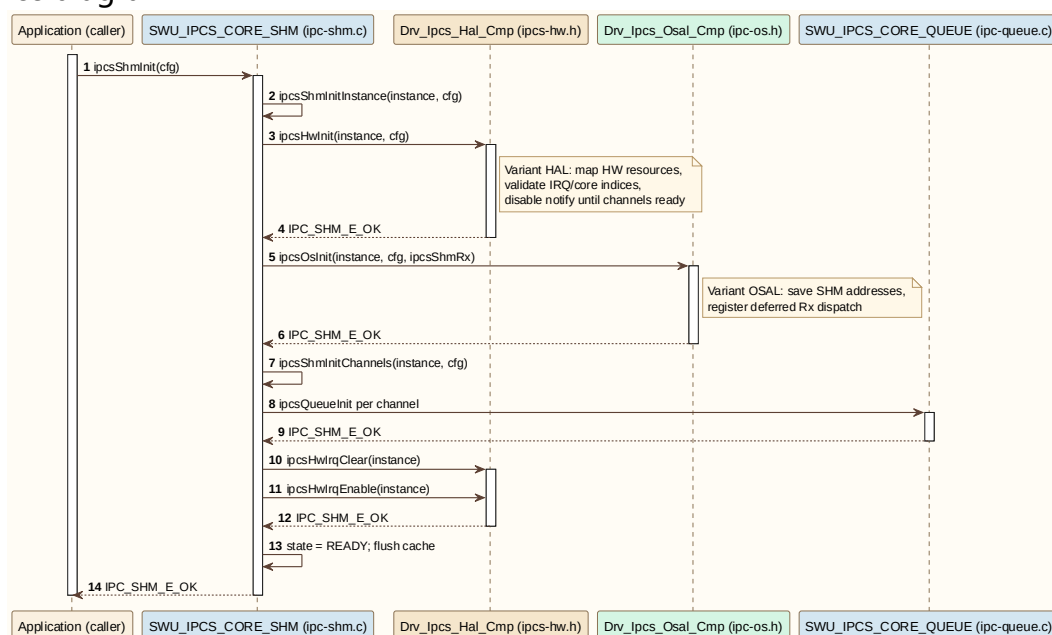
CORE_SHM、OSAL、HAL

IRQ 路径 vs ipcsShmPollChannels

5.39.1 Initialization Sequence (CORE-S01) 初始化流程

按架构：应用调用 ipcsShmInit → 逐 instance 调用 HAL 初始化、OSAL 初始化、channel 初始化（具体 HAL/OSAL 实现见第 4/5 章）。

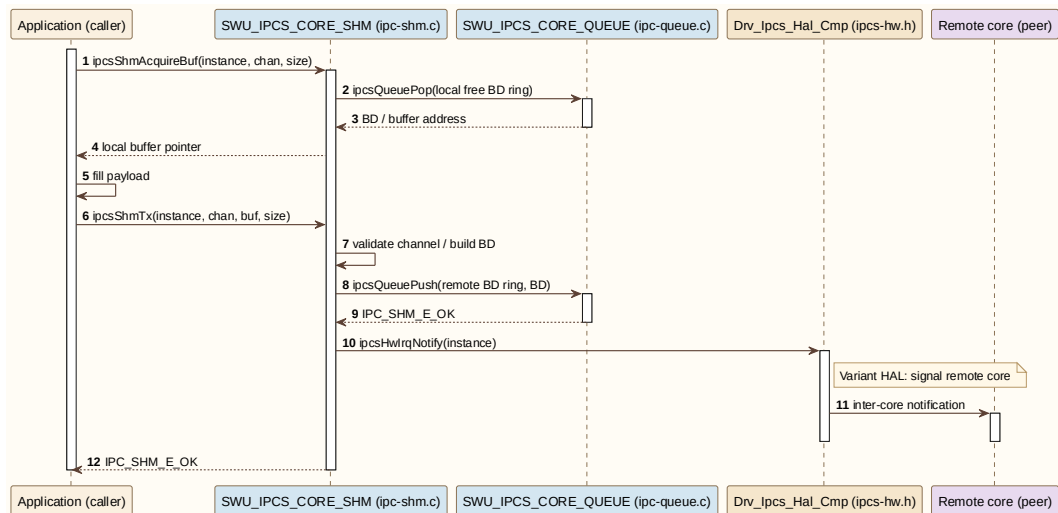
sequence diagram



5.39.2 Managed Transmit Sequence (CORE-S02) Managed 发送流程

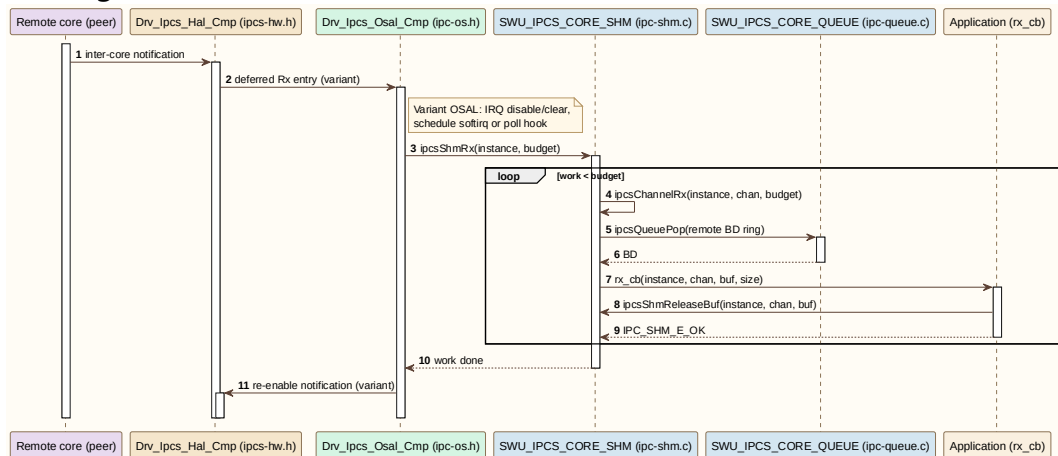
ipcsShmAcquireBuf → 填充数据 → ipcsShmTx → queue push BD → HAL 通知远端。

sequence diagram



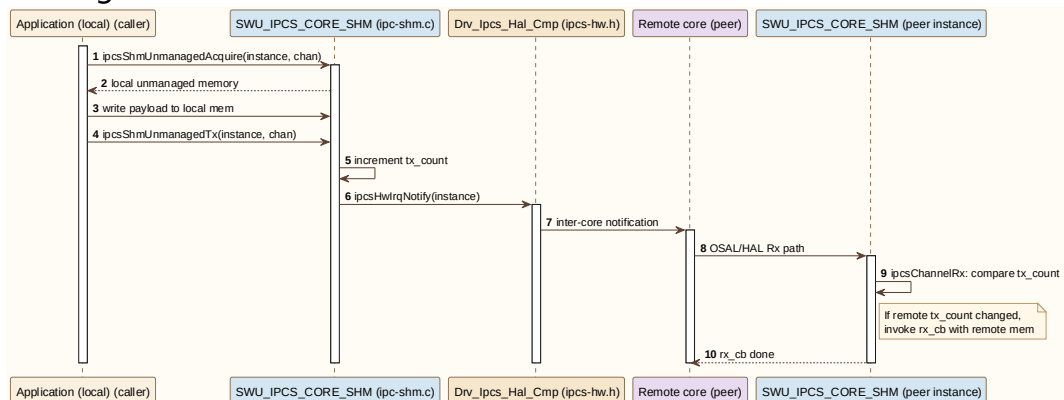
5.39.3 Managed Receive and Release Sequence (CORE-S03) Managed 接收与释放流程

OSAL 触发 `ipcsShmRx` → `ipcsChannelRx` → 应用回调 → `ipcsShmReleaseBuf`.
sequence diagram



5.39.4 Unmanaged Sequence (CORE-S04) Unmanaged 收发流程

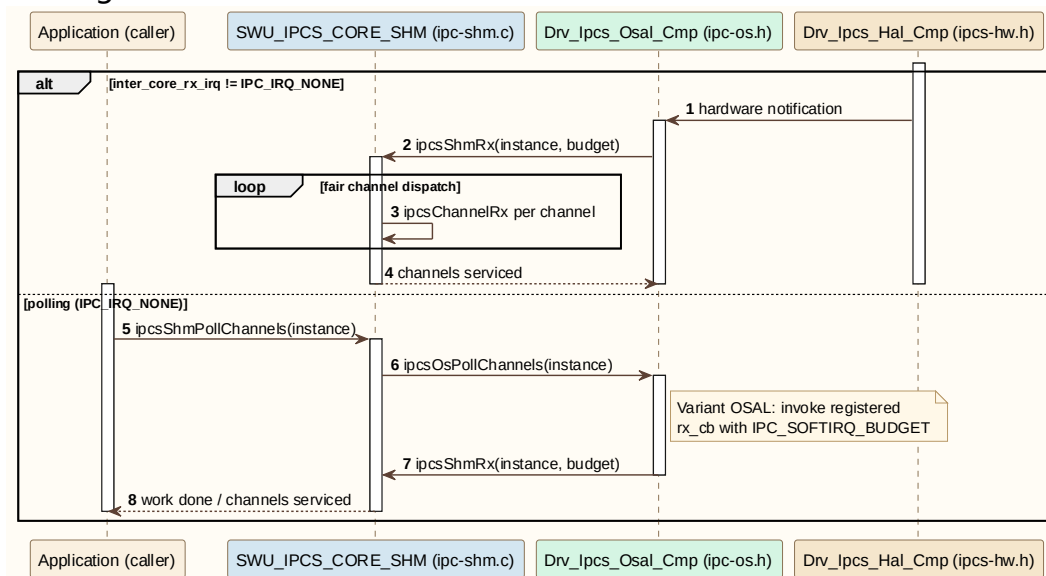
`ipcsShmUnmanagedAcquire` / `ipcsShmUnmanagedTx`; 接收侧检查 `tx_count`.
sequence diagram



5.39.5 Interrupt and Polling Sequence (CORE-S05) 中断与轮询流程

OSAL 注册 hardirq/softirq 或 polling (ipcsShmPollChannels) ; Core 按预算分发 channel。

sequence diagram



6 RTOS VARIANT DETAILED DESIGN RTOS 详设

6.1 DEFINITION AND SCOPE 定义与范围

RTOS 部署变体在单地址空间内实现 Drv_Ipcs_Osal_Cmp 与 Drv_Ipcs_Hal_Cmp: OS 实现三选一 (FreeRTOS、ThreadX、AUTOSAR OS) , HAL 位于 ipcs/mcu/hw/ 并由三种 OS 实现共用。通信核心仍使用第 4 章 ipcs/ipcs_cores 与 ipc-shm.h 对外 API。

6.2 FILE STRUCTURE 文件结构

6.2.1	6.2.2
6.2.3	6.2.4
6.2.5	6.2.6
6.2.7	6.2.8
6.2.9	6.2.10
6.2.11	6.2.12
6.2.13	6.2.14
6.2.15	6.2.16 File List 文件列表

组件

文件

Drv_Ipcs_Hal_Cmp
ipcs/mcu/hw/ipc-hw-platform.h
Drv_Ipcs_Hal_Cmp
ipcs/mcu/hw/ipc-hw.c
Drv_Ipcs_Hal_Cmp
ipcs/mcu/hw/ipc-hw.h
Drv_Ipcs_Osal_Cmp
ipcs/mcu/os/autosar/ipc-os-autosar.c
Drv_Ipcs_Osal_Cmp
ipcs/mcu/os/freertos/ipc-os-freertos.c
Drv_Ipcs_Osal_Cmp
ipcs/mcu/os/ipc-os.h
Drv_Ipcs_Osal_Cmp
ipcs/mcu/os/threadx/ipc-os-threadx.c

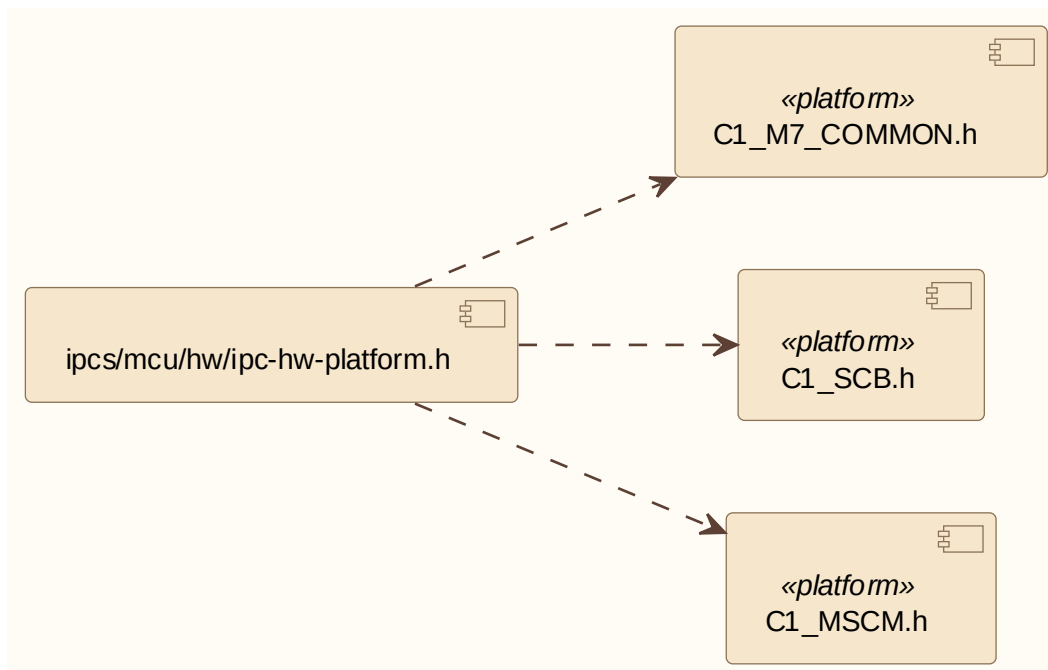
6.2.17 ipc-hw-platform.h

描述:

ipcs/mcu/hw/ipc-hw-platform.h 属于 Drv_Ipcs_Hal_Cmp / 平台定义。

依赖关系：

C1_M7_COMMON.h、C1_SCB.h、C1_MSCM.h (与 ipcs/mcu/hw/ipc-hw-platform.h 一致)



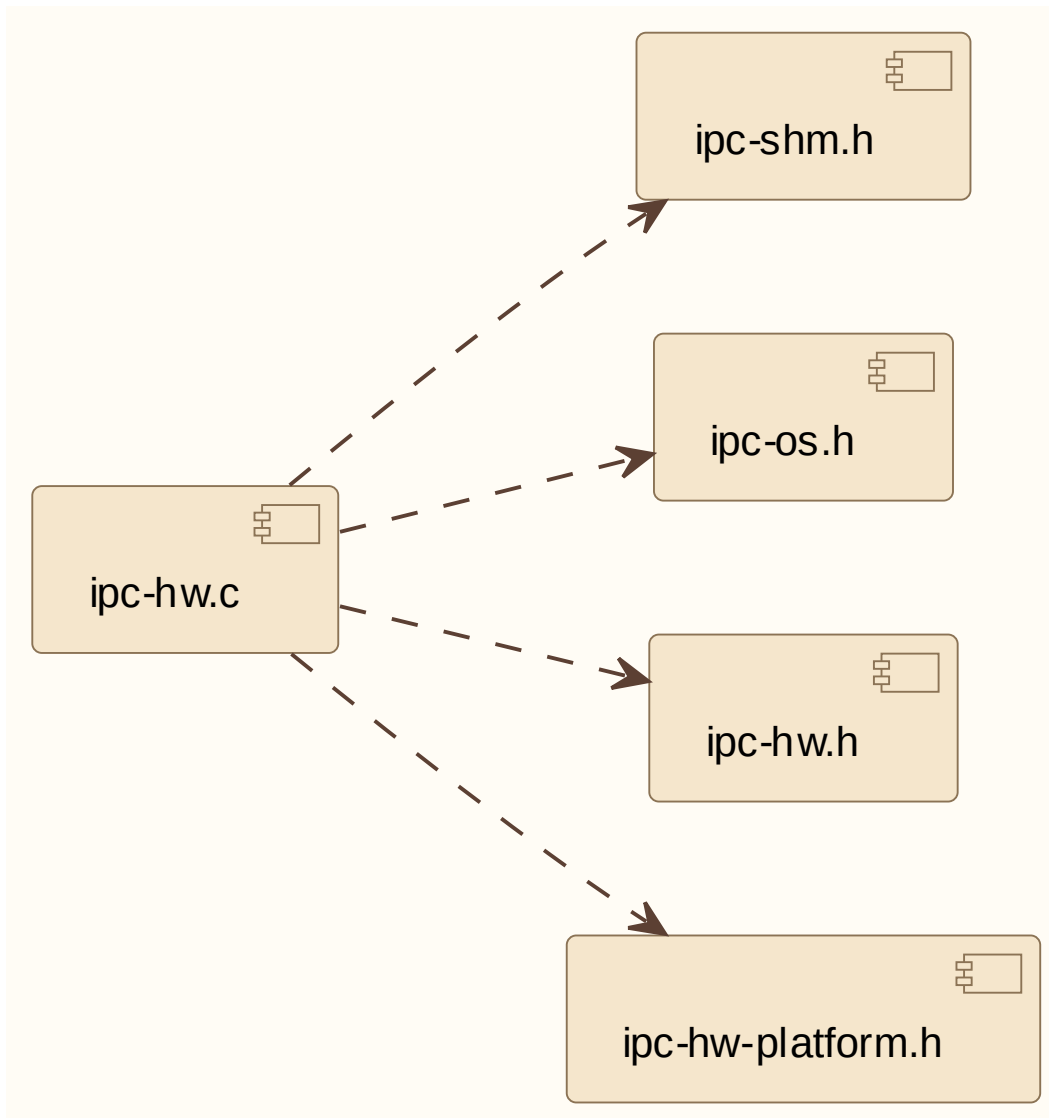
6. 2. 18 ipc-hw.c

描述：

ipcs/mcu/hw/ipc-hw.c 属于 Drv_Ipcs_Hal_Cmp / IPCS-HAL。

依赖关系：

ipc-shm.h、ipc-os.h、ipc-hw.h、ipc-hw-platform.h (与 ipcs/mcu/hw/ipc-hw.c 中 #include 顺序一致)



6.2.19 ipc-hw.h

描述:

ipcs/mcu/hw/ipc-hw.h 属于 Drv_Ipcs_Hal_Cmp / IPCS-HAL。

依赖关系:

本头文件未 #include 工程内其他头文件（仅 HAL API 声明）。



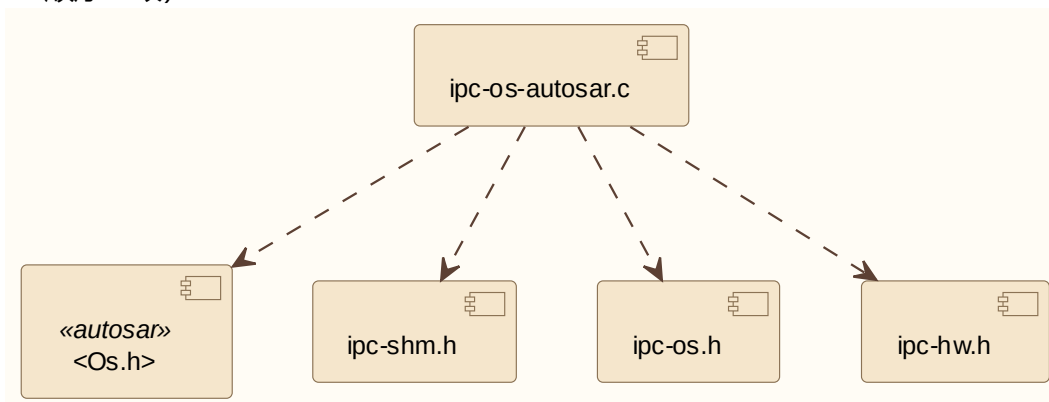
6.2.20 ipc-os-autosar.c

描述:

ipcs/mcu/os/autosar/ipc-os-autosar.c 属于 Drv_Ipcs_Osal_Cmp / AUTOSAR OS 实现。

依赖关系:

Os.h、ipc-shm.h、ipc-os.h、ipc-hw.h (与 ipcs/mcu/os/autosar/ipc-os-autosar.c 中 #include 顺序一致)



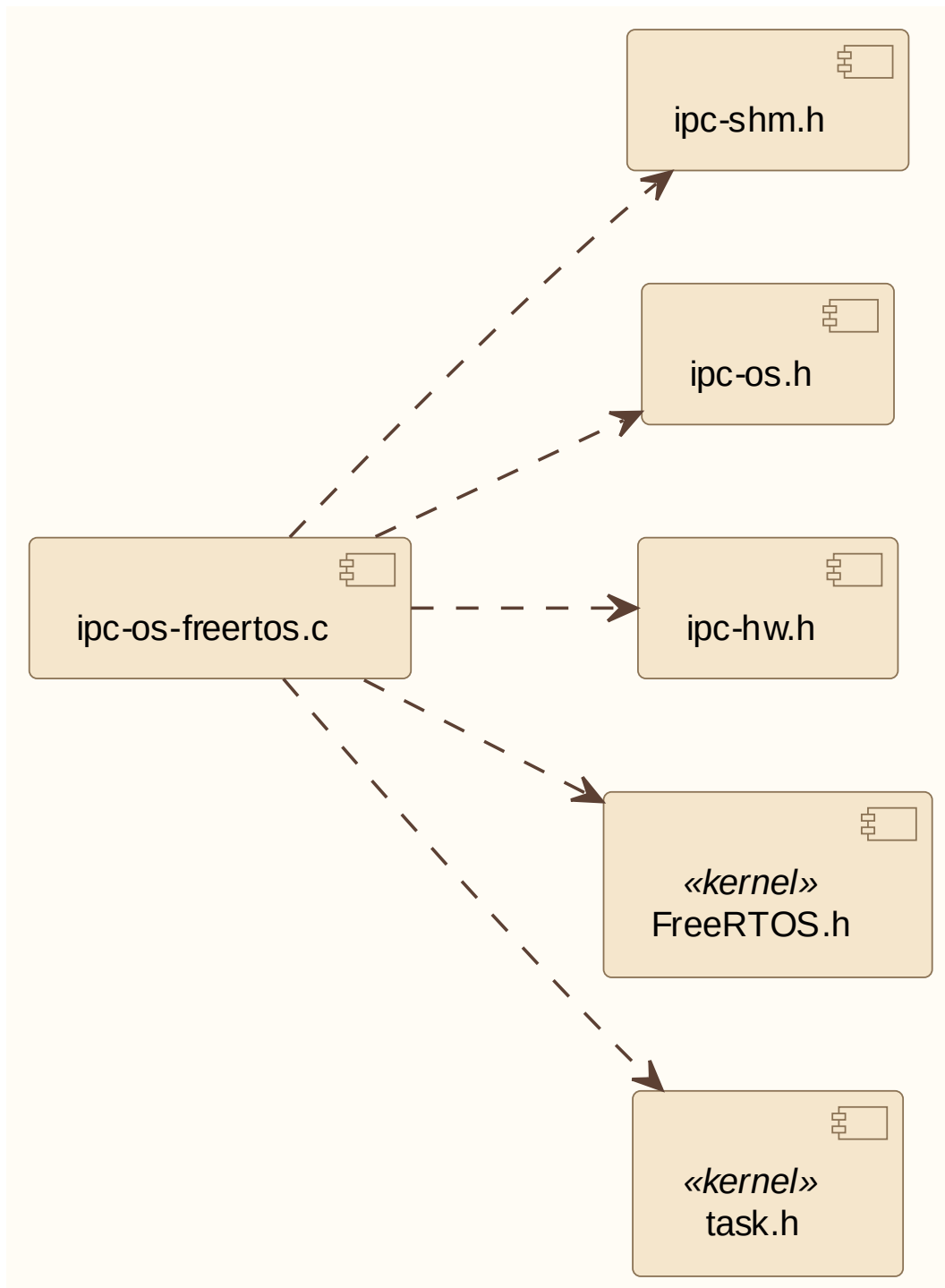
6.2.21 ipc-os-freertos.c

描述:

ipcs/mcu/os/freertos/ipc-os-freertos.c 属于 Drv_Ipcs_Osal_Cmp / FreeRTOS 实现。

依赖关系:

ipc-shm.h、ipc-os.h、ipc-hw.h、FreeRTOS.h、task.h (与 ipcs/mcu/os/freertos/ipc-os-freertos.c 一致)



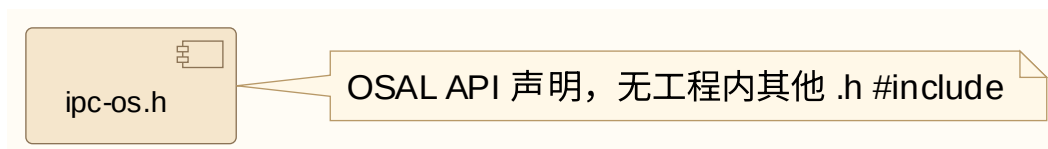
6.2.22 ipc-os.h

描述：

ipcs/mcu/os/ipc-os.h 属于 Drv_Ipcs_Osal_Cmp / IPCS-OSAL。

依赖关系：

本头文件未 #include 工程内其他头文件（OSAL 宏与 API 声明）。



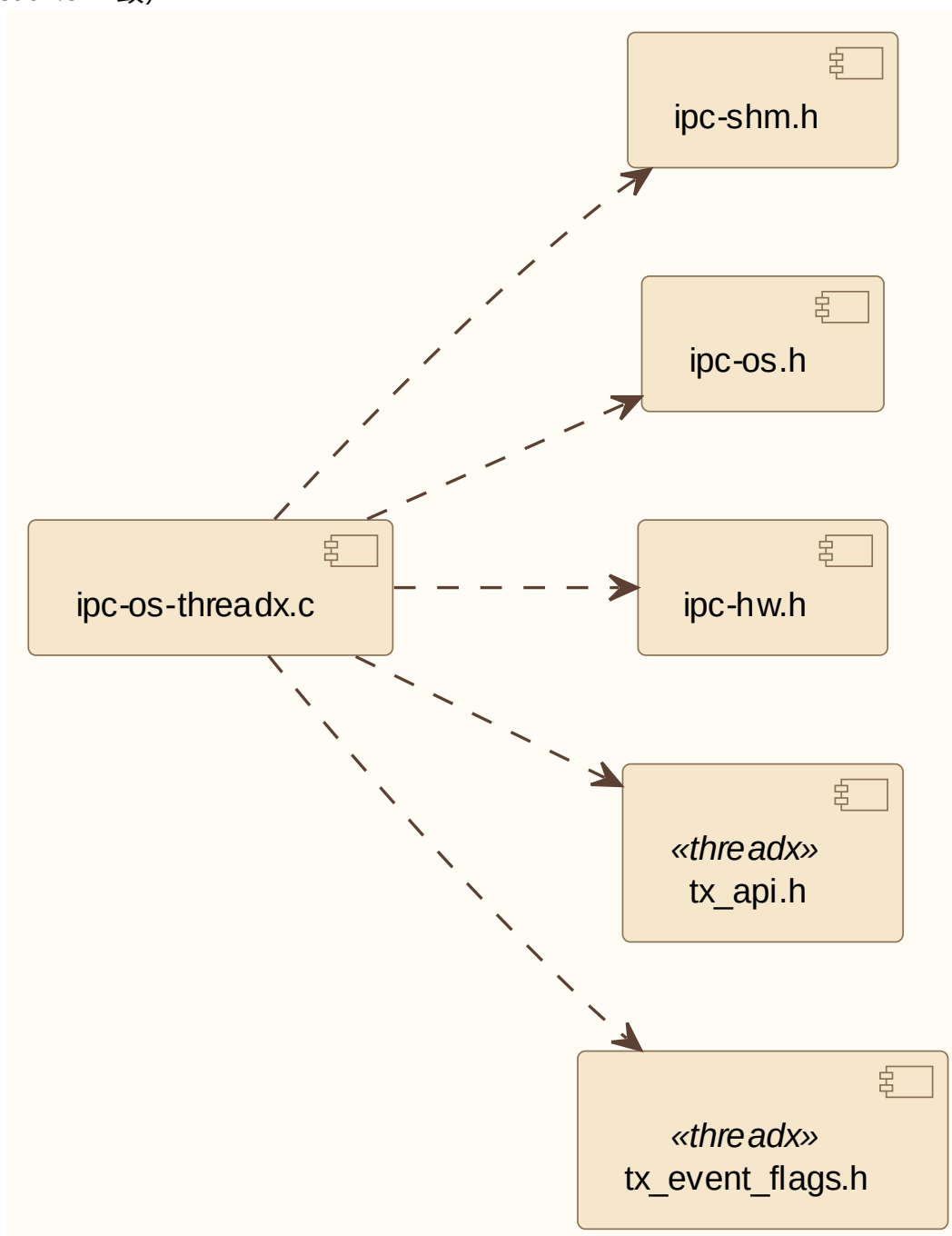
6.2.23 `ipc-os-threadx.c`

描述:

`ipcs/mcu/os/threadx/ipc-os-threadx.c` 属于 `Drv_Ipcs_Osal_Cmp / ThreadX` 实现。

依赖关系:

`ipc-shm.h`、`ipc-os.h`、`ipc-hw.h`、`tx_api.h`、`tx_event_flags.h` (与 `ipcs/mcu/os/threadx/ipc-os-threadx.c` 一致)

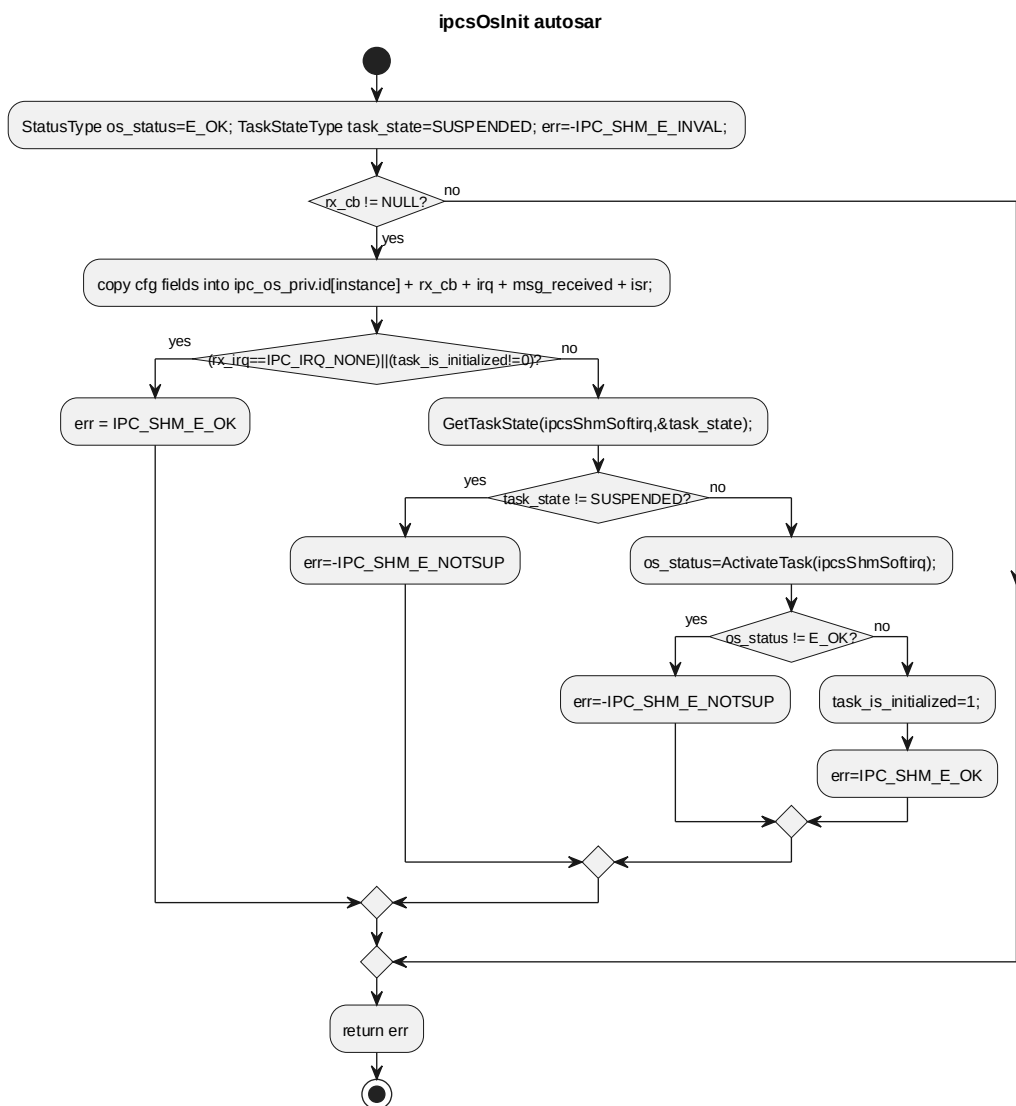


6.3 SWU_IPCS_OSAL_AUTOSAR DESIGN 单元设计

6.3.1 ipcsOsInit

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR			
函数说明	OS specific initialization code			
函数原型	sint32 ipcsOsInit(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg, sint32 (*rx_cb)(const uint8, sint32))			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	cfg	const struct IPCS_SHM_CFG_TYPE *	configuration parameters
	I	rx_cb	sint32 (*rx_cb)(const uint8, sint32)	rx callback to be called from rx softirq
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK on success, -IPC_SHM_E_NOTSUP if softirq task not in	
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

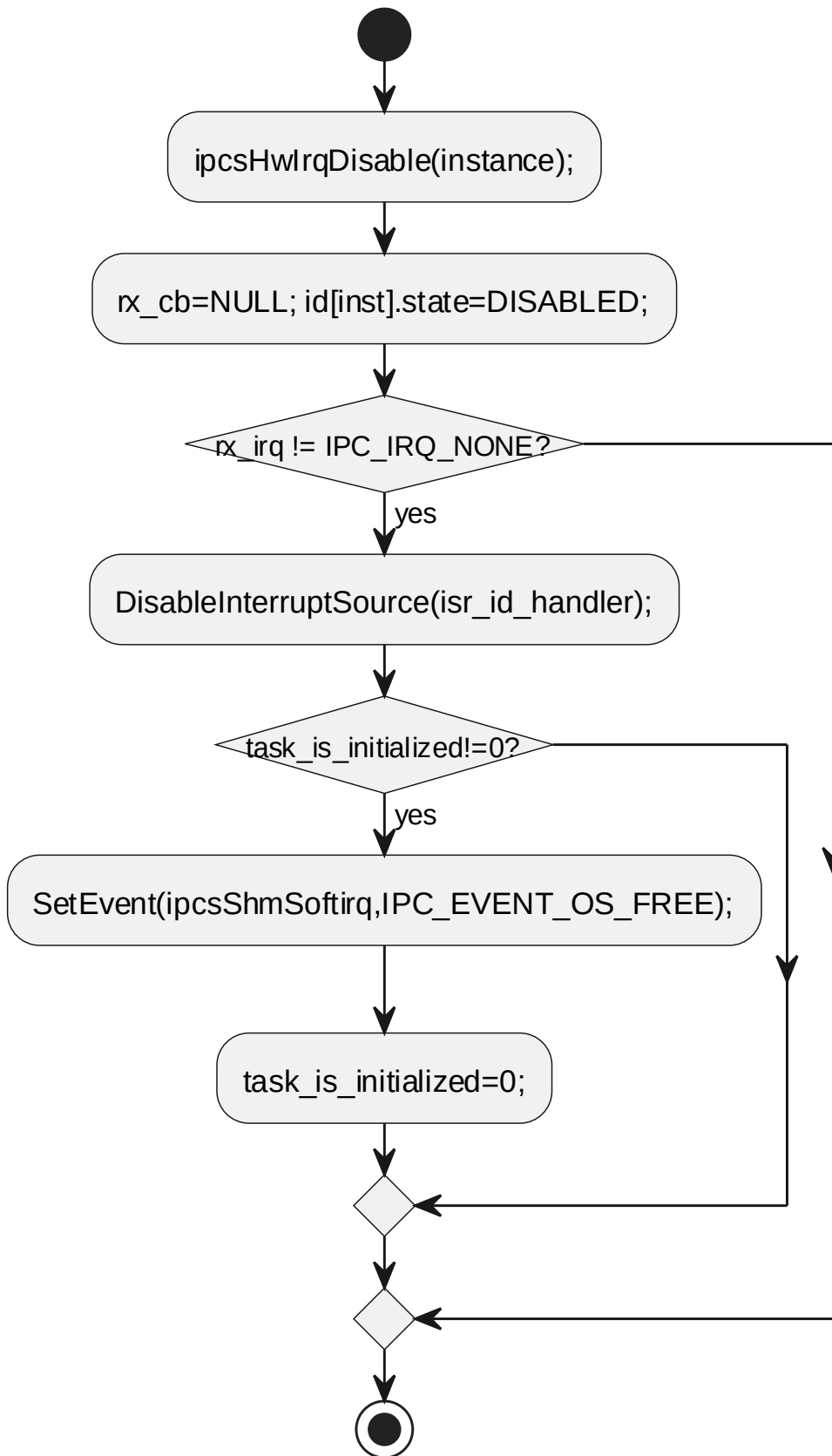


6.3.2 ipcsOsFree

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR			
函数说明	free OS specific resources			
函数原型	void ipcsOsFree(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

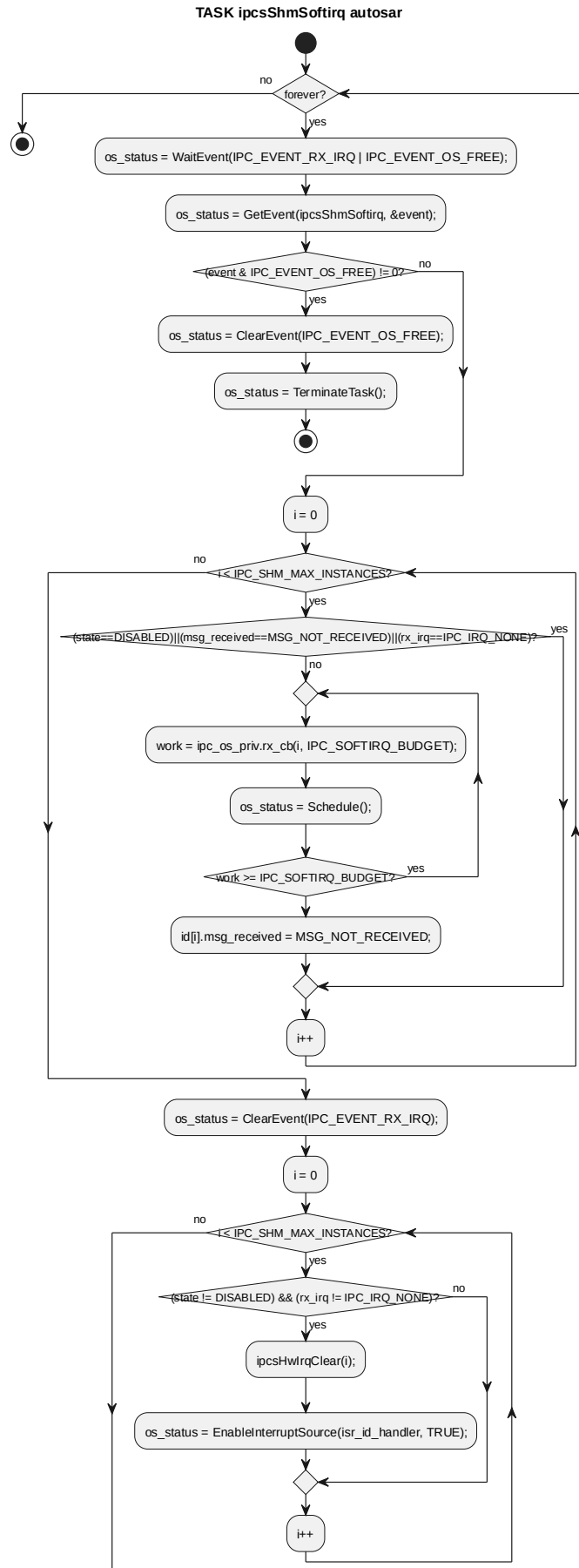
ipcsOsFree autosar



6.3.3 ipcsShmSoftirq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR			
函数说明	task acting as deferred interrupt handler			
函数原型	TASK(ipcsShmSoftirq)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	-	-	-	-
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	-			

processing flow

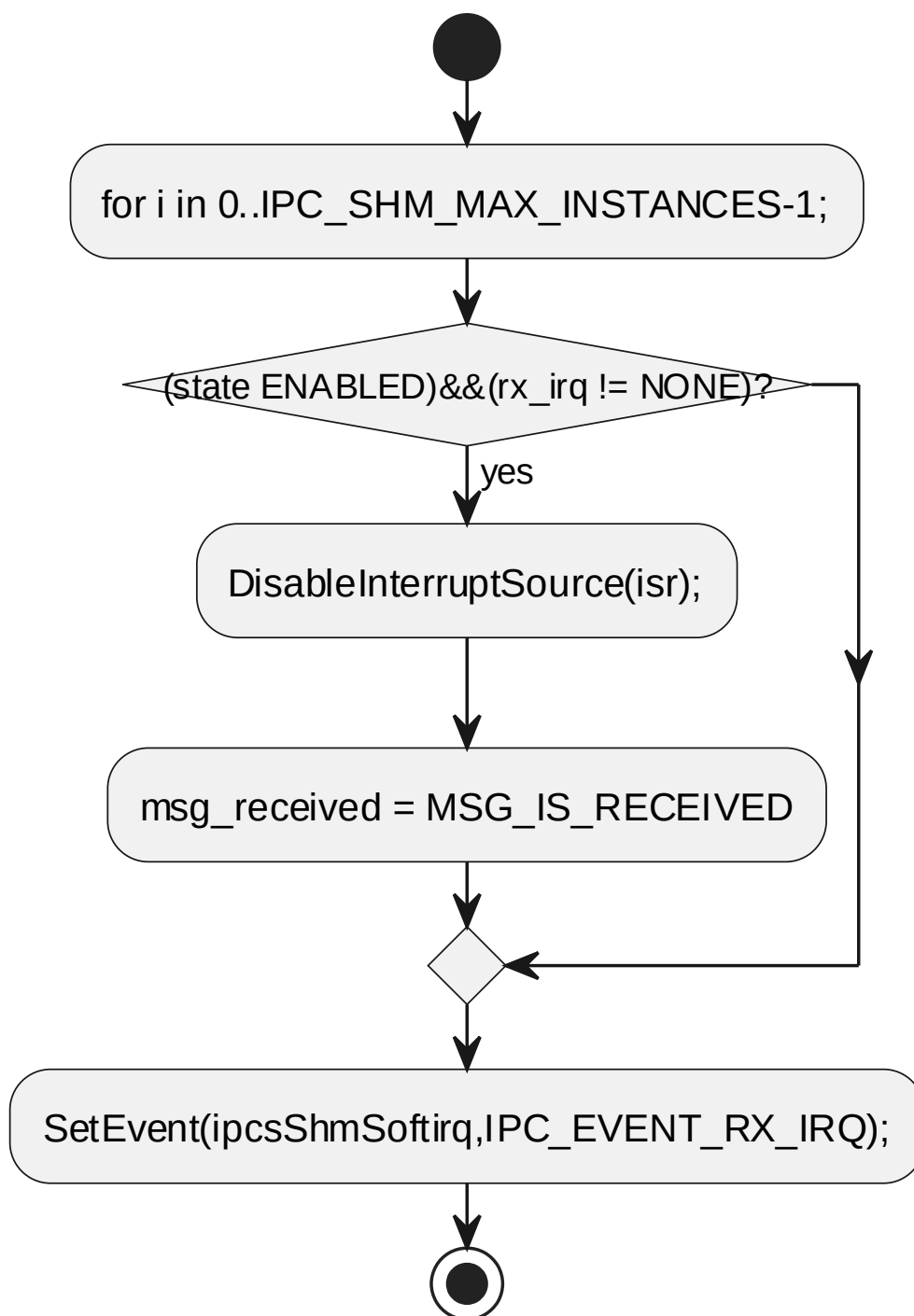


6.3.4 ipcsShmHardirq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR			
函数说明	driver interrupt service routine			
函数原型	void ipcsShmHardirq(void)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	-	-	-	-
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

ipcsShmHardirq autosar



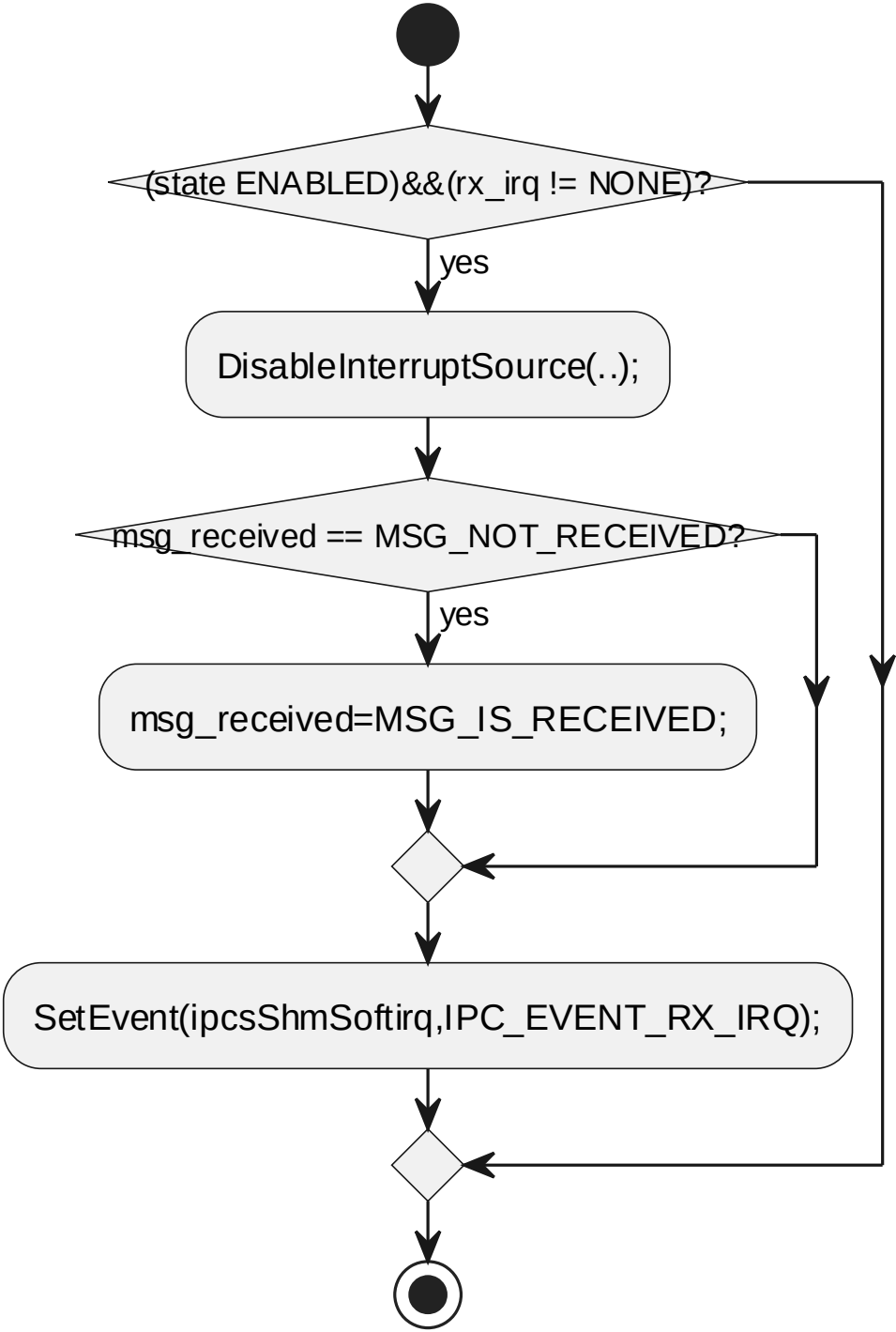
6.3.5 ipcsShmHardirqInstance

对应软件架构 ID	Drv_Ipcs_Osal_Cmp
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR
函数说明	driver interrupt service routine

函数原型	void ipcsShmHardirqInstance(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

ipcsShmHardirqInstance autosar

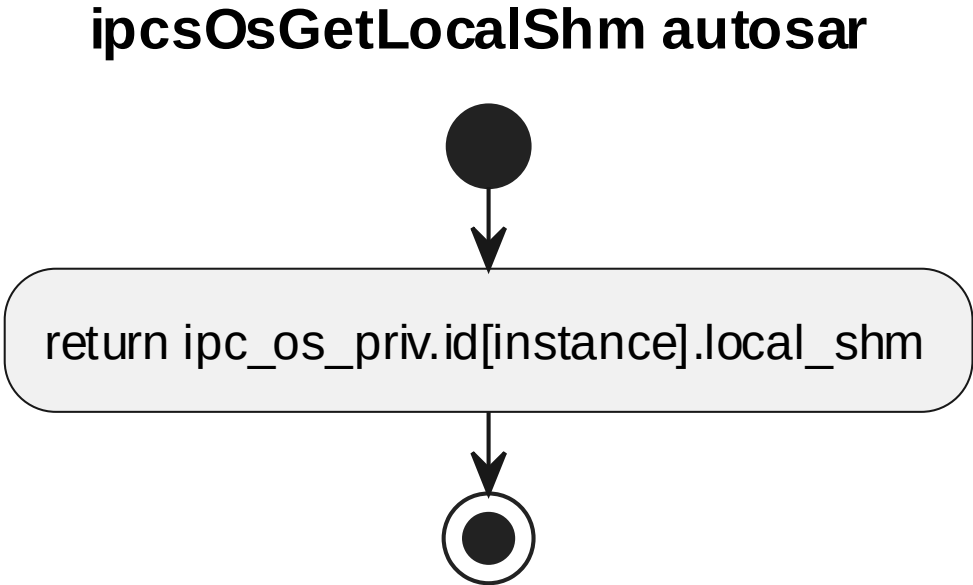


6.3.6 ipcs0sGetLocalShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR
函数说明	get local shared mem address

函数原型	uintptr_t ipcsOsGetLocalShm(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	uintptr_t		源码返回值	
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow



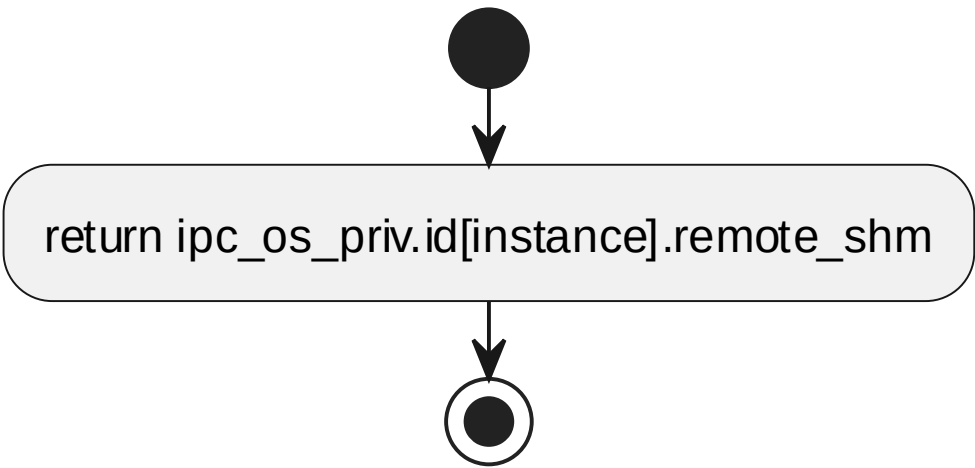
6.3.7 ipcsOsGetRemoteShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR			
函数说明	get remote shared mem address			
函数原型	uintptr_t ipcsOsGetRemoteShm(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	uintptr_t		源码返回值	

函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c
函数声明文件	ipcs/mcu/os/ipc-os.h

processing flow

ipcsOsGetRemoteShm autosar

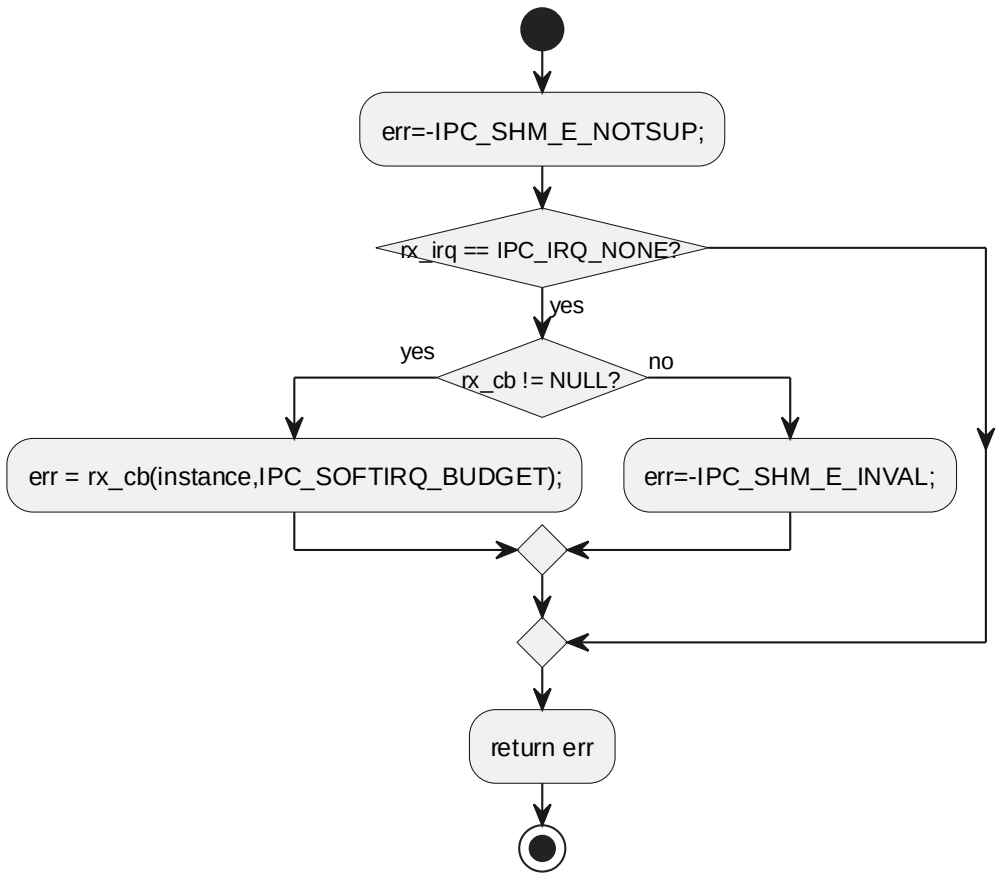


6.3.8 ipcs0sPollChannels

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_AUTOSAR			
函数说明	invoke rx callback configured at initialization			
函数原型	sint32 ipcsOsPollChannels(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	sint32		work done, error code otherwise	
函数定义文件	ipcs/mcu/os/autosar/ipc-os-autosar.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

ipcsOsPollChannels autosar



6.4 SWU_IPCS_OSAL_FREERTOS DESIGN 单元设计

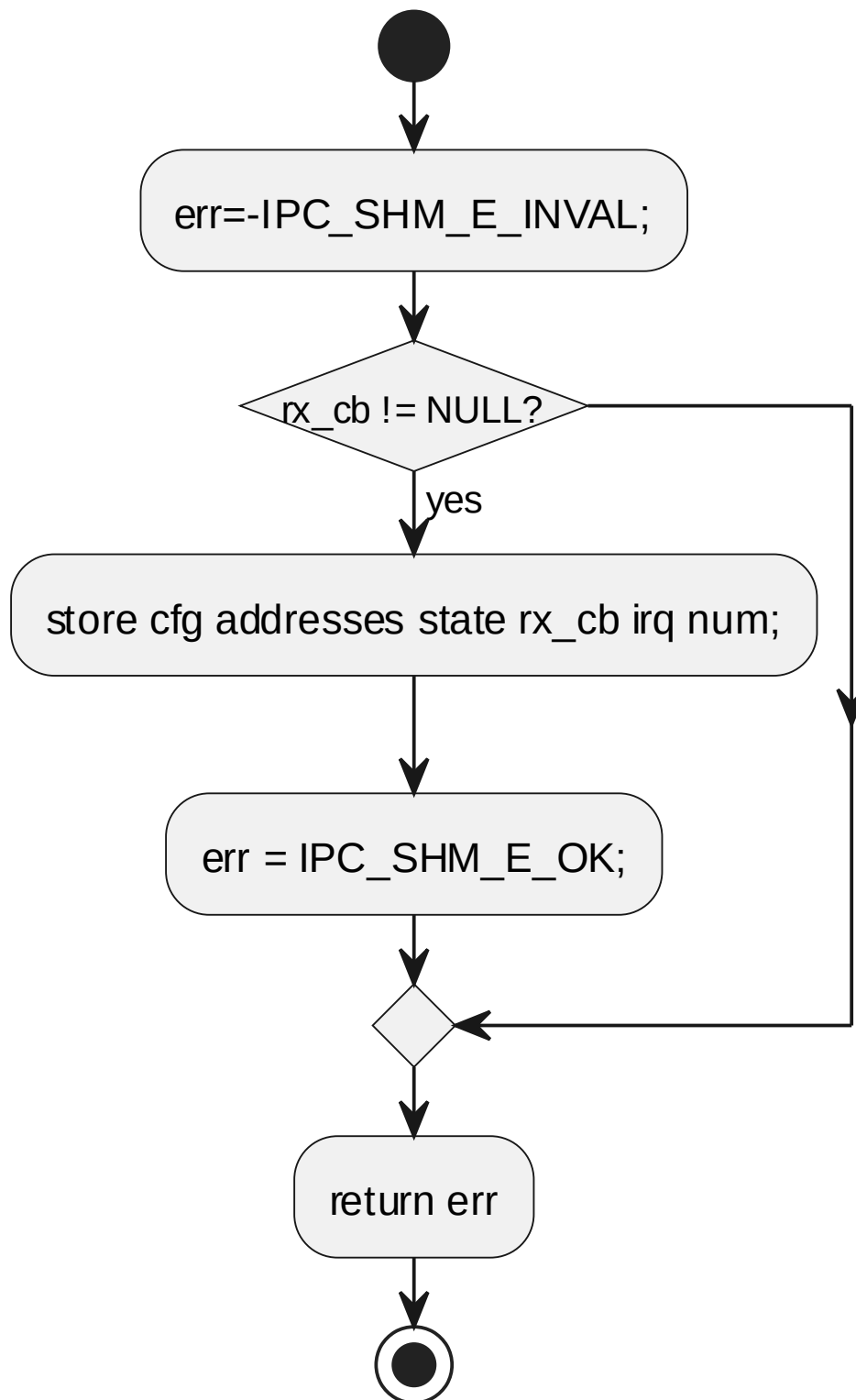
6.4.1 ipcs0sInit

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	OS specific initialization code			
函数原型	sint32 ipcsOsInit(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg, sint32 (*rx_cb)(const uint8, sint32))			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	cfg	const struct IPCS_SHM_CFG_TYPE *	configuration parameters
	I	rx_cb	sint32 (*rx_cb)(const uint8, sint32)	rx callback to be called from rx softirq

返回值	数据类型	说明
	sint32	IPC_SHM_E_OK on success, -IPC_SHM_E_NOMEM if the softirq task creation
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c	
函数声明文件	ipcs/mcu/os/ipc-os.h	

processing flow

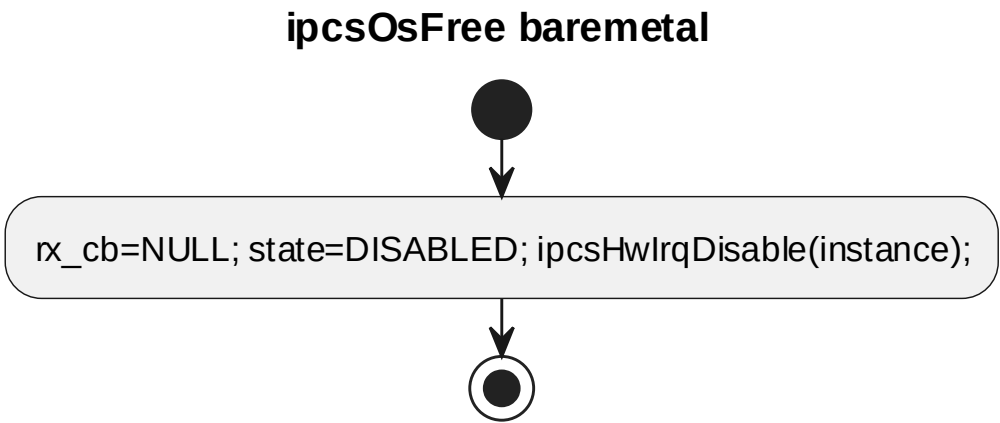
ipcsOsInit baremetal



6.4.2 ipcsOsFree

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	free OS specific resources			
函数原型	void ipcsOsFree(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

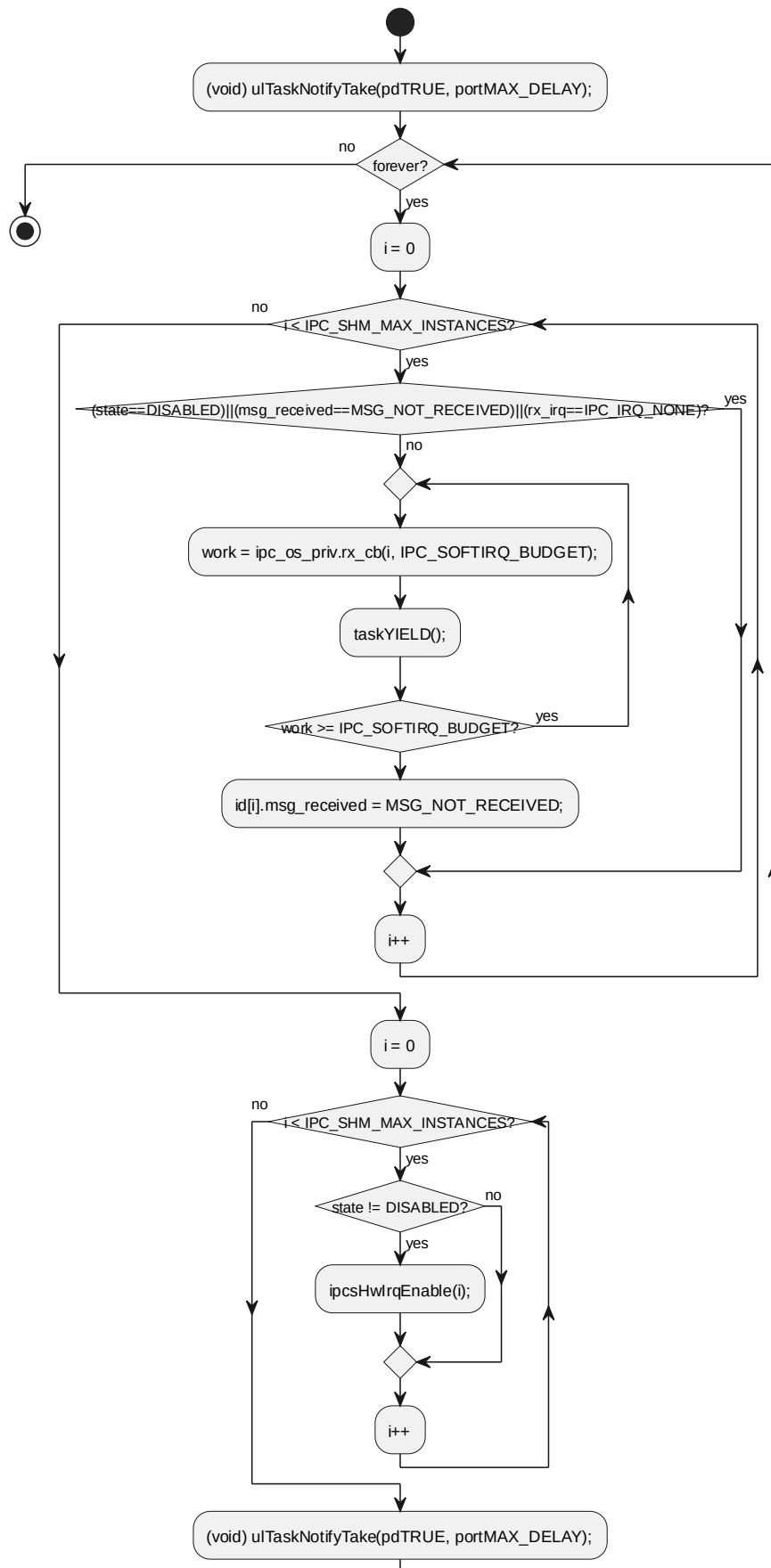


6.4.3 ipcsShmSoftirq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	task acting as deferred interrupt handler			
函数原型	static void ipcsShmSoftirq(void)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	-	-	-	-
返回值	数据类型		说明	
	void		源码返回值	
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	-			

processing flow

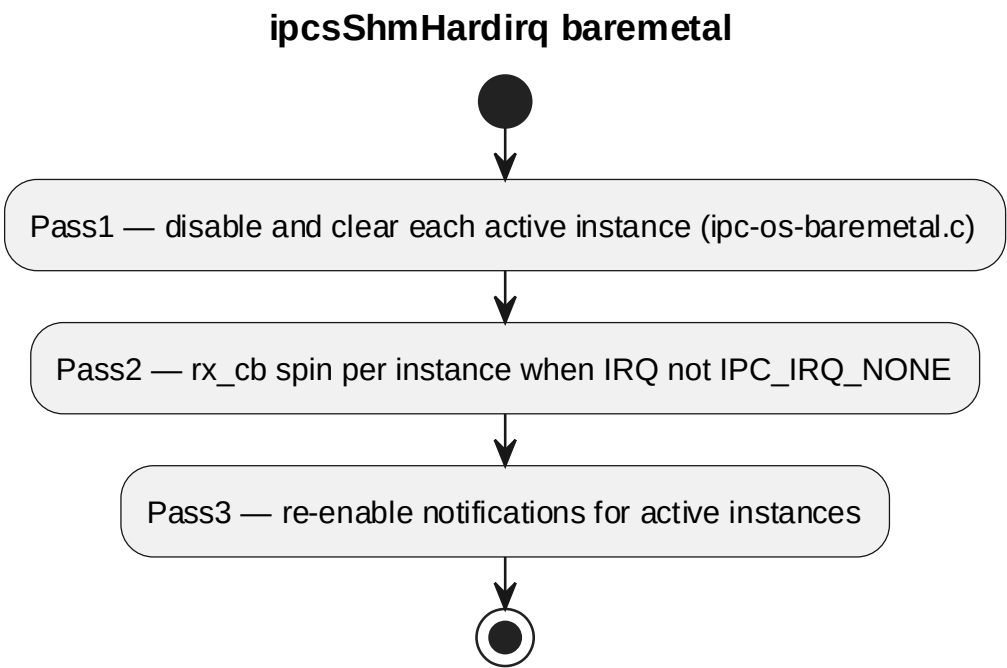
ipcsShmSoftirq FreeRTOS



6.4.4 ipcsShmHardirq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	driver interrupt service routine			
函数原型	void ipcsShmHardirq(void)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	-	-	-	-
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow



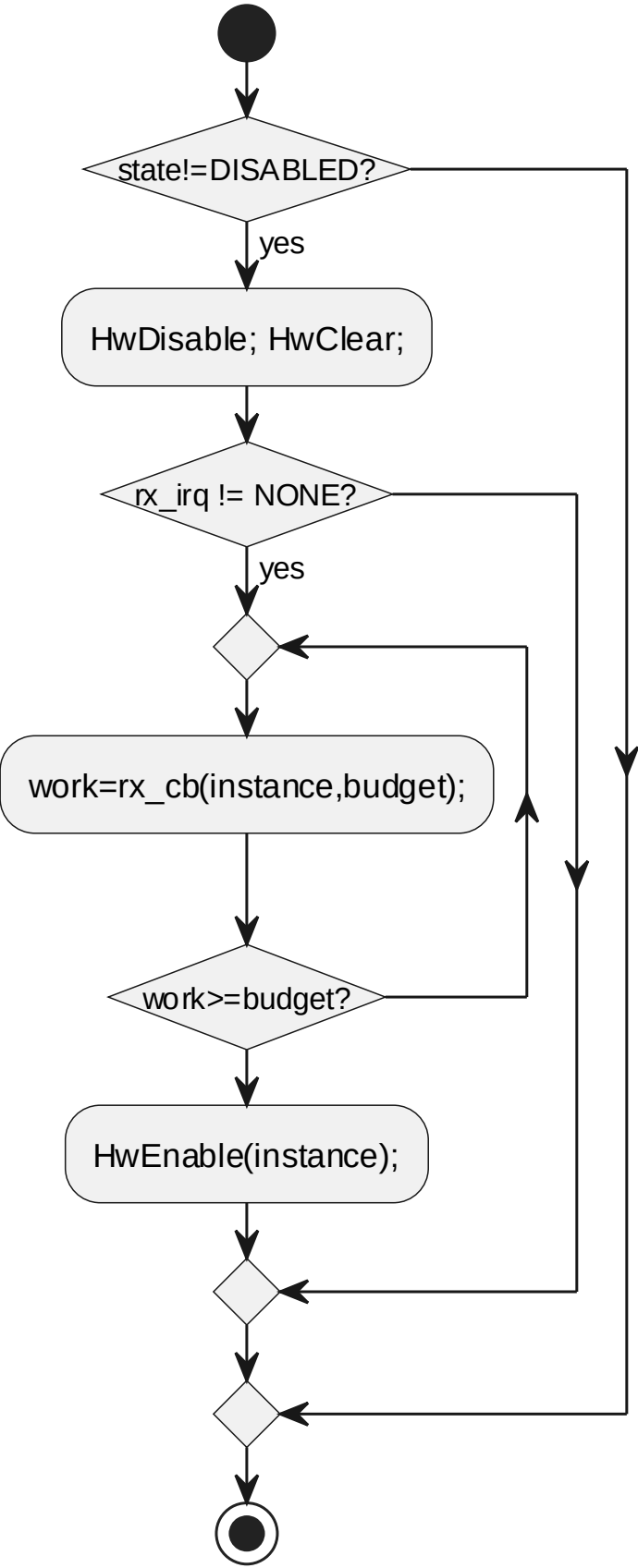
6.4.5 ipcsShmHardirqInstance

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	driver interrupt service routine			
函数原型	void ipcsShmHardirqInstance(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance

				索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

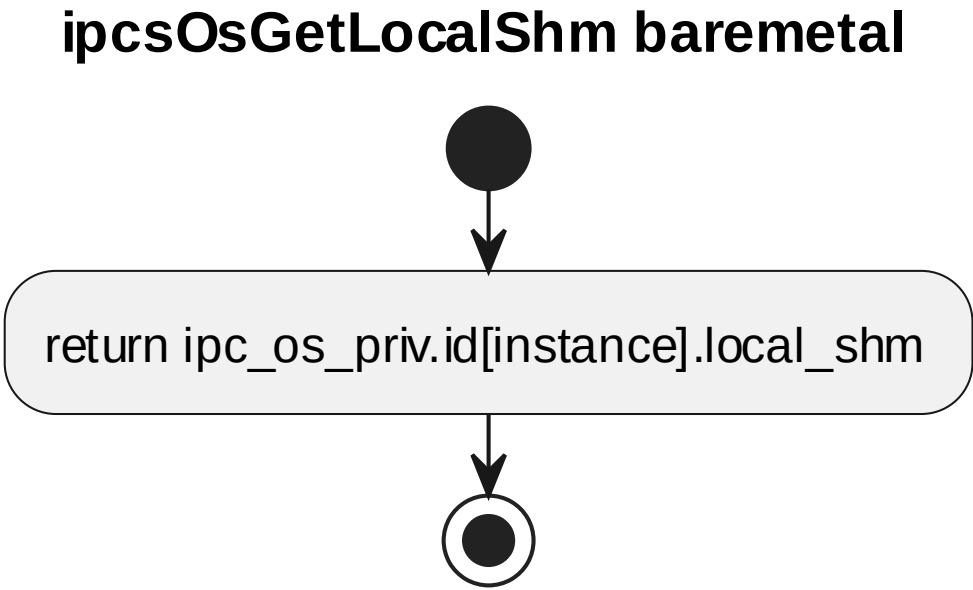
ipcsShmHardirqInstance baremetal



6. 4. 6 ipcsOsGetLocalShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	get local shared mem address			
函数原型	uintptr_t ipcsOsGetLocalShm(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	uintptr_t		源码返回值	
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow



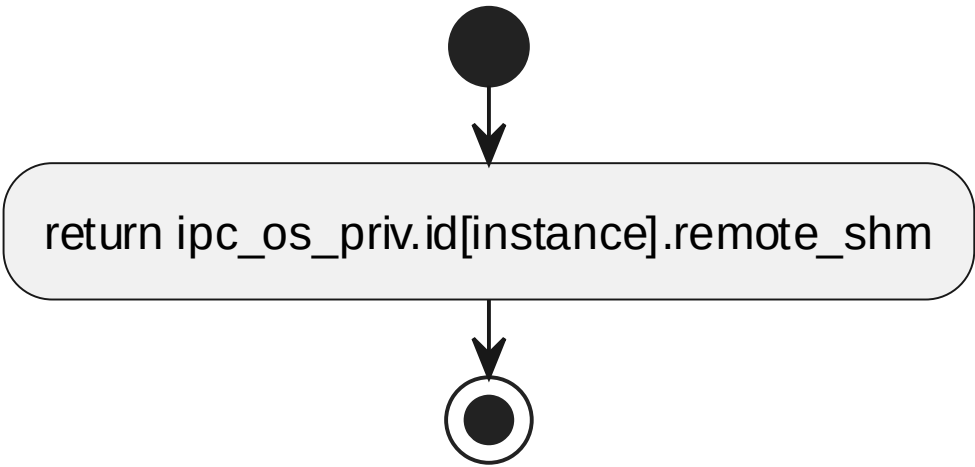
6. 4. 7 ipcsOsGetRemoteShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	get remote shared mem address			
函数原型	uintptr_t ipcsOsGetRemoteShm(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明

	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	uintptr_t		源码返回值	
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

ipcsOsGetRemoteShm baremetal

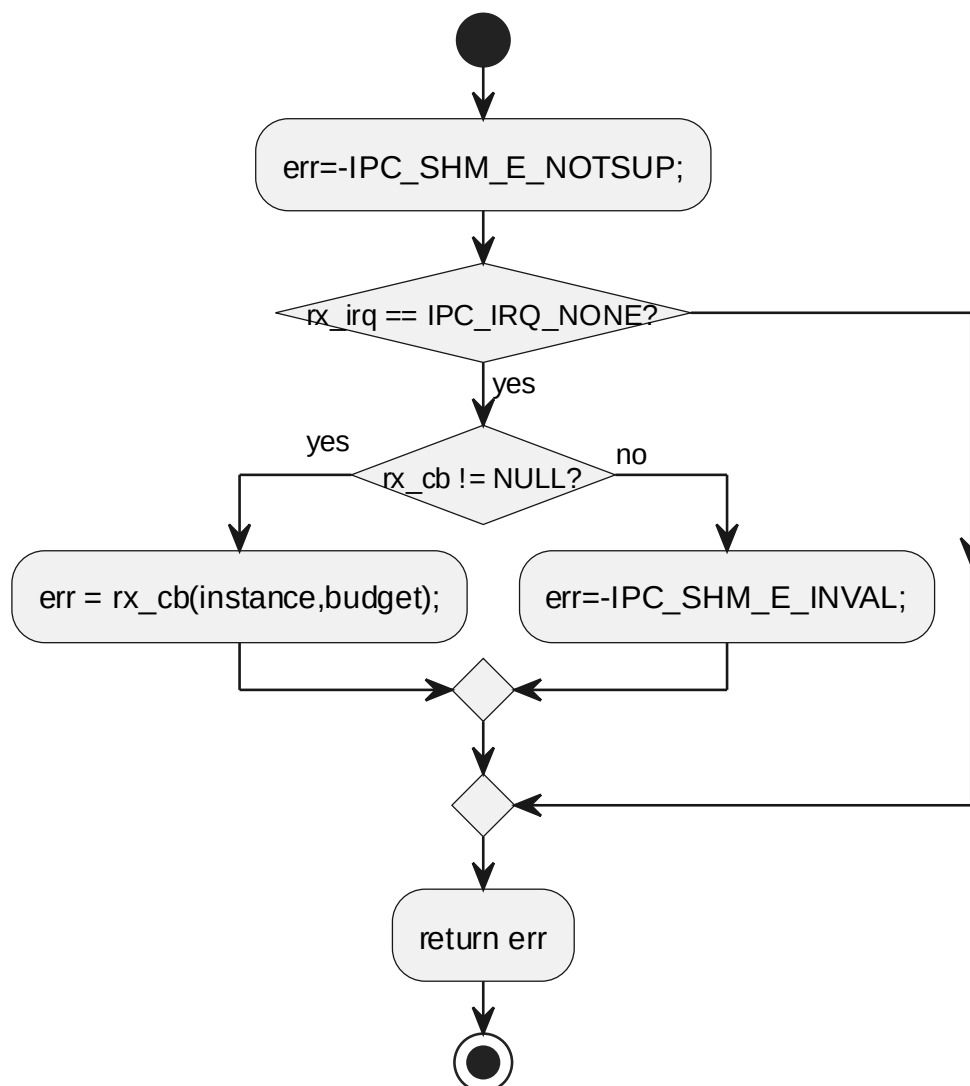


6. 4. 8 ipcsOsPollChannels

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_FREERTOS			
函数说明	invoke rx callback configured at initialization			
函数原型	sint32 ipcsOsPollChannels(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	sint32		work done, error code otherwise	
函数定义文件	ipcs/mcu/os/freertos/ipc-os-freertos.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

ipcsOsPollChannels baremetal



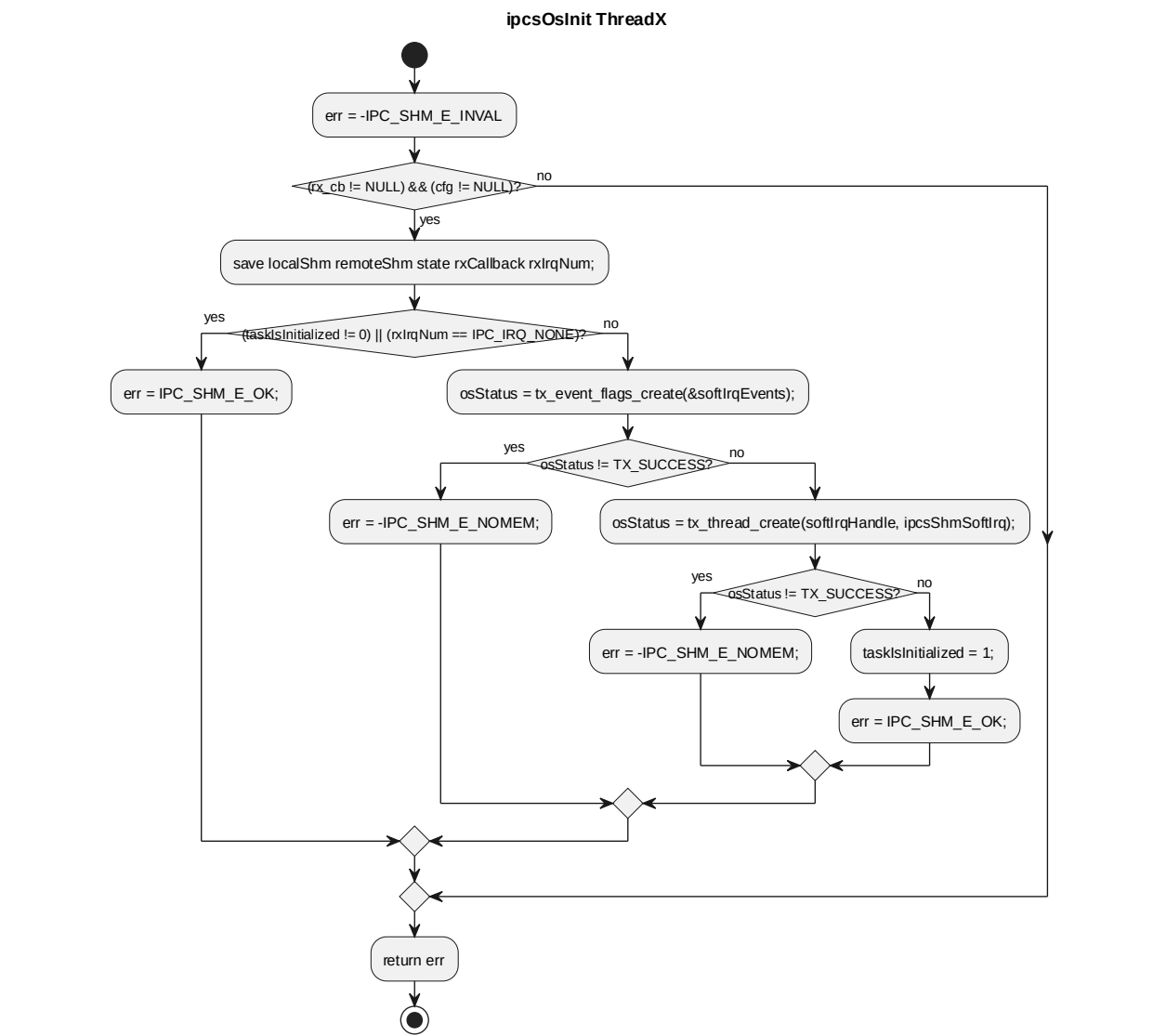
6.5 SWU_IPCS_OSAL_THREADX DESIGN 单元设计

6.5.1 ipcs0sInit

对应软件架构 ID	Drv_lpcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_THREADX			
函数说明	OS specific initialization code. When inter_core_rx_irq is disabled by passing IPC_IRQ_NONE, the softirq task will not be created.			
函数原型	sint32 ipcsOsInit(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg, sint32 (*rx_cb)(const uint8 instance, sint32 budget))			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance

				索引
	l	cfg	const struct IPCS_SHM_CFG_ TYPE *	configuration parameters
	l	rx_cb	sint32 (*rx_cb) (const uint8 instance, sint32 budget)	rx callback to be called from rx softirq
返回值	数据类型		说明	
	sint32		IPC_SHM_E_OK on success, -IPC_SHM_E_NOMEM if softirq task or event flags creation failed, -IPC_SHM_E_INVALID for invalid rx_cb or cfg	
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

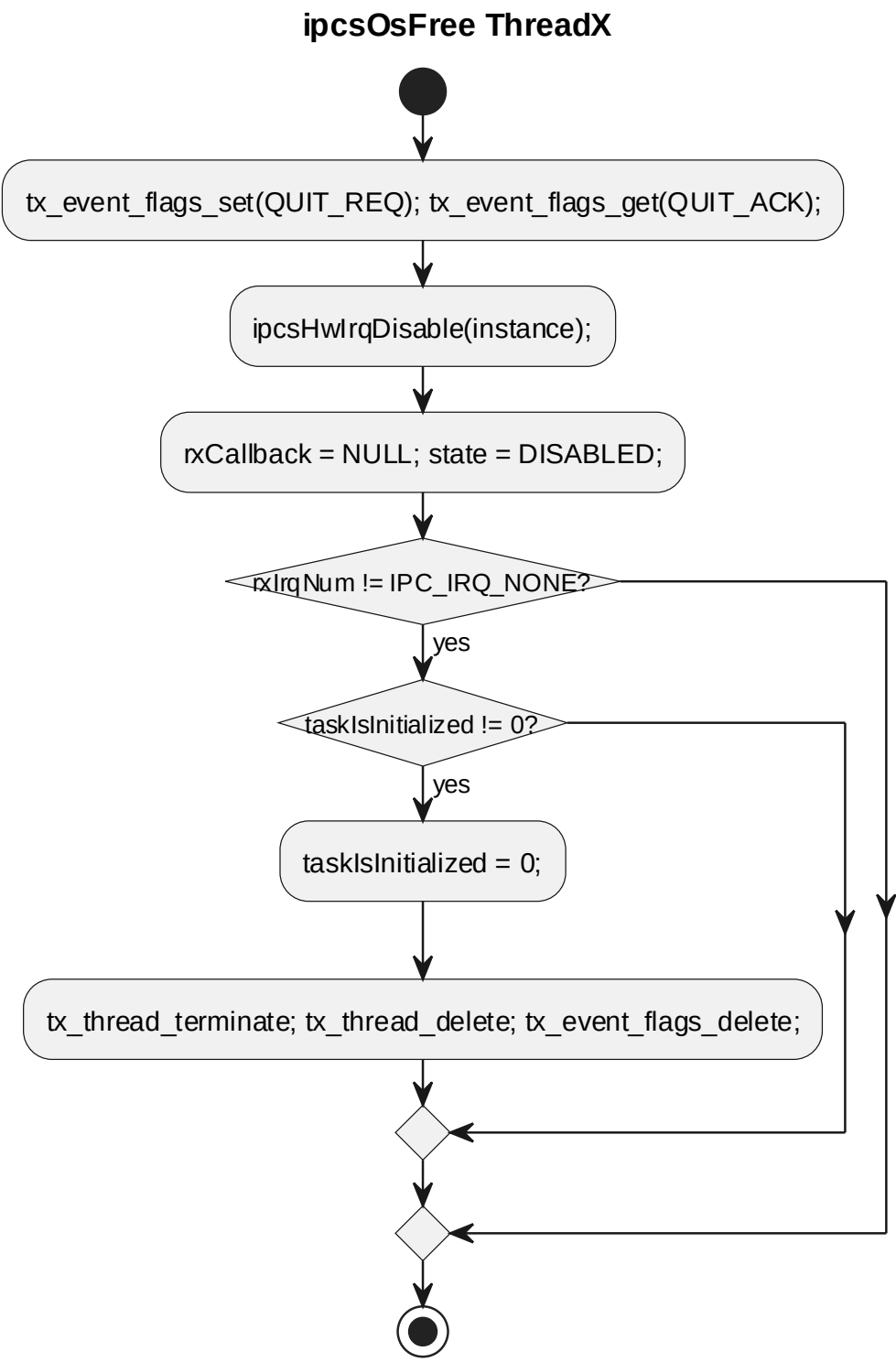
processing flow



6.5.2 ipcsOsFree

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_THREADX			
函数说明	free OS specific resources			
函数原型	void ipcsOsFree(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow



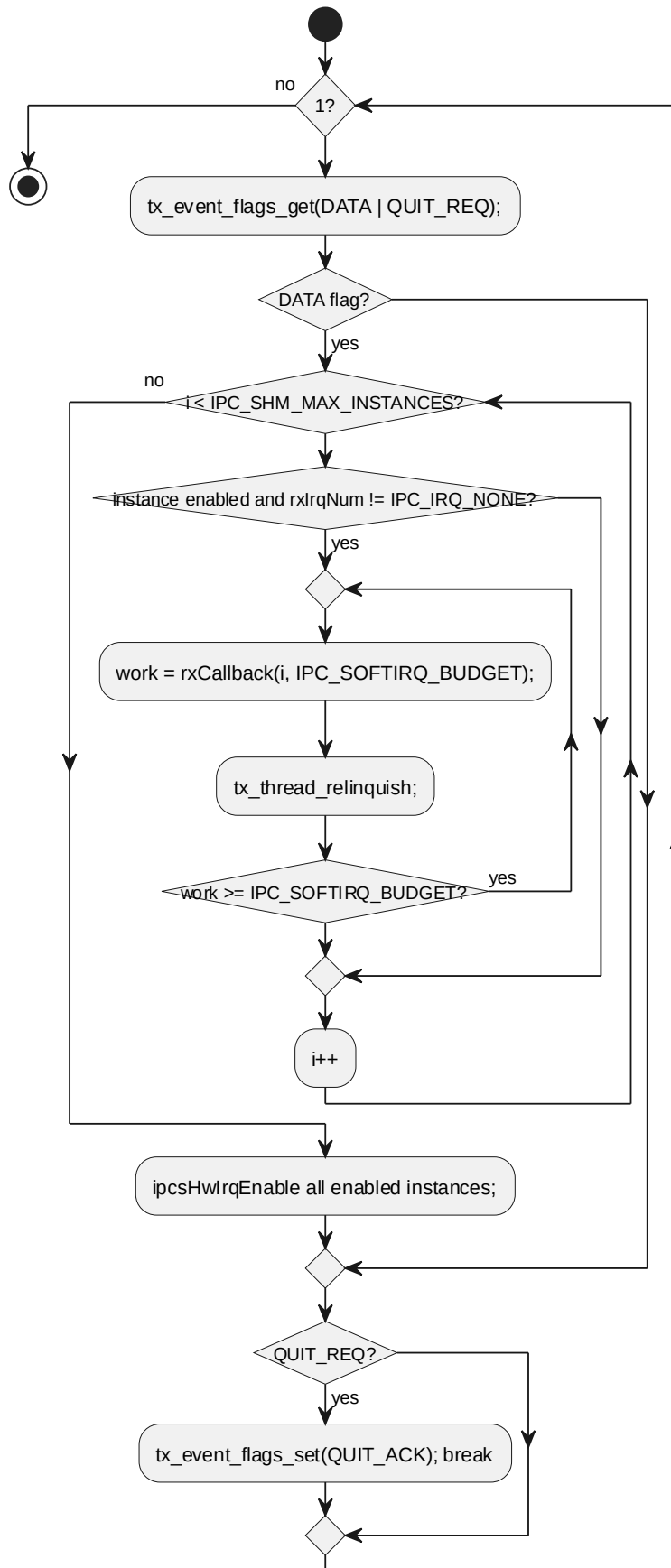
6.5.3 ipcsShmSoftIrq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp
软件单元 ID	SWU_IPCS_OSAL_THREADX
函数说明	task acting as deferred interrupt handler. Waits on event flags, calls upper

	layer rx callback registered with ipcsOsInit(), re-enables IRQ when work is done. Terminates when ipcsOsFree() sets quit request.			
函数原型	static void ipcsShmSoftIrq(uint32 ullInput)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	l	ullInput	uint32	ThreadX thread entry parameter (unused)
返回值	数据类型		说明	
	void		-	
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c			
函数声明文件	-			

processing flow

ipcsShmSoftIrq ThreadX

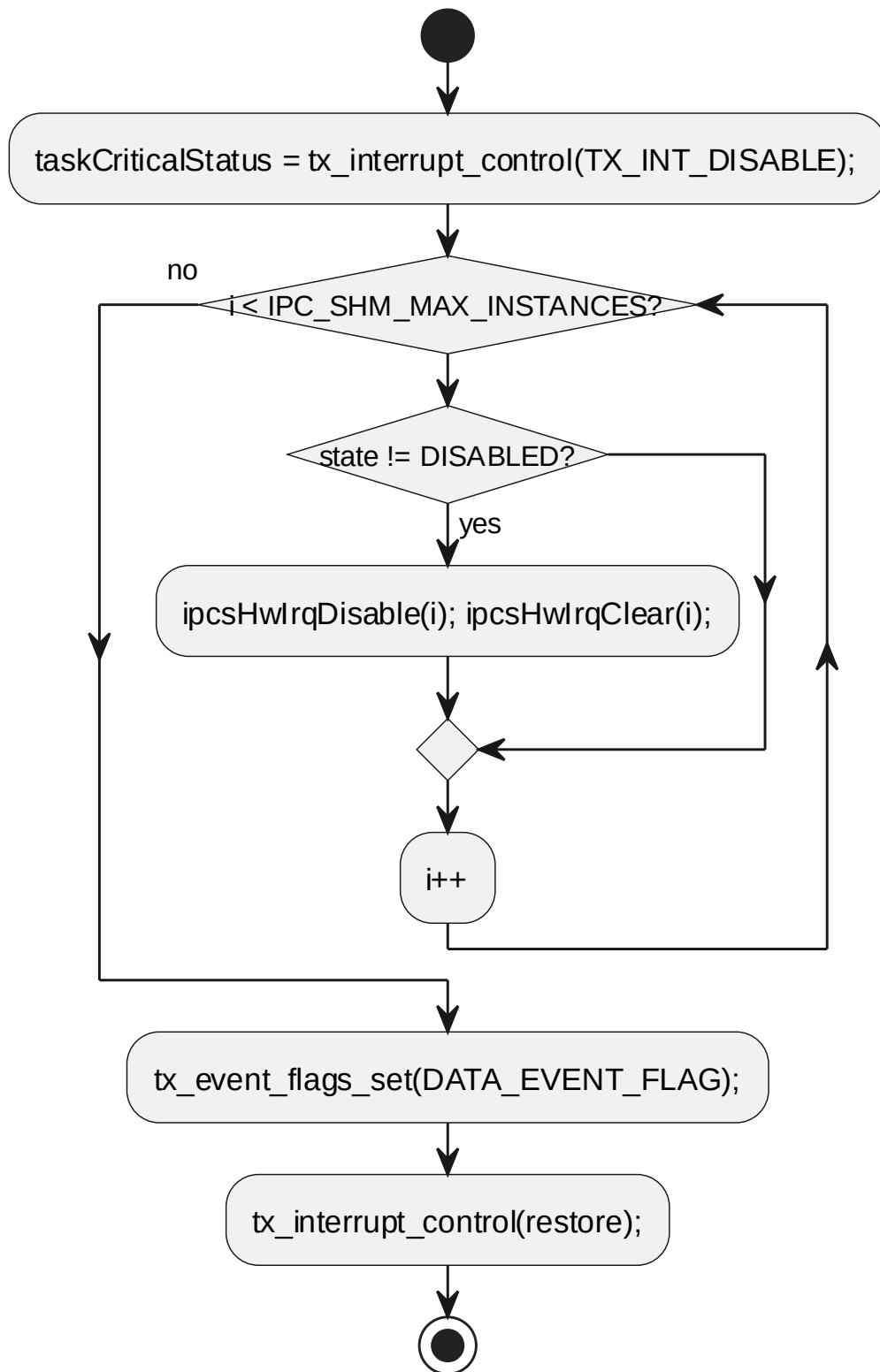


6.5.4 ipcsShmHardIrq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_THREADX			
函数说明	driver interrupt service routine			
函数原型	void ipcsShmHardIrq(void)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	-	-	-	-
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

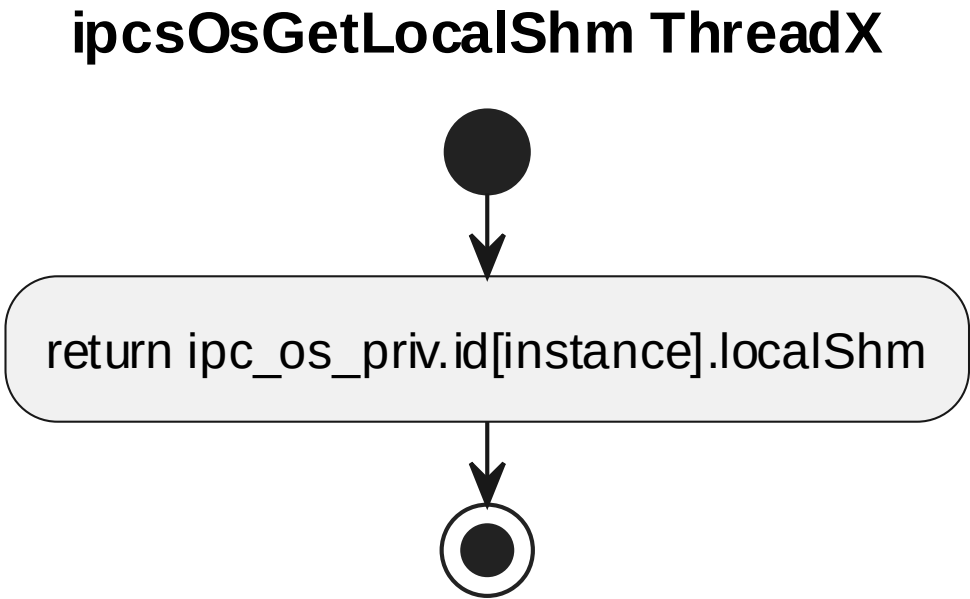
ipcsShmHardIrq ThreadX



6.5.5 ipcsOsGetLocalShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_THREADX			
函数说明	get local shared mem address			
函数原型	uintptr_t ipcsOsGetLocalShm(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	uintptr_t		local shared memory address for instance	
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow



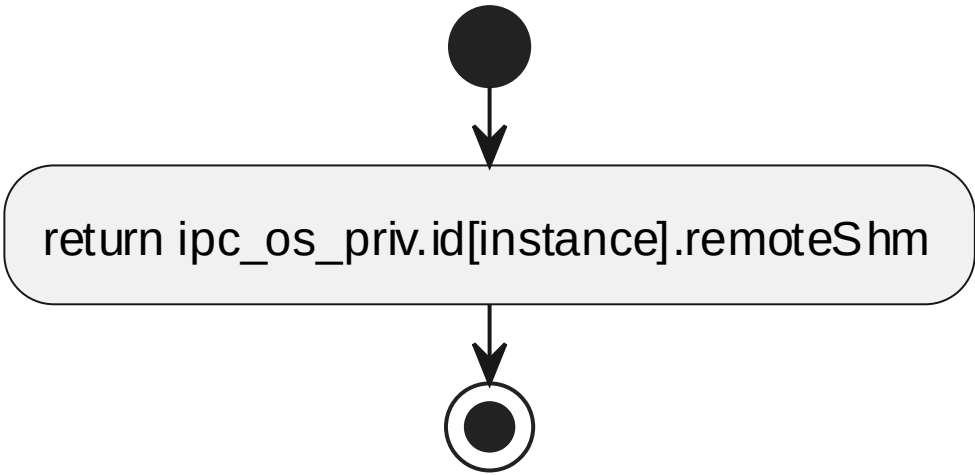
6.5.6 ipcsOsGetRemoteShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp
软件单元 ID	SWU_IPCS_OSAL_THREADX
函数说明	get remote shared mem address
函数原型	uintptr_t ipcsOsGetRemoteShm(const uint8 instance)
制约条件	-

输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	uintptr_t		remote shared memory address for instance	
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c			
函数声明文件	ipcs/mcu/os/ipc-os.h			

processing flow

ipcsOsGetRemoteShm ThreadX

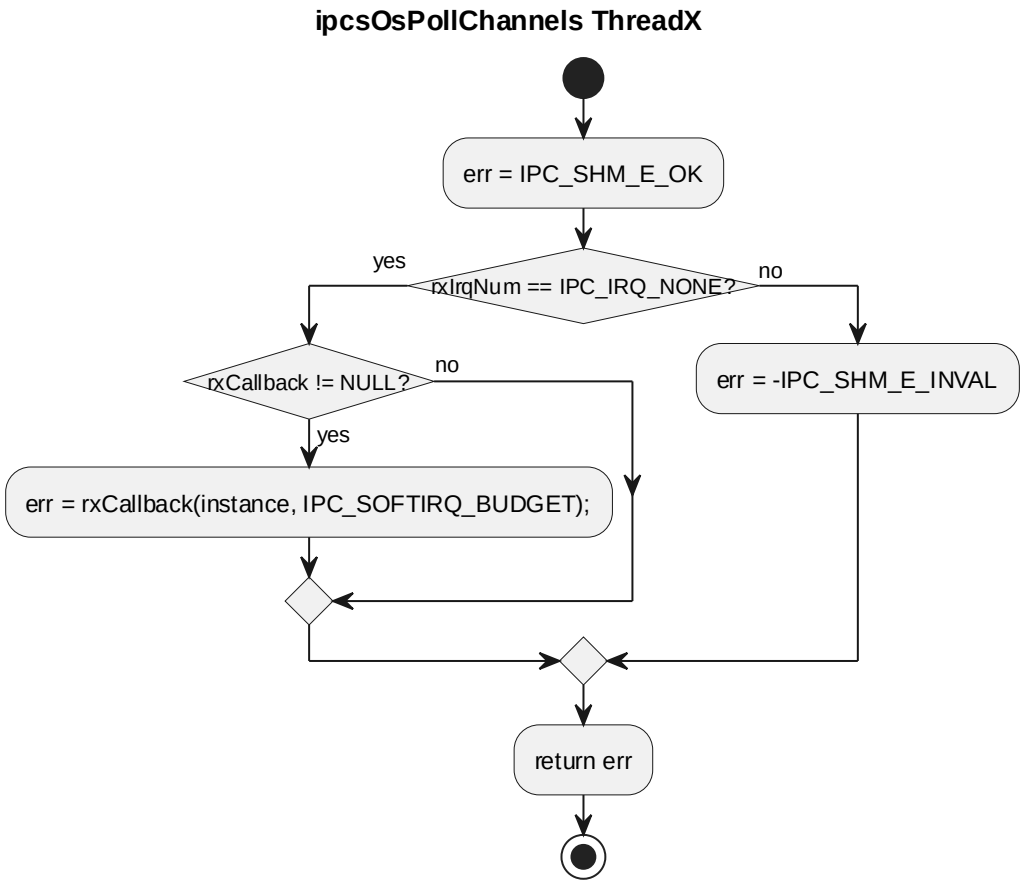


6.5.7 ipcsOsPollChannels

对应软件架构ID	Drv_Ipcs_Osal_Cmp			
软件单元 ID	SWU_IPCS_OSAL_THREADX			
函数说明	invoke rx callback configured at initialization. The softirq task handles rx operation when rx interrupt is configured.			
函数原型	sint32 ipcsOsPollChannels(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	sint32		work done when rx_irq is IPC_IRQ_NONE; IPC_SHM_E_OK if	

		rx_irq configured but callback not invoked; -IPC_SHM_E_INVALID when rx interrupt is configured
函数定义文件	ipcs/mcu/os/threadx/ipc-os-threadx.c	
函数声明文件	ipcs/mcu/os/ipc-os.h	

processing flow



6.6 SWU_IPCS_HAL_MCU DESIGN 单元设计

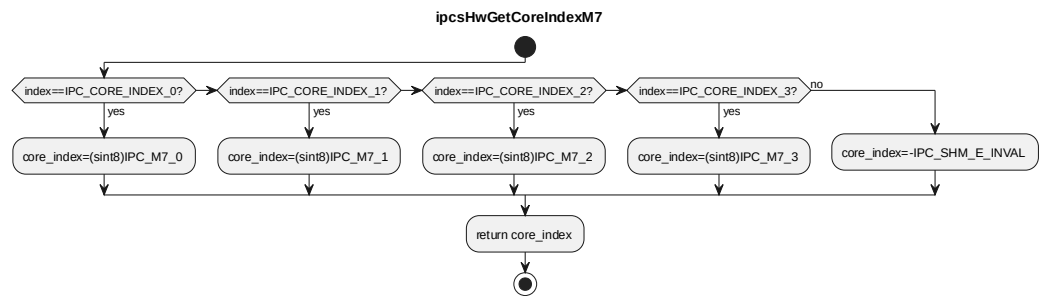
本节严格按照 reference.md 的内部函数表格格式描述内部函数。除 3.3 中列出的 9 个对外接口之外，其余内部函数、跨组件调用接口和 OS task 单元均作为内部接口。

6.6.1 ipcsHwGetCoreIndexM7

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	Validate and get core index if core type is m7			
函数原型	static sint8 ipcsHwGetCoreIndexM7(uint8 index)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	index	uint8	core 或 IRQ 配

				置索引
返回值	数据类型		说明	
	sint8		core_index if core type is m7	
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	-			

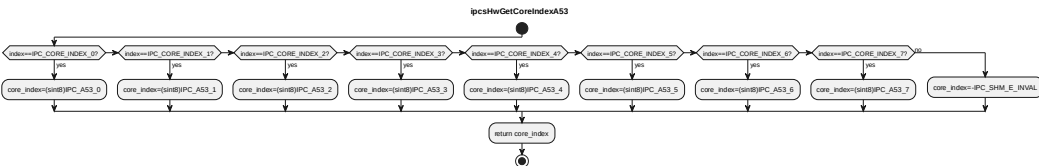
processing flow



6.6.2 ipcsHwGetCoreIndexA53

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	Validate and get core index if core type is a53			
函数原型	static sint8 ipcsHwGetCoreIndexA53(uint8 index)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	index	uint8	core 或 IRQ 配置索引
返回值	数据类型		说明	
	sint8		core_index if core type is a53	
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	-			

processing flow

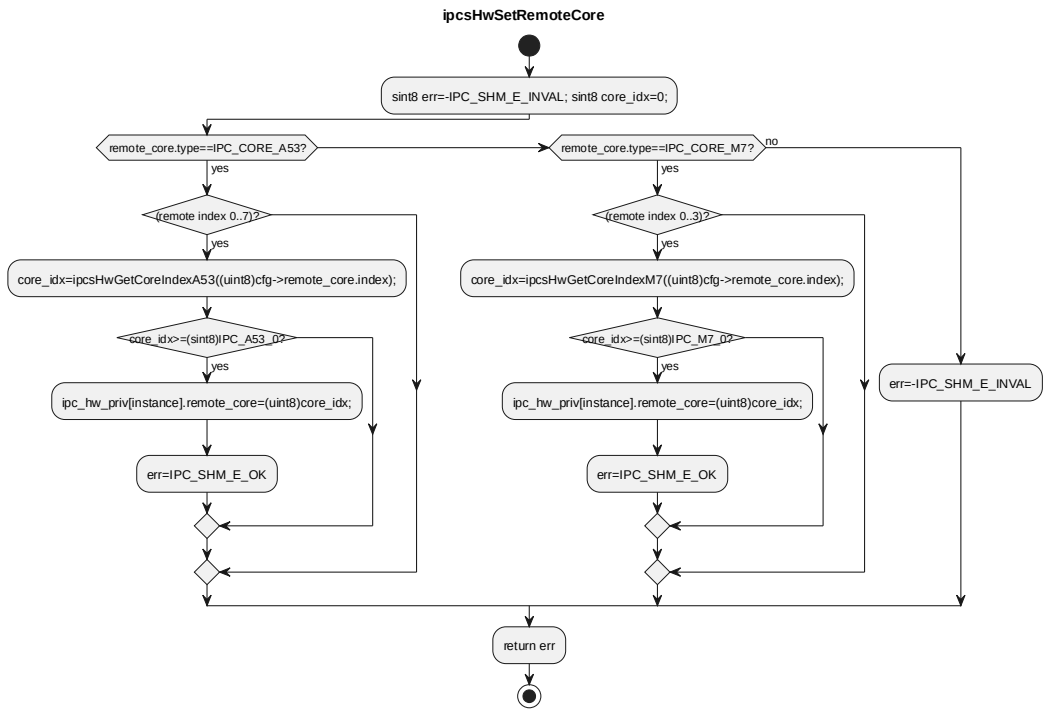


6.6.3 ipcsHwSetRemoteCore

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	get remote core for platform private data			
函数原型	static sint8 ipcsHwSetRemoteCore(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)			

制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	configuration parameters
	I	cfg	const struct IPCS_SHM_CFG_TYPE *	: Local core type from ipcf configuration
返回值	数据类型		说明	
	sint8		IPC_SHM_E_OK for success, -IPC_SHM_E_INVALID for invalid core	
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	-			

processing flow

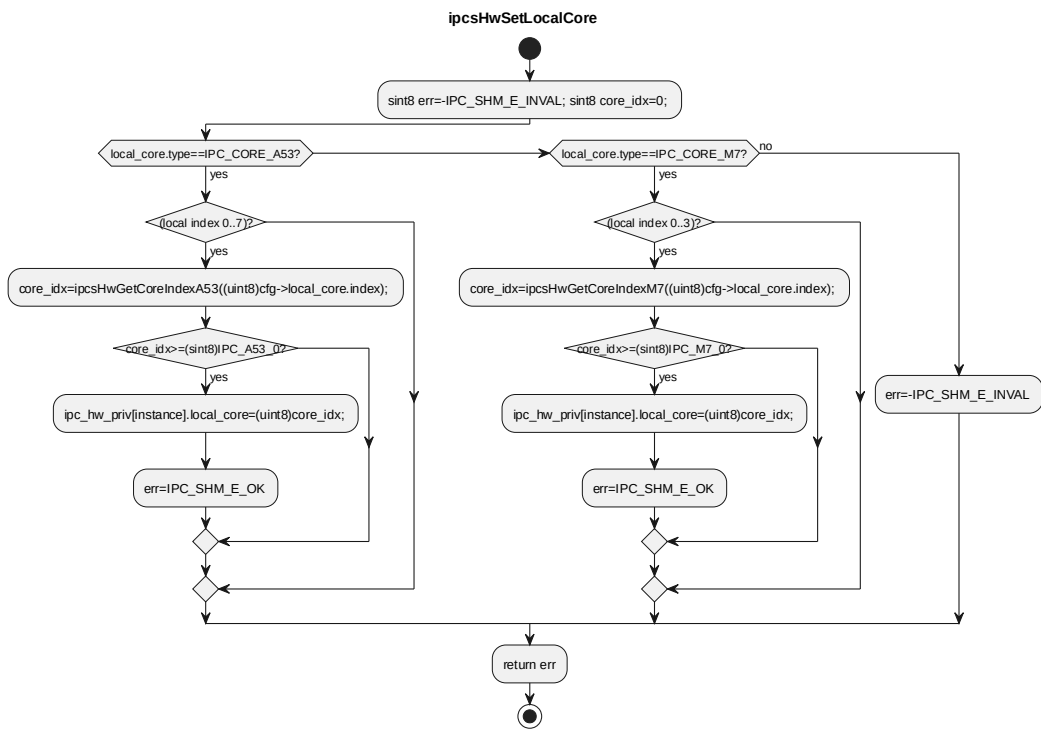


6.6.4 ipcsHwSetLocalCore

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	get local core for platform private data			
函数原型	static sint8 ipcsHwSetLocalCore(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	configuration parameters

	I	cfg	const struct IPCS_SHM_CFG_ TYPE *	: Local core type from ipcf configuration
返回值	数据类型		说明	
	sint8		IPC_SHM_E_OK for success, -IPC_SHM_E_INVALID for invalid core	
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	-			

processing flow

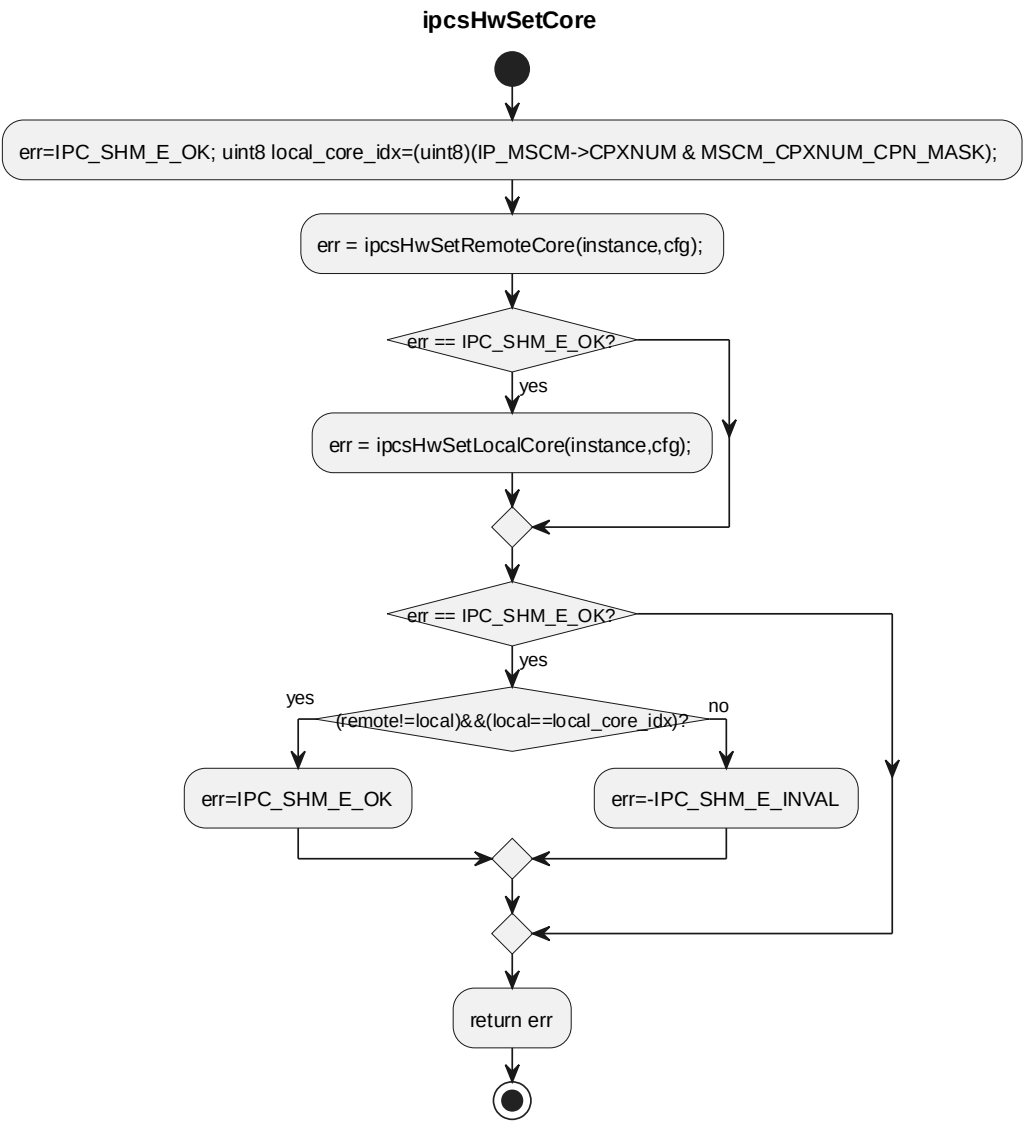


6.6.5 ipcsHwSetCore

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	get local and remote core for platform private data			
函数原型	static sint8 ipcsHwSetCore(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	configuration parameters
	I	cfg	const struct IPCS_SHM_CFG_ TYPE *	: Local core type from ipcf configuration

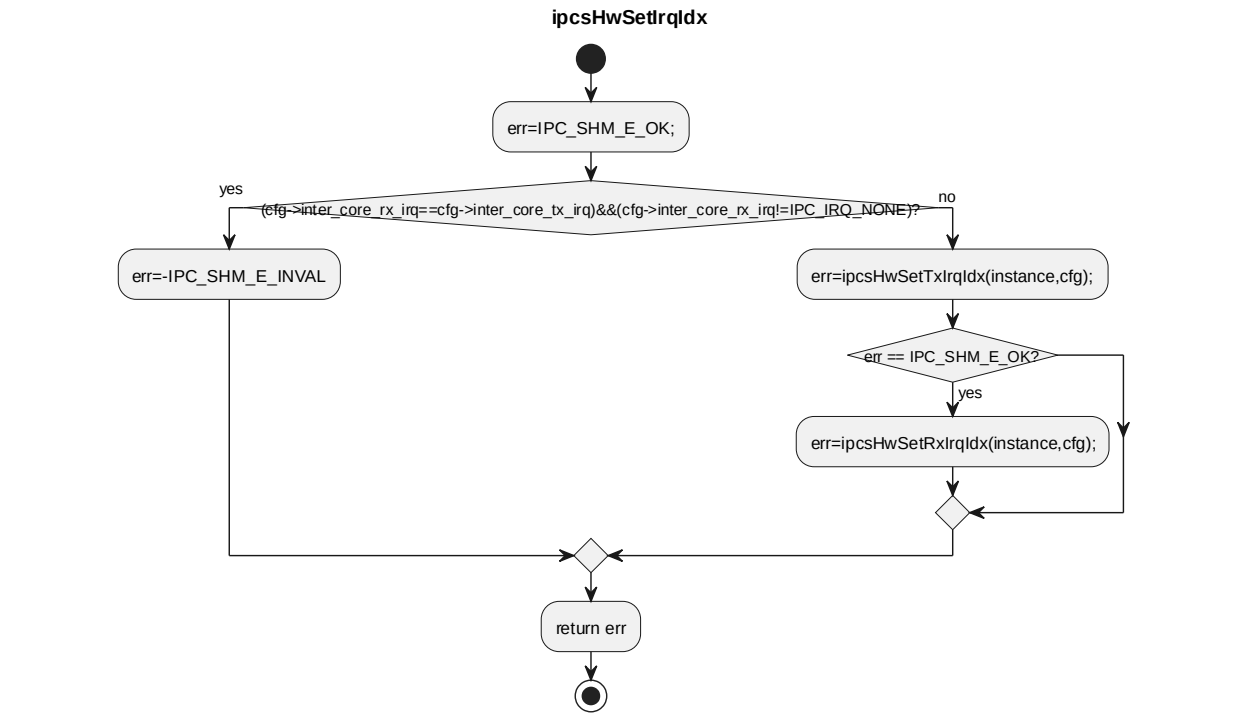
返回值	数据类型	说明
	sint8	IPC_SHM_E_OK for success, -IPC_SHM_E_INVALID for invalid core
函数定义文件	ipcs/mcu/hw/ipc-hw.c	
函数声明文件	-	

processing flow



6.6.6 ipcsHwSetTxIrqIdx

对应软件架构 ID	Drv_Ipcs_Hal_Cmp
软件单元 ID	SWU_IPCS_HAL_MCU
函数说明	get tx irq and msi index for platform private data
函数原型	static sint8 ipcsHwSetTxIrqIdx(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)
制约条件	-

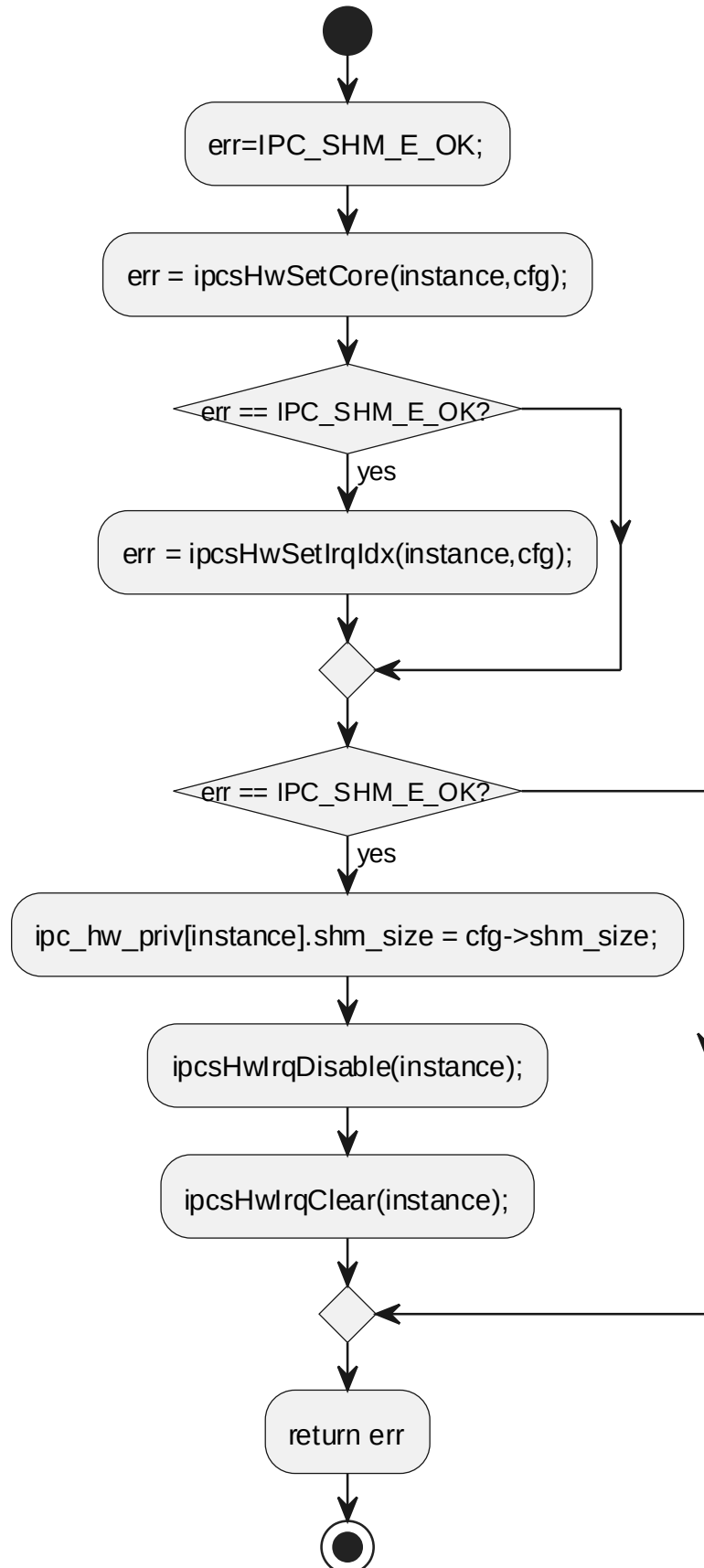


6.6.9 ipcsHwInit

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	platform specific initialization			
函数原型	sint8 ipcsHwInit(const uint8 instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
	I	cfg	const struct IPCS_SHM_CFG_TYPE *	configuration parameters
返回值	数据类型		说明	
	sint8		IPC_SHM_E_OK for success, -IPC_SHM_E_INVALID for either inter core	
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

processing flow

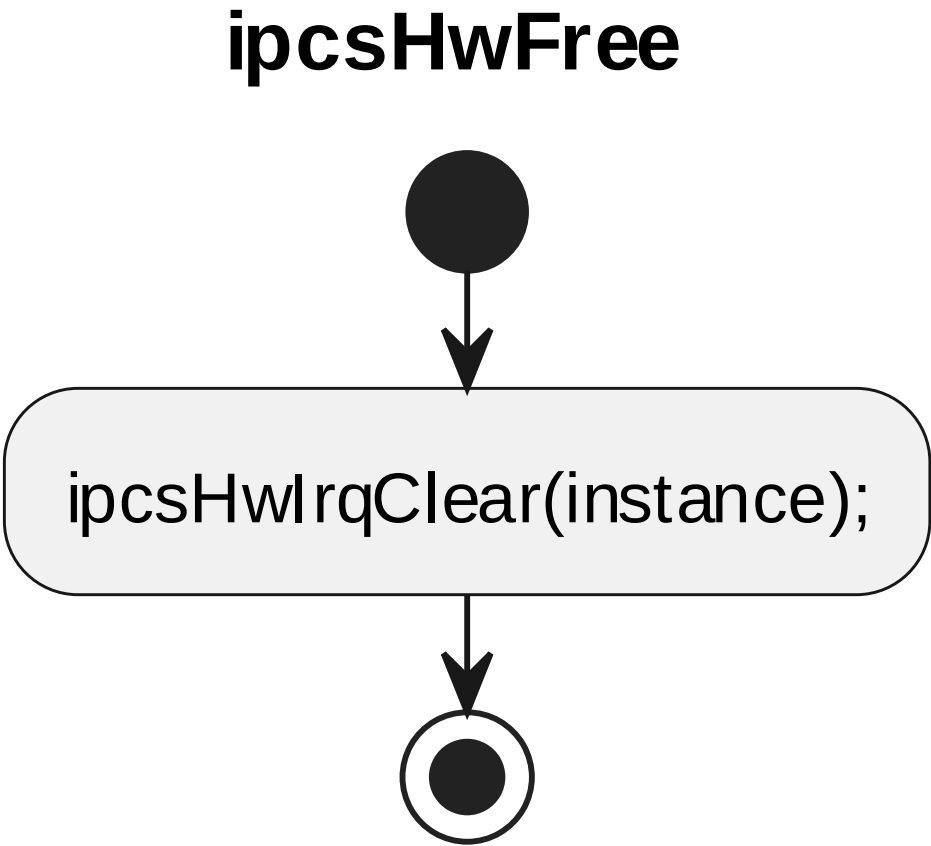
ipcsHwInit



6. 6. 10 ipcsHwFree

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	free hw resources			
函数原型	void ipcsHwFree(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

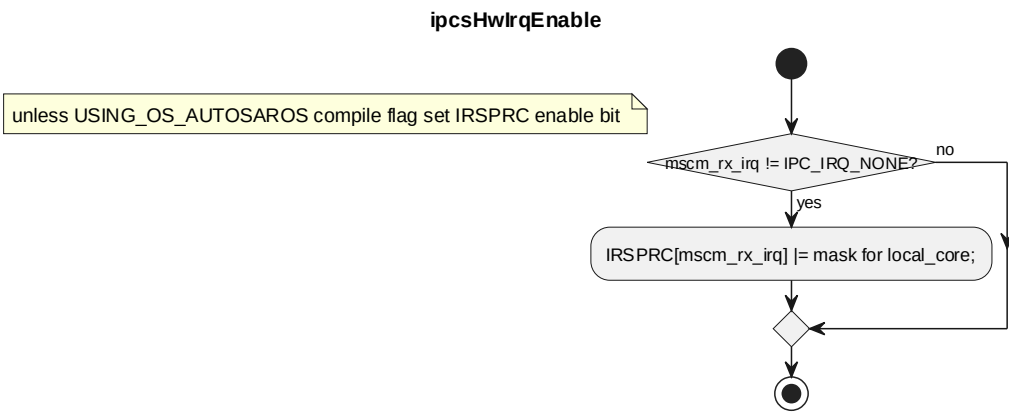
processing flow



6. 6. 11 ipcsHwIrqEnable

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	enable notifications from remote			
函数原型	void ipcsHwIrqEnable(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

processing flow

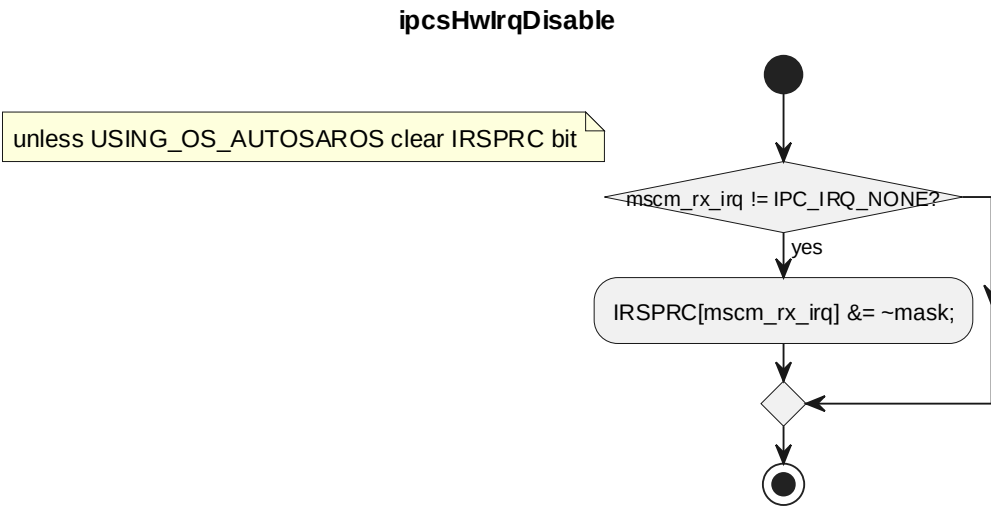


6. 6. 12 ipcsHwIrqDisable

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	disable notifications from remote			
函数原型	void ipcsHwIrqDisable(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			

函数声明文件	ipcs/mcu/hw/ipc-hw.h
--------	----------------------

processing flow

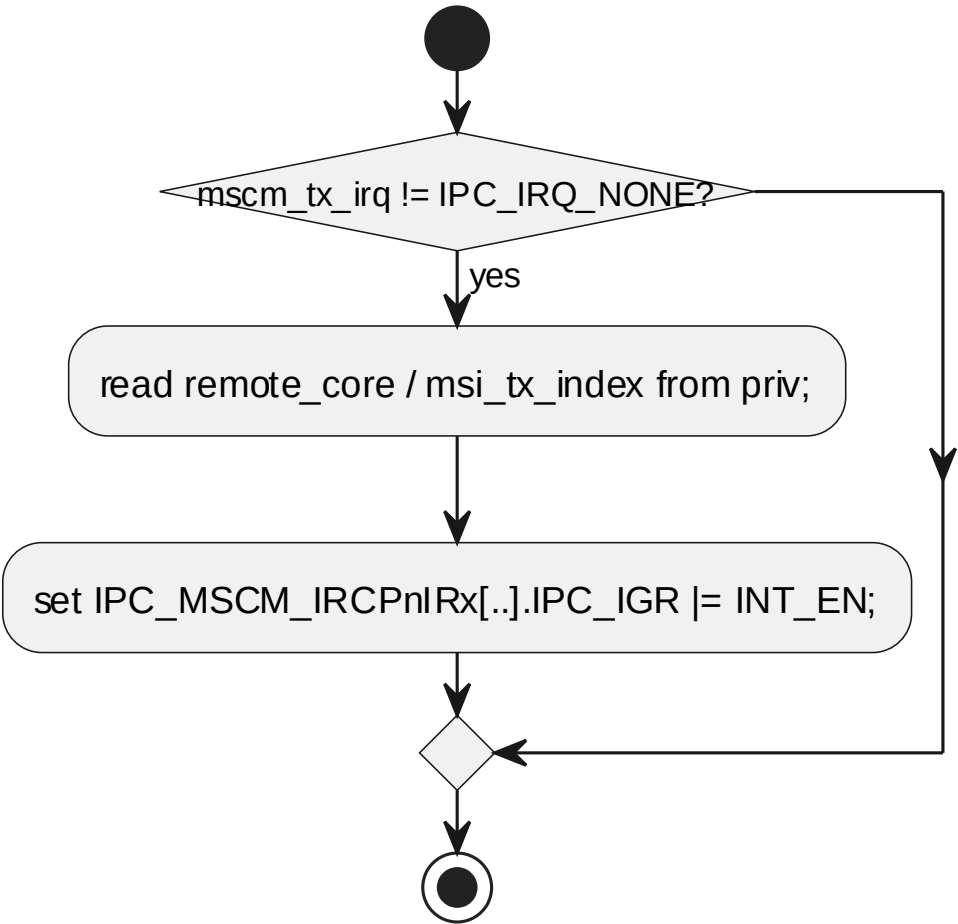


6. 6. 13 ipcsHwIrqNotify

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	notify remote that data is available			
函数原型	void ipcsHwIrqNotify(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

processing flow

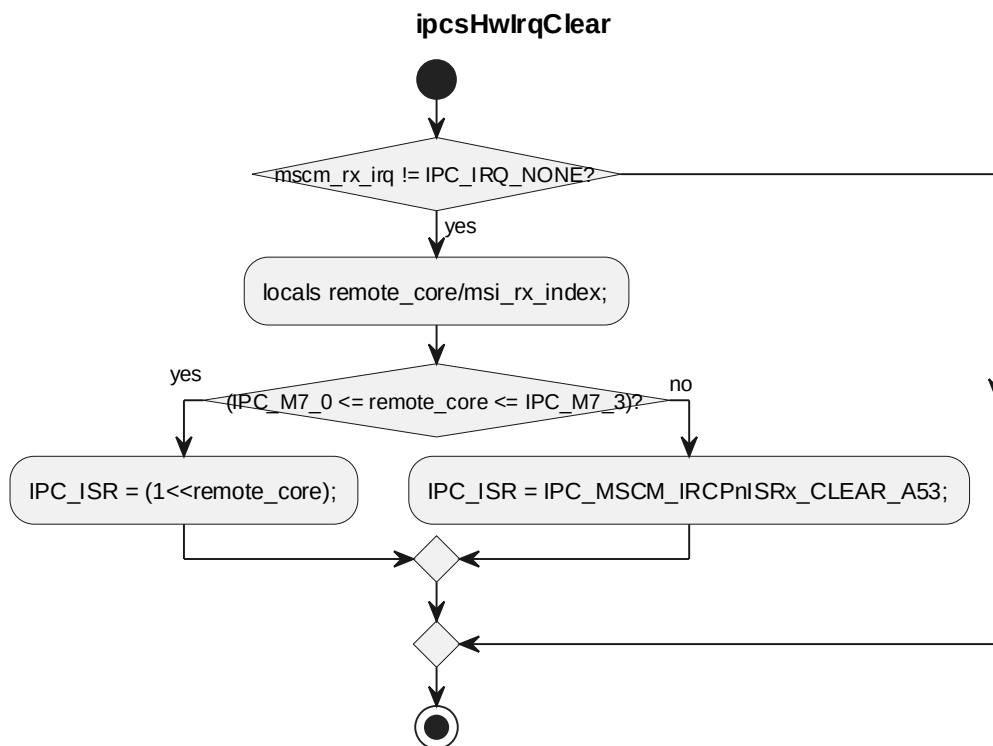
ipcsHwIrqNotify



6.6.14 ipcsHwIrqClear

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	clear available data notification			
函数原型	void ipcsHwIrqClear(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	IPCS shared memory instance 索引
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

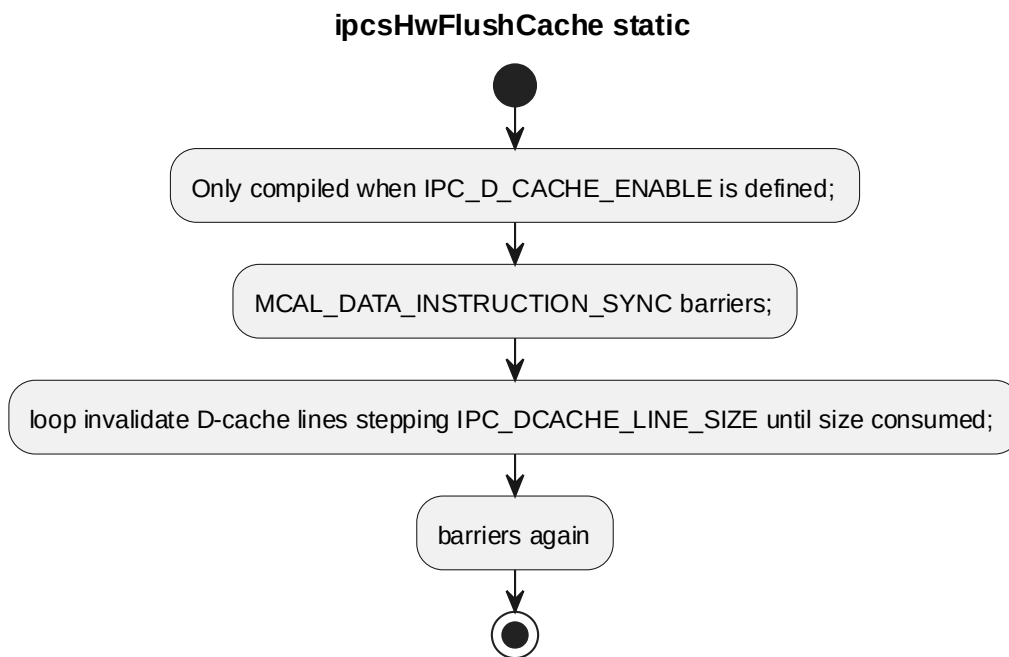
processing flow



6. 6. 15 ipcsHwFlushCache

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	-			
函数原型	static void ipcsHwFlushCache(uint32 data_addr, uint32 data_size)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	data_addr	uint32	源码参数
	I	data_size	uint32	复制数据大小，单位为 byte
返回值	数据类型		说明	
	void		源码返回值	
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	-			

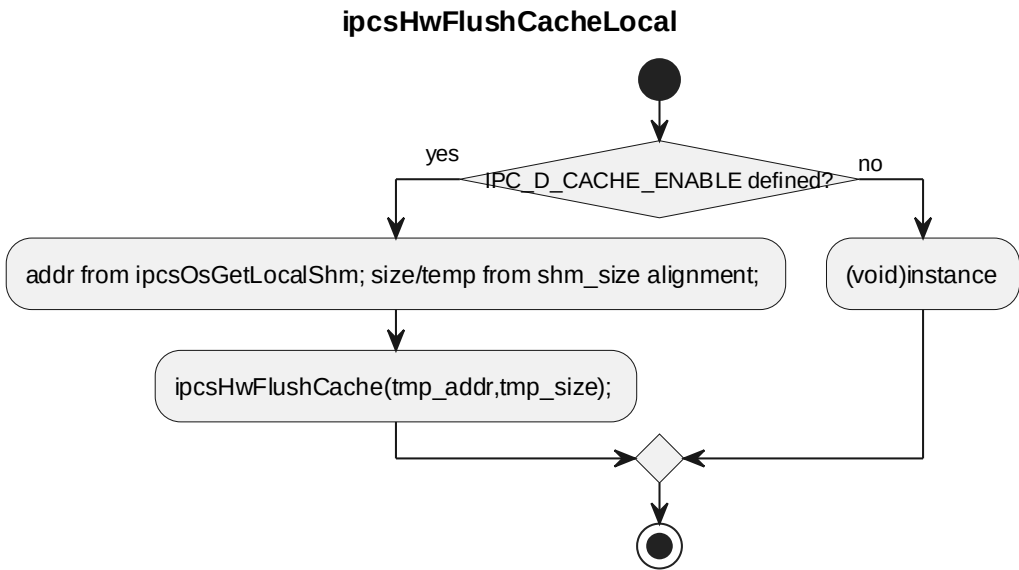
processing flow



6. 6. 16 ipcsHwFlushCacheLocal

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	Clear and invalidate cache content			
函数原型	void ipcsHwFlushCacheLocal(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

processing flow

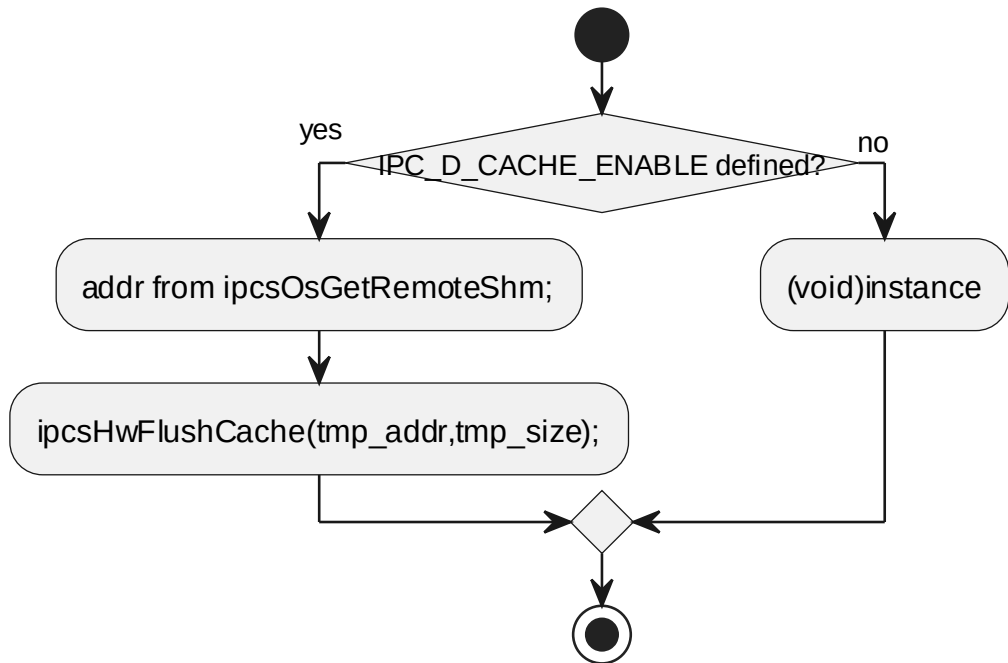


6. 6. 17 ipcsHwFlushCacheRemote

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_MCU			
函数说明	Clear and invalidate cache content			
函数原型	void ipcsHwFlushCacheRemote(const uint8 instance)			
制约条件	-			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8	instance id
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mcu/hw/ipc-hw.c			
函数声明文件	ipcs/mcu/hw/ipc-hw.h			

processing flow

ipcsHwFlushCacheRemote



6. 7	6. 8	6. 9	6. 10	6. 11
6. 12	6. 13	6. 14	6. 15	6. 16
6. 17	6. 18	6. 19	6. 20	6. 21
6. 22	6. 23	6. 24	6. 25	6. 26
6. 27	6. 28	6. 29	6. 30	6. 31 GLOBAL VARIABLE S 全局变 量

全局变量名称
全局变量类型
全局变量范围
全局变量描述
全局变量的存储 RAM 区
ipc_os_priv
static struct IPCS_OS_PRIV_TYPE_TYPE
ipcs/mcu/os/autosar/ipc-os-autosar.c
AUTOSAR OS 实现私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)
ipc_os_priv

static struct IPCS_OS_PRIV_TYPE_TYPE
ipcs/mcu/os/freertos/ipc-os-freertos.c
FreeRTOS 实现私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)
ipc_os_priv
static struct (见 章节 5.8.7)
ipcs/mcu/os/threadx/ipc-os-threadx.c
ThreadX 实现私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)
ipc_hw_priv
static struct IPCS_HW_PRIV_TYPE_TYPE [IPC_SHM_MAX_INSTANCES]
ipcs/mcu/hw/ipc-hw.c
每 instance 平台 HAL 私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)
6.32 DATA TYPES 类型定义

6.32.1	6.32.2	6.32.3
6.32.4	6.32.5	6.32.6
6.32.7	6.32.8	6.32.9
6.32.10	6.32.11	6.32.12
6.32.13	6.32.14	6.32.15
6.32.16	6.32.17	6.32.18 struct IPCS_OS_PRIV_INSTANCE_TYPE (FreeRTOS 实现)

定义于 ipcs/mcu/os/freertos/ipc-os-freertos.c。
Type
Name
Description
uintptr_t
local_shm
local shared memory address
uintptr_t

remote_shm
remote shared memory address
sint32
state
state of instance
sint32
rx_irq_num
rx interrupt number
sint32
msg_received
state to indicate notification received for a new message

6. 32. 19	6. 32. 20	6. 32. 21
6. 32. 22	6. 32. 23	6. 32. 24
6. 32. 25	6. 32. 26	6. 32. 27
6. 32. 28	6. 32. 29	6. 32. 30 struct IPCS_OS_PRIV_TYPE_ TYPE (AUTOSAR 实 现)

定义于 ipcs/mcu/os/autosar/ipc-os-autosar.c。

Type

Name

Description

struct IPCS_OS_PRIV_INSTANCE_TYPE

id[IPC_SHM_MAX_INSTANCES]

private data per instance

sint32 (*rx_cb)(const uint8 instance, sint32 budget)

rx_cb

upper layer rx callback

sint32

task_is_initialized

flag to know if the softirq task is initialized

6. 32. 31	6. 32. 32	6. 32. 33
6. 32. 34	6. 32. 35	6. 32. 36
6. 32. 37	6. 32. 38	6. 32. 39
6. 32. 40	6. 32. 41	6. 32. 42
6. 32. 43	6. 32. 44	6. 32. 45 struct IPCS_OS_PRIV_TYPE_ TYPE (FreeRTOS 实 现)

定义于 ipcs/mcu/os/freertos/ipc-os-freertos.c。

Type

Name

Description

struct IPCS_OS_PRIV_INSTANCE_TYPE

id[IPC_SHM_MAX_INSTANCES]

private data per instance

sint32 (*rx_cb)(const uint8 instance, sint32 budget)

rx_cb

upper layer rx callback

TaskHandle_t

softirq_handle

rx task handle used by the ISR to notify the rx task

sint32

task_is_initialized

flag to know if the softirq task is initialized

6. 32. 46	6. 32. 47	6. 32. 48
6. 32. 49	6. 32. 50	6. 32. 51
6. 32. 52	6. 32. 53	6. 32. 54
6. 32. 55	6. 32. 56	6. 32. 57
6. 32. 58	6. 32. 59	6. 32. 60
6. 32. 61	6. 32. 62	6. 32. 63 IPC_OS_PRIV_ INSTANCE_T (ThreadX 实现)

定义于 ipcs/mcu/os/threadx/ipc-os-threadx.c (typedef struct) 。

Type

Name

Description

uintptr_t

localShm

local shared memory address

uintptr_t

remoteShm

remote shared memory address

sint32

state

state of u8Instance

sint32

rxIrqNum

rx interrupt number

sint32 (*rxCallback)(const uint8 u8Instance, sint32 budget)

rxCallback

upper layer rx callback registered per instance

6. 32. 64	6. 32. 65	6. 32. 66
6. 32. 67	6. 32. 68	6. 32. 69
6. 32. 70	6. 32. 71	6. 32. 72
6. 32. 73	6. 32. 74	6. 32. 75
6. 32. 76	6. 32. 77	6. 32. 78
6. 32. 79	6. 32. 80	6. 32. 81 ThreadX ipc_os_priv 聚合类 型

ipc_os_priv 在 ipcs/mcu/os/threadx/ipc-os-threadx.c 中为匿名 static struct, 成员如下。

Type

Name

Description

IPC_OS_PRIV_INSTANCE_T

id[IPC_SHM_MAX_INSTANCES]

private data per u8Instance

TX_EVENT_FLAGS_GROUP

softIrqEvents

event flags used by hardirq to notify softirq thread

uint8

ipcSoftIrqStack[IPC_SOFTIRQ_STACK_SIZE]

softirq thread stack

TX_THREAD

softIrqHandle

rx task handle used by the ISR to notify the rx task

sint32

taskIsInitialized

flag to know if the softirq task is initialized

6. 32. 82	6. 32. 83	6. 32. 84
6. 32. 85	6. 32. 86	6. 32. 87
6. 32. 88	6. 32. 89	6. 32. 90
6. 32. 91	6. 32. 92	6. 32. 93
6. 32. 94	6. 32. 95	6. 32. 96
6. 32. 97	6. 32. 98	6. 32. 99
6. 32. 100	6. 32. 101	6. 32. 102
6. 32. 103	6. 32. 104	6. 32. 105 struct IPCS_HW_PRIV_TYPE_ TYPE

定义于 ipcs/mcu/hw/ipc-hw.c。

Type

Name

Description

uint8

msi_tx_irq

MSI index of inter-core interrupt corresponds to mscm_tx_irq

uint8

msi_rx_irq

MSI index of inter-core interrupt corresponds to mscm_rx_irq

uint8

remote_core

remote core to trigger the interrupt on

uint8

local_core

local core on where this instance is running

uint32

shm_size

local/remote shared memory size

sint16

mscm_tx_irq

核间中断控制器 inter-core interrupt reserved for shm driver tx

sint16

mscm_rx_irq

核间中断控制器 inter-core interrupt reserved for shm driver rx

6. 32. 106	6. 32. 107
6. 32. 108	6. 32. 109
6. 32. 110	6. 32. 111
6. 32. 112	6. 32. 113
6. 32. 114	6. 32. 115
6. 32. 116	6. 32. 117
6. 32. 118	6. 32. 119
6. 32. 120	6. 32. 121
6. 32. 122	6. 32. 123
6. 32. 124	6. 32. 125
6. 32. 126	6. 32. 127
6. 32. 128	6. 32. 129
6. 32. 130	6. 32. 131 enum IPCS_PROCESSOR_IDX_E

定义于 ipcs/mcu/hw/ipc-hw-platform.h。

Name

Description

IPC_A53_0
ARM Cortex-A53 cluster 0 core 0
IPC_A53_1
ARM Cortex-A53 cluster 1 core 1
IPC_A53_2
ARM Cortex-A53 cluster 1 core 0
IPC_A53_3
ARM Cortex-A53 cluster 1 core 1
IPC_M7_0
ARM Cortex-M7 core 0
IPC_M7_1
ARM Cortex-M7 core 1
IPC_M7_2
ARM Cortex-M7 core 2
IPC_M7_3
ARM Cortex-M7 core 2
IPC_A53_4
ARM Cortex-A53 cluster 0 core 2
IPC_A53_5
ARM Cortex-A53 cluster 0 core 3
IPC_A53_6
ARM Cortex-A53 cluster 1 core 2
IPC_A53_7
ARM Cortex-A53 cluster 1 core 3

6. 32. 132	6. 32. 133	6. 32. 134
6. 32. 135	6. 32. 136	6. 32. 137
6. 32. 138	6. 32. 139	6. 32. 140 IPCS_MSCM_IR CP_IR_TYPE

定义于 ipcs/mcu/hw/ipc-hw-platform.h (typedef struct) 。

Type

Name

Description

volatile uint32

IPC_ISR

Interrupt Router CPn Interruptx Status Register

volatile uint32

IPC_IGR

Interrupt Router CPn Interruptx Generation Register

6. 32. 141	6. 32. 142	6. 32. 143
6. 32. 144	6. 32. 145	6. 32. 146 IPCS_MSCM_IR CPnIRx_TYPE

定义于 ipcs/mcu/hw/ipc-hw-platform.h (typedef struct) 。

Type

Name

Description

IPCS_MSCM_IRCP_IR_TYPE

IRCPnIRx[IPC_MSCM_CPX_COUNT][IPC_MSCM_MSI_COUNT]

memory-mapped 核间中断控制器 interrupt router register array

6. 33 DYNAMIC DETAILED DESIGN 动态详细设计

RTOS 部署变体与 Linux 部署变体共用 SHM / OSAL / HAL 三层固定接口契约 (ipc-shm.h、ipc-os.h、ipc-hw.h; 架构见 章节 3.1) 。Core 层通过同一组 ipcsOs*、ipcsHw* 原型调用 OSAL 与 HAL; FreeRTOS、ThreadX、AUTOSAR OS 三套实现及共用 SWU_IPCS_HAL_MCU 不改变 该边界。因此, 与 章节 4.7 重复的跨单元 UML 序列图不在本节再次展开。

7 LINUX VARIANT DETAIL DESIGN LINUX 详设

7. 1 DEFINITION AND SCOPE 定义与范围

Linux 部署变体通过 ipcs/mpu 实现 Drv_Ipcs_Linux_Adapt_Cmp 与 Drv_Ipcs_Hal_Cmp: UIO、CDEV 为用户侧代理加内核 Backend; 全内核形态将 OSAL 与 HAL 均置于内核模块 (见 章节 3.3) 。通信核心仍使用第 4 章 ipcs/ipcs_cores。

7.2 FILE STRUCTURE 文件结构

7.2.1	7.2.2
7.2.3	7.2.4
7.2.5	7.2.6
7.2.7	7.2.8
7.2.9	7.2.10
7.2.11	7.2.12
7.2.13	7.2.14
7.2.15	7.2.16
7.2.17	7.2.18
7.2.19	7.2.20
7.2.21	7.2.22
7.2.23	7.2.24
7.2.25	7.2.26
7.2.27	7.2.28 File List 文件列表

组件

文件

Drv_Ipcs_Linux_Adapt_Cmp

ipcs/mpu/os_uio/ipc-os.c

Drv_Ipcs_Linux_Adapt_Cmp

ipcs/mpu/os_uio/ipc-os.h
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_cdev/ipc-os.c
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_cdev/ipc-os.h
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_kernel/ipc-os.c
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_kernel/ipc-os.h
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_kernel/ipc-uio.c
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_kernel/ipc-uio.h
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_kernel/ipc-cdev.c
Drv_Ipcs_Linux_Adapt_Cmp
ipcs/mpu/os_kernel/ipc-cdev.h
Drv_Ipcs_Hal_Cmp
ipcs/mpu/hw/c1/ipc-hw.c
Drv_Ipcs_Hal_Cmp
ipcs/mpu/hw/c1/ipc-hw-platform.h
Drv_Ipcs_Hal_Cmp
ipcs/mpu/hw/ipc-hw.h

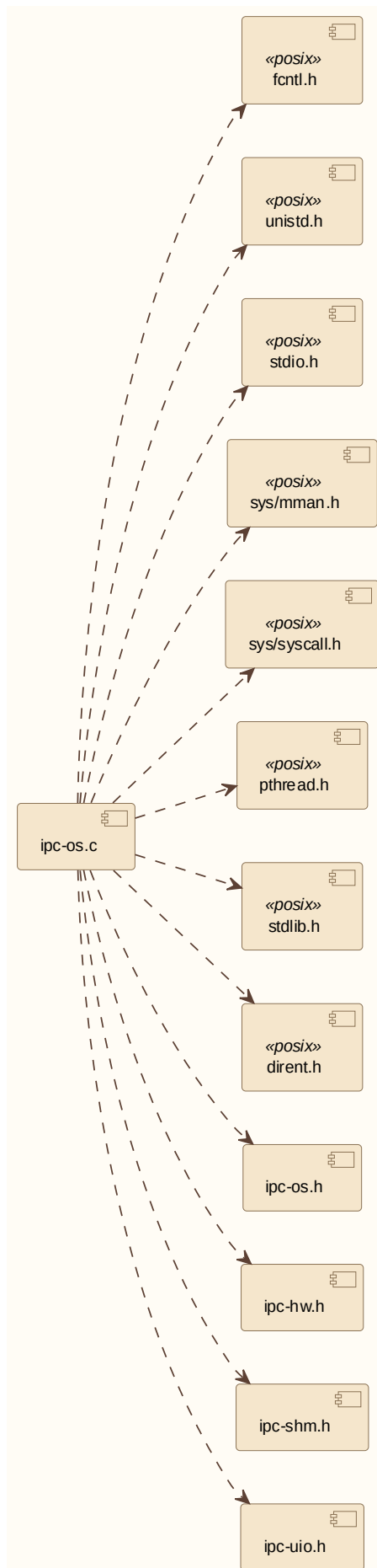
7.2.29 ipc-os.c (UIO User Proxy) UIO 用户侧代理

描述:

ipcs/mpu/os_uio/ipc-os.c 属于 Drv_Ipcs_Linux_Adapt_Cmp / UIO 用户侧代理。

依赖关系:

fcntl.h、unistd.h、stdio.h、sys/mman.h、sys/syscall.h、pthread.h、stdlib.h、dirent.h、ipc-os.h、ipc-hw.h、ipc-shm.h、ipc-uio.h (与 ipcs/mpu/os_uio/ipc-os.c 中 #include 顺序一致)



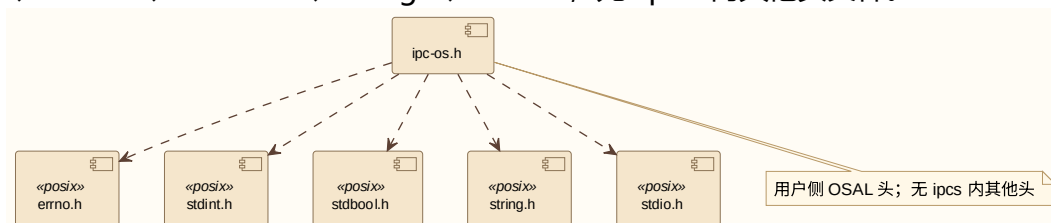
7.2.30 ipc-os.h (UIO User Proxy) UIO 用户侧头文件

描述:

ipcs/mpu/os_uio/ipc-os.h 属于 Drv_Ipcs_Linux_Adapt_Cmp / UIO 用户侧 OSAL 头。

依赖关系:

errno.h、stdint.h、stdbool.h、string.h、stdio.h; 无 ipcs 内其他头文件。



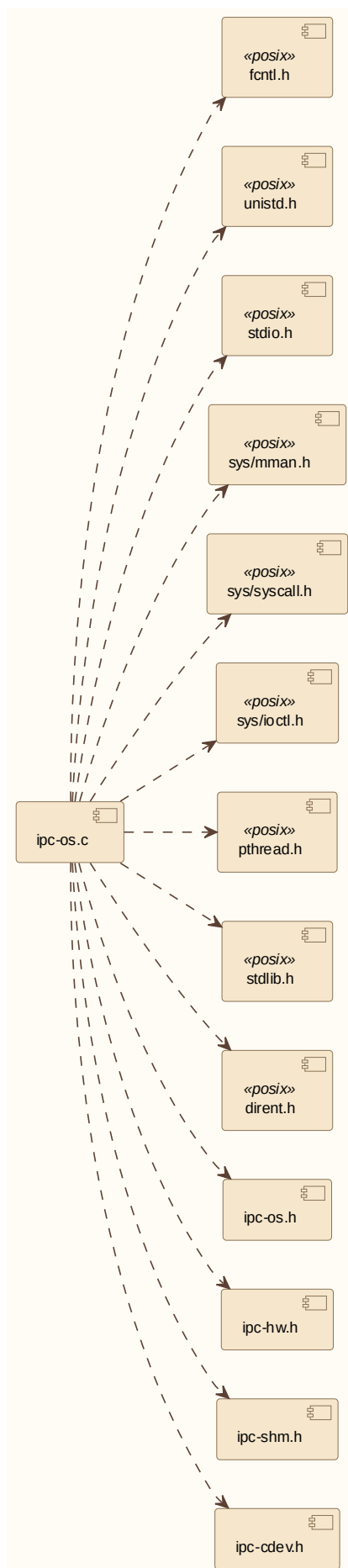
7.2.31 ipc-os.c (CDEV User Proxy) CDEV 用户侧代理

描述:

ipcs/mpu/os_cdev/ipc-os.c 属于 Drv_Ipcs_Linux_Adapt_Cmp / CDEV 用户侧代理。

依赖关系:

fcntl.h、unistd.h、stdio.h、sys/mman.h、sys/syscall.h、sys/ioctl.h、pthread.h、stdlib.h、dirent.h、ipc-os.h、ipc-hw.h、ipc-shm.h、ipc-cdev.h (与 ipcs/mpu/os_cdev/ipc-os.c 一致)



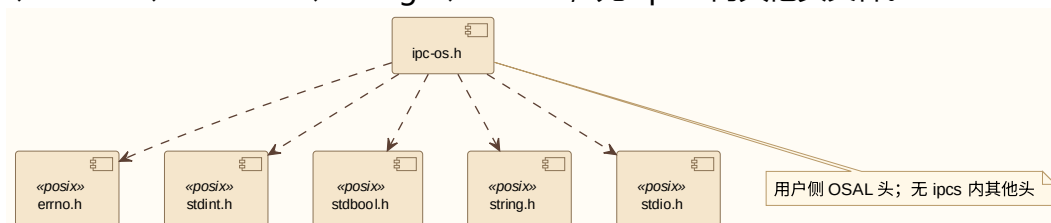
7.2.32 ipc-os.h (CDEV User Proxy) CDEV 用户侧头文件

描述:

ipcs/mpu/os_cdev/ipc-os.h 属于 Drv_Ipcs_Linux_Adapt_Cmp / CDEV 用户侧 OSAL 头。

依赖关系:

errno.h、stdint.h、stdbool.h、string.h、stdio.h; 无 ipcs 内其他头文件。



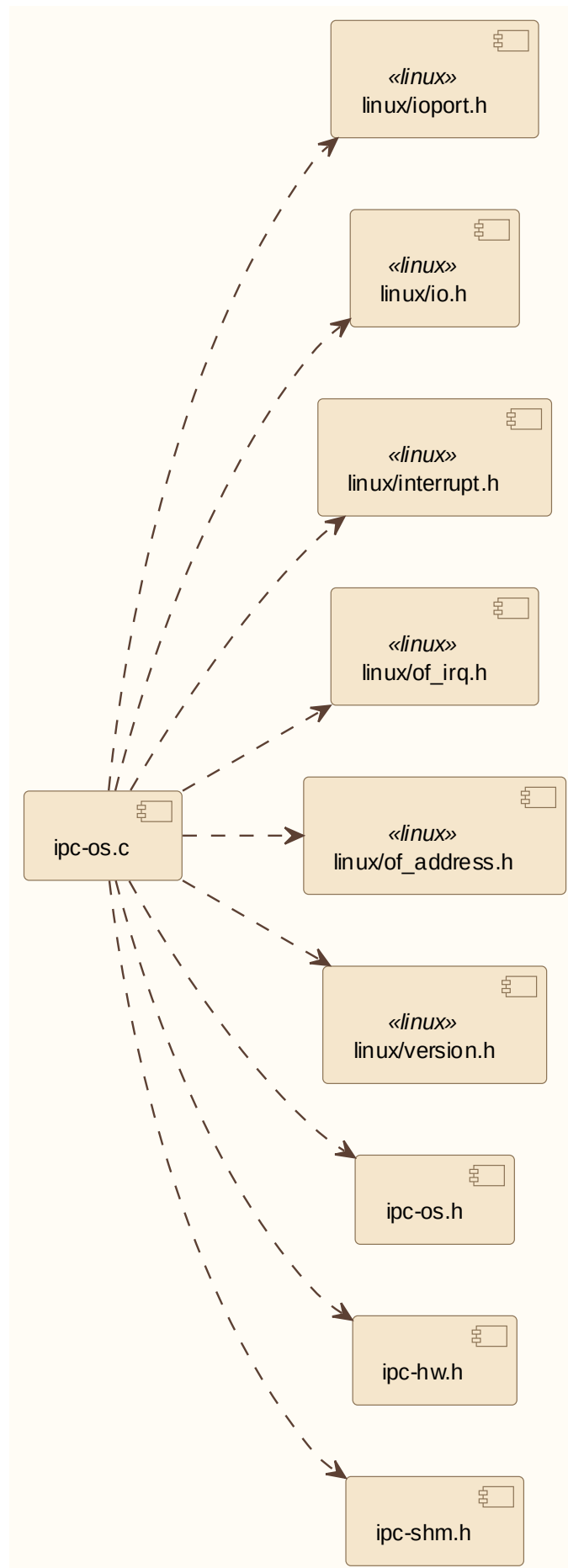
7.2.33 ipc-os.c (In-Kernel OSAL) 全内核 OSAL

描述:

ipcs/mpu/os_kernel/ipc-os.c 属于 Drv_Ipcs_Linux_Adapt_Cmp / 全内核 OSAL 实现。

依赖关系:

linux/ioport.h、linux/io.h、linux/interrupt.h、linux/of_irq.h、linux/of_address.h、linux/version.h、ipc-os.h、ipc-hw.h、ipc-shm.h (与 ipcs/mpu/os_kernel/ipc-os.c 一致)



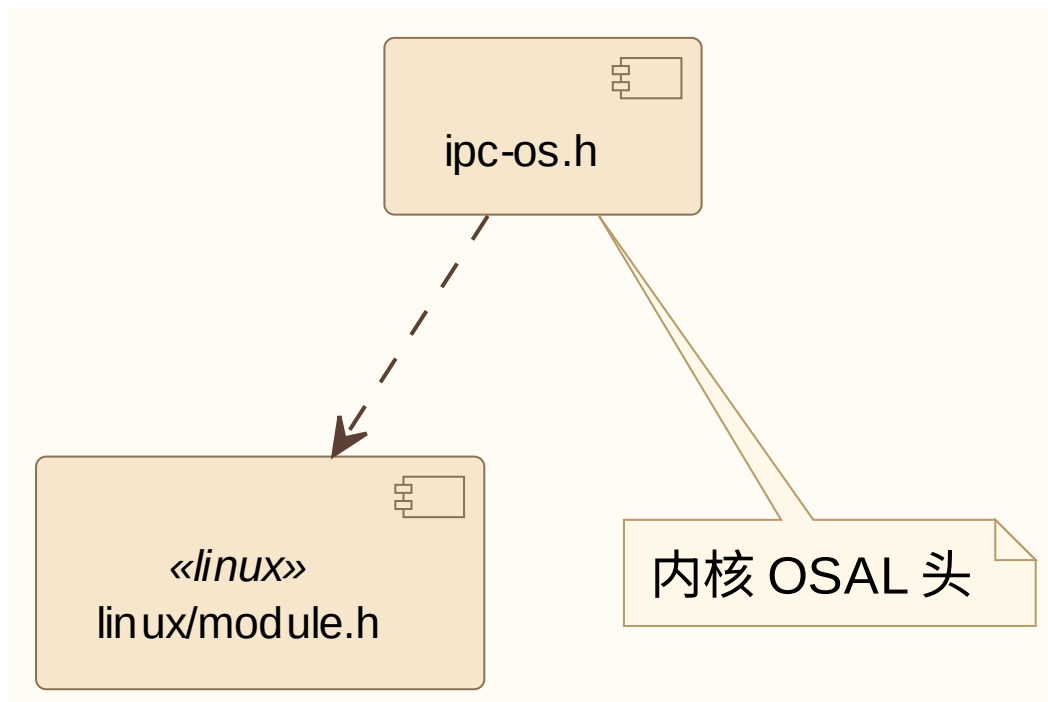
7.2.34 ipc-os.h (In-Kernel OSAL) 全内核 OSAL 头文件

描述:

ipcs/mpu/os_kernel/ipc-os.h 属于 Drv_Ipcs_Linux_Adapt_Cmp / 内核 OSAL 头。

依赖关系:

linux/module.h



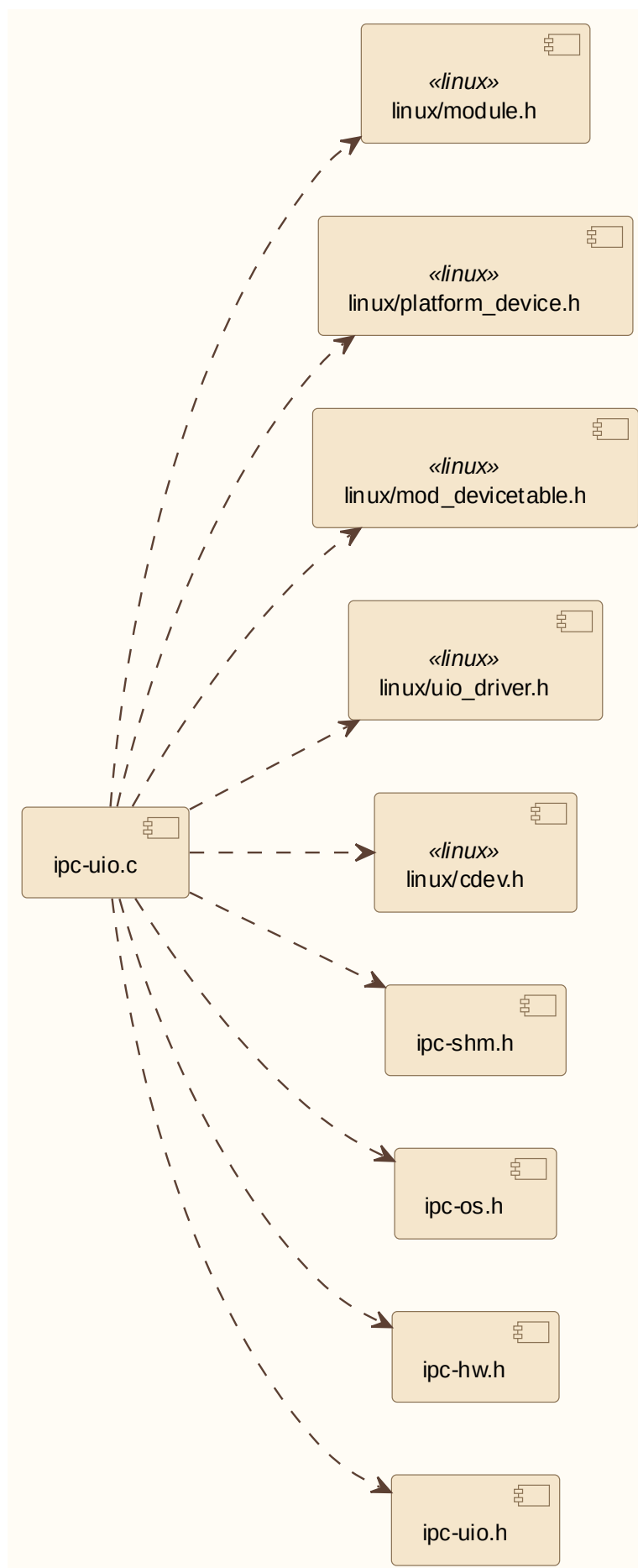
7.2.35 ipc-uio.c

描述:

ipcs/mpu/os_kernel/ipc-uio.c 属于 Drv_Ipcs_Linux_Adapt_Cmp / UIO 内核 Backend。

依赖关系:

linux/module.h、linux/platform_device.h、linux/mod_devicetable.h、linux/uio_driver.h、linux/cdev.h、ipc-shm.h、ipc-os.h、ipc-hw.h、ipc-uio.h (与 ipcs/mpu/os_kernel/ipc-uio.c 一致)



7.2.36 ipc-uio.h

描述:

ipcs/mpu/os_kernel/ipc-uio.h 属于 Drv_Ipcs_Linux_Adapt_Cmp / UIO 命令与宏定义。

依赖关系:

本头文件无 #include (UIO 命令宏)。



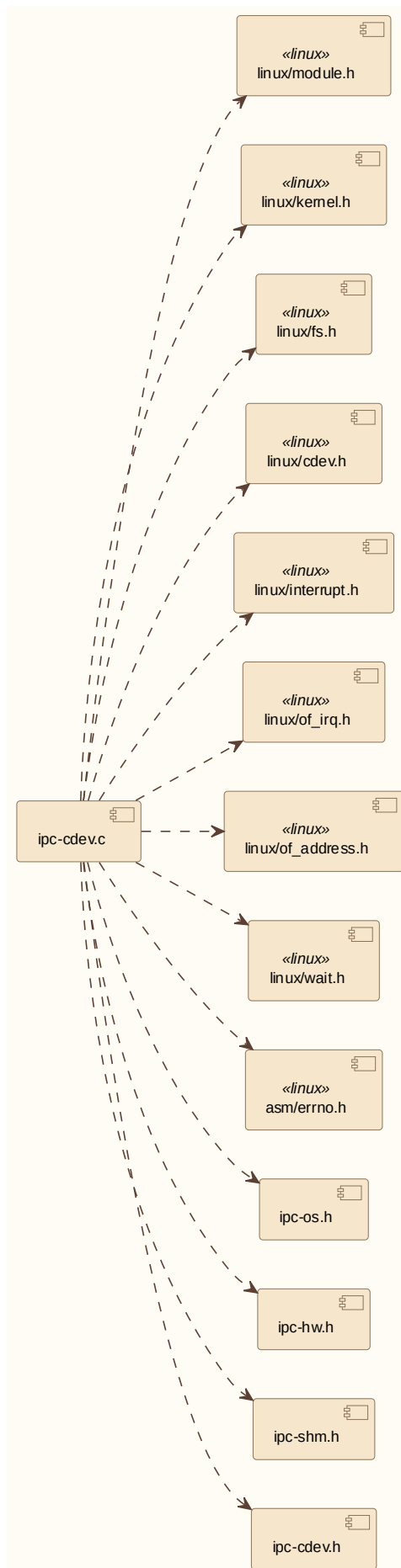
7.2.37 ipc-cdev.c

描述:

ipcs/mpu/os_kernel/ipc-cdev.c 属于 Drv_Ipcs_Linux_Adapt_Cmp / CDEV 内核 Backend。

依赖关系:

linux/module.h、linux/kernel.h、linux/fs.h、linux/cdev.h、linux/interrupt.h、linux/of_irq.h、linux/of_address.h、linux/wait.h、asm/errno.h、ipc-os.h、ipc-hw.h、ipc-shm.h、ipc-cdev.h (与 ipcs/mpu/os_kernel/ipc-cdev.c 一致)



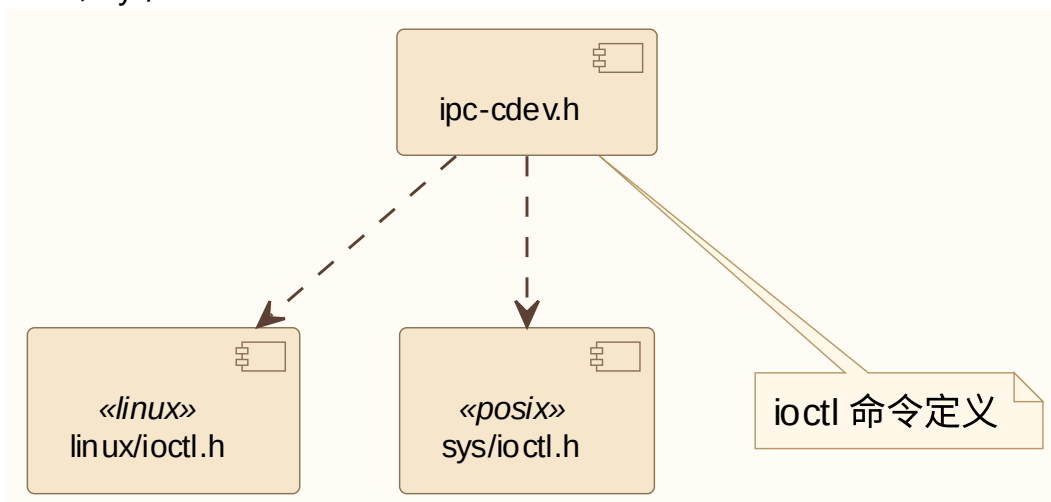
7. 2. 38 ipc-cdev.h

描述:

ipcs/mpu/os_kernel/ipc-cdev.h 属于 Drv_Ipcs_Linux_Adapt_Cmp / CDEV ioctl 定义。

依赖关系:

linux/ioctl.h、 sys/ioctl.h



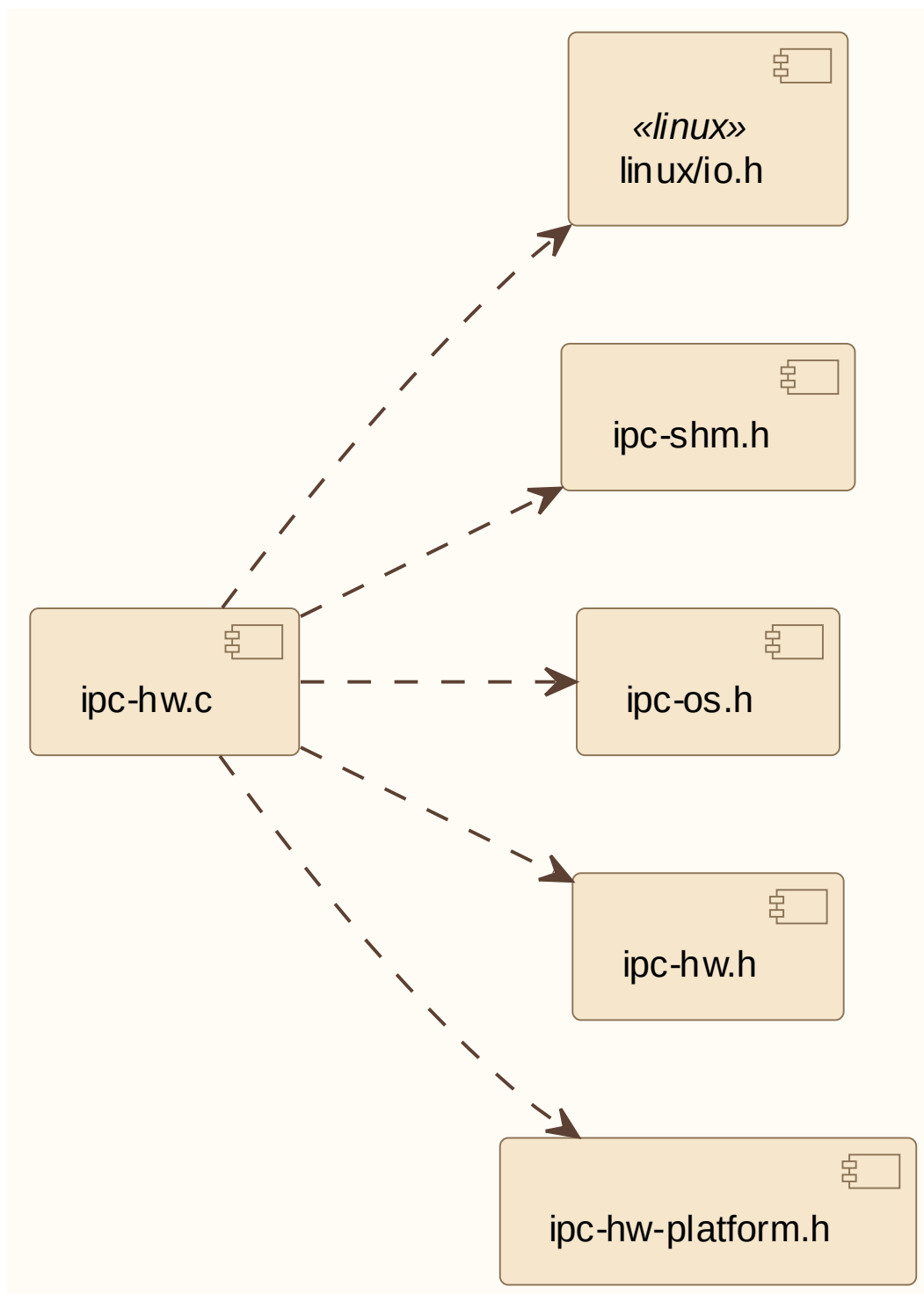
7. 2. 39 ipc-hw.c

描述:

ipcs/mpu/hw/c1/ipc-hw.c 属于 Drv_Ipcs_Hal_Cmp / Linux HAL 实现。

依赖关系:

linux/io.h、 ipc-shm.h、 ipc-os.h、 ipc-hw.h、 ipc-hw-platform.h (与 ipcs/mpu/hw/c1/ipc-hw.c 一致)



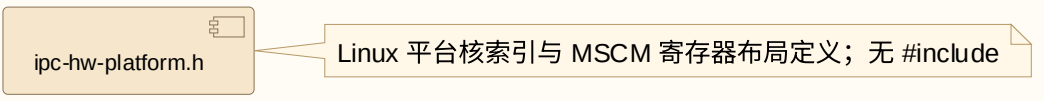
7.2.40 ipc-hw-platform.h

描述:

ipcs/mpu/hw/c1/ipc-hw-platform.h 属于 Drv_Ipcs_Hal_Cmp / Linux 平台定义。

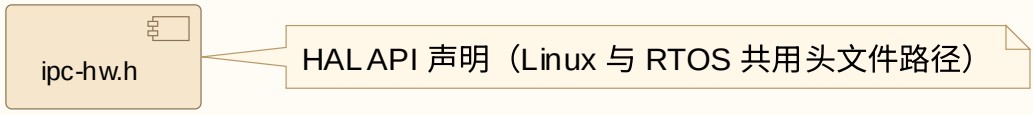
依赖关系:

本头文件无 #include (核索引与 核间中断控制器 寄存器布局宏)。



7. 2. 41 ipc-hw.h

描述：
ipcs/mpu/hw/ipc-hw.h 属于 Drv_Ipcs_Hal_Cmp / HAL API 声明（Linux 构建包含路径）。
依赖关系：
本头文件无 #include（HAL API 声明）。

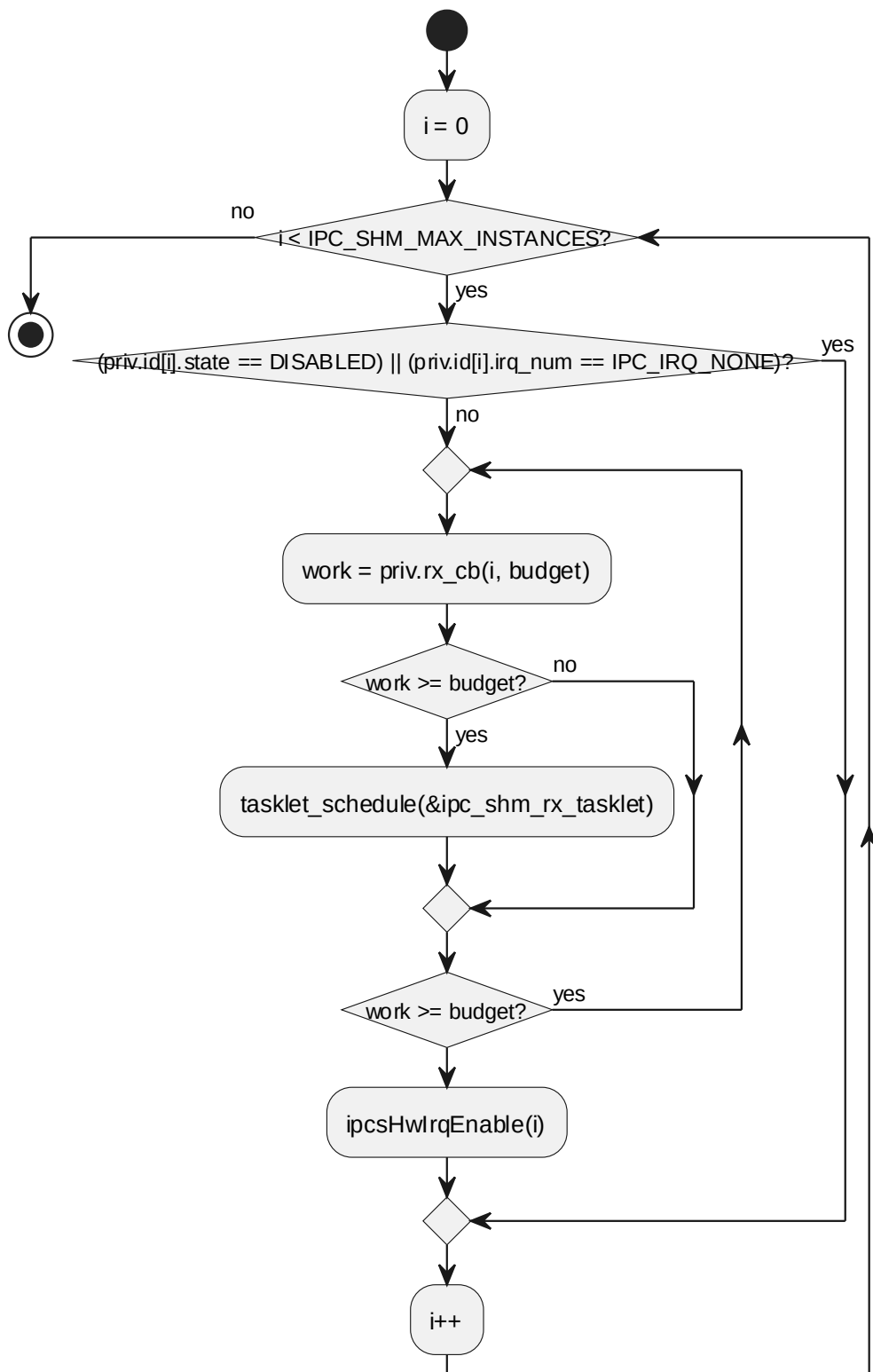


7. 3 SWU_IPCS_LINUX_OS_KERN DESIGN 单元设计

7.3.1 ipcsShmSoftirq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_KERN			
函数说明	延迟收包处理，遍历实例并调用上层 rx_cb，完成后重新使能 IRQ。			
函数原型	static void ipcsShmSoftirq(unsigned long arg)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	l	arg	unsigned long arg	[IN] arg
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

processing flow

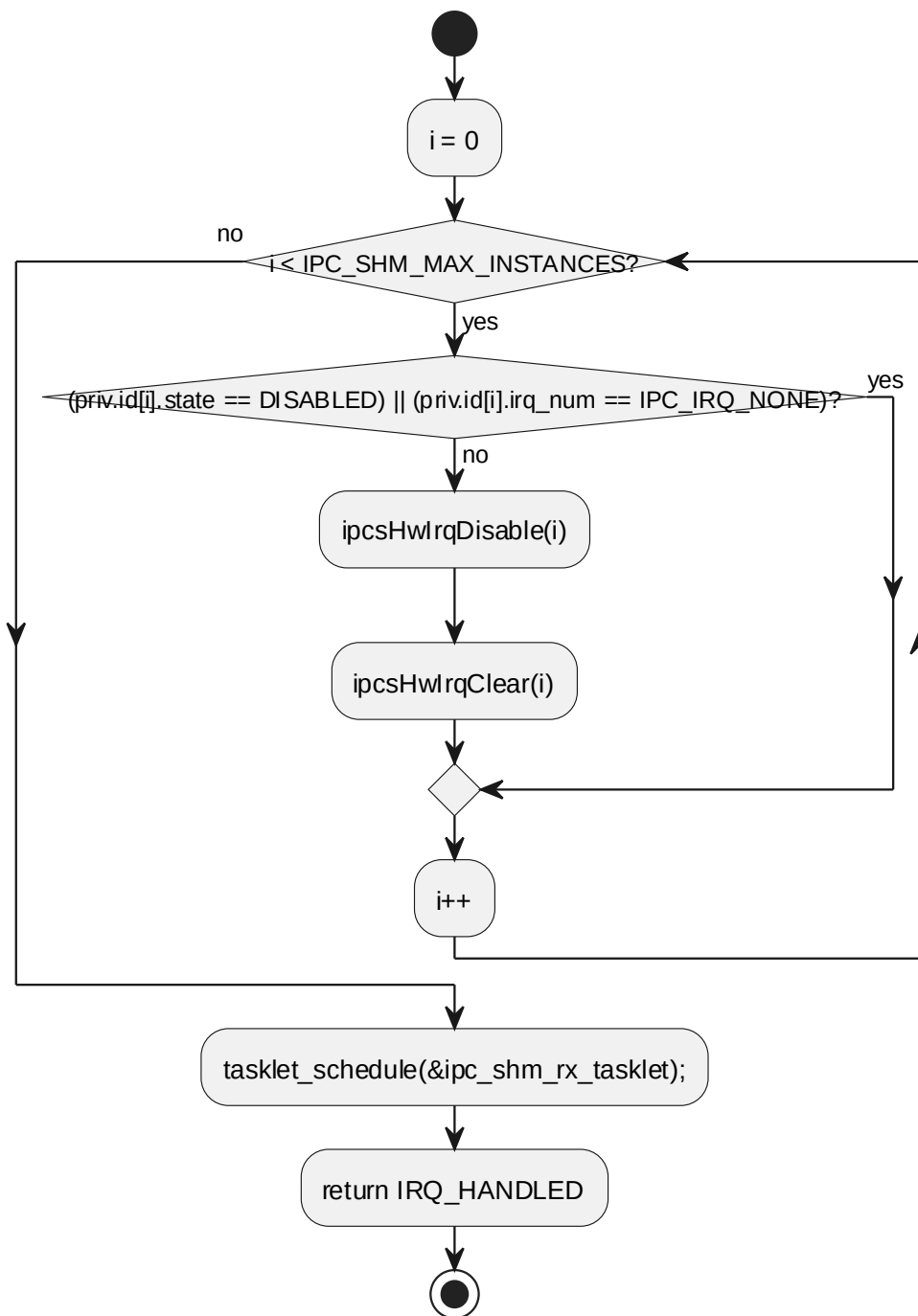


7.3.2 ipcsShmHardirq

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp
软件单元 ID	SWU_IPCS_LINUX_OS_KERN
函数说明	硬中断处理，禁止并清除远端通知，中断后续处理交给 tasklet 或等待队列。

函数原型	static irqreturn_t ipcsShmHardirq(int irq, void *dev)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	irq	int irq	[IN] irq
	I	dev	void *dev	[IN] dev
返回值	数据类型		说明	
	irqreturn_t		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

processing flow

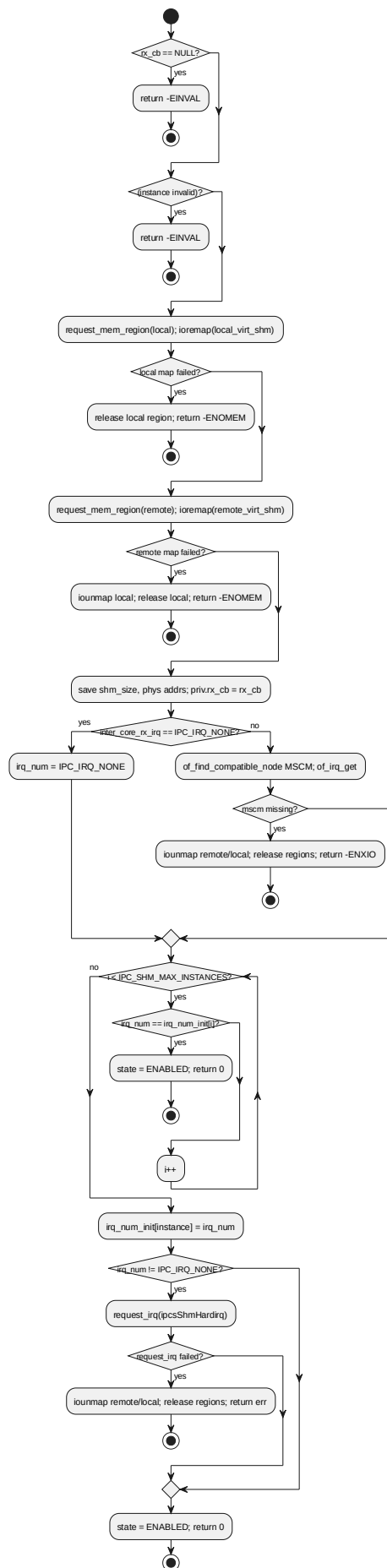


7.3.3 ipcsOsInit

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp
软件单元 ID	SWU_IPCS_LINUX_OS_KERN
函数说明	初始化指定实例的 Linux OSAL 资源，建立共享内存映射、记录回调并配置接收中断。
函数原型	int ipcsOsInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg, int (*rx_cb)(const uint8_t, int))
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行

输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_ TYPE *cfg	[IN] cfg
	I	rx_cb	int (*rx_cb)(const uint8_t, int)	[IN] rx_cb
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

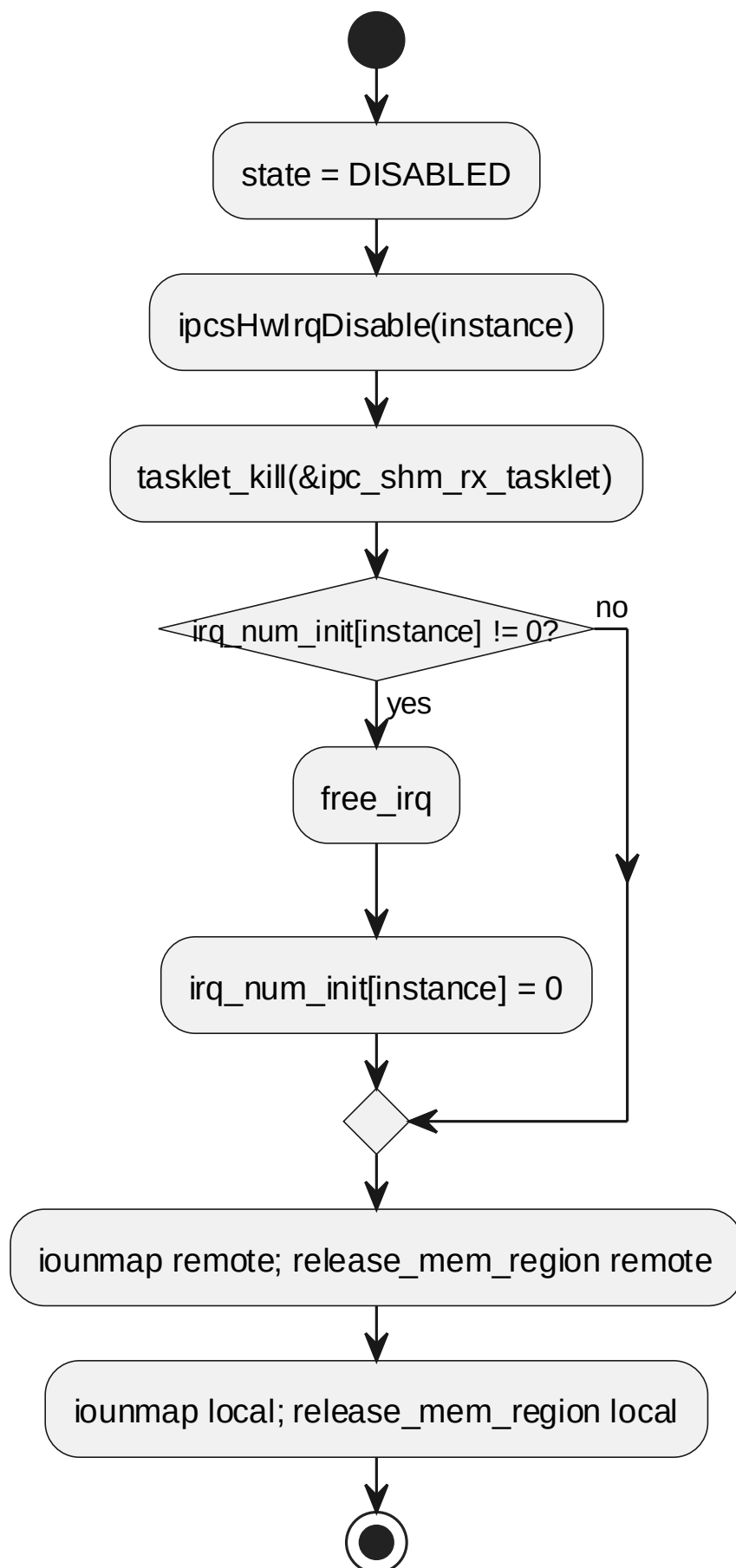
processing flow



7.3.4 ipcsOsFree

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_KERN			
函数说明	释放指定实例 OSAL 资源，关闭线程/设备、解除映射并清理状态。			
函数原型	void ipcsOsFree(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

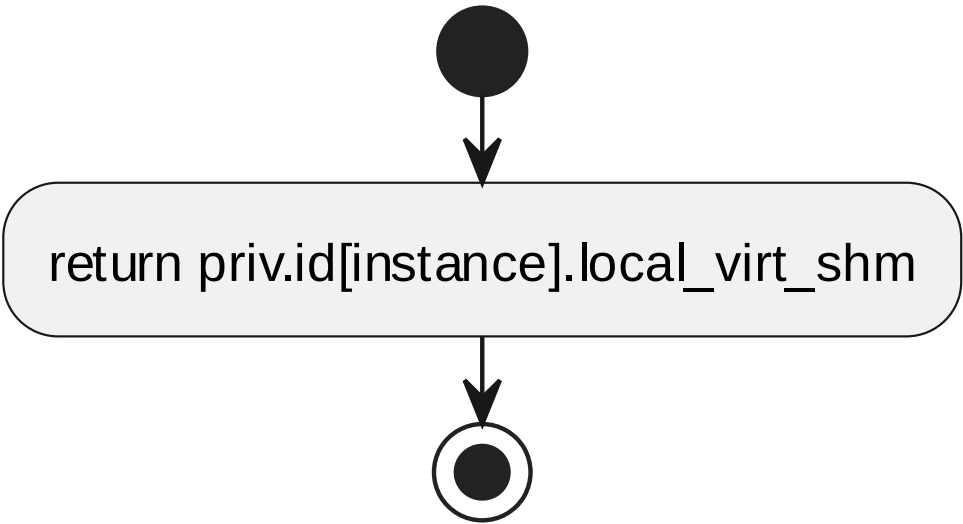
processing flow



7.3.5 ipcsOsGetLocalShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_KERN			
函数说明	返回本地共享内存虚拟地址。			
函数原型	uintptr_t ipcsOsGetLocalShm(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	uintptr_t		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

processing flow

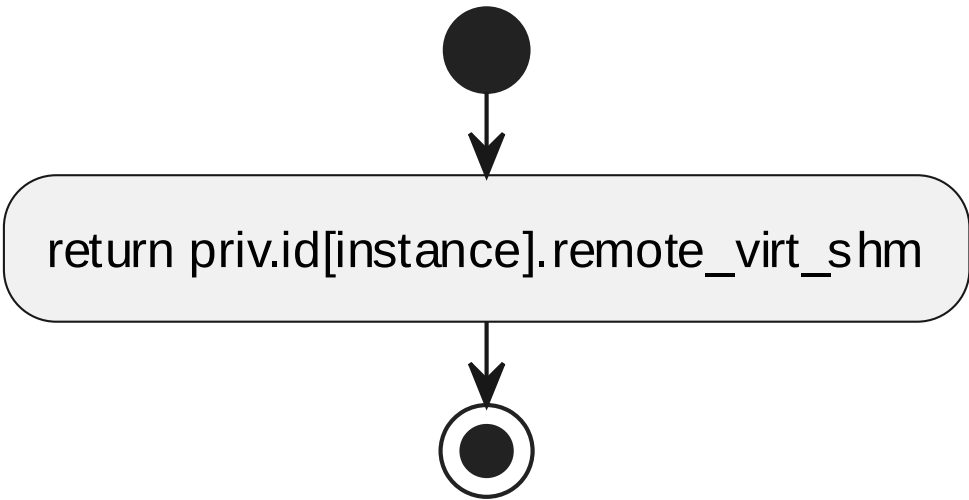


7.3.6 ipcsOsGetRemoteShm

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_KERN			
函数说明	返回远端共享内存虚拟地址。			
函数原型	uintptr_t ipcsOsGetRemoteShm(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	uintptr_t		-	

函数定义文件	ipcs/mpu/os_kernel/ipc-os.c
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h

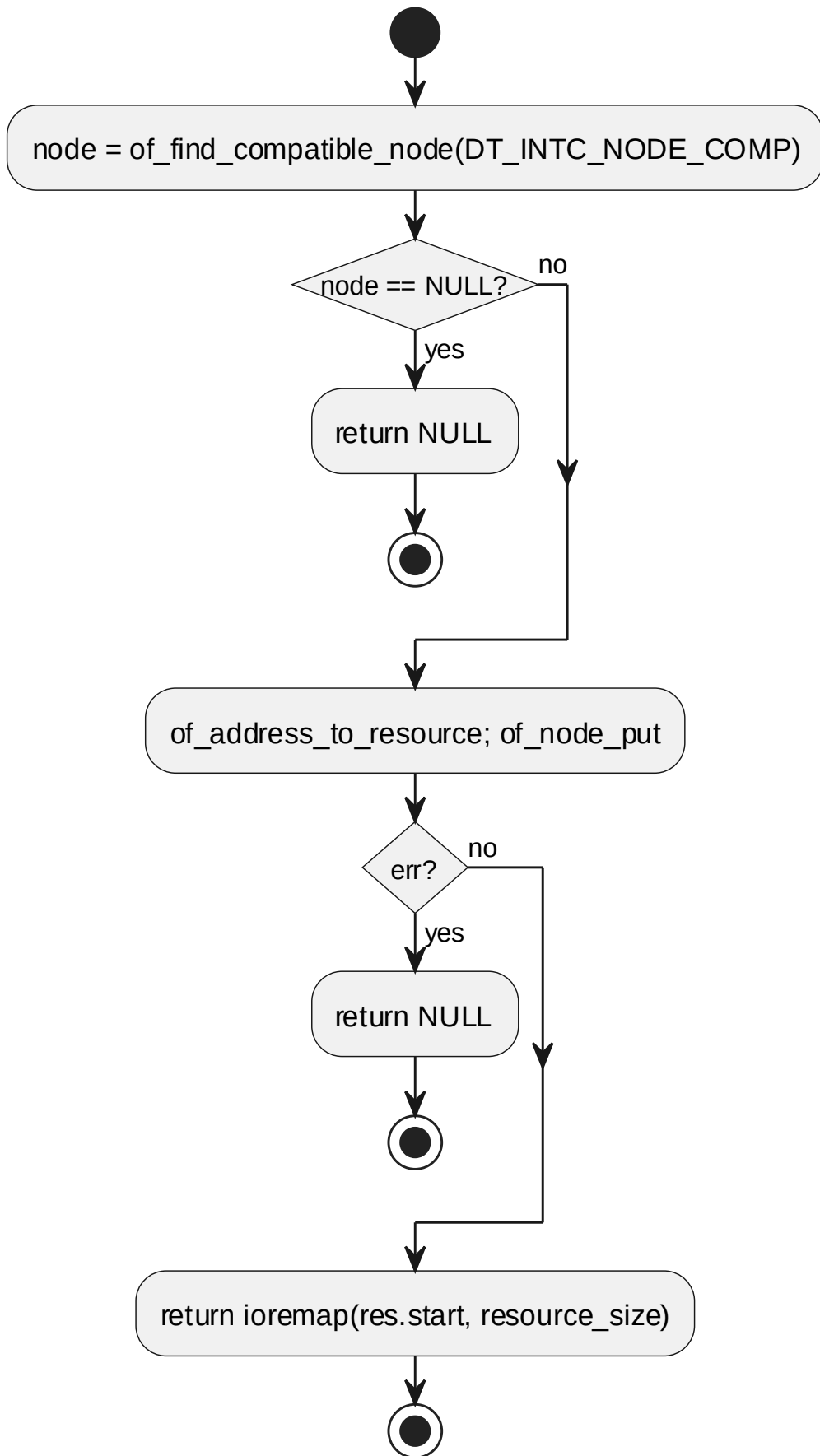
processing flow



7.3.7 ipcOsMapIntc

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_OS_KERN	
函数说明	映射或返回中断控制器寄存器空间。	
函数原型	void *ipcOsMapIntc(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	void *	-
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h	

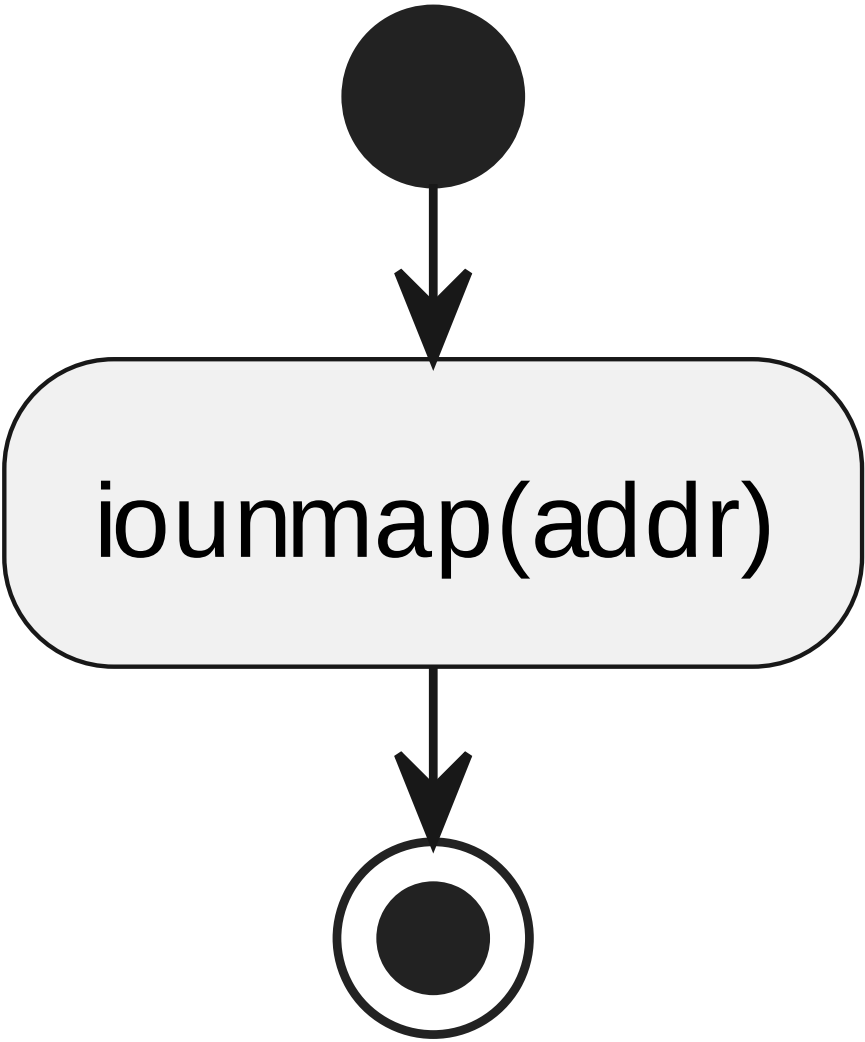
processing flow



7.3.8 ipcsOsUnmapIntc

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_KERN			
函数说明	释放中断控制器寄存器映射或提供对应空实现。			
函数原型	void ipcsOsUnmapIntc(void *addr)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	addr	void *addr	[IN] addr
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

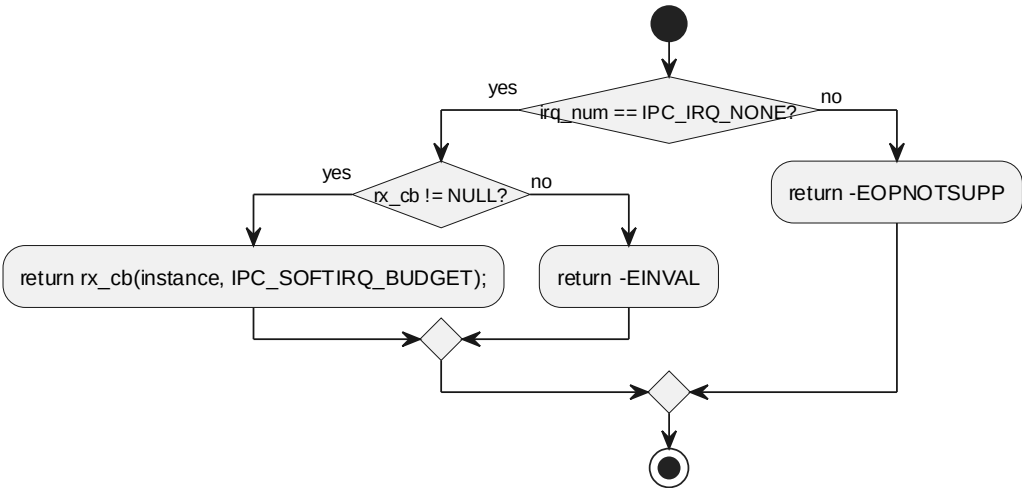
processing flow



7.3.9 ipcsOsPollChannels

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_KERN			
函数说明	在轮询模式下触发 rx_cb 处理接收通道。			
函数原型	int ipcsOsPollChannels(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h			

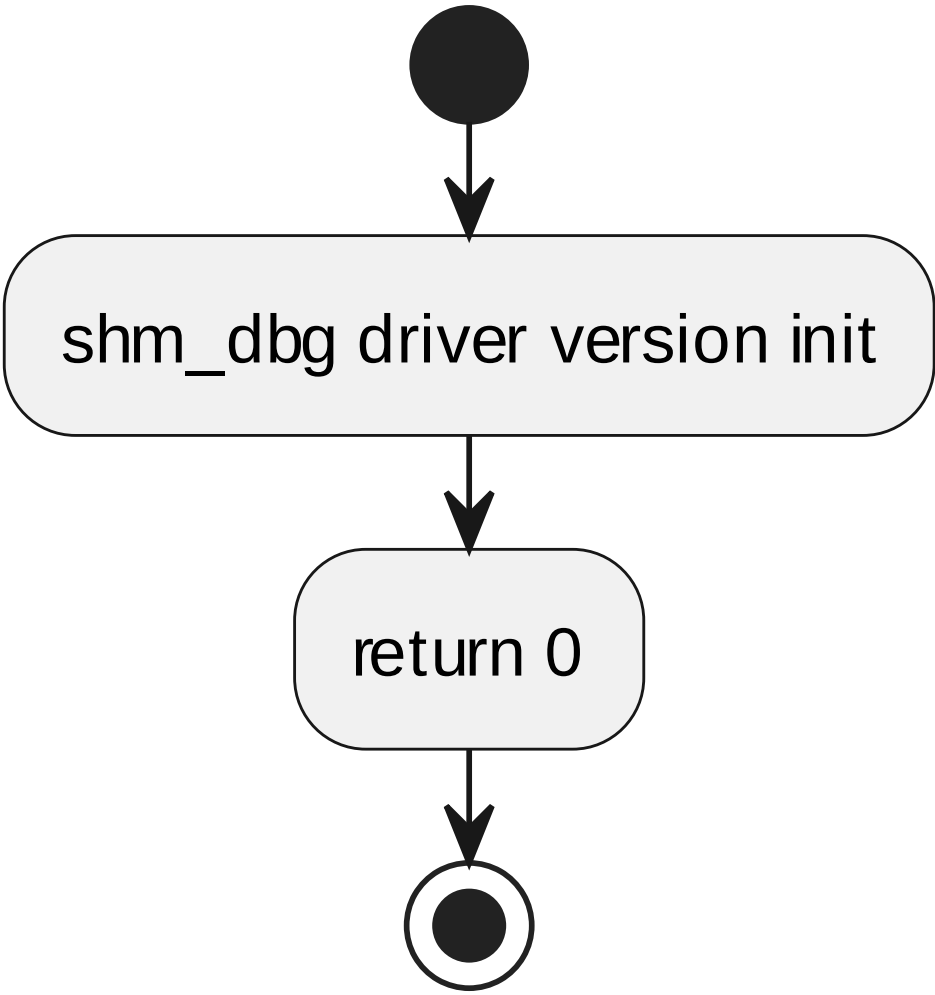
processing flow



7.3.10 shm_mod_init

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_OS_KERN	
函数说明	Linux 全内核模块初始化入口。	
函数原型	static int __init shm_mod_init(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	int __init	-
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h	

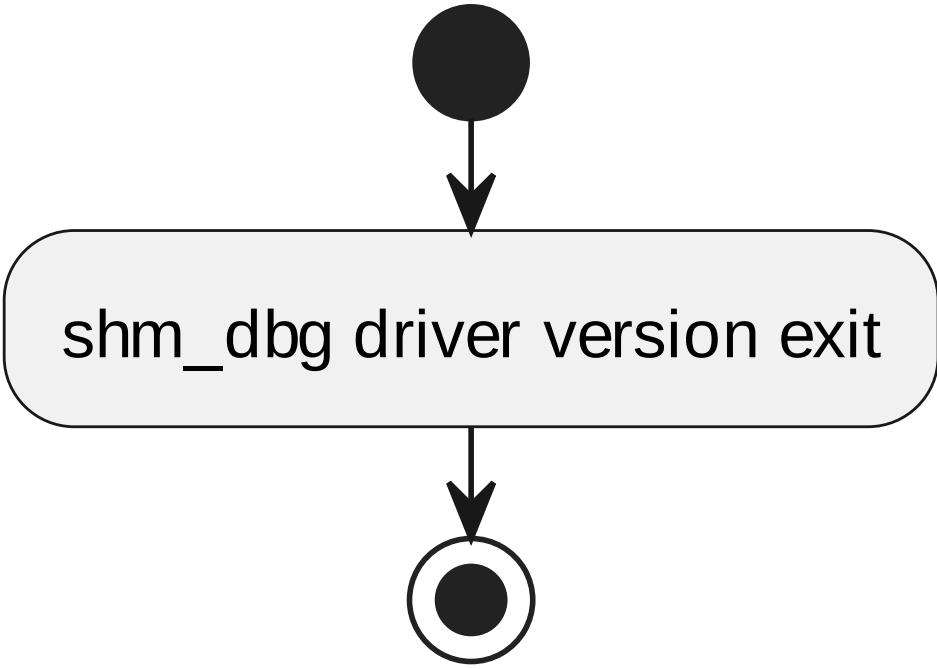
processing flow



7.3.11 shm_mod_exit

对应软件架构 ID	Drv_Ipcs_Osal_Cmp / Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_OS_KERN	
函数说明	Linux 全内核模块退出口。	
函数原型	static void __exit shm_mod_exit(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	void __exit	-
函数定义文件	ipcs/mpu/os_kernel/ipc-os.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-os.h	

processing flow

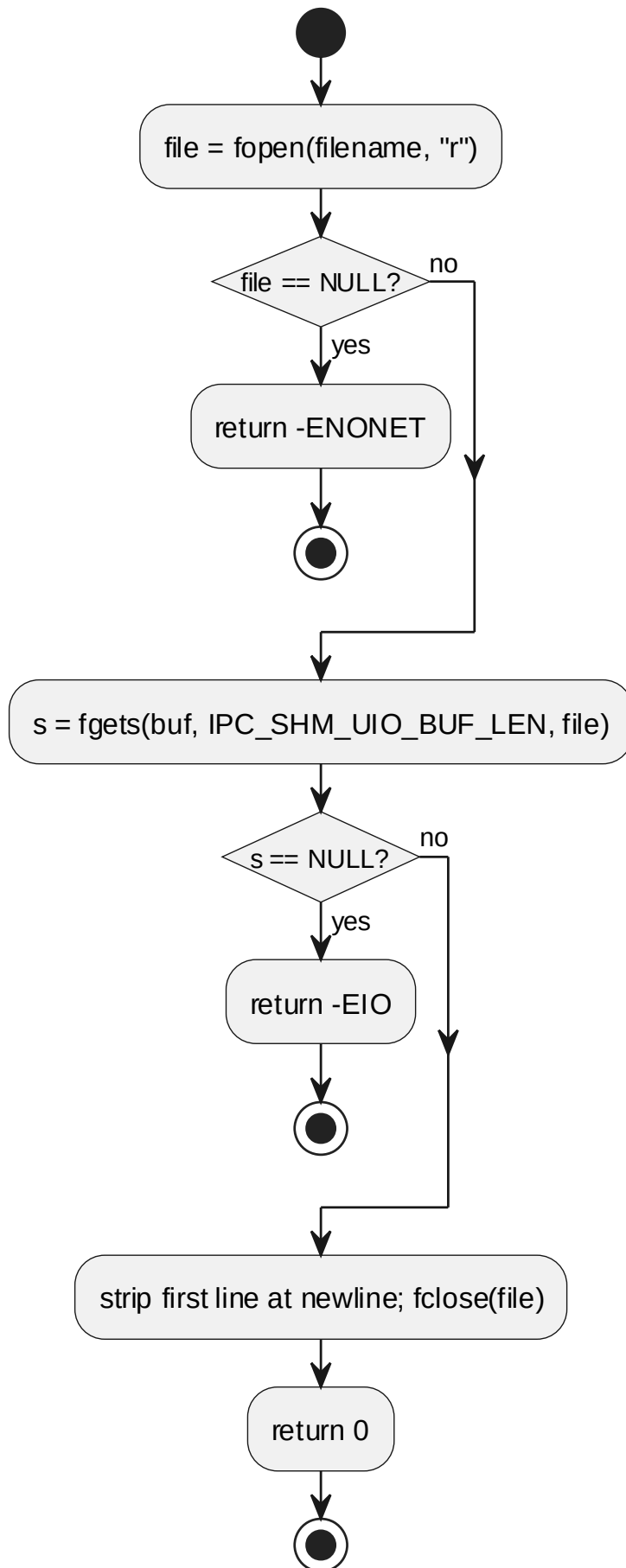


7.4 SWU_IPCS_LINUX_OS_UIO DESIGN 单元设计

7.4.1 line_from_file

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	读取 sysfs 文件中的一行内容。			
函数原型	static int line_from_file(char *filename, char *buf)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	filename	char *filename	[IN] filename
	I	buf	char *buf	[IN] buf
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

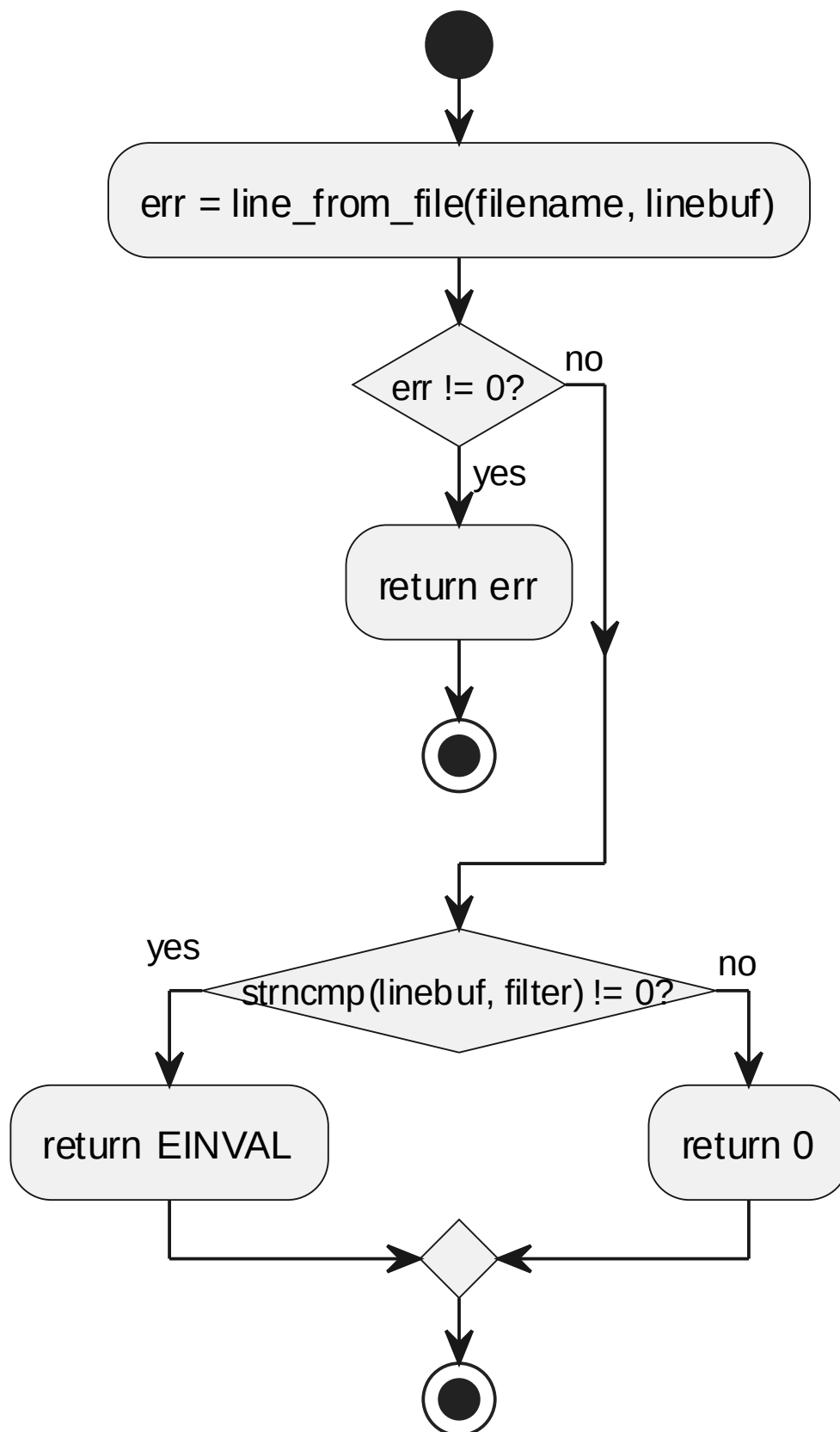
processing flow



7.4.2 line_match

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	比较 sysfs 文件内容与目标过滤字符串。			
函数原型	static int line_match(char *filename, char *filter)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	filename	char *filename	[IN] filename
	I	filter	char *filter	[IN] filter
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

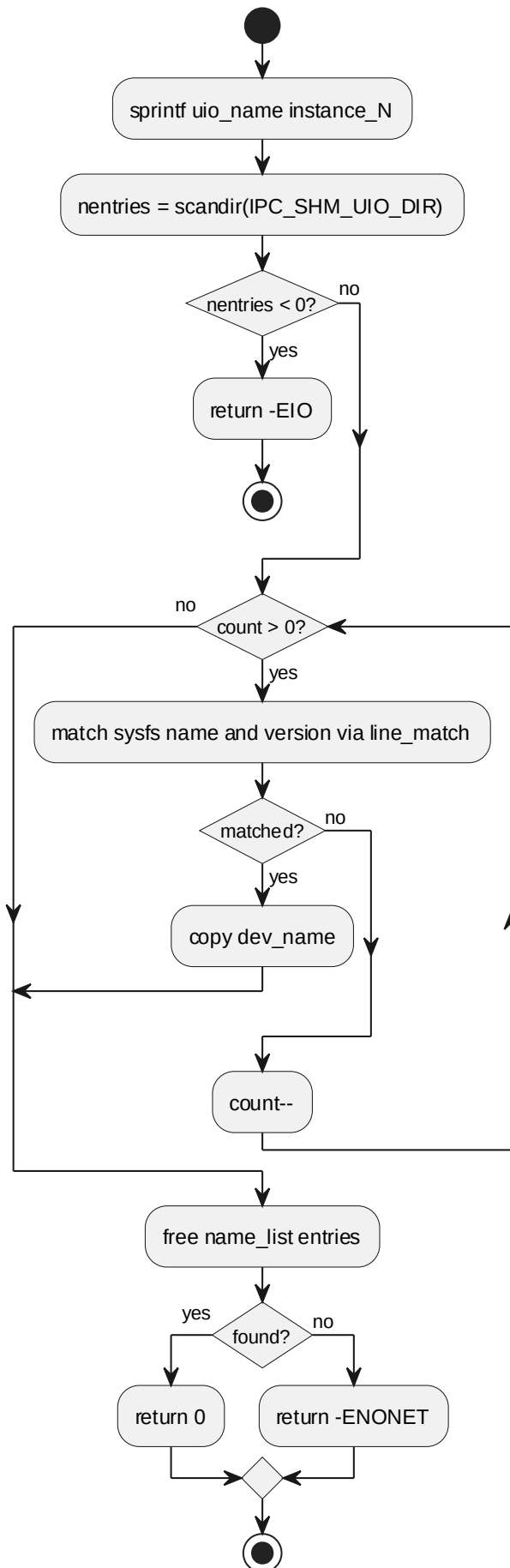
processing flow



7.4.3 get_uio_dev_name

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	在 sysfs 中查找匹配实例的 UIO 设备名。			
函数原型	static int get_uio_dev_name(char *dev_name, const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	dev_name	char *dev_name	[IN] dev_name
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

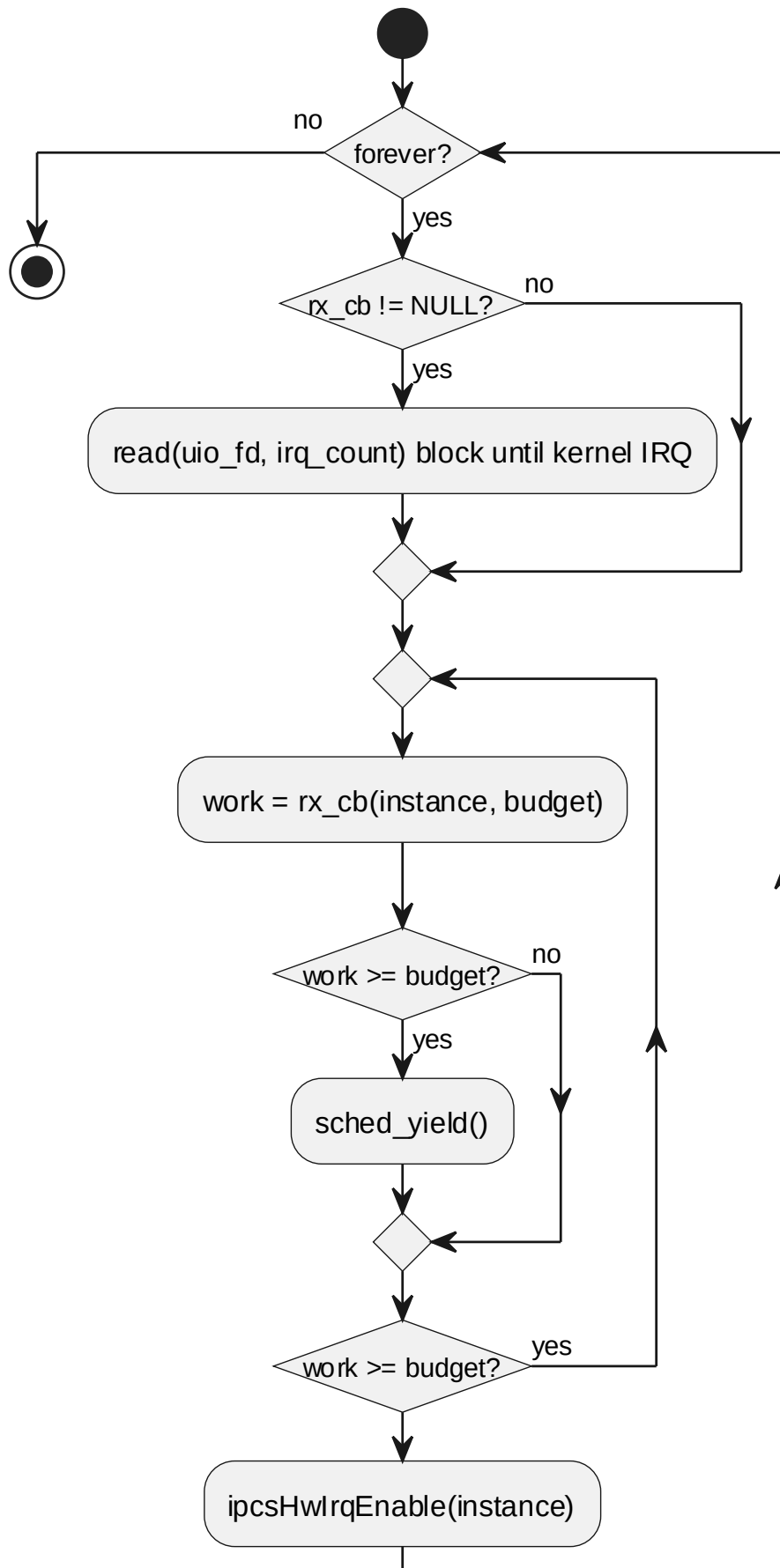
processing flow



7.4.4 ipcsShmSoftirq

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	延迟收包处理，遍历实例并调用上层 rx_cb，完成后重新使能 IRQ。			
函数原型	static void *ipcsShmSoftirq(void *arg)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	l	arg	void *arg	[IN] arg
返回值	数据类型		说明	
	void *		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

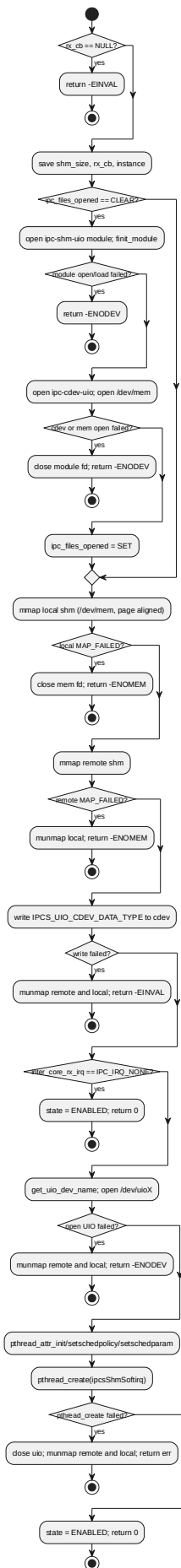
processing flow



7.4.5 ipcsOsInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	初始化指定实例的 Linux OSAL 资源，建立共享内存映射、记录回调并配置接收中断。			
函数原型	int ipcsOsInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg, int (*rx_cb)(const uint8_t, int))			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_ TYPE *cfg	[IN] cfg
	I	rx_cb	int (*rx_cb)(const uint8_t, int)	[IN] rx_cb
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

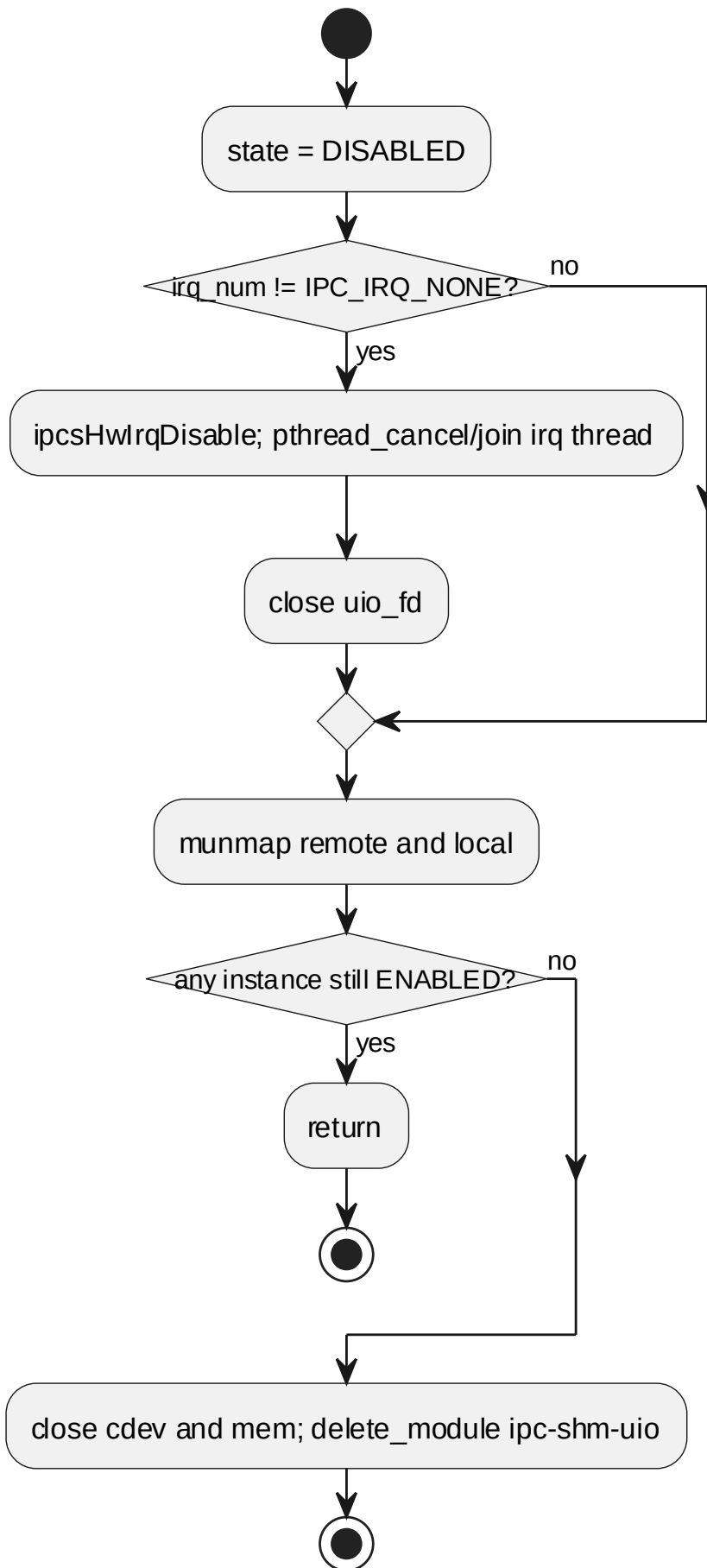
processing flow



7.4.6 ipcsOsFree

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	释放指定实例 OSAL 资源，关闭线程/设备、解除映射并清理状态。			
函数原型	void ipcsOsFree(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

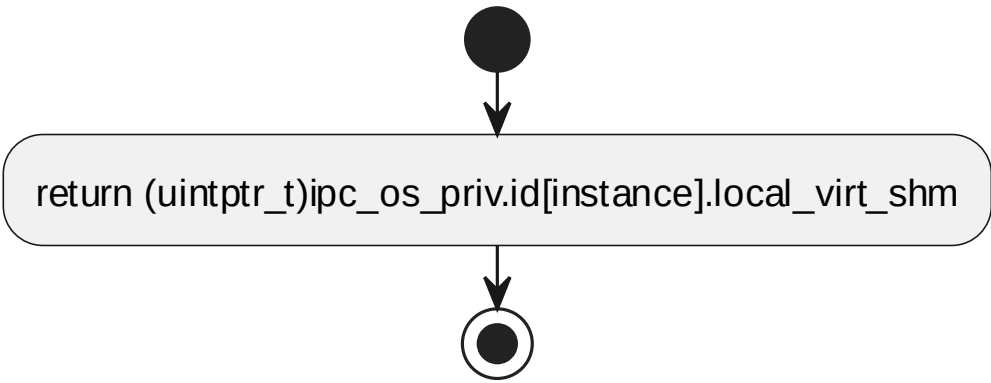
processing flow



7.4.7 ipcsOsGetLocalShm

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	返回本地共享内存虚拟地址。			
函数原型	uintptr_t ipcsOsGetLocalShm(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	uintptr_t		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

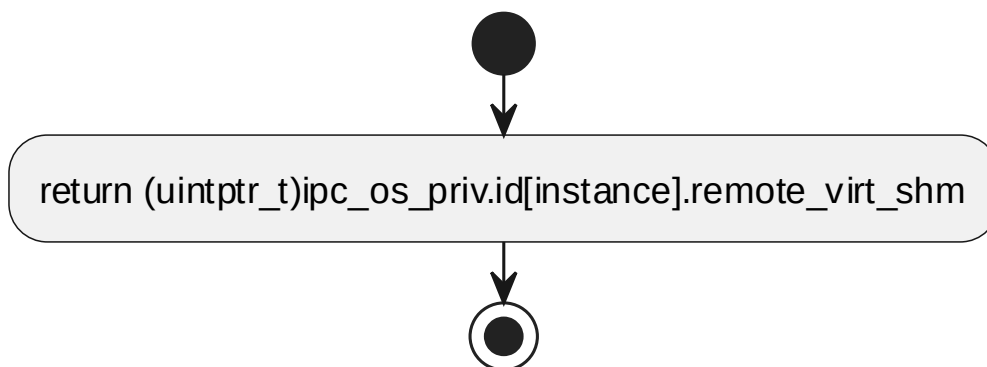
processing flow



7.4.8 ipcsOsGetRemoteShm

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	返回远端共享内存虚拟地址。			
函数原型	uintptr_t ipcsOsGetRemoteShm(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	uintptr_t		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

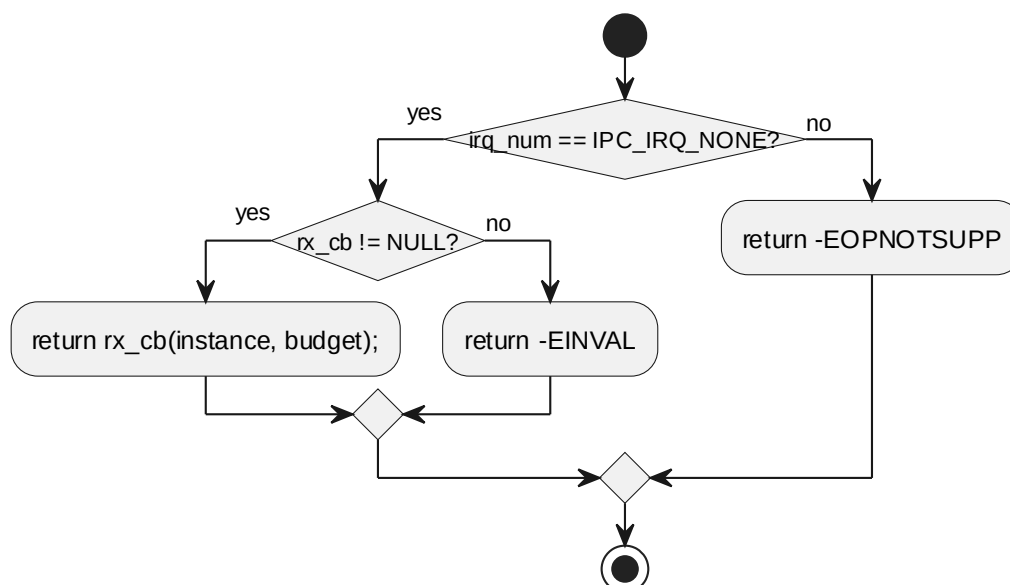
processing flow



7.4.9 ipcsOsPollChannels

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	在轮询模式下触发 rx_cb 处理接收通道。			
函数原型	int ipcsOsPollChannels(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

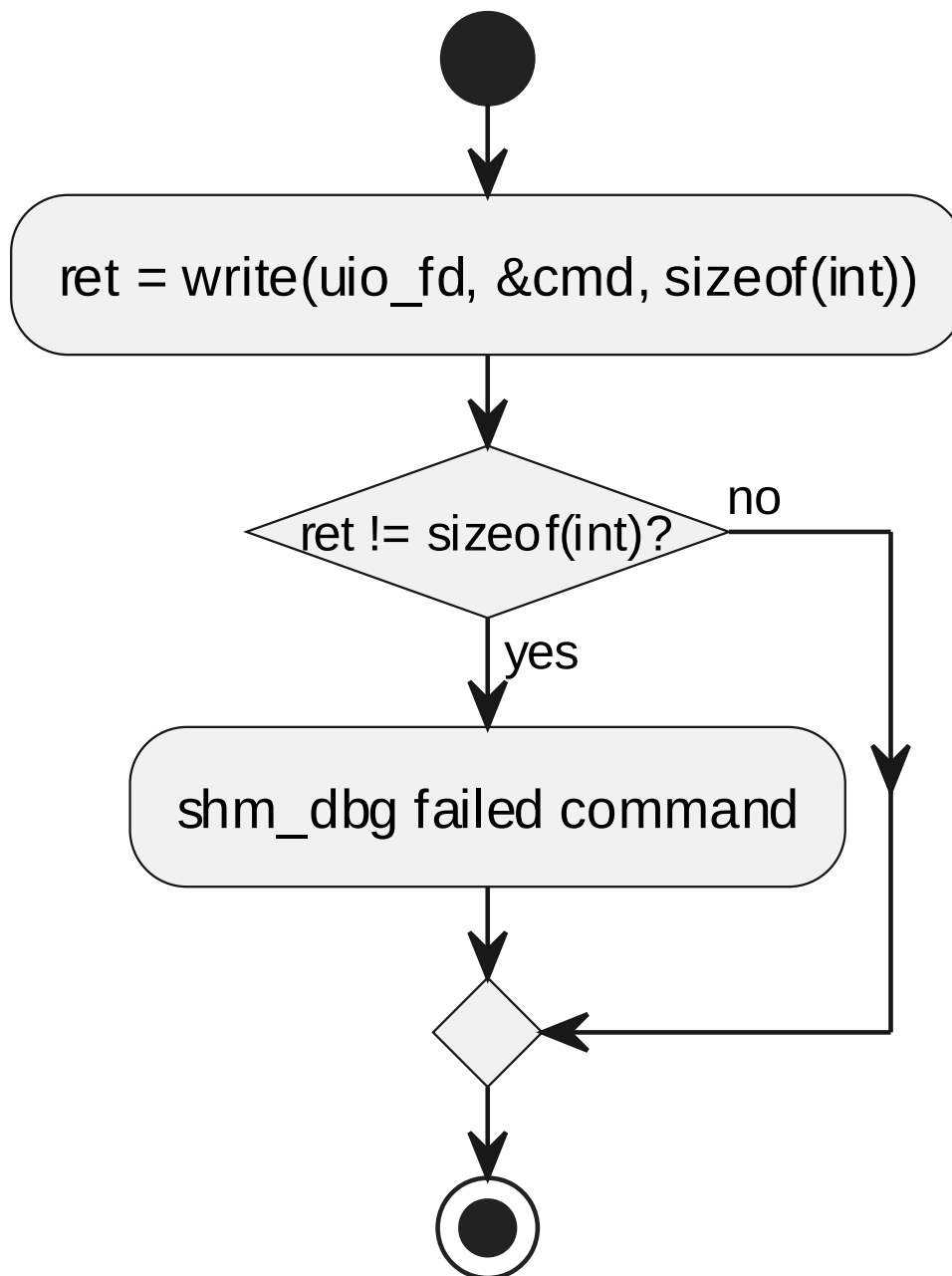
processing flow



7. 4. 10 ipcsSendUioCmd

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	向 UIO fd 写入命令，代理 IRQ 使能、禁止或通知。			
函数原型	static void ipcsSendUioCmd(uint32_t uio_fd, int32_t cmd)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	uio_fd	uint32_t uio_fd	[IN] uio_fd
	I	cmd	int32_t cmd	[IN] cmd
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

processing flow

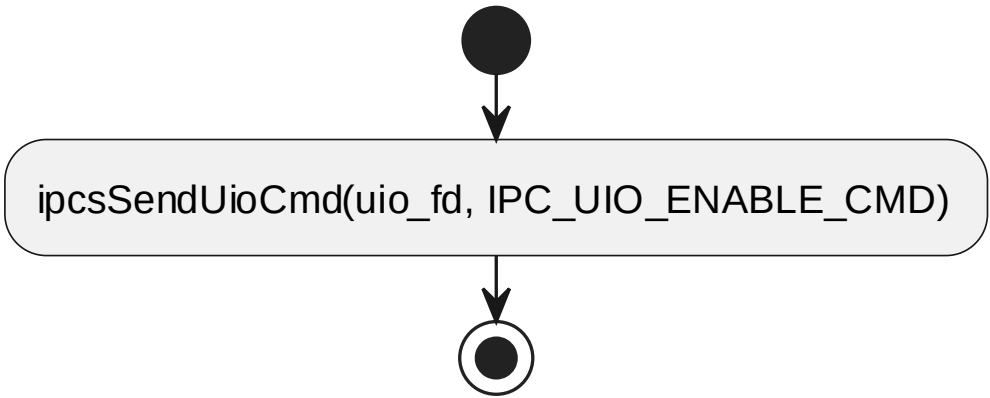


7.4.11 ipcsHwIrqEnable

对应软件架构ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	使能指定实例接收中断；用户侧为转发代理，内核侧访问硬件。			
函数原型	void ipcsHwIrqEnable(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance

返回值	数据类型	说明
	-	
函数定义文件	ipcs/mpu/os_uio/ipc-os.c	
函数声明文件	ipcs/mpu/os_uio/ipc-os.h	

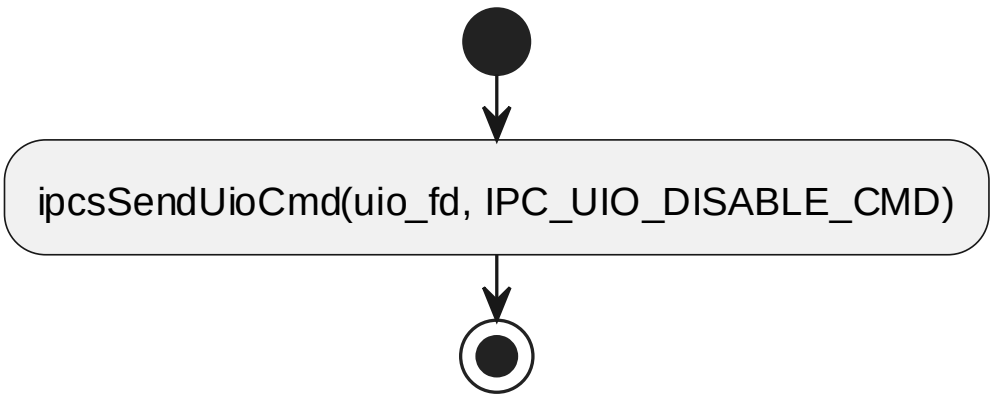
processing flow



7. 4. 12 ipcsHwIrqDisable

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	禁止指定实例接收中断；用户侧为转发代理，内核侧访问硬件。			
函数原型	void ipcsHwIrqDisable(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

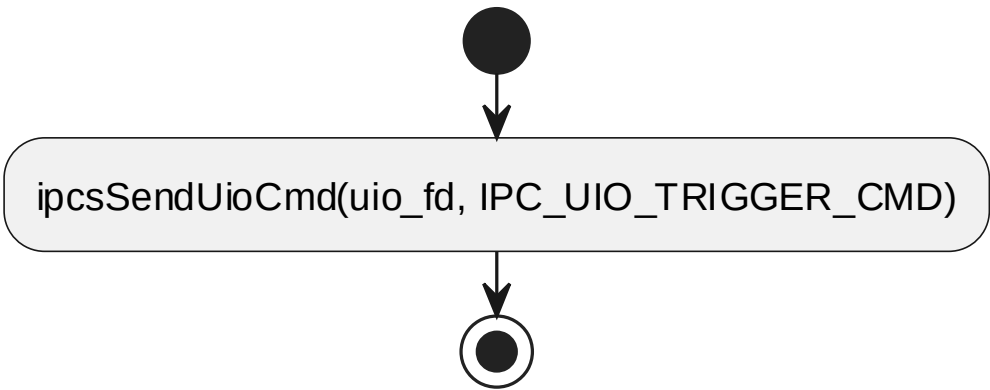
processing flow



7. 4. 13 ipcsHwIrqNotify

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	通知远端有数据可用；用户侧为转发代理，内核侧触发硬件中断。			
函数原型	void ipcsHwlrqNotify(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

processing flow

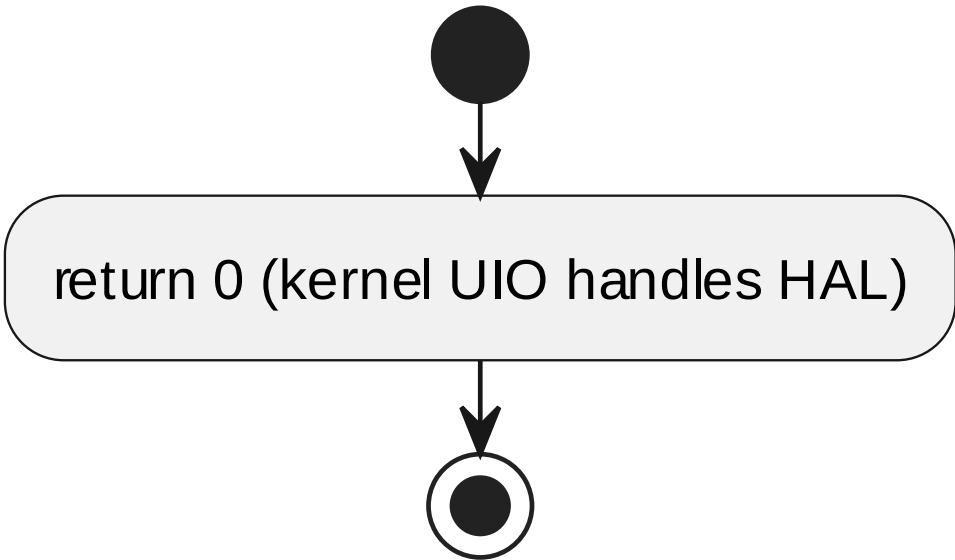


7. 4. 14 ipcsHwInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	初始化 HAL 资源；用户侧为空实现，内核侧映射并配置 核间中断控制器/IRQ。			
函数原型	int ipcsHwInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_ TYPE *cfg	[IN] cfg
返回值	数据类型		说明	

	int	-
函数定义文件	ipcs/mpu/os_uio/ipc-os.c	
函数声明文件	ipcs/mpu/os_uio/ipc-os.h	

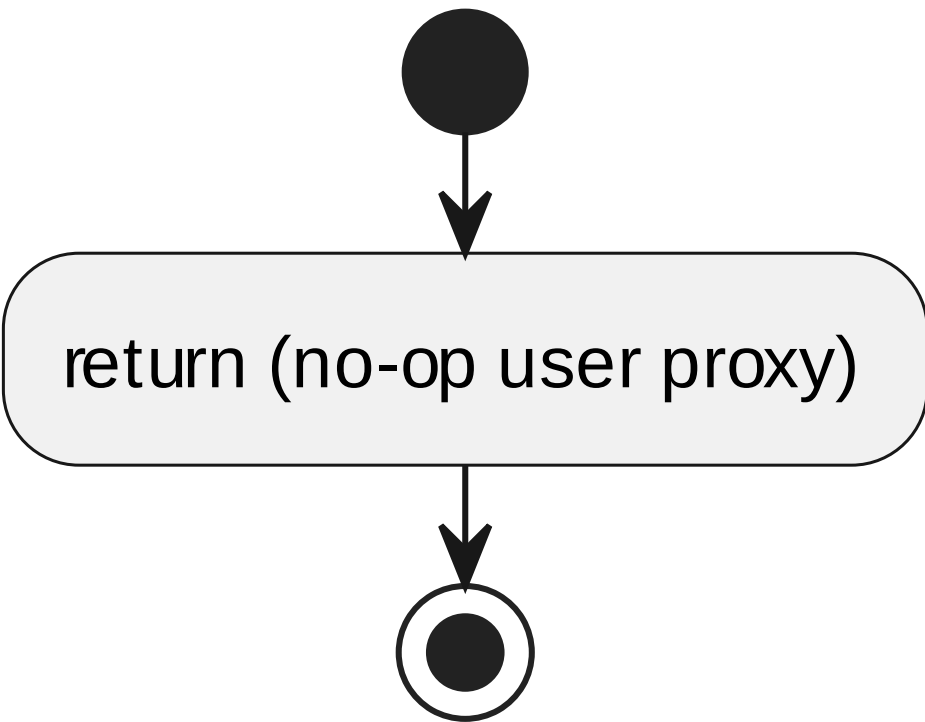
processing flow



7. 4. 15 ipcsHwFree

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_UIO			
函数说明	释放 HAL 资源；用户侧为空实现，内核侧释放映射状态。			
函数原型	void ipcsHwFree(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_uio/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_uio/ipc-os.c			
函数声明文件	ipcs/mpu/os_uio/ipc-os.h			

processing flow

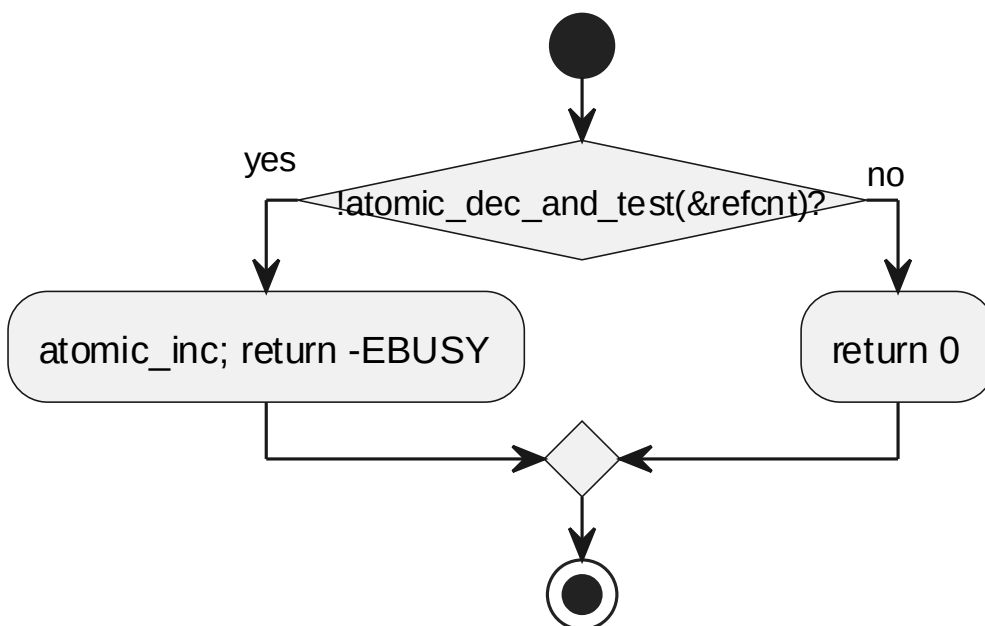


7.5 SWU_IPCS_LINUX_UIO_KO DESIGN 单元设计

7.5.1 ipcsShmUioOpen

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	处理 UIO 设备打开请求并维护引用计数。			
函数原型	static int ipcsShmUioOpen(struct uio_info *info, struct inode *inode)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	info	struct uio_info *info	[IN] info
	I	inode	struct inode *inode	[IN] inode
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

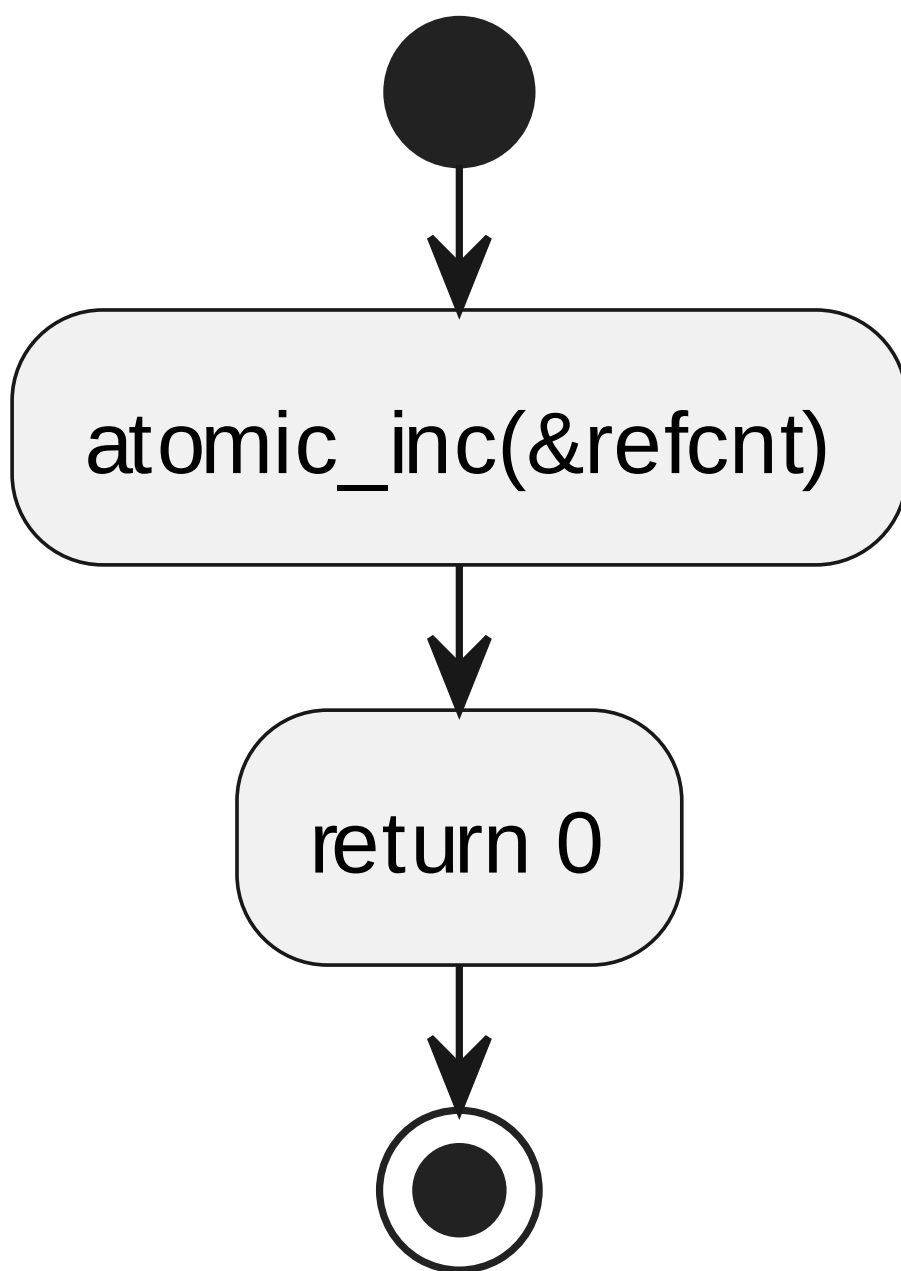
processing flow



7.5.2 ipcsShmUioRelease

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	处理 UIO 设备关闭请求并恢复引用计数。			
函数原型	static int ipcsShmUioRelease(struct uio_info *info, struct inode *inode)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	info	struct uio_info *info	[IN] info
	I	inode	struct inode *inode	[IN] inode
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

processing flow

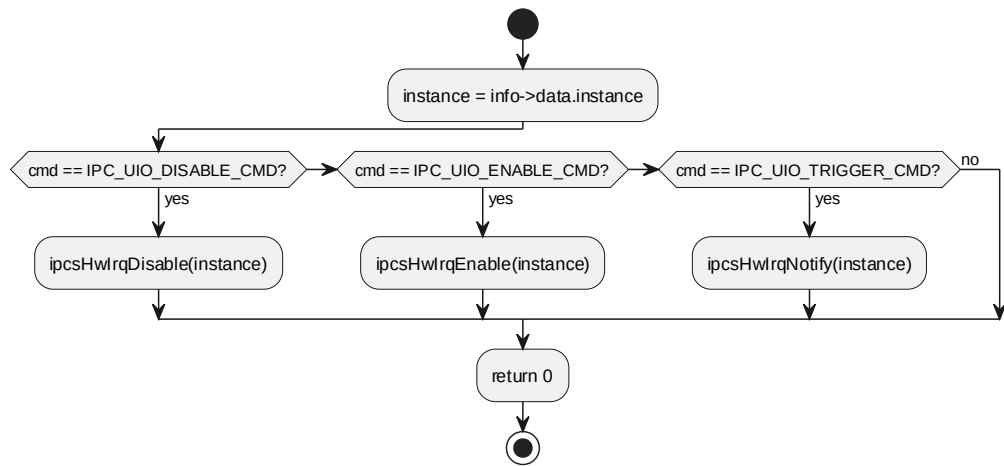


7.5.3 ipcsShmUioIrqcontrol

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	处理 UIO irqcontrol 命令并调用 HAL IRQ 操作。			
函数原型	static int ipcsShmUioIrqcontrol(struct uio_info *dev_info, int cmd)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明

	I	dev_info	struct uio_info *dev_info	[IN] dev_info
	I	cmd	int cmd	[IN] cmd
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

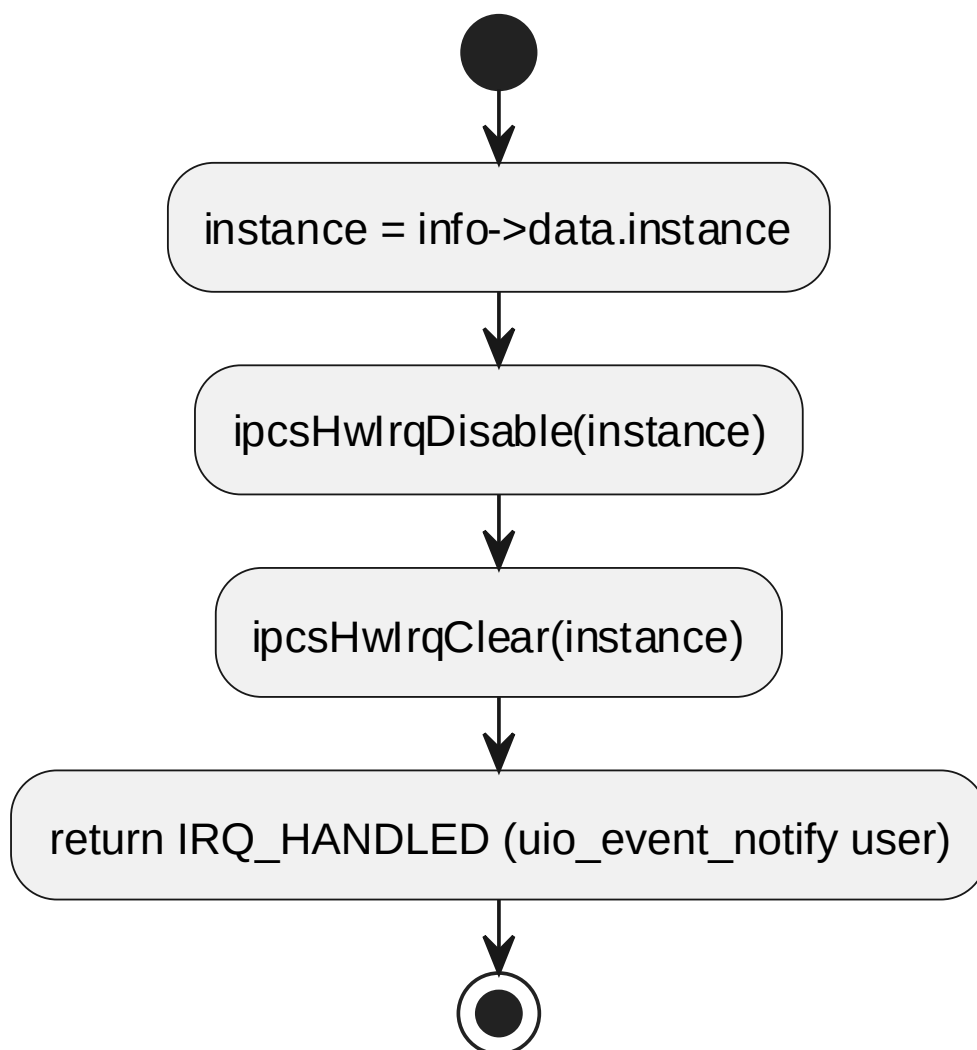
processing flow



7.5.4 ipcsShmUioHandler

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	UIO 中断处理，禁止并清除 IRQ，返回 IRQ_HANDLED 唤醒用户态。			
函数原型	static irqreturn_t ipcsShmUioHandler(int irq, struct uio_info *dev_info)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	irq	int irq	[IN] irq
	I	dev_info	struct uio_info *dev_info	[IN] dev_info
返回值	数据类型		说明	
	irqreturn_t		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

processing flow

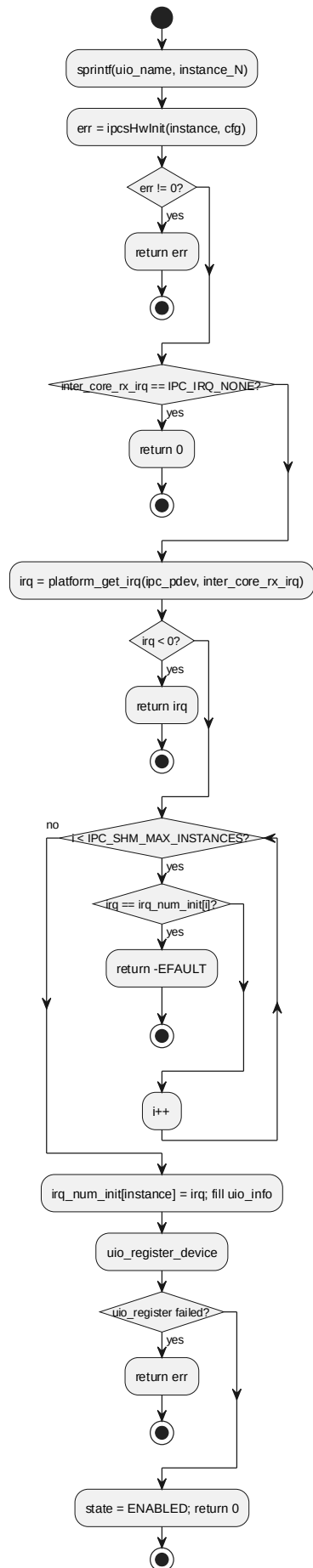


7.5.5 ipcsUioInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	根据用户配置初始化 UIO 实例、HAL 与 IRQ，并注册 UIO 设备。			
函数原型	static int ipcsUioInit(struct IPCS_UIO_CDEV_DATA_TYPE *data)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	data	struct IPCS_UIO_CDEV_DATA_TYPE *data	[IN] data
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			

函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h
--------	------------------------------

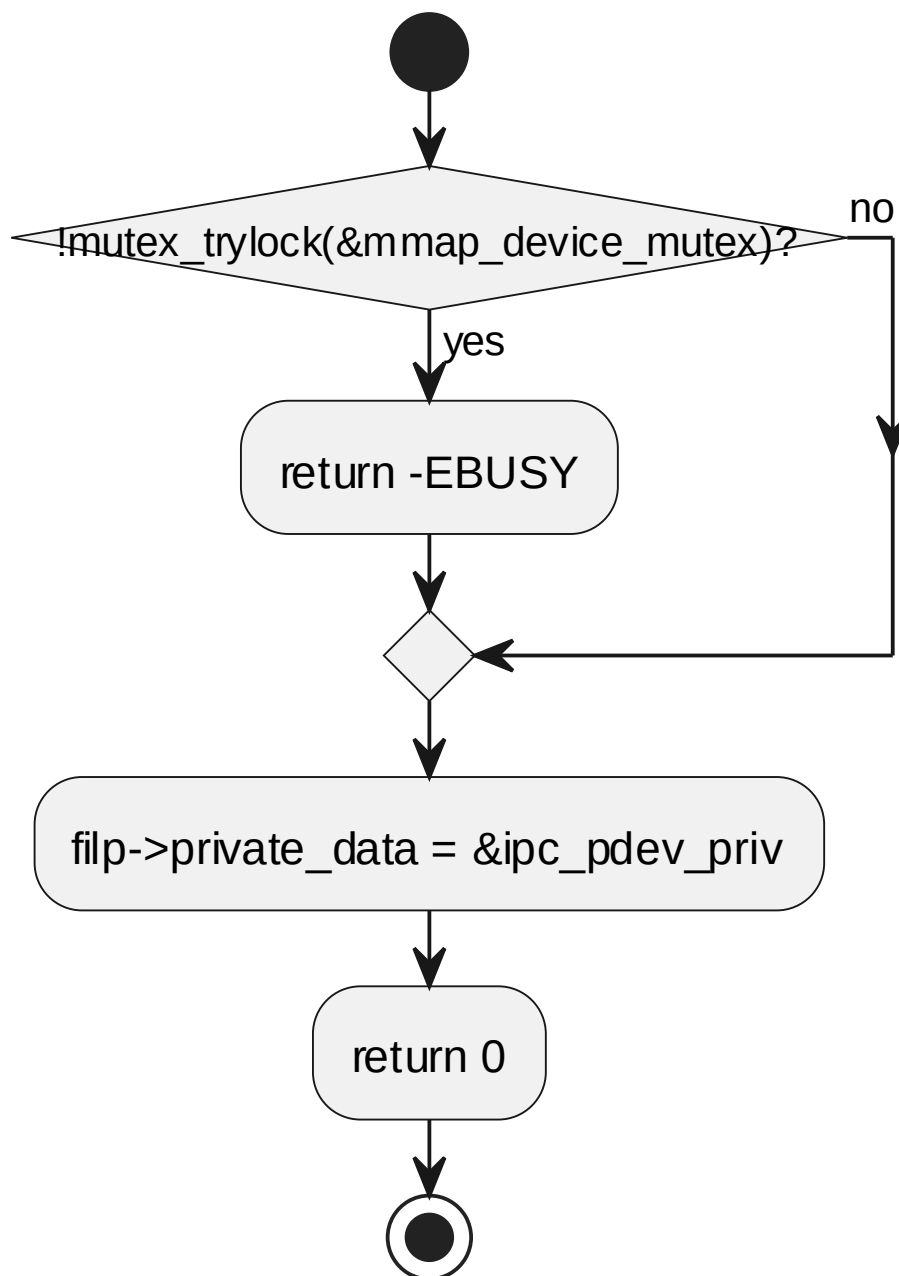
processing flow



7.5.6 ipcsCdevOpen

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	处理字符设备打开请求。			
函数原型	static int ipcsCdevOpen(struct inode *inode, struct file *filp)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	inode	struct inode *inode	[IN] inode
	I	filp	struct file *filp	[IN] filp
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

processing flow

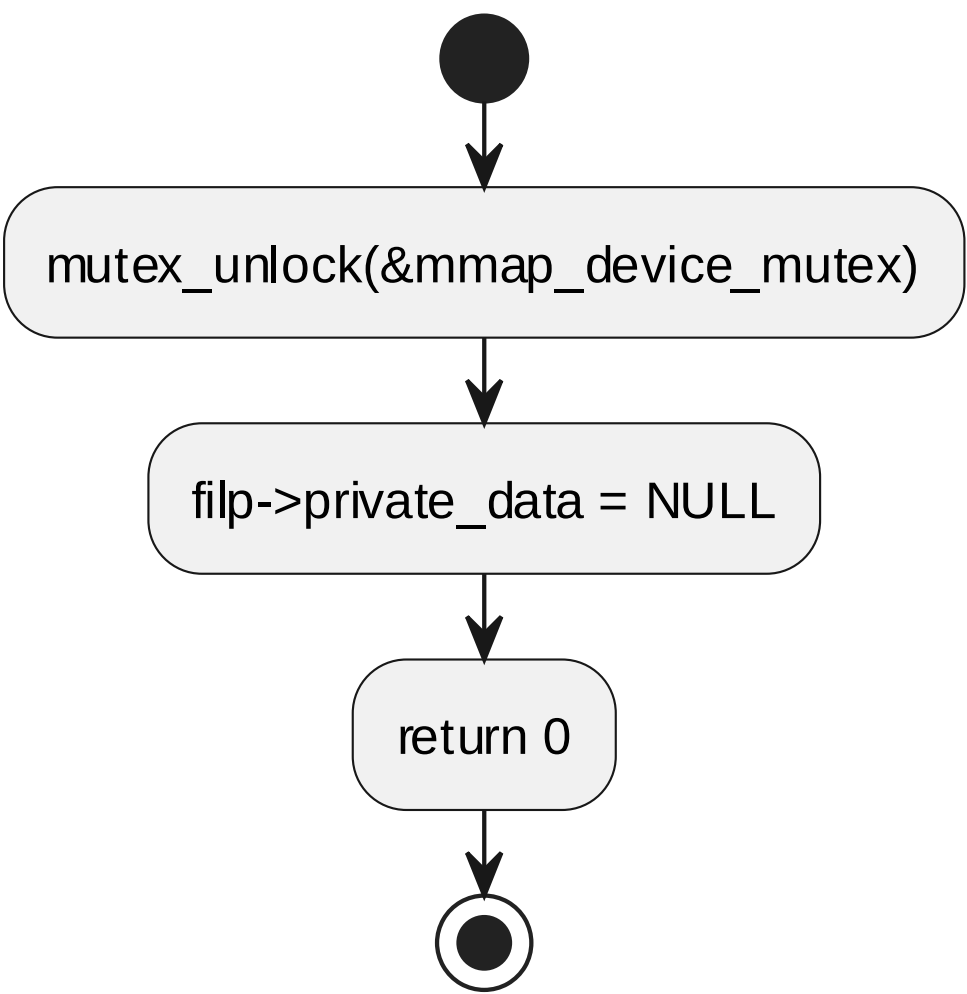


7.5.7 ipcsCdevRelease

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	处理字符设备关闭请求。			
函数原型	static int ipcsCdevRelease(struct inode *inode, struct file *filp)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	inode	struct inode	[IN] inode

			*inode	
	l	filp	struct file *filp	[IN] filp
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

processing flow

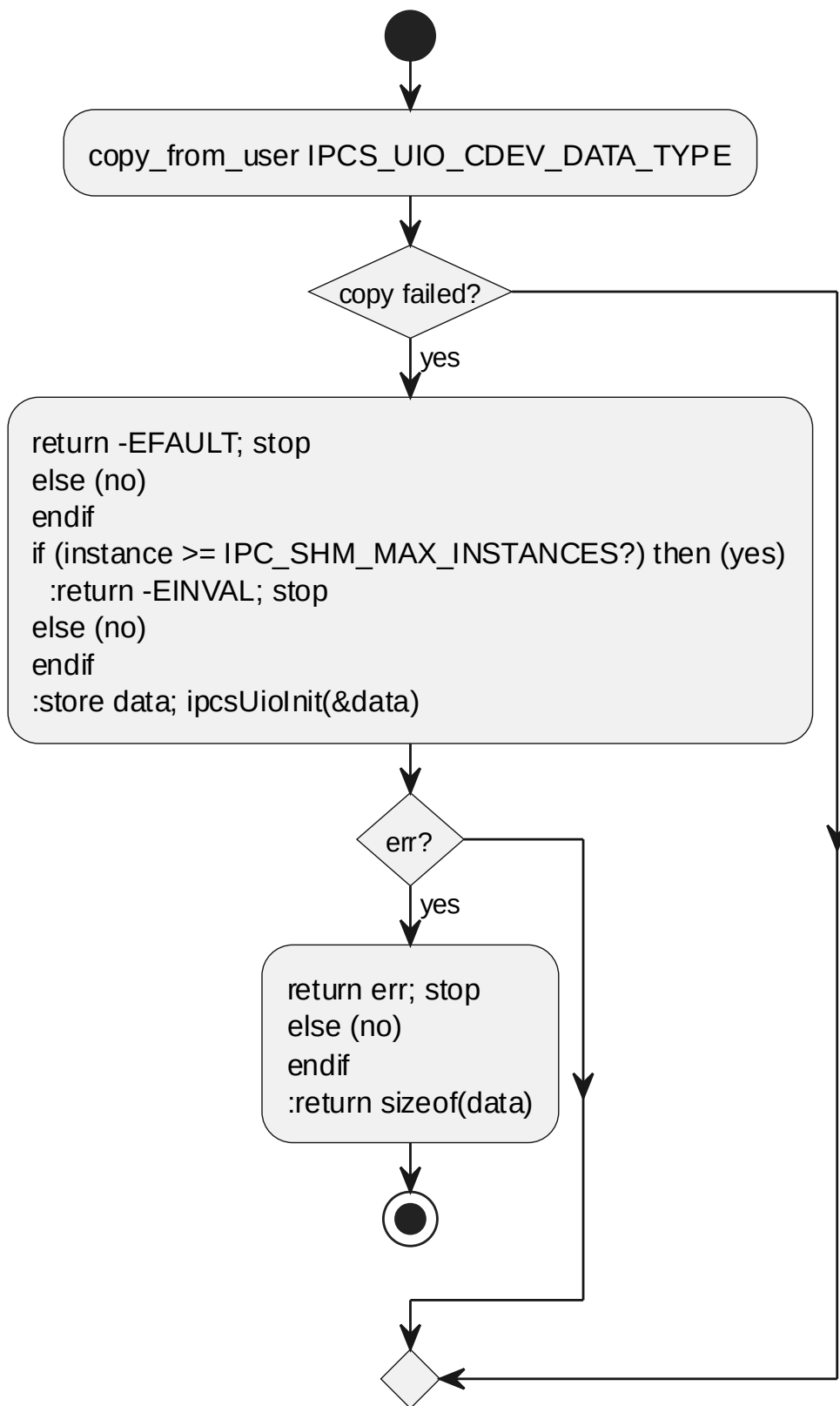


7.5.8 ipcscdevwrite

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp
软件单元 ID	SWU_IPCS_LINUX_UIO_KO
函数说明	接收用户侧 UIO 配置并初始化对应 UIO 设备。
函数原型	static ssize_t ipcscdevwrite(struct file *file, const char __user *user_buffer, size_t size, loff_t *offset)
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行

输入/输出参数	I/O	参数名	数据类型	说明
	l	file	struct file *file	[IN] file
	l	user_buffer	const char __user *user_buffer	[IN] user_buffer
	l	size	size_t size	[IN] size
	l	offset	loff_t *offset	[IN] offset
返回值	数据类型		说明	
	ssize_t		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

processing flow

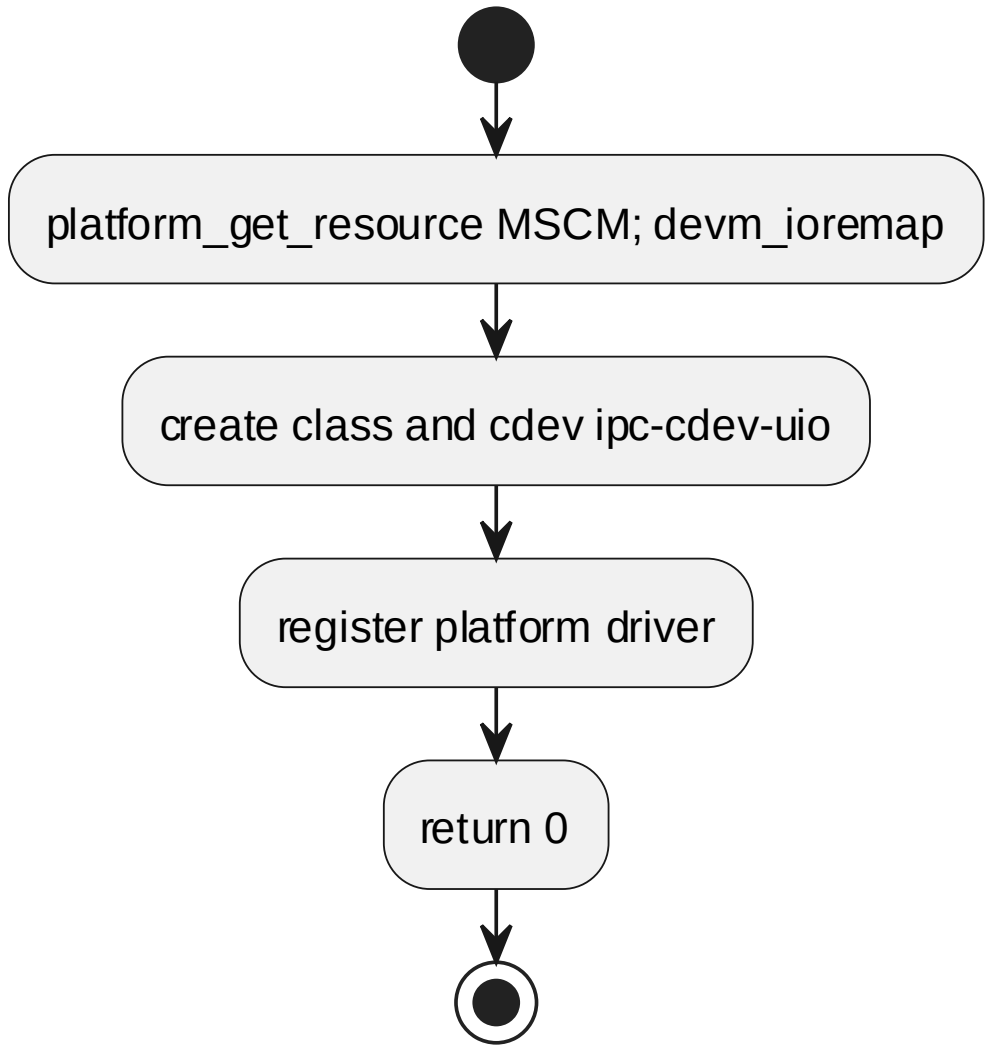


7.5.9 ipcsShmUioProbe

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp
-----------	--------------------------

软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	平台驱动 probe，映射 核间中断控制器 资源并创建设备节点。			
函数原型	static int ipcsShmUioProbe(struct platform_device *pdev)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	pdev	struct platform_device *pdev	[IN] pdev
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

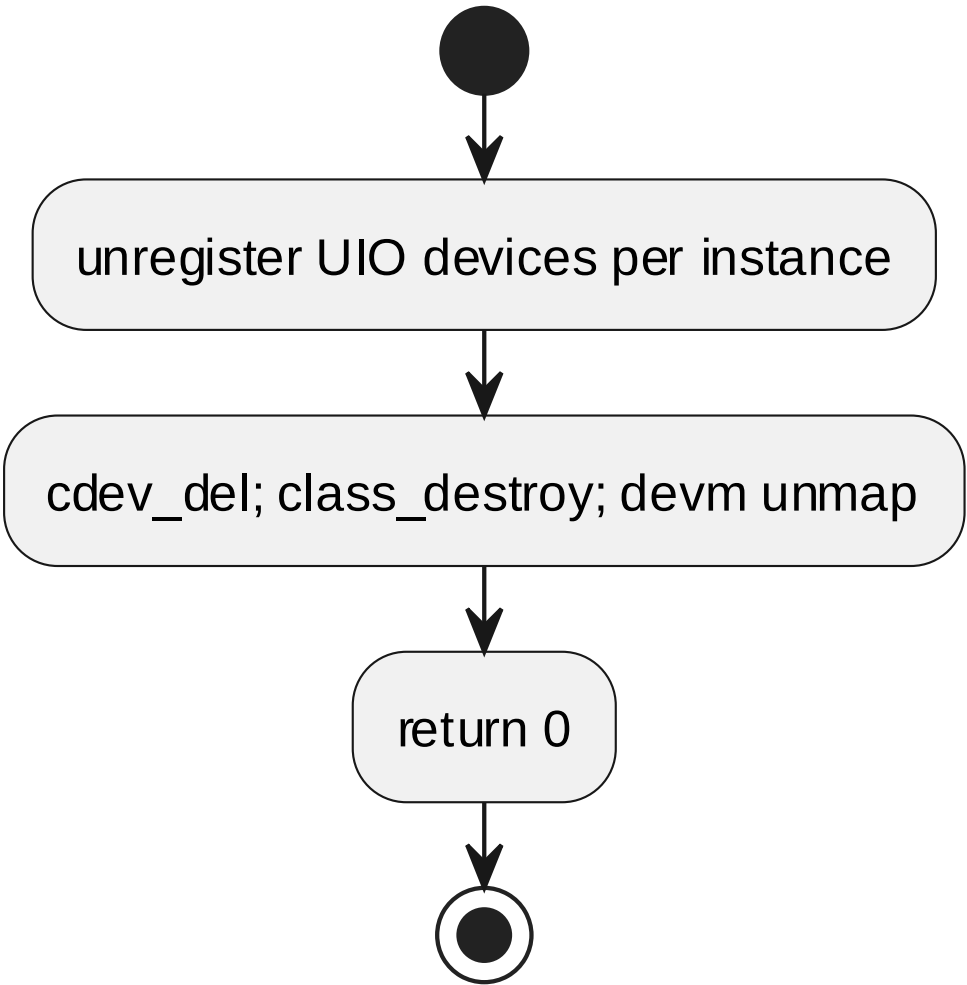
processing flow



7. 5. 10 ipcsShmUioRemove

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	平台驱动 remove，注销设备并释放 UIO 实例。			
函数原型	static int ipcsShmUioRemove(struct platform_device *pdev)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	pdev	struct platform_device *pdev	[IN] pdev
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

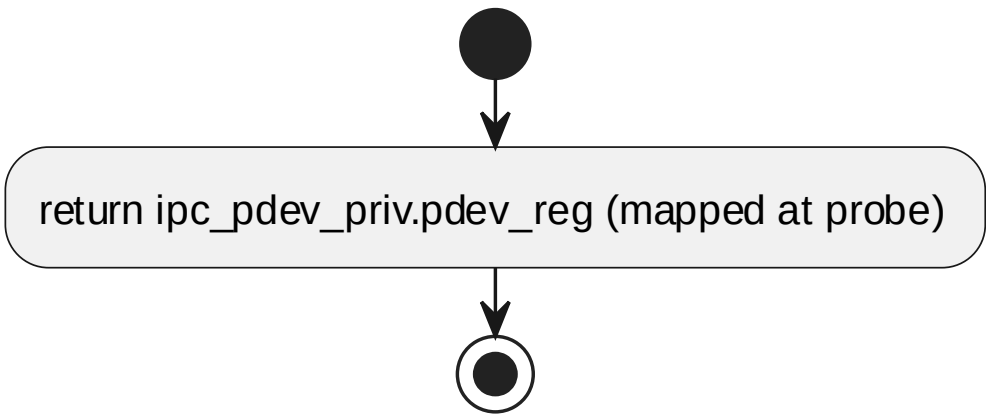
processing flow



7. 5. 11 ipcs0sMapIntc

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_UIO_KO	
函数说明	映射或返回中断控制器寄存器空间。	
函数原型	void *ipcsOsMapIntc(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	void *	-
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h	

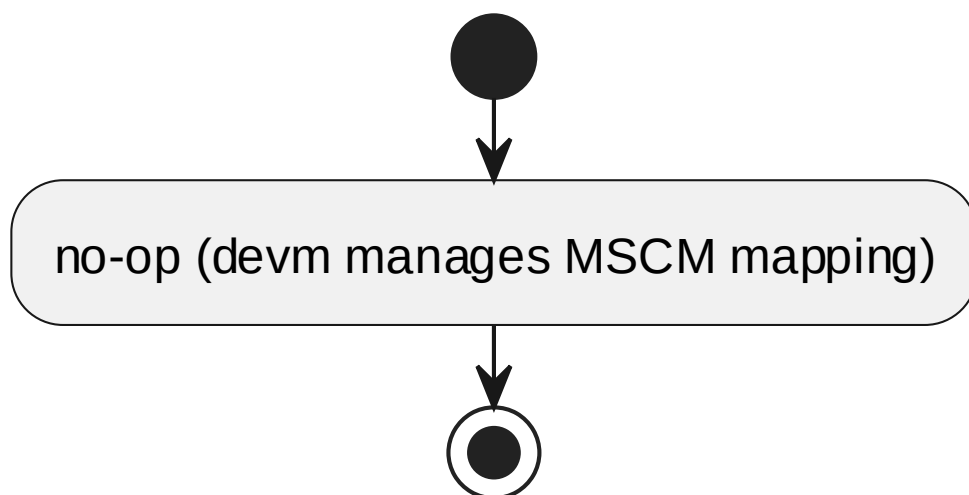
processing flow



7. 5. 12 ipcs0sUnmapIntc

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_UIO_KO			
函数说明	释放中断控制器寄存器映射或提供对应空实现。			
函数原型	void ipcsOsUnmapIntc(void *addr)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-uio.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	addr	void *addr	[IN] addr
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_kernel/ipc-uio.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-uio.h			

processing flow

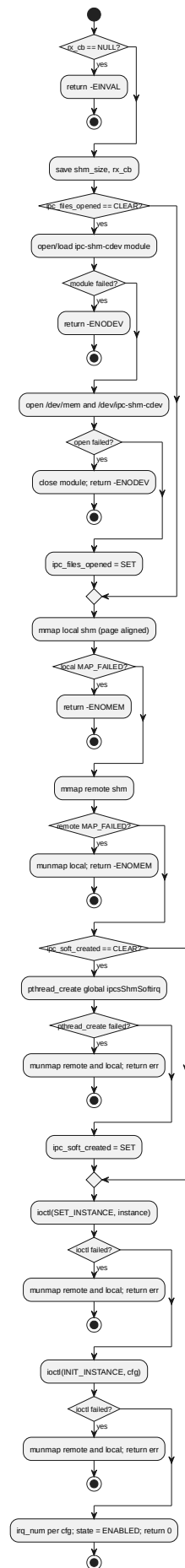


7.6 SWU_IPCS_LINUX_OS_CDEV DESIGN 单元设计

7.6.1 ipcsOsInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	初始化指定实例的 Linux OSAL 资源，建立共享内存映射、记录回调并配置接收中断。			
函数原型	int ipcsOsInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg, int (*rx_cb)(const uint8_t, int))			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_ TYPE *cfg	[IN] cfg
	I	rx_cb	int (*rx_cb)(const uint8_t, int)	[IN] rx_cb
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

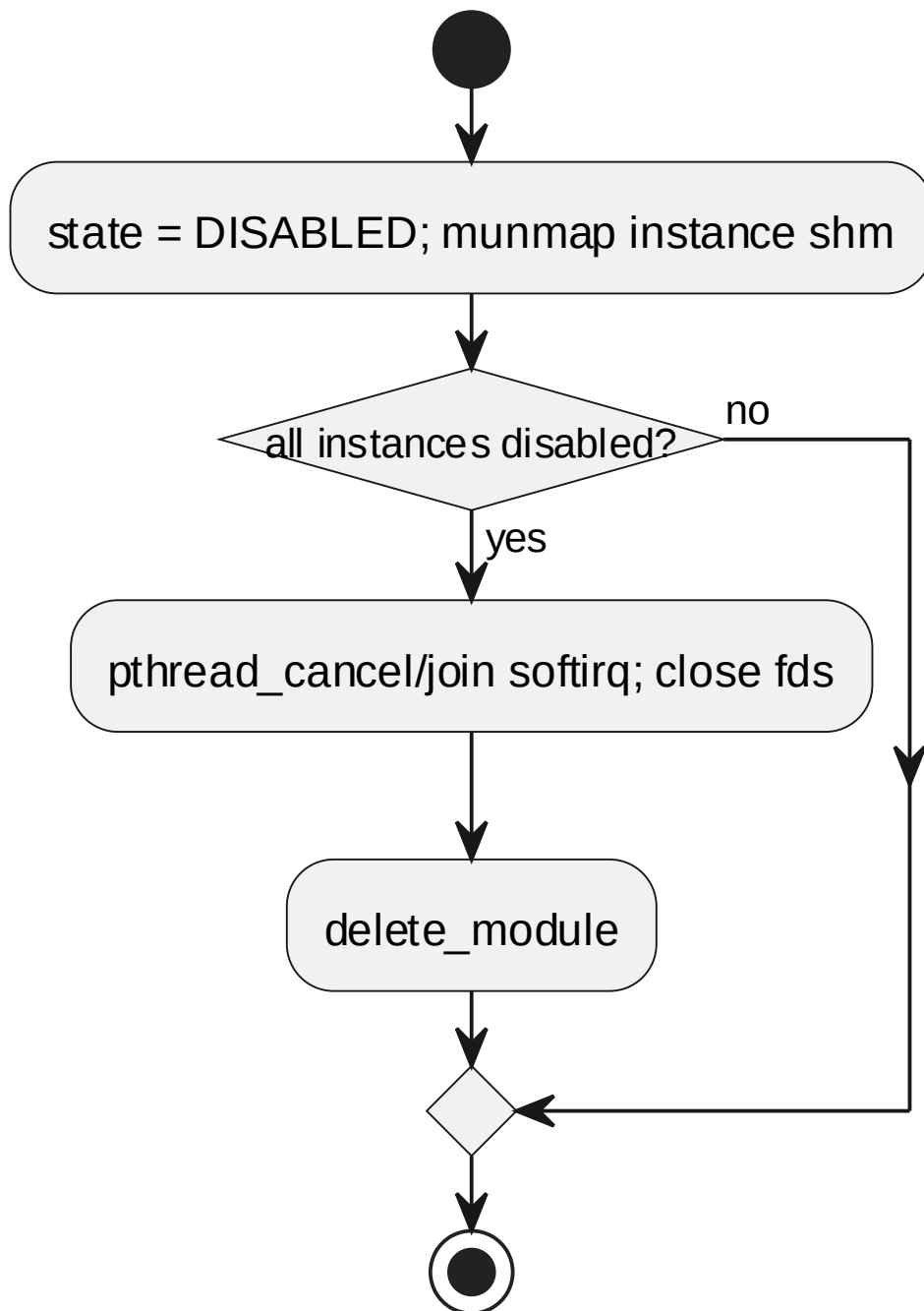
processing flow



7.6.2 ipcsOsFree

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	释放指定实例 OSAL 资源，关闭线程/设备、解除映射并清理状态。			
函数原型	void ipcsOsFree(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

processing flow

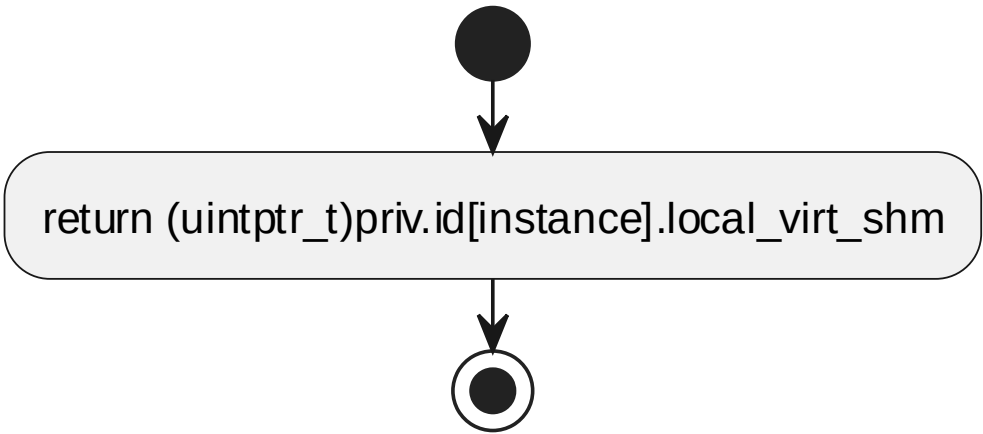


7.6.3 ipcOsGetLocalShm

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	返回本地共享内存虚拟地址。			
函数原型	uintptr_t ipcOsGetLocalShm(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t	[IN] instance

			instance	
返回值	数据类型		说明	
	uintptr_t		-	
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

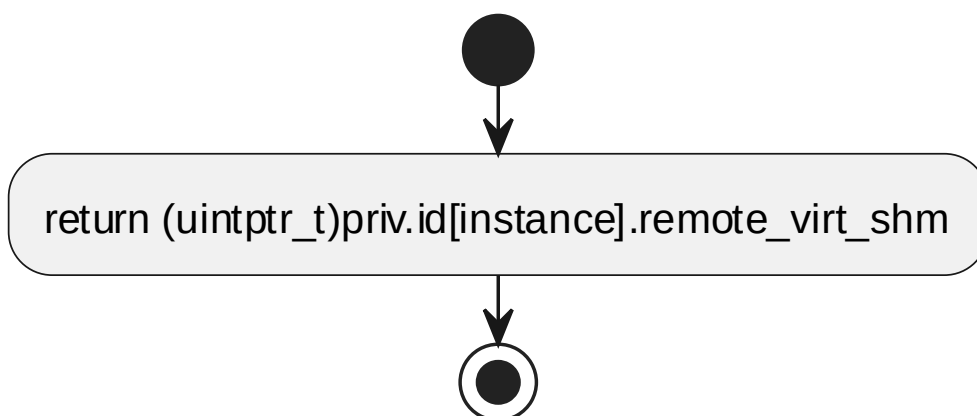
processing flow



7.6.4 ipcsOsGetRemoteShm

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	返回远端共享内存虚拟地址。			
函数原型	uintptr_t ipcsOsGetRemoteShm(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	uintptr_t		-	
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

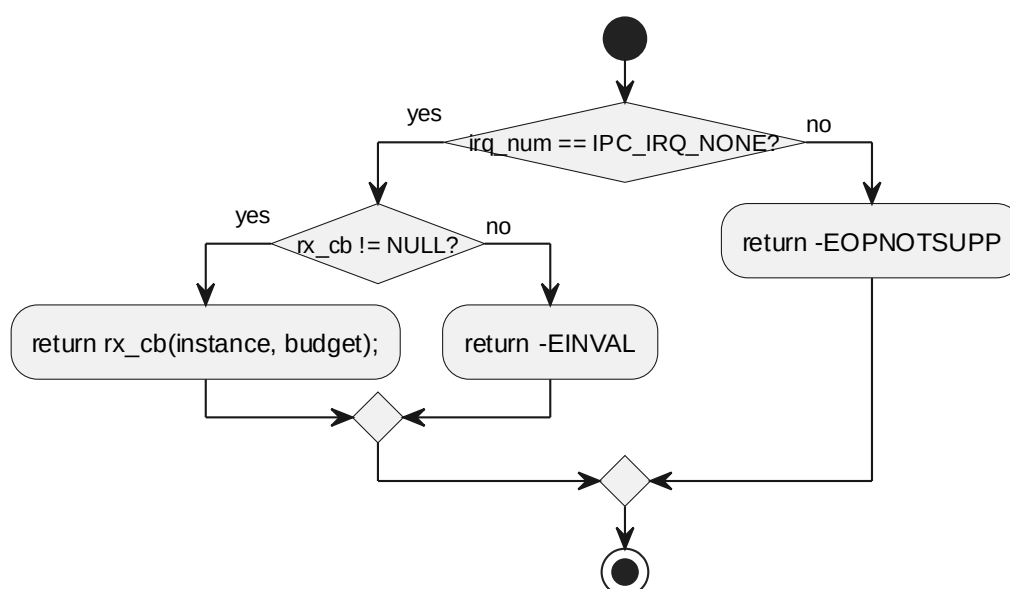
processing flow



7.6.5 ipcsOsPollChannels

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	在轮询模式下触发 rx_cb 处理接收通道。			
函数原型	int ipcsOsPollChannels(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

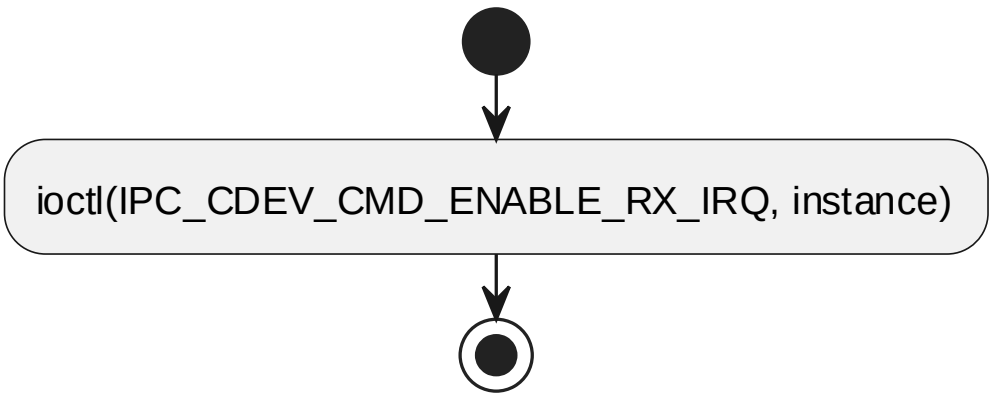
processing flow



7.6.6 ipcsHwIrqEnable

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	使能指定实例接收中断；用户侧为转发代理，内核侧访问硬件。			
函数原型	void ipcsHwIrqEnable(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

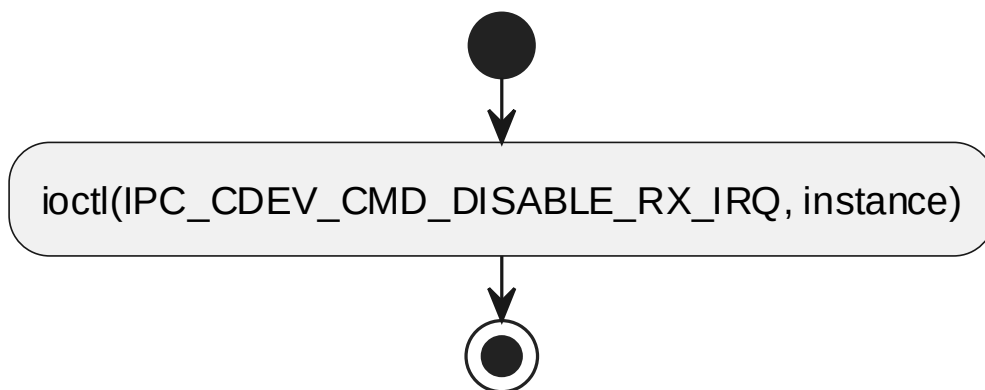
processing flow



7.6.7 ipcsHwIrqDisable

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	禁止指定实例接收中断；用户侧为转发代理，内核侧访问硬件。			
函数原型	void ipcsHwIrqDisable(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

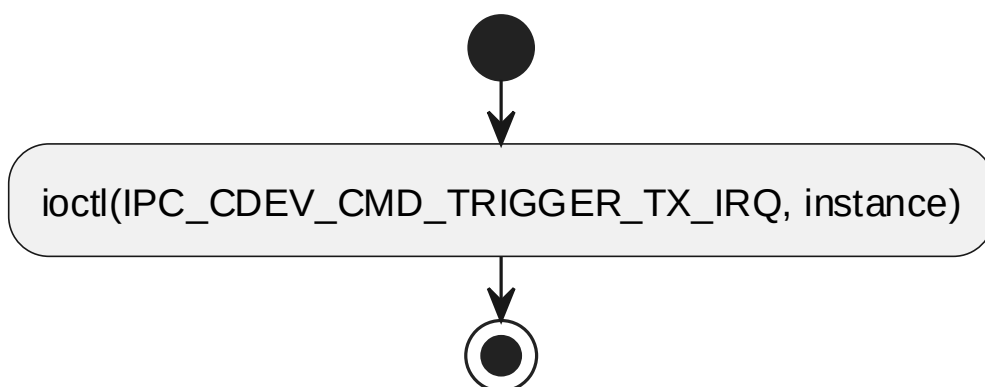
processing flow



7.6.8 ipcshwIrqNotify

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	通知远端有数据可用；用户侧为转发代理，内核侧触发硬件中断。			
函数原型	void ipcsHwIrqNotify(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

processing flow

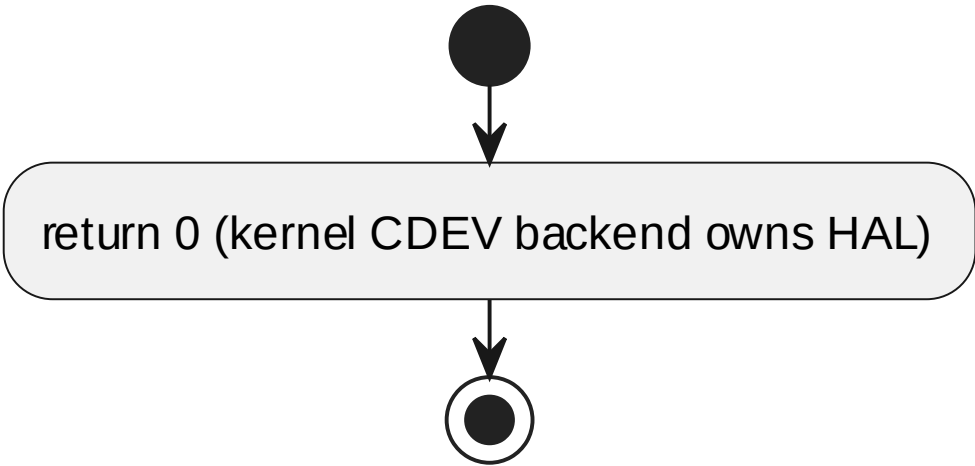


7.6.9 ipcshwInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	初始化 HAL 资源；用户侧为空实现，内核侧映射并配置 核间中断控制器/IRQ。			

函数原型	int ipcsHwInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_TYPE *cfg	[IN] cfg
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

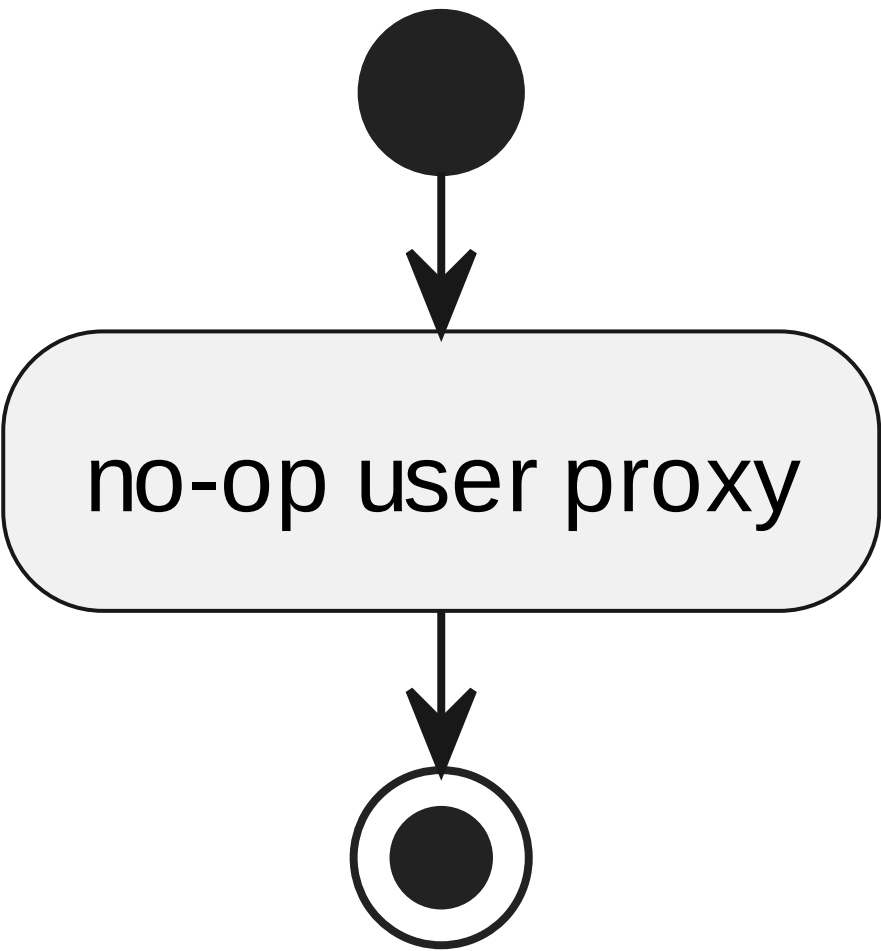
processing flow



7. 6. 10 ipcsHwFree

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_OS_CDEV			
函数说明	释放 HAL 资源；用户侧为空实现，内核侧释放映射状态。			
函数原型	void ipcsHwFree(const uint8_t instance)			
制约条件	按 `ipcs/mpu/os_cdev/ipc-os.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_cdev/ipc-os.c			
函数声明文件	ipcs/mpu/os_cdev/ipc-os.h			

processing flow

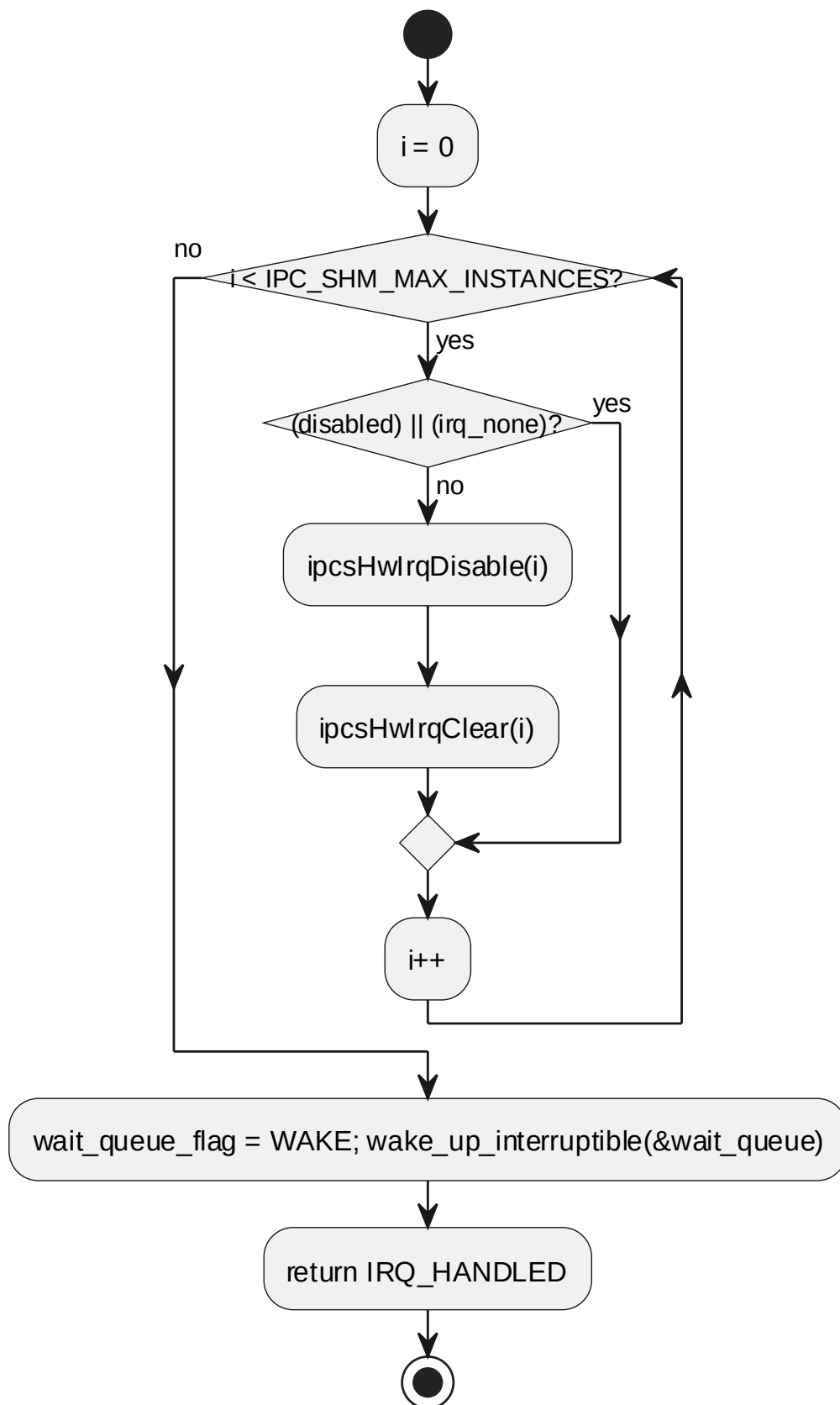


7. 7 SWU_IPCS_LINUX_CDEV_KO DESIGN 单元设计

7. 7. 1 ipcsShmHardirq

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO			
函数说明	硬中断处理，禁止并清除远端通知，中断后续处理交给 tasklet 或等待队列。			
函数原型	static irqreturn_t ipcsShmHardirq(int irq, void *dev)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	irq	int irq	[IN] irq
	I	dev	void *dev	[IN] dev
返回值	数据类型		说明	
	irqreturn_t		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c			

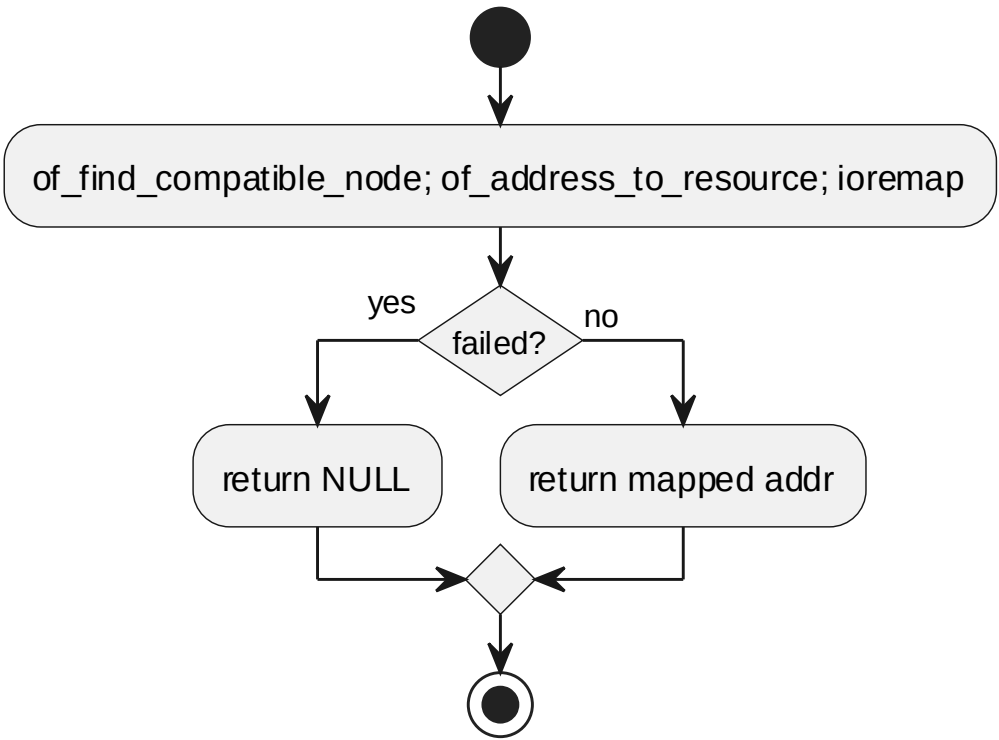
processing flow



7.7.2 ipcsOsMapIntc

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO	
函数说明	映射或返回中断控制器寄存器空间。	
函数原型	void *ipcsOsMapIntc(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	void *	-
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h	

processing flow

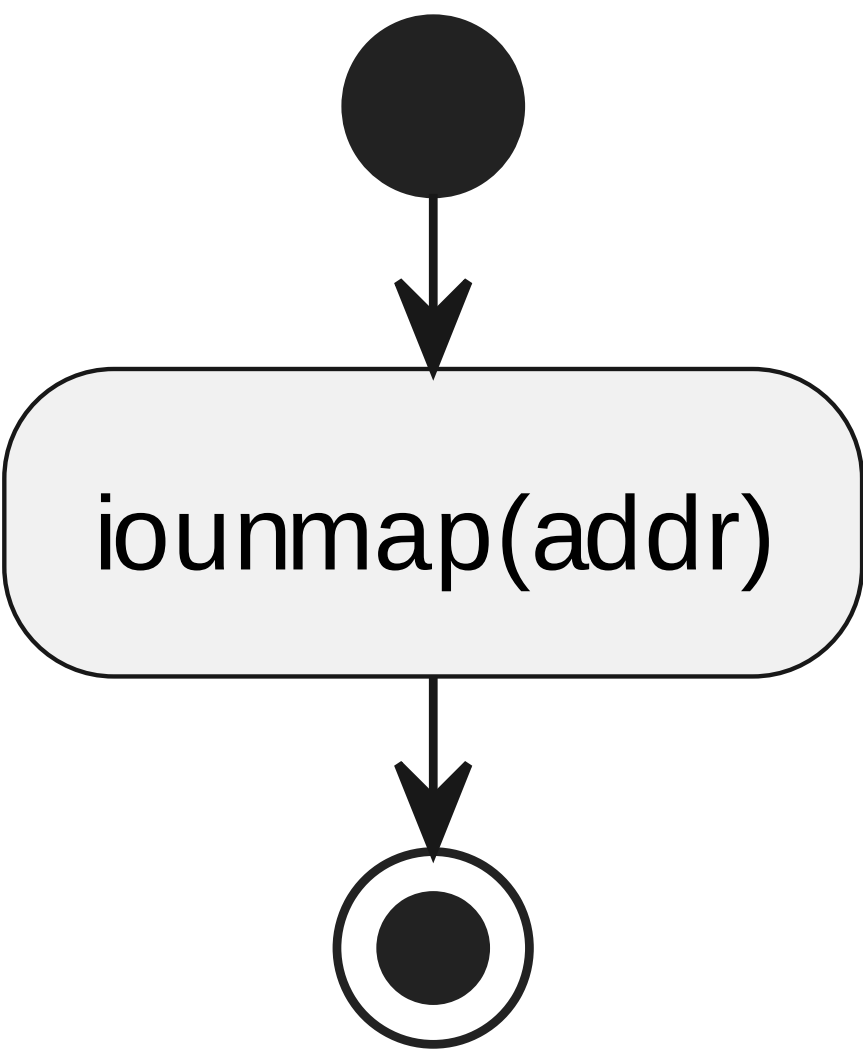


7.7.3 ipcsOsUnmapIntc

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO			
函数说明	释放中断控制器寄存器映射或提供对应空实现。			
函数原型	void ipcsOsUnmapIntc(void *addr)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明

	l	addr	void *addr	[IN] addr
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h			

processing flow

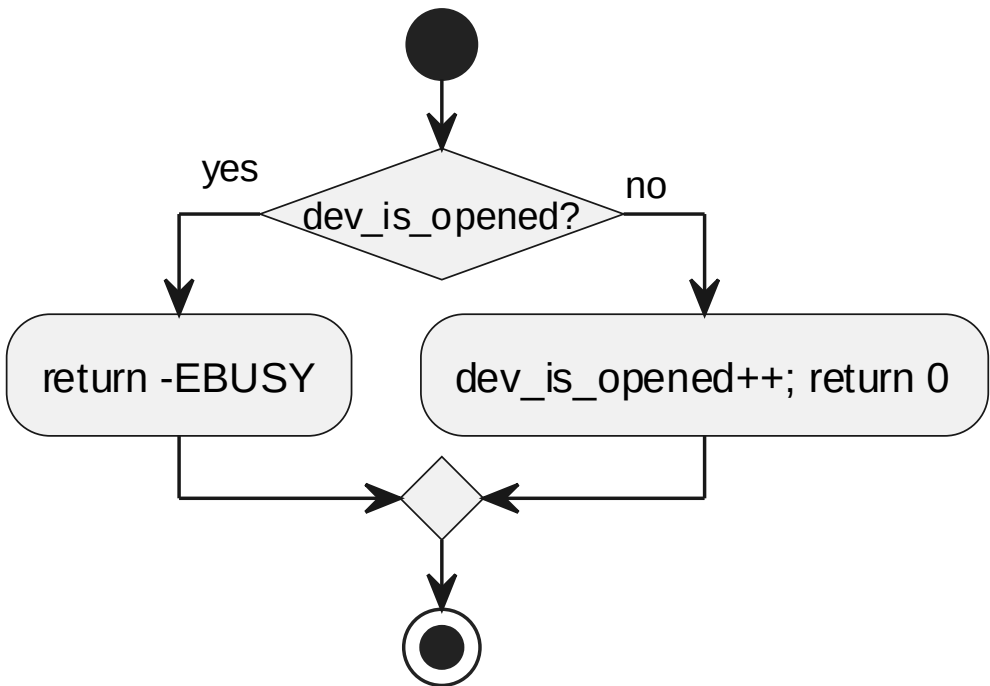


7.7.4 ipcsCdevOpen

对应软件架构ID	Drv_Ipcs_Linux_Adapt_Cmp
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO
函数说明	处理字符设备打开请求。
函数原型	static int ipcsCdevOpen(struct inode *inode, struct file *file)
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执

	行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	inode	struct inode *inode	[IN] inode
	I	file	struct file *file	[IN] file
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h			

processing flow

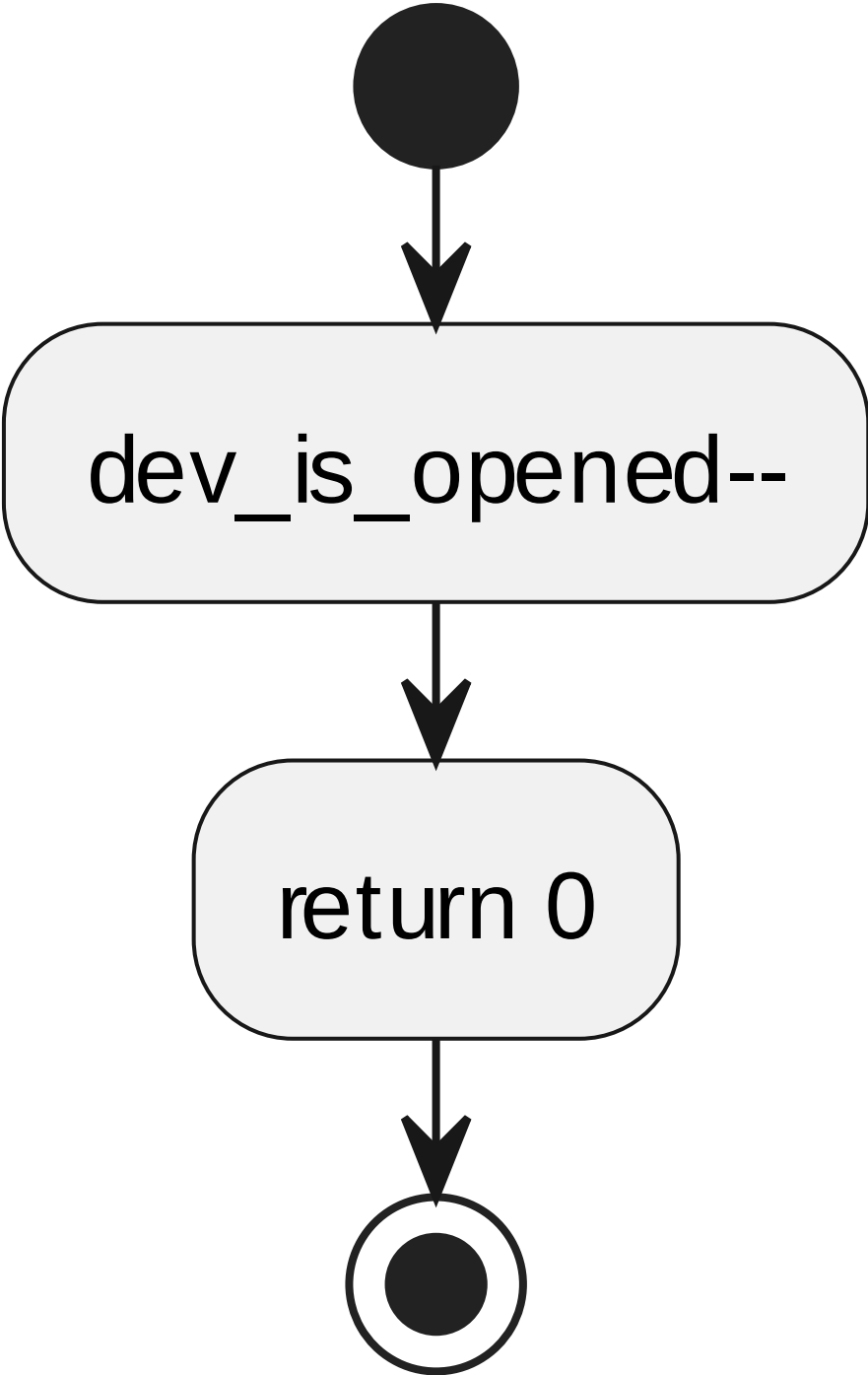


7.7.5 ipcsCdevRelease

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO			
函数说明	处理字符设备关闭请求。			
函数原型	static int ipcsCdevRelease(struct inode *inode, struct file *file)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	inode	struct inode *inode	[IN] inode
	I	file	struct file *file	[IN] file
返回值	数据类型		说明	

	int	-
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h	

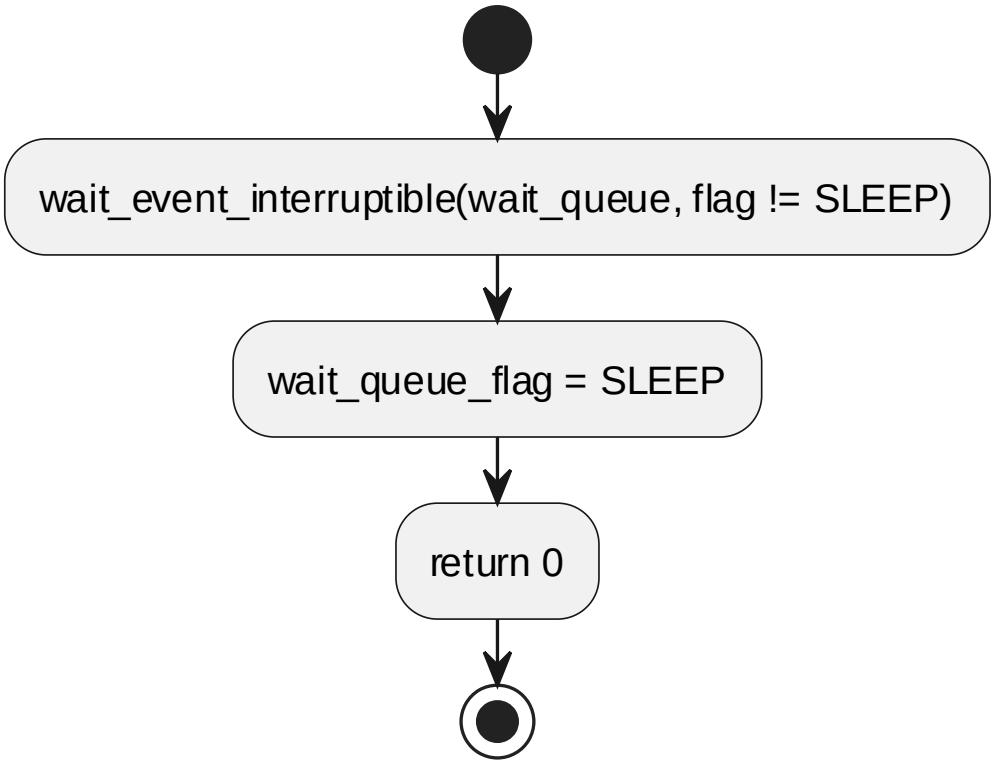
processing flow



7.7.6 ipcsCdevRead

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO			
函数说明	阻塞等待内核接收中断唤醒。			
函数原型	static ssize_t ipcsCdevRead(struct file *file, char __user *user_buffer, size_t size, loff_t *offset)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	file	struct file *file	[IN] file
	I	user_buffer	char __user *user_buffer	[IN] user_buffer
	I	size	size_t size	[IN] size
	I	offset	loff_t *offset	[IN] offset
返回值	数据类型		说明	
	ssize_t		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h			

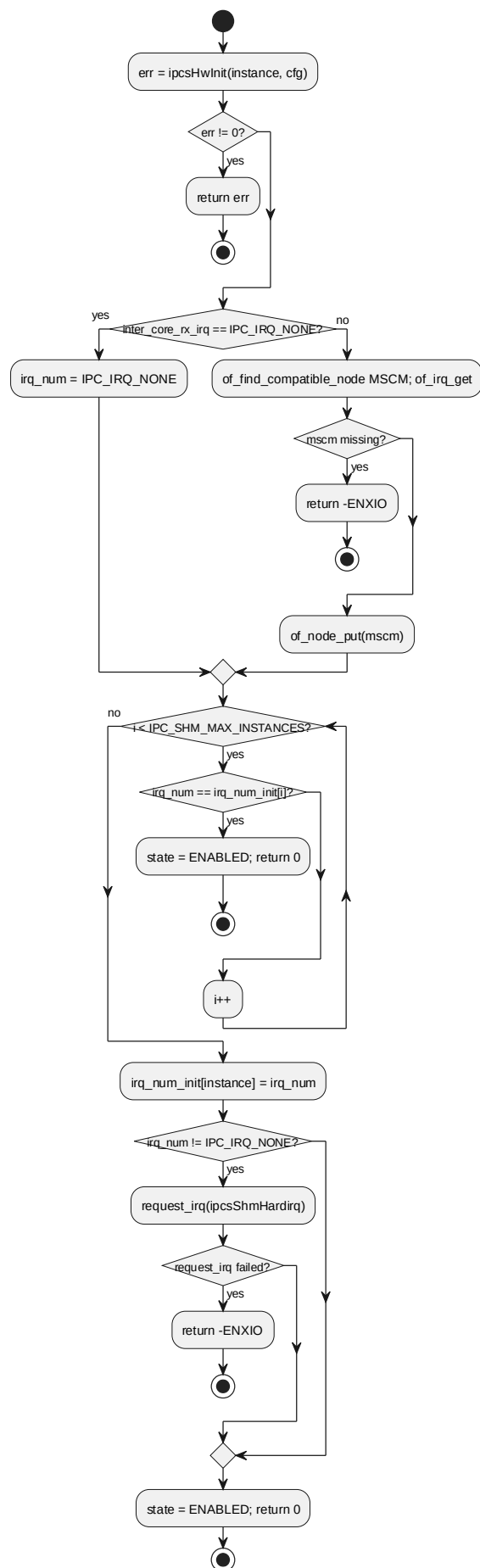
processing flow



7.7.7 ipcsCdevOsInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO			
函数说明	初始化 CDEV 后端实例、HAL 和接收 IRQ。			
函数原型	static int ipcsCdevOsInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_TYPE *cfg	[IN] cfg
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h			

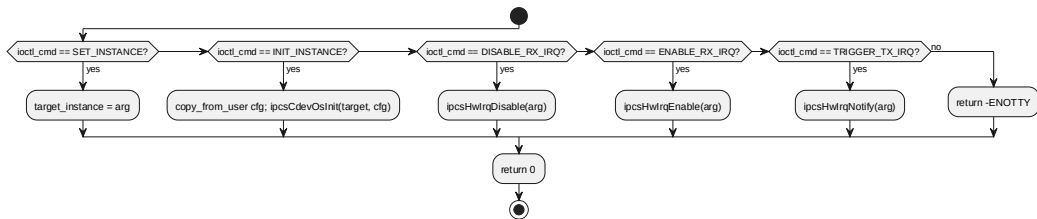
processing flow



7.7.8 ipcsCdevIoctl

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp			
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO			
函数说明	处理 CDEV 用户侧 ioctl 命令，包括实例初始化和 IRQ 操作代理。			
函数原型	static long ipcsCdevIoctl(struct file *file, unsigned int ioctl_cmd, unsigned long ioctl_arg)			
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	file	struct file *file	[IN] file
	I	ioctl_cmd	unsigned int ioctl_cmd	[IN] ioctl_cmd
	I	ioctl_arg	unsigned long ioctl_arg	[IN] ioctl_arg
返回值	数据类型		说明	
	long		-	
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c			
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h			

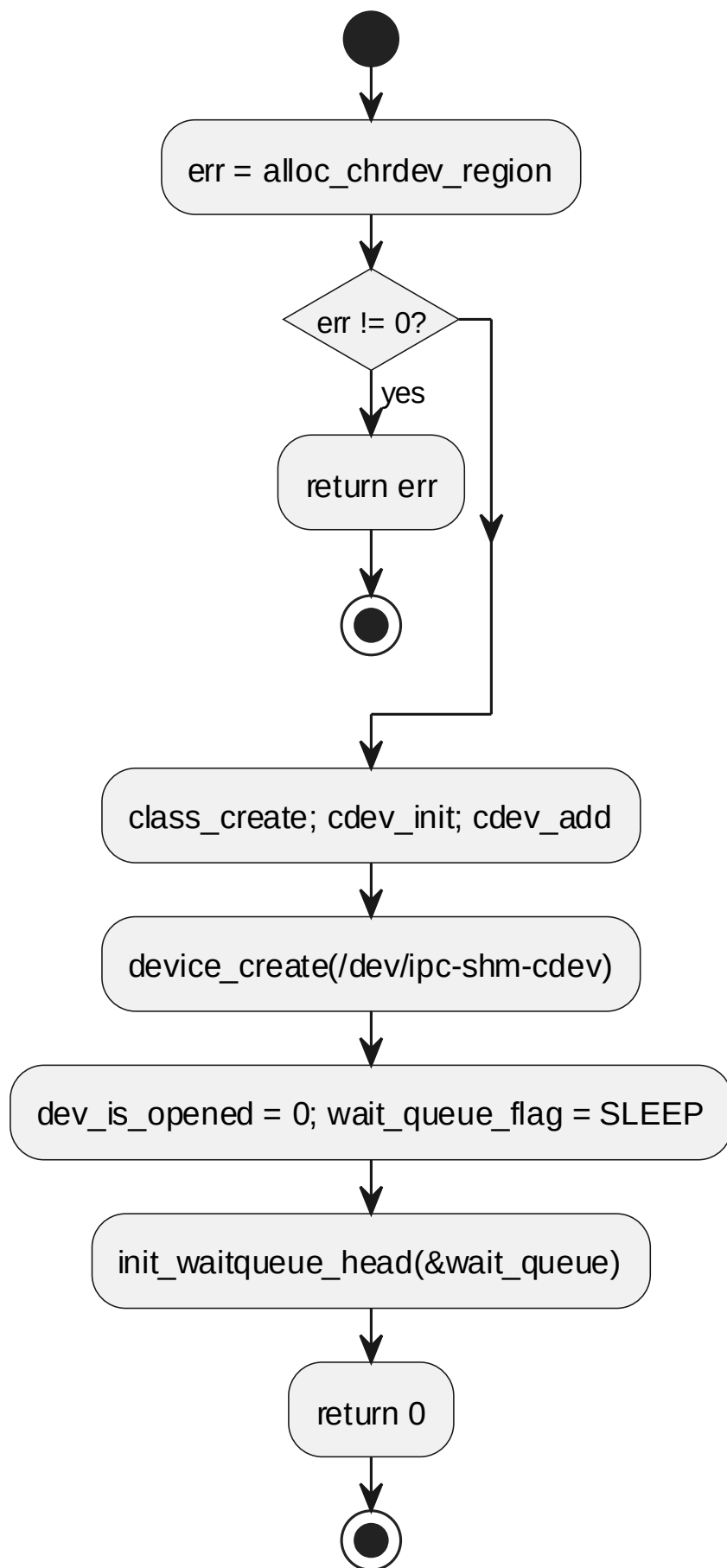
processing flow



7.7.9 ipcsCdevInit

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO	
函数说明	CDEV 模块初始化，创建字符设备和 wait queue。	
函数原型	static int ipcsCdevInit(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	int	-
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h	

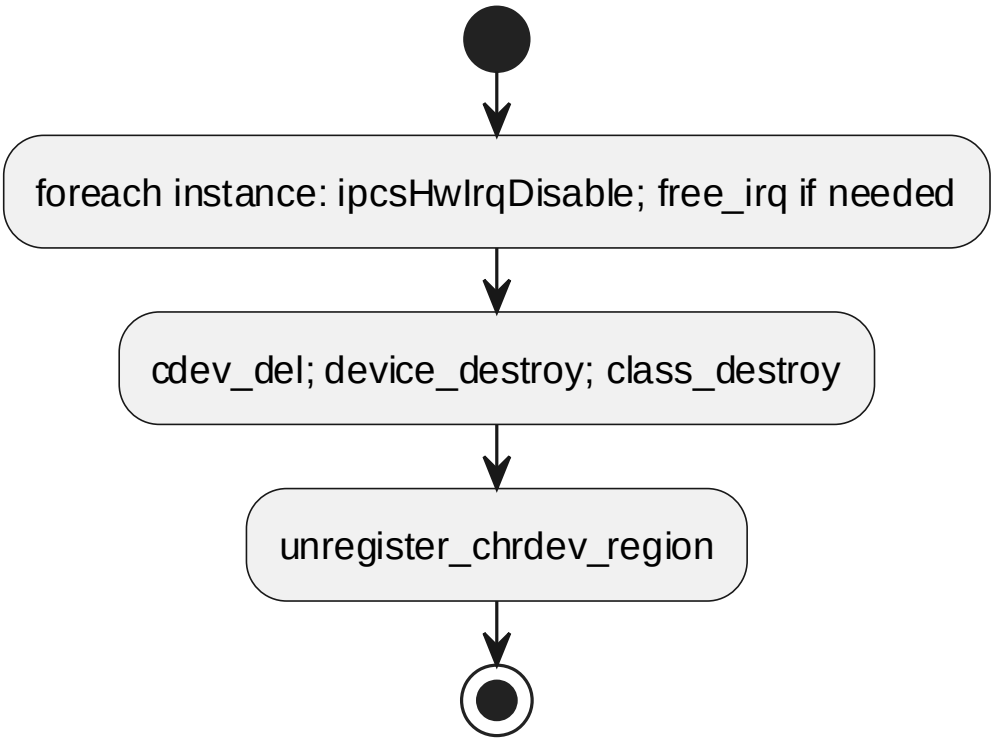
processing flow



7. 7. 10 ipcsCdevClean

对应软件架构 ID	Drv_Ipcs_Linux_Adapt_Cmp	
软件单元 ID	SWU_IPCS_LINUX_CDEV_KO	
函数说明	CDEV 模块清理，禁止 IRQ、释放中断并销毁字符设备。	
函数原型	static void ipcsCdevClean(void)	
制约条件	按 `ipcs/mpu/os_kernel/ipc-cdev.c` 中的入参检查、实例状态和内核资源状态执行	
输入/输出参数	-	
返回值	数据类型	说明
	-	
函数定义文件	ipcs/mpu/os_kernel/ipc-cdev.c	
函数声明文件	ipcs/mpu/os_kernel/ipc-cdev.h	

processing flow



7. 8 SWU_IPCS_HAL_LINUX DESIGN 单元设计

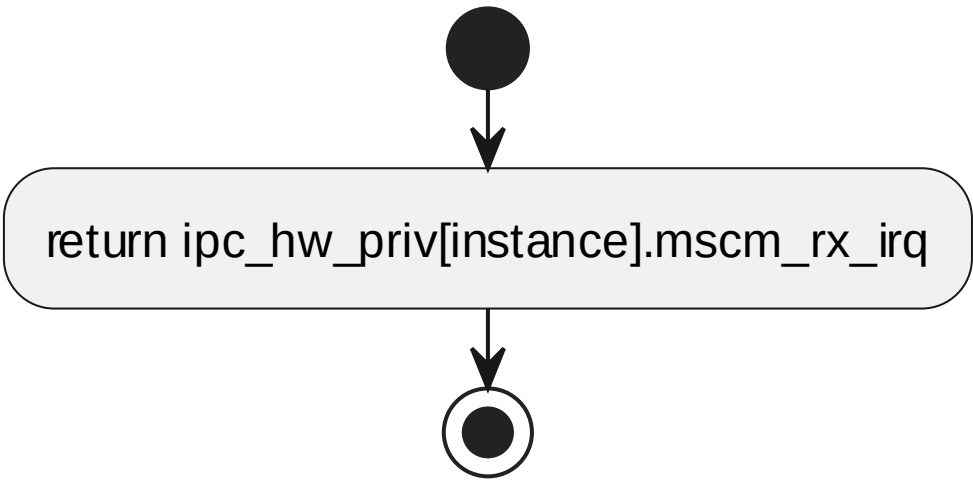
Linux 内核侧 HAL，完成 核间中断控制器 映射、核索引解析、IRQ 使能/禁止/通知/清除等硬件操作。

7. 8. 1 ipcsHwGetRxIrq

对应软件架构 ID	Drv_Ipcs_Hal_Cmp	
软件单元 ID	SWU_IPCS_HAL_LINUX	
函数说明	返回指定实例使用的 核间中断控制器 接收中断索引。	

函数原型	int ipcsHwGetRxIrq(const uint8_t instance)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

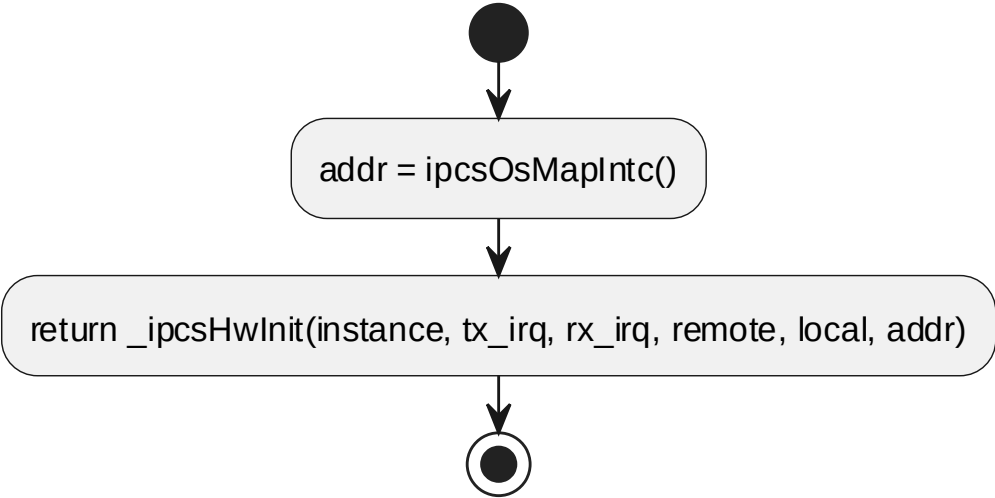
processing flow



7.8.2 ipcsHwInit

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	初始化 HAL 资源；用户侧为空实现，内核侧映射并配置 核间中断控制器/IRQ。			
函数原型	int ipcsHwInit(const uint8_t instance, const struct IPCS_SHM_CFG_TYPE *cfg)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	cfg	const struct IPCS_SHM_CFG_TYPE *cfg	[IN] cfg
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

processing flow

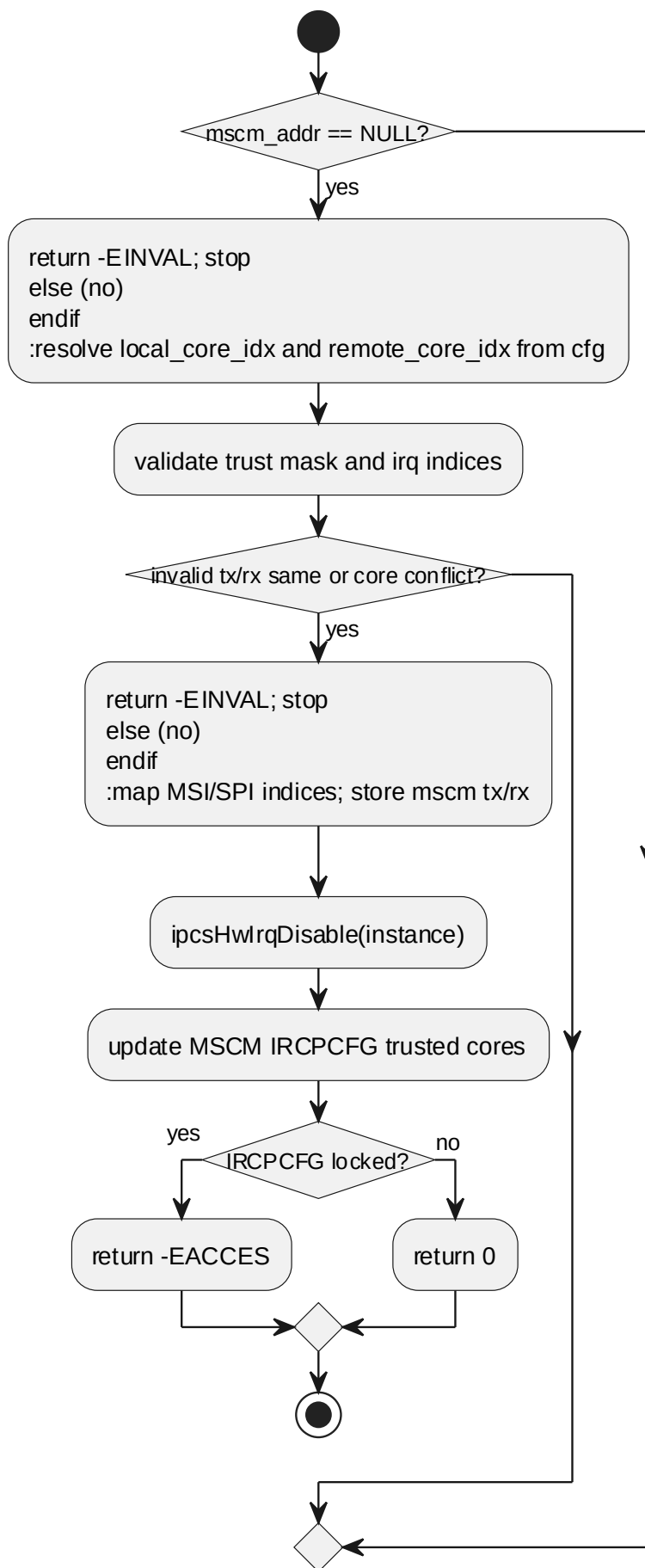


7.8.3 _ipcsHwInit

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	HAL 底层初始化，供 Linux UIO 等内核路径复用。			
函数原型	int _ipcsHwInit(const uint8_t instance, int tx_irq, int rx_irq, const struct IPCS_SHM_REMOTE_CORE_TYPE *remote_core, const struct IPCS_SHM_LOCAL_CORE_TYPE *local_core, void *mscm_addr)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
	I	tx_irq	int tx_irq	[IN] tx_irq
	I	rx_irq	int rx_irq	[IN] rx_irq
	I	remote_core	const struct IPCS_SHM_REMOTE_CORE_TYPE *remote_core	[IN] remote_core
	I	local_core	const struct IPCS_SHM_LOCAL_CORE_TYPE *local_core	[IN] local_core
	I	mscm_addr	void *mscm_addr	[IN] mscm_addr
返回值	数据类型		说明	
	int		-	
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			

函数声明文件	ipcs/mpu/hw/ipc-hw.h
--------	----------------------

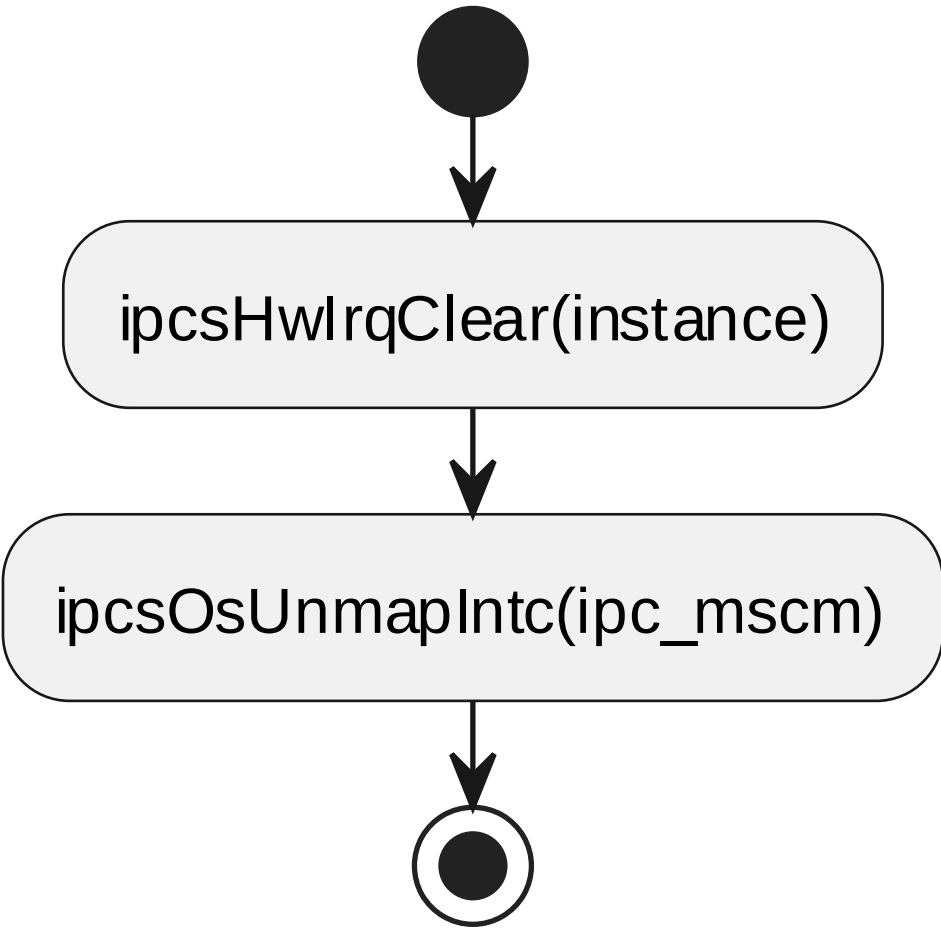
processing flow



7.8.4 ipcsHwFree

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	释放 HAL 资源；用户侧为空实现，内核侧释放映射状态。			
函数原型	void ipcsHwFree(const uint8_t instance)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

processing flow

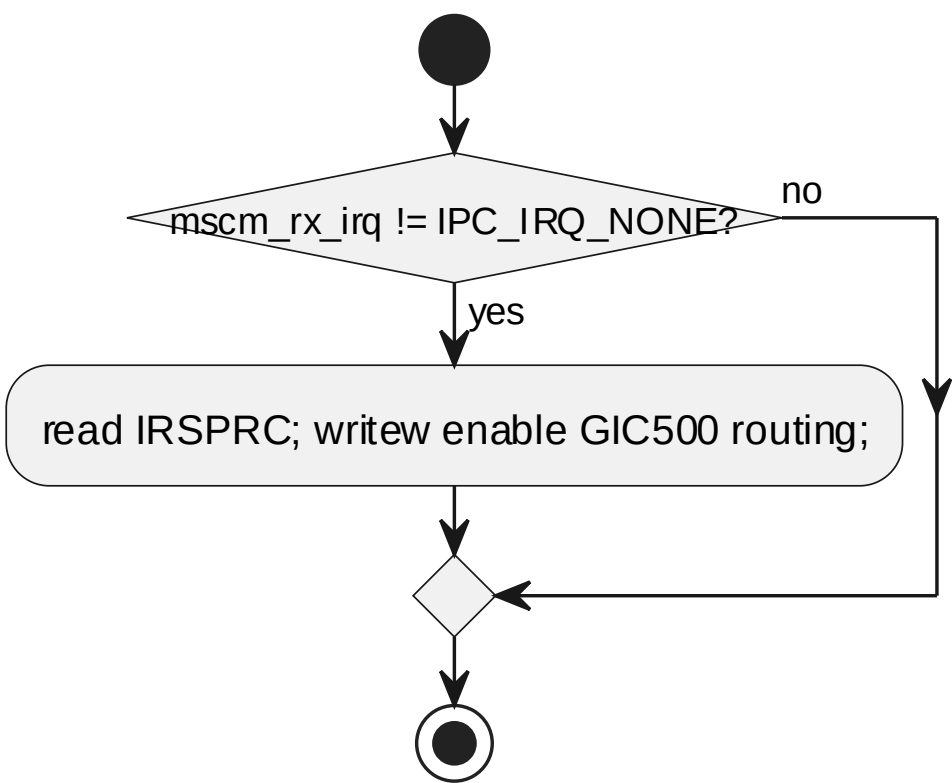


7.8.5 ipcsHwIrqEnable

对应软件架构 ID	Drv_Ipcs_Hal_Cmp
-----------	------------------

软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	使能指定实例接收中断；用户侧为转发代理，内核侧访问硬件。			
函数原型	void ipcsHwIrqEnable(const uint8_t instance)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

processing flow

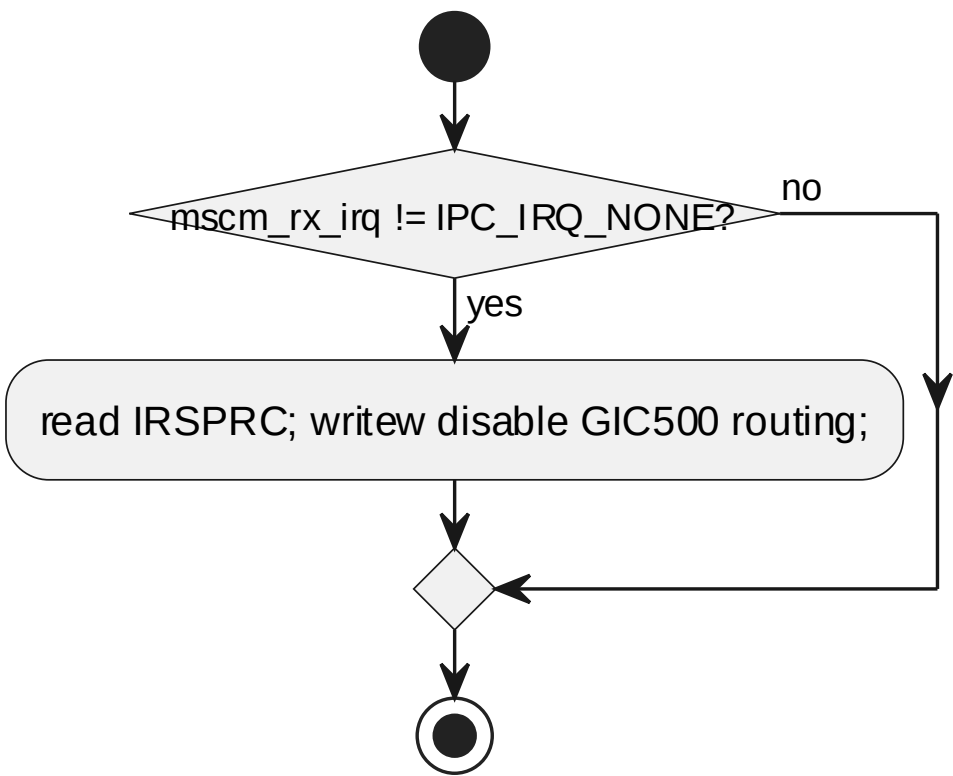


7.8.6 ipcsHwIrqDisable

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	禁止指定实例接收中断；用户侧为转发代理，内核侧访问硬件。			
函数原型	void ipcsHwIrqDisable(const uint8_t instance)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t	[IN] instance

			instance	
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

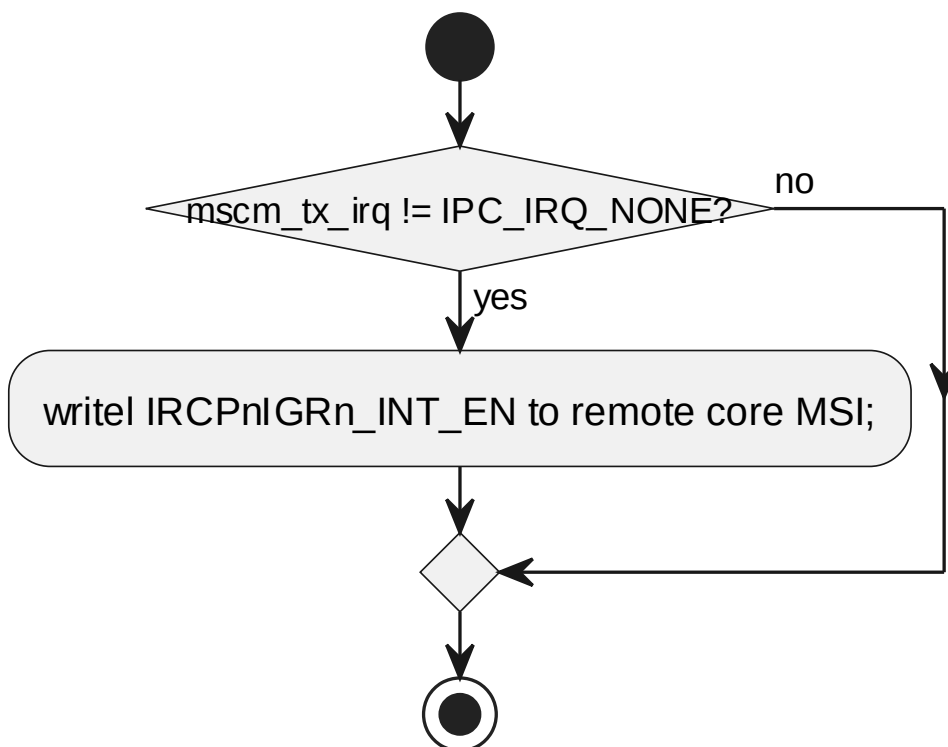
processing flow



7.8.7 ipcsHwIrqNotify

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	通知远端有数据可用；用户侧为转发代理，内核侧触发硬件中断。			
函数原型	void ipcsHwIrqNotify(const uint8_t instance)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

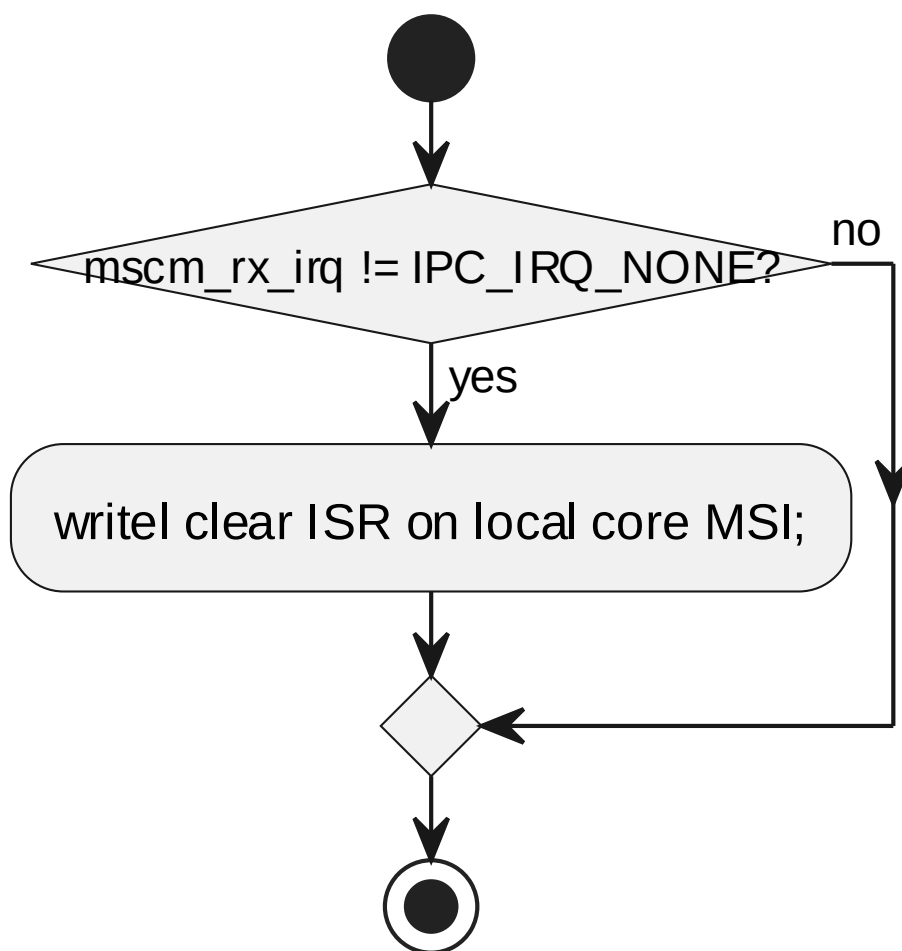
processing flow



7.8.8 ipcsHwIrqClear

对应软件架构 ID	Drv_Ipcs_Hal_Cmp			
软件单元 ID	SWU_IPCS_HAL_LINUX			
函数说明	清除指定实例接收中断状态。			
函数原型	void ipcsHwIrqClear(const uint8_t instance)			
制约条件	按 `ipcs/mpu/hw/c1/ipc-hw.c` 中的入参检查、实例状态和内核资源状态执行			
输入/输出参数	I/O	参数名	数据类型	说明
	I	instance	const uint8_t instance	[IN] instance
返回值	数据类型		说明	
	-			
函数定义文件	ipcs/mpu/hw/c1/ipc-hw.c			
函数声明文件	ipcs/mpu/hw/ipc-hw.h			

processing flow



7. 9	7. 10	7. 11	7. 12
7. 13	7. 14	7. 15	7. 16
7. 17	7. 18	7. 19	7. 20
7. 21	7. 22	7. 23	7. 24
7. 25	7. 26	7. 27	7. 28
7. 29	7. 30	7. 31	7. 32
7. 33	7. 34	7. 35	7. 36
7. 37	7. 38	7. 39	7. 40
7. 41	7. 42	7. 43	7. 44 <i>DYNAMIC DETAILED DESIGN 动态详 细设计</i>

场景 ID

场景名称

涉及软件单元

设计依据

LIN-S01

UIO 初始化

CORE_SHM、LINUX_OS_UIO、LINUX_UIO_KO、HAL_LINUX

ipcsOsInit (os_uio) + ipcsCdevWrite/ipcsUiolnit

LIN-S02

CDEV 初始化

CORE_SHM、LINUX_OS_CDEV、LINUX_CDEV_KO、HAL_LINUX

ipcsOsInit (os_cdev) + ioctl INIT

LIN-S03

全内核初始化

CORE_SHM、LINUX_OS_KERN、HAL_LINUX

ipcsOsInit (os_kernel)

LIN-S04

UIO 发送通知

CORE_SHM、LINUX_OS_UIO、LINUX_UIO_KO、HAL_LINUX

ipcsSendUioCmd / ipcsShmUiolrqcontrol

LIN-S05

CDEV 发送通知

CORE_SHM、LINUX_OS_CDEV、LINUX_CDEV_KO、HAL_LINUX

ioctl(TRIGGER_TX_IRQ)

LIN-S06

UIO 接收唤醒

HAL_LINUX、LINUX_UIO_KO、LINUX_OS_UIO、CORE_SHM

ipcsShmUioHandler + pthread read

LIN-S07

CDEV 接收唤醒

HAL_LINUX、LINUX_CDEV_KO、LINUX_OS_CDEV、CORE_SHM

ipcsShmHardirq + wait_queue

LIN-S08

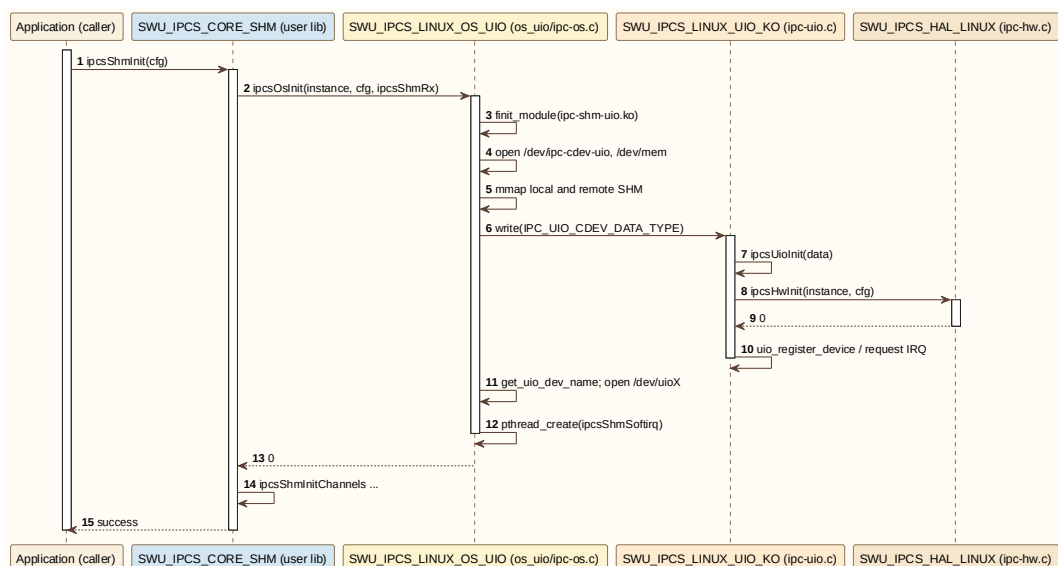
全内核接收

HAL_LINUX、LINUX_OS_KERN、CORE_SHM

ipcsShmHardirq + tasklet

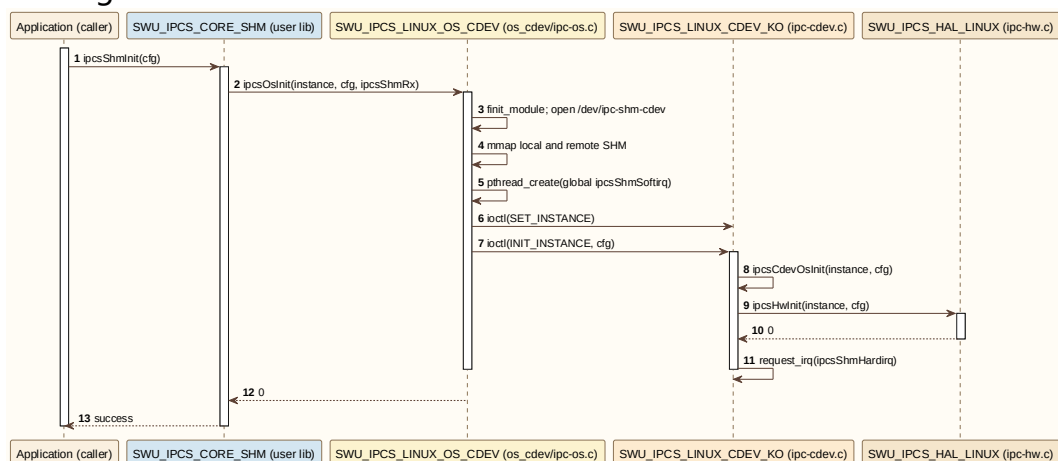
7.44.1 UIO Initialization (LIN-S01) UIO 初始化

sequence diagram



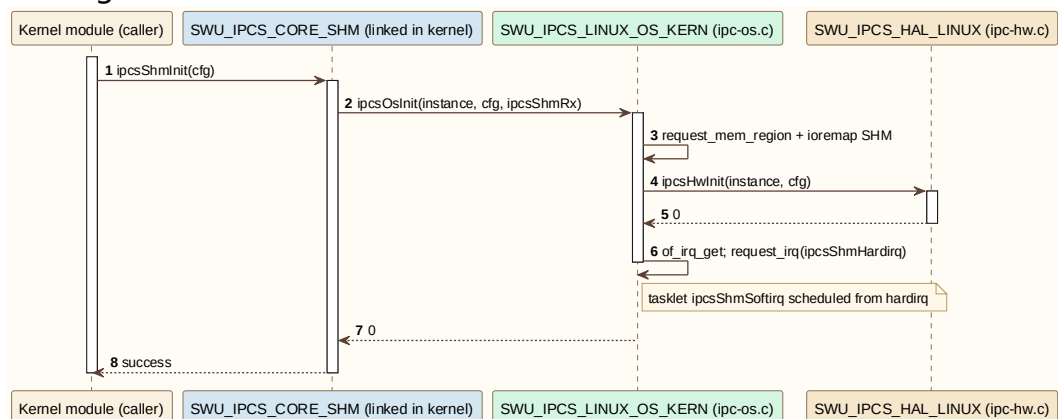
7.44.2 CDEV Initialization (LIN-S02) CDEV 初始化

sequence diagram



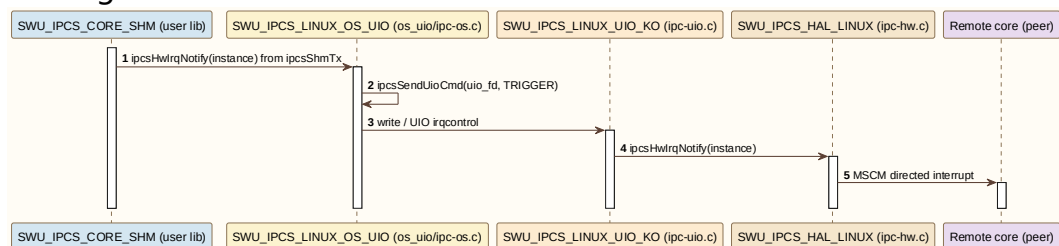
7.44.3 In-Kernel Initialization (LIN-S03) 全内核初始化

sequence diagram



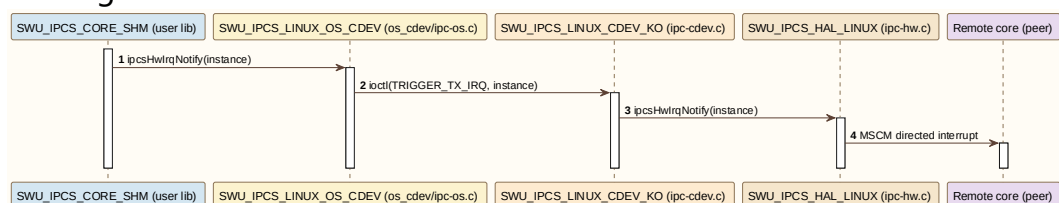
7.44.4 UIO Transmit Notify (LIN-S04) UIO 发送通知

sequence diagram



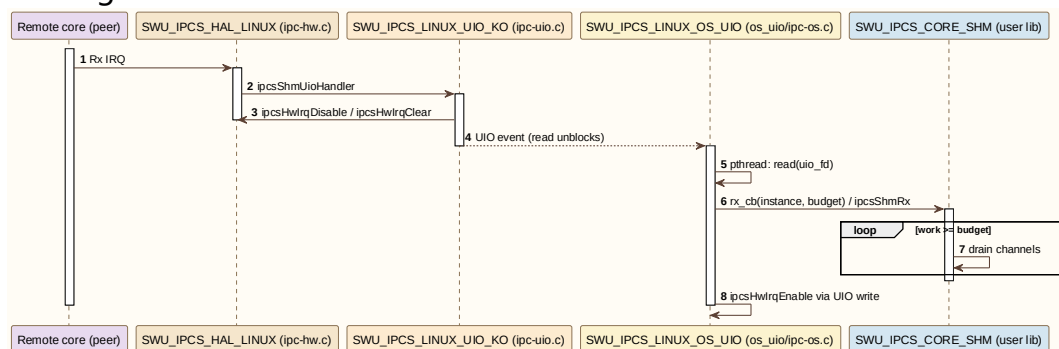
7.44.5 CDEV Transmit Notify (LIN-S05) CDEV 发送通知

sequence diagram



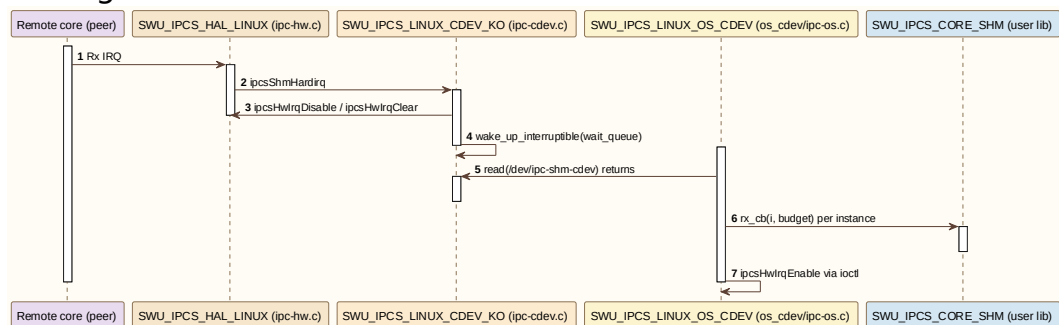
7.44.6 UIO Receive Wakeup (LIN-S06) UIO 接收唤醒

sequence diagram



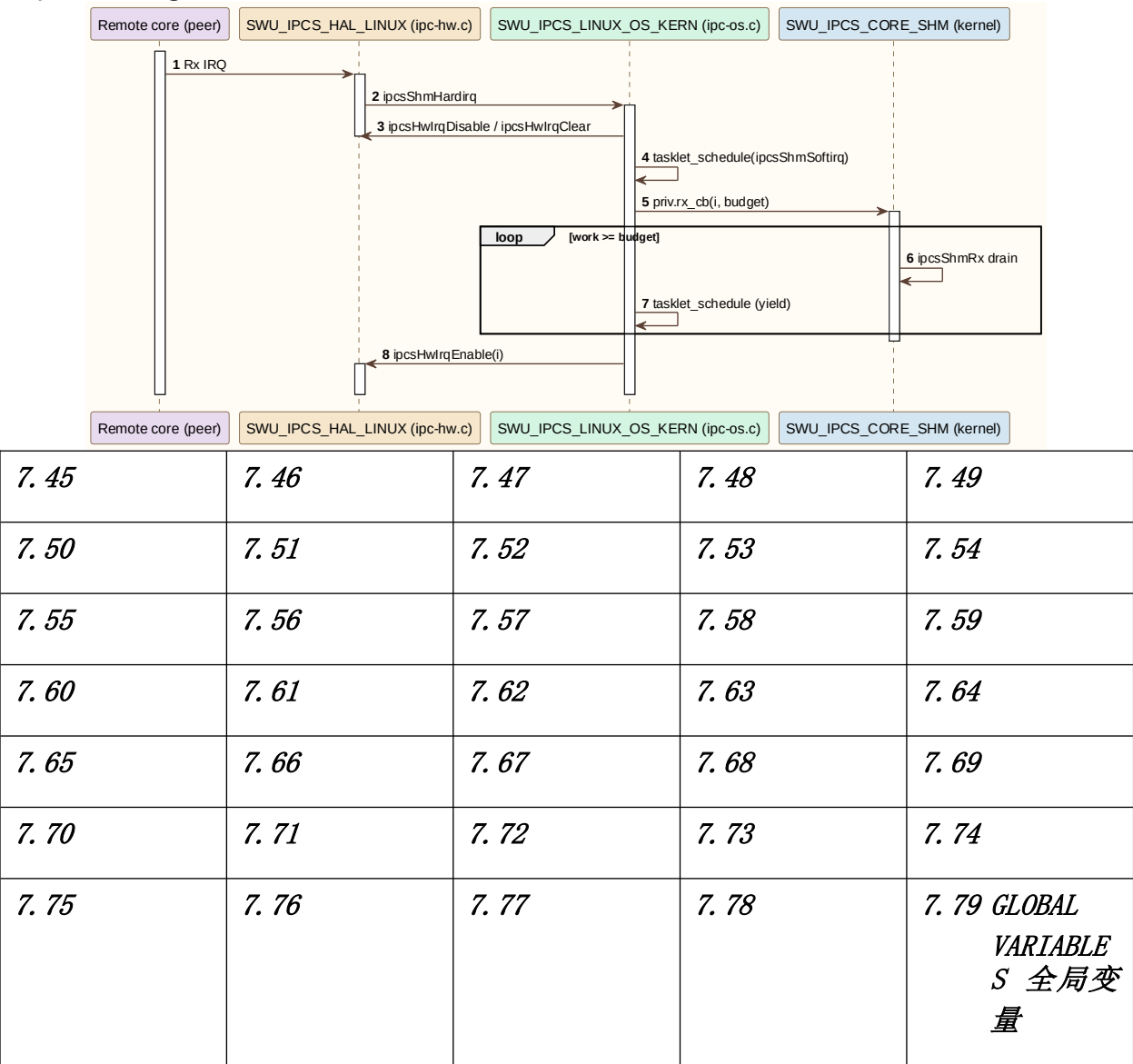
7.44.7 CDEV Receive Wakeup (LIN-S07) CDEV 接收唤醒

sequence diagram



7. 44. 8 In-Kernel Receive (LIN-S08) 全内核接收

sequence diagram



全局变量名称

全局变量类型

全局变量范围

全局变量描述

全局变量的存储 RAM 区

ipc_os_priv

static struct IPCS_OS_PRIV_TYPE_TYPE

ipcs/mpu/os_uio/ipc-os.c

UIO 用户侧 OSAL 代理私有数据

编译器/链接器默认 RAM (.bss, 未指定 section)

priv

static struct IPCS_OS_PRIV_TYPE

ipcs/mpu/os_cdev/ipc-os.c
CDEV 用户侧 OSAL 代理私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)
priv
static struct IPCS_OS_PRIV_TYPE
ipcs/mpu/os_kernel/ipc-os.c
全内核 OSAL 实现私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)
ipc_pdev_priv
struct IPCS_PDEV_PRIV_TYPE_TYPE
ipcs/mpu/os_kernel/ipc-uio.c
UIO 内核 Backend 平台设备与 cdev/UIO 实例状态
编译器/链接器默认 RAM (.bss, 未指定 section)
ipc_cdev_priv
struct IPCS_CDEV_PRIV_TYPE_TYPE
ipcs/mpu/os_kernel/ipc-cdev.c
CDEV 内核 Backend 字符设备与 wait queue 状态
编译器/链接器默认 RAM (.bss, 未指定 section)
ipc_hw_priv
static struct IPCS_HW_PRIV_TYPE_TYPE [IPC_SHM_MAX_INSTANCES]
ipcs/mpu/hw/c1/ipc-hw.c
每 instance Linux HAL 私有数据
编译器/链接器默认 RAM (.bss, 未指定 section)

7. 80 DATA TYPES 类型定义

7. 80. 1	7. 80. 2
7. 80. 3	7. 80. 4
7. 80. 5	7. 80. 6 enum IPCS_STATUS_E (UIO 用户侧)

定义于 ipcs/mpu/os_uio/ipc-os.c。

Name
Description
IPC_STATUS_CLEAR
0
IPC_STATUS_SET
1

7. 80. 7	7. 80. 8
7. 80. 9	7. 80. 10
7. 80. 11	7. 80. 12 enum IPCS_STATUS_E (CDEV 用户侧)

定义于 ipcs/mpu/os_cdev/ipc-os.c (枚举名与 UIO 实现相同, 取值一致)。

Name

Description

IPC_STATUS_CLEAR

0

IPC_STATUS_SET

1

7. 80. 13	7. 80. 14	7. 80. 15
7. 80. 16	7. 80. 17	7. 80. 18
7. 80. 19	7. 80. 20	7. 80. 21
7. 80. 22	7. 80. 23	7. 80. 24
7. 80. 25	7. 80. 26	7. 80. 27
7. 80. 28	7. 80. 29	7. 80. 30
7. 80. 31	7. 80. 32	7. 80. 33
7. 80. 34	7. 80. 35	7. 80. 36
7. 80. 37	7. 80. 38	7. 80. 39
7. 80. 40	7. 80. 41	7. 80. 42
7. 80. 43	7. 80. 44	7. 80. 45
7. 80. 46	7. 80. 47	7. 80. 48
7. 80. 49	7. 80. 50	7. 80. 51
7. 80. 52	7. 80. 53	7. 80. 54 struct IPCS_OS_PRIV_INSTA NCE_TYPE (UIO 用户 侧)

定义于 ipcs/mpu/os_uio/ipc-os.c。

Type

Name

Description

uint8_t
state
state to indicate whether instance is initialized
uint8_t
instance
target instance
int
irq_num
target instance interrupt index
int
uio_fd
UIO device file descriptor
void *
local_virt_shm
local ShM virtual address
void *
remote_virt_shm
remote ShM virtual address
void *
local_shm_map
local ShM mapped page address
void *
remote_shm_map
remote ShM mapped page address
size_t
local_shm_offset
local ShM offset in mapped page
size_t
remote_shm_offset
remote ShM offset in mapped page
size_t
shm_size
local/remote ShM size
int (*rx_cb)(const uint8_t instance, int budget)
rx_cb
upper layer Rx callback function
pthread_t
irq_thread_id
Rx interrupt thread id

7. 80. 55	7. 80. 56	7. 80. 57
7. 80. 58	7. 80. 59	7. 80. 60
7. 80. 61	7. 80. 62	7. 80. 63
7. 80. 64	7. 80. 65	7. 80. 66
7. 80. 67	7. 80. 68	7. 80. 69 struct IPCS_OS_PRIV_TYPE_ TYPE (UIO 用户侧)

定义于 ipcs/mpu/os_uio/ipc-os.c。

Type

Name

Description

uint8_t

ipc_files_opened

indicate whether device files are opened

int

ipc_cdev_fd

kernel character device file descriptor

int

dev_mem_fd

MEM device file descriptor

struct IPCS_OS_PRIV_INSTANCE_TYPE

id[IPC_SHM_MAX_INSTANCES]

private data per instance

7. 80. 70	7. 80. 71	7. 80. 72
7. 80. 73	7. 80. 74	7. 80. 75
7. 80. 76	7. 80. 77	7. 80. 78
7. 80. 79	7. 80. 80	7. 80. 81
7. 80. 82	7. 80. 83	7. 80. 84
7. 80. 85	7. 80. 86	7. 80. 87
7. 80. 88	7. 80. 89	7. 80. 90
7. 80. 91	7. 80. 92	7. 80. 93
7. 80. 94	7. 80. 95	7. 80. 96
7. 80. 97	7. 80. 98	7. 80. 99 struct IPCS_OS_PRIV_I NSTANCE_TYPE (CDEV 用 户 侧)

定义于 ipcs/mpu/os_cdev/ipc-os.c.

Type

Name

Description

uint8_t

state

state to indicate whether instance is initialized

int

irq_num

rx interrupt number using to check for polling

size_t

shm_size

local/remote ShM size

void *
 local_virt_shm
 local ShM virtual address
 void *
 remote_virt_shm
 remote ShM virtual address
 void *
 local_shm_map
 local ShM mapped page address
 void *
 remote_shm_map
 remote ShM mapped page address
 size_t
 local_shm_offset
 local ShM offset in mapped page
 size_t
 remote_shm_offset
 remote ShM offset in mapped page

7. 80. 100	7. 80. 101	7. 80. 102
7. 80. 103	7. 80. 104	7. 80. 105
7. 80. 106	7. 80. 107	7. 80. 108
7. 80. 109	7. 80. 110	7. 80. 111
7. 80. 112	7. 80. 113	7. 80. 114
7. 80. 115	7. 80. 116	7. 80. 117
7. 80. 118	7. 80. 119	7. 80. 120
7. 80. 121	7. 80. 122	7. 80. 123 struct IPCS_OS_PRIV_TYPE (CDEV 用户侧)

定义于 ipcs/mpu/os_cdev/ipc-os.c。

Type	Name	Description
uint8_t	ipc_files_opened	indicate whether device files are opened
uint8_t	ipc_soft_created	indicate whether ipcsShmSoftirq thread is opened
int	ipc_usr_fd	ipc-shm-usr kernel device file descriptor
int	dev_mem_fd	MEM device file descriptor
pthread_t	irq_thread_id	Rx interrupt thread id
struct IPCS_OS_PRIV_INSTANCE_TYPE	id[IPC_SHM_MAX_INSTANCES]	private data per instance
int (*rx_cb)(const uint8_t instance, int budget)	rx_cb	upper layer rx callback function

7. 80. 124	7. 80. 125	7. 80. 126
7. 80. 127	7. 80. 128	7. 80. 129
7. 80. 130	7. 80. 131	7. 80. 132
7. 80. 133	7. 80. 134	7. 80. 135
7. 80. 136	7. 80. 137	7. 80. 138
7. 80. 139	7. 80. 140	7. 80. 141
7. 80. 142	7. 80. 143	7. 80. 144
7. 80. 145	7. 80. 146	7. 80. 147 struct IPCS_OS_PRIV_I NSTANCE_TYPE （ 全 内 核 OSAL）

定义于 ipcs/mpu/os_kernel/ipc-os.c。

Type

Name

Description

int

shm_size

local/remote shared memory size

uintptr_t

local_phys_shm

local shared memory physical address

uintptr_t

remote_phys_shm

remote shared memory physical address

uintptr_t

local_virt_shm

local shared memory virtual address

uintptr_t

remote_virt_shm

remote shared memory virtual address

int

irq_num

Linux IRQ number

int

state

state to indicate whether instance is initialized

7. 80. 148	7. 80. 149	7. 80. 150
7. 80. 151	7. 80. 152	7. 80. 153
7. 80. 154	7. 80. 155	7. 80. 156
7. 80. 157	7. 80. 158	7. 80. 159 struct IPCS_OS_PRIV_TYPE （全内核 OSAL）

定义于 ipcs/mpu/os_kernel/ipc-os.c.

Type

Name

Description

struct IPCS_OS_PRIV_INSTANCE_TYPE

id[IPC_SHM_MAX_INSTANCES]

private data per instance

int (*rx_cb)(const uint8_t instance, int budget)

rx_cb

upper layer rx callback

int

irq_num_init[IPC_SHM_MAX_INSTANCES]

array to save all initialized irq

7. 80. 160	7. 80. 161	7. 80. 162
7. 80. 163	7. 80. 164	7. 80. 165
7. 80. 166	7. 80. 167	7. 80. 168 struct IPCS_UIO_CDEV_DATA _TYPE

定义于 ipcs/mpu/os_kernel/ipc-uio.h。

Type

Name

Description

uint8_t

instance

target instance

struct IPCS_SHM_CFG_TYPE

cfg

instance configuration passed from user space

7. 80. 169	7. 80. 170	7. 80. 171
7. 80. 172	7. 80. 173	7. 80. 174
7. 80. 175	7. 80. 176	7. 80. 177
7. 80. 178	7. 80. 179	7. 80. 180
7. 80. 181	7. 80. 182	7. 80. 183
7. 80. 184	7. 80. 185	7. 80. 186
7. 80. 187	7. 80. 188	7. 80. 189
7. 80. 190	7. 80. 191	7. 80. 192 struct IPCS_UIO_PRIV_TYPE _TYPE

定义于 ipcs/mpu/os_kernel/ipc-uio.c。

Type

Name

Description

int

state

instance state (initialized or not)

int

irq_num

rx interrupt number using to request irq
struct device *
dev
Linux device capabilities
atomic_t
refcnt
reference counter to allow a single UIO device open at a time
struct uio_info
info
UIO device capabilities
struct IPCS_UIO_CDEV_DATA_TYPE
data
Chrdev private data to get user space configuration
char
uio_name[32]
UIO device name

7. 80. 193	7. 80. 194	7. 80. 195
7. 80. 196	7. 80. 197	7. 80. 198
7. 80. 199	7. 80. 200	7. 80. 201
7. 80. 202	7. 80. 203	7. 80. 204
7. 80. 205	7. 80. 206	7. 80. 207
7. 80. 208	7. 80. 209	7. 80. 210
7. 80. 211	7. 80. 212	7. 80. 213
7. 80. 214	7. 80. 215	7. 80. 216 struct IPCS_PDEV_PRIV_TYP E_TYPE

定义于 ipcs/mpu/os_kernel/ipc-uio.c。
Type
Name
Description

dev_t

major

chrdev major number which is allocated dynamically

int

irq_num_init[IPC_SHM_MAX_INSTANCES]

interrupt index for each instance

struct platform_device *

ipc_pdev

Linux platform device capabilities

void __iomem *

pdev_reg

Platform device register

struct class *

cdev_class

class use to create device file in /dev

struct cdev

cdev

variable use for character device

struct IPCS_UIO_PRIV_TYPE_TYPE

uio_id[IPC_SHM_MAX_INSTANCES]

IPCS SHM UIO device data for each instance

7. 80. 217	7. 80. 218	7. 80. 219
7. 80. 220	7. 80. 221	7. 80. 222
7. 80. 223	7. 80. 224	7. 80. 225 struct IPCS_OS_PRIV_I NSTANCE_TYPE （ CDEV 内 核 Backend）

定义于 ipcs/mpu/os_kernel/ipc-cdev.c.

Type

Name

Description

int

state

instance state (initialized or not)

int

irq_num

rx interrupt number using to request irq

7. 80. 226	7. 80. 227	7. 80. 228
7. 80. 229	7. 80. 230	7. 80. 231
7. 80. 232	7. 80. 233	7. 80. 234
7. 80. 235	7. 80. 236	7. 80. 237
7. 80. 238	7. 80. 239	7. 80. 240
7. 80. 241	7. 80. 242	7. 80. 243
7. 80. 244	7. 80. 245	7. 80. 246
7. 80. 247	7. 80. 248	7. 80. 249
7. 80. 250	7. 80. 251	7. 80. 252
7. 80. 253	7. 80. 254	7. 80. 255 struct IPCS_CDEV_PRIV_TYP E_TYPE

定义于 ipcs/mpu/os_kernel/ipc-cdev.c。

Type

Name

Description

char

dev_is_opened

to check if device is open or not

uint8_t

target_instance

instance to be initialized

uint8_t

wait_queue_flag

flag to wake up the wait queue

int

irq_num_init[IPC_SHM_MAX_INSTANCES]
 array to save all initialized irq
 wait_queue_head_t
 wait_queue
 variable use for wait queue operation
 dev_t
 dev_major_num
 major number is dynamically allocated when initialize
 struct class *
 ipc_class
 class use to create device file in /dev
 struct cdev
 ipc_cdev
 variable use for character device
 struct IPCS_OS_PRIV_INSTANCE_TYPE
 instance_id[IPC_SHM_MAX_INSTANCES]
 private data per instance

7. 80. 256	7. 80. 257	7. 80. 258
7. 80. 259	7. 80. 260	7. 80. 261
7. 80. 262	7. 80. 263	7. 80. 264
7. 80. 265	7. 80. 266	7. 80. 267
7. 80. 268	7. 80. 269	7. 80. 270
7. 80. 271	7. 80. 272	7. 80. 273
7. 80. 274	7. 80. 275	7. 80. 276
7. 80. 277	7. 80. 278	7. 80. 279
7. 80. 280	7. 80. 281	7. 80. 282 struct IPCS_HW_PRIV_TYPE_ TYPE (Linux HAL)

定义于 ipcs/mpu/hw/c1/ipc-hw.c。

Type

Name

Description

uint8_t

msi_tx_irq

MSI index of inter-core interrupt corresponds to mscm_tx_irq

uint8_t

msi_rx_irq

MSI index of inter-core interrupt corresponds to mscm_rx_irq

uint8_t

spi_index

shared peripheral interrupts index

int

mscm_tx_irq

核间中断控制器 inter-core interrupt reserved for shm driver tx

int

mscm_rx_irq

核间中断控制器 inter-core interrupt reserved for shm driver rx

int

remote_core

index of remote core to trigger the interrupt on

int

local_core

index of the local core targeted by remote

struct IPCS_MSCM_REGS_TYPE *

ipc_mscm

pointer to memory-mapped hardware peripheral 核间中断控制器

7. 80. 283	7. 80. 284
7. 80. 285	7. 80. 286
7. 80. 287	7. 80. 288
7. 80. 289	7. 80. 290
7. 80. 291	7. 80. 292
7. 80. 293	7. 80. 294
7. 80. 295	7. 80. 296
7. 80. 297	7. 80. 298
7. 80. 299	7. 80. 300
7. 80. 301	7. 80. 302
7. 80. 303	7. 80. 304
7. 80. 305	7. 80. 306
7. 80. 307	7. 80. 308 enum IPCS_C1_PROCESSOR_IDX_E

定义于 ipcs/mpu/hw/c1/ipc-hw-platform.h。

Name

Description

IPC_A53_0

ARM Cortex-A53 cluster 0 core 0

IPC_A53_1

ARM Cortex-A53 cluster 1 core 1

IPC_A53_2

ARM Cortex-A53 cluster 1 core 0
IPC_A53_3
ARM Cortex-A53 cluster 1 core 1
IPC_M7_0
ARM Cortex-M7 core 0
IPC_M7_1
ARM Cortex-M7 core 1
IPC_M7_2
ARM Cortex-M7 core 2
IPC_M7_3
ARM Cortex-M7 core 2
IPC_A53_4
ARM Cortex-A53 cluster 0 core 2
IPC_A53_5
ARM Cortex-A53 cluster 0 core 3
IPC_A53_6
ARM Cortex-A53 cluster 1 core 2
IPC_A53_7
ARM Cortex-A53 cluster 1 core 3

7. 80. 309	7. 80. 310	7. 80. 311
7. 80. 312	7. 80. 313	7. 80. 314
7. 80. 315	7. 80. 316	7. 80. 317
7. 80. 318	7. 80. 319	7. 80. 320
7. 80. 321	7. 80. 322	7. 80. 323
7. 80. 324	7. 80. 325	7. 80. 326
7. 80. 327	7. 80. 328	7. 80. 329
7. 80. 330	7. 80. 331	7. 80. 332
7. 80. 333	7. 80. 334	7. 80. 335
7. 80. 336	7. 80. 337	7. 80. 338
7. 80. 339	7. 80. 340	7. 80. 341
7. 80. 342	7. 80. 343	7. 80. 344
7. 80. 345	7. 80. 346	7. 80. 347
7. 80. 348	7. 80. 349	7. 80. 350
7. 80. 351	7. 80. 352	7. 80. 353
7. 80. 354	7. 80. 355	7. 80. 356

7. 80. 309	7. 80. 310	7. 80. 311
7. 80. 357	7. 80. 358	7. 80. 359
7. 80. 360	7. 80. 361	7. 80. 362
7. 80. 363	7. 80. 364	7. 80. 365
7. 80. 366	7. 80. 367	7. 80. 368
7. 80. 369	7. 80. 370	7. 80. 371
7. 80. 372	7. 80. 373	7. 80. 374
7. 80. 375	7. 80. 376	7. 80. 377
7. 80. 378	7. 80. 379	7. 80. 380
7. 80. 381	7. 80. 382	7. 80. 383
7. 80. 384	7. 80. 385	7. 80. 386
7. 80. 387	7. 80. 388	7. 80. 389
7. 80. 390	7. 80. 391	7. 80. 392
7. 80. 393	7. 80. 394	7. 80. 395
7. 80. 396	7. 80. 397	7. 80. 398
7. 80. 399	7. 80. 400	7. 80. 401

7. 80. 309	7. 80. 310	7. 80. 311
7. 80. 402	7. 80. 403	7. 80. 404
7. 80. 405	7. 80. 406	7. 80. 407
7. 80. 408	7. 80. 409	7. 80. 410
7. 80. 411	7. 80. 412	7. 80. 413
7. 80. 414	7. 80. 415	7. 80. 416
7. 80. 417	7. 80. 418	7. 80. 419
7. 80. 420	7. 80. 421	7. 80. 422
7. 80. 423	7. 80. 424	7. 80. 425
7. 80. 426	7. 80. 427	7. 80. 428
7. 80. 429	7. 80. 430	7. 80. 431
7. 80. 432	7. 80. 433	7. 80. 434
7. 80. 435	7. 80. 436	7. 80. 437
7. 80. 438	7. 80. 439	7. 80. 440
7. 80. 441	7. 80. 442	7. 80. 443
7. 80. 444	7. 80. 445	7. 80. 446

7. 80. 309	7. 80. 310	7. 80. 311
7. 80. 447	7. 80. 448	7. 80. 449
7. 80. 450	7. 80. 451	7. 80. 452
7. 80. 453	7. 80. 454	7. 80. 455
7. 80. 456	7. 80. 457	7. 80. 458
7. 80. 459	7. 80. 460	7. 80. 461
7. 80. 462	7. 80. 463	7. 80. 464
7. 80. 465	7. 80. 466	7. 80. 467
7. 80. 468	7. 80. 469	7. 80. 470
7. 80. 471	7. 80. 472	7. 80. 473
7. 80. 474	7. 80. 475	7. 80. 476
7. 80. 477	7. 80. 478	7. 80. 479
7. 80. 480	7. 80. 481	7. 80. 482
7. 80. 483	7. 80. 484	7. 80. 485
7. 80. 486	7. 80. 487	7. 80. 488
7. 80. 489	7. 80. 490	7. 80. 491

7. 80. 309	7. 80. 310	7. 80. 311
7. 80. 492	7. 80. 493	7. 80. 494
7. 80. 495	7. 80. 496	7. 80. 497
7. 80. 498	7. 80. 499	7. 80. 500
7. 80. 501	7. 80. 502	7. 80. 503
7. 80. 504	7. 80. 505	7. 80. 506
7. 80. 507	7. 80. 508	7. 80. 509
7. 80. 510	7. 80. 511	7. 80. 512
7. 80. 513	7. 80. 514	7. 80. 515
7. 80. 516	7. 80. 517	7. 80. 518
7. 80. 519	7. 80. 520	7. 80. 521
7. 80. 522	7. 80. 523	7. 80. 524 struct IPCS_MSCM_REGS_TYP E

定义于 ipcs/mpu/hw/c1/ipc-hw-platform.h (核间中断控制器 Peripheral Register Structure) 。

Type

Name

Description

volatile const uint32_t

CPXTYPE

Processor X Type (CPXTYPE): CPU-specific read returning the accessing core personality (Cortex-A53 or Cortex-M7 ASCII ID plus Arm rYpZ revision byte). Read by Cortex-A53/M7 only; other bus masters read zero; writes terminate with an error (RM §8.4.2).

volatile const uint32_t

CPXNUM

Processor X Number (CPXNUM): CPU-specific read returning the logical processor number of the accessing core. Read by Cortex-A53/M7 only; other bus masters read zero; writes terminate with an error (RM §8.4.3).

volatile const uint32_t

CPXREV

Processor X Revision (CPXREV): CPU-specific read returning the logical core revision per Arm rYpZ nomenclature. Read by Cortex-A53/M7 only; other bus masters read zero; writes terminate with an error (RM §8.4.4).

volatile const uint32_t

CPXCFG[IPC_MSCM_CPnCFG_COUNT]

Processor X Configuration 0–3 (CPXCFG0–CPXCFG3): CPU-specific L1 cache, L2 cache, tightly coupled memory, and processor-option information for the accessing core. Read by Cortex-A53/M7 only; other bus masters read zero; writes terminate with an error (RM §8.4.5–§8.4.8).

uint8_t

RESERVED00[IPC_MSCM_RESERVED00_COUNT]

Reserved address space between CPX and CP0 register blocks; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP0TYPE

Processor Type (CP0TYPE): processor personality for CP0; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP0NUM

Processor Number (CP0NUM): logical processor number for CP0; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP0REV

Processor Revision (CP0REV): Arm rYpZ revision for CP0; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP0CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP0 (CP0CFG0–CP0CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED01[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP0 and CP1 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP1TYPE

Processor Type (CP1TYPE): processor personality for CP1; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP1NUM

Processor Number (CP1NUM): logical processor number for CP1; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP1REV

Processor Revision (CP1REV): Arm rYpZ revision for CP1; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP1CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP1 (CP1CFG0–CP1CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED02[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP1 and CP2 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP2TYPE

Processor Type (CP2TYPE): processor personality for CP2; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP2NUM

Processor Number (CP2NUM): logical processor number for CP2; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP2REV

Processor Revision (CP2REV): Arm rYpZ revision for CP2; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP2CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP2 (CP2CFG0–CP2CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED03[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP2 and CP3 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP3TYPE

Processor Type (CP3TYPE): processor personality for CP3; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP3NUM

Processor Number (CP3NUM): logical processor number for CP3; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP3REV

Processor Revision (CP3REV): Arm rYpZ revision for CP3; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP3CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP3 (CP3CFG0–CP3CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED04[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP3 and CP4 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP4TYPE

Processor Type (CP4TYPE): processor personality for CP4; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP4NUM

Processor Number (CP4NUM): logical processor number for CP4; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP4REV

Processor Revision (CP4REV): Arm rYpZ revision for CP4; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP4CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP4 (CP4CFG0–CP4CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED05[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP4 and CP5 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP5TYPE

Processor Type (CP5TYPE): processor personality for CP5; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP5NUM

Processor Number (CP5NUM): logical processor number for CP5; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP5REV

Processor Revision (CP5REV): Arm rYpZ revision for CP5; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP5CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP5 (CP5CFG0–CP5CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED06[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP5 and CP6 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP6TYPE

Processor Type (CP6TYPE): processor personality for CP6; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP6NUM

Processor Number (CP6NUM): logical processor number for CP6; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP6REV

Processor Revision (CP6REV): Arm rYpZ revision for CP6; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP6CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP6 (CP6CFG0–CP6CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED07[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP6 and CP7 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP7TYPE

Processor Type (CP7TYPE): processor personality for CP7; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP7NUM

Processor Number (CP7NUM): logical processor number for CP7; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP7REV

Processor Revision (CP7REV): Arm rYpZ revision for CP7; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP7CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP7 (CP7CFG0–CP7CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED08[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP7 and CP8 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP8TYPE

Processor Type (CP8TYPE): processor personality for CP8; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP8NUM

Processor Number (CP8NUM): logical processor number for CP8; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP8REV

Processor Revision (CP8REV): Arm rYpZ revision for CP8; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP8CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP8 (CP8CFG0–CP8CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED09[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP8 and CP9 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP9TYPE

Processor Type (CP9TYPE): processor personality for CP9; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP9NUM

Processor Number (CP9NUM): logical processor number for CP9; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP9REV

Processor Revision (CP9REV): Arm rYpZ revision for CP9; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP9CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP9 (CP9CFG0–CP9CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED010[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP9 and CP10 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP10TYPE

Processor Type (CP10TYPE): processor personality for CP10; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP10NUM

Processor Number (CP10NUM): logical processor number for CP10; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP10REV

Processor Revision (CP10REV): Arm rYpZ revision for CP10; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP10CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP10 (CP10CFG0–CP10CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED011[IPC_MSCM_RESERVED00_COUNT]

Reserved padding between CP10 and CP11 register groups; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile const uint32_t

CP11TYPE

Processor Type (CP11TYPE): processor personality for CP11; same fields and access rules as CPXTYPE (RM §8.4.9). Read-only; writes terminate with an error.

volatile const uint32_t

CP11NUM

Processor Number (CP11NUM): logical processor number for CP11; same fields and access rules as CPXNUM (RM §8.4.10). Read-only; writes terminate with an error.

volatile const uint32_t

CP11REV

Processor Revision (CP11REV): Arm rYpZ revision for CP11; same fields and access rules as CPXREV (RM §8.4.11). Read-only; writes terminate with an error.

volatile const uint32_t

CP11CFG[IPC_MSCM_CPnCFG_COUNT]

Processor Configuration 0–3 for CP11 (CP11CFG0–CP11CFG3): L1/L2 cache, tightly coupled memory, and processor options; same fields as CPXCFG (RM §8.4.12–§8.4.15). Read-only; writes terminate with an error.

uint8_t

RESERVED012[IPC_MSCM_RESERVED01_COUNT]

Reserved address space between CP11 registers and interrupt-router configuration block; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile uint32_t

IRPCCFG

Interrupt Router Configuration (IRPCCFG): designates trusted cores allowed to access and manage outstanding MSIs; privileged 32-bit read/write from Cortex-A53/M7 (RM §8.4.16).

uint8_t

RESERVED013[IPC_MSCM_RESERVED02_COUNT]

Reserved address space between IRPCCFG and IRNMIC; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile uint32_t

IRNMIC

Interrupt Router Nonmaskable Interrupt Control (IRNMIC): per-core enable fields routing the NMI request; disabled when all fields are zero; privileged access (RM §8.4.17).

uint8_t

RESERVED14[IPC_MSCM_RESERVED03_COUNT]

Reserved address space between IRNMIC and IRSPRC array; accesses are not permitted and terminate with an error (RM §8.4 note).

volatile uint16_t

IRSPRC[IPC_MSCM_IRSPRC_COUNT]

Interrupt Router Shared Peripheral Routing Control (IRSPRC0–IRSPRC239): halfword array selecting GIC500 and Cortex-M7 steering for each shared peripheral interrupt; privileged read/write (RM §8.4.18).

struct { volatile uint32_t IPC_ISR; volatile uint32_t IPC_IGR; }

IRCPnIRx[IPC_MSCM_CP_COUNT][IPC_MSCM_IRQ_COUNT]

Interrupt Router CPn Interruptx register pairs (IRCPnISRx / IRCPnIGRx): IPC_ISR holds MSI status (W1C bitmap); IPC_IGR initiates MSI to another core; array indexed by target CP and interrupt index (RM §8.4.19–§8.4.20).

8 TRACE & CONSISTENCY 双向追溯一致性

8.1 TRACEABILITY STATEMENT 追溯策略声明

本章节建立本模块软件详细设计与上游（软件需求、软件架构）及下游（物理源代码）之间的双向追溯关系，满足 ASPICE SWE.3.BP4 要求。

需求—架构组件分配依据架构设计规范的章节 2.4；架构组件—软件单元映射依据本文档 章节 2.1–章节 2.2；单元行为与接口依据 章节 4–章节 6。本矩阵用于证明：已分配至本驱动的架构组件与需求在详细设计单元及物理实现中均有对应实体，并支持变更影响分析。

<i>8. 2</i>	<i>8. 3</i>	<i>8. 4</i>	<i>8. 5</i>	<i>8. 6</i>
<i>8. 7</i>	<i>8. 8</i>	<i>8. 9</i>	<i>8. 10</i>	<i>8. 11</i>
<i>8. 12</i>	<i>8. 13</i>	<i>8. 14</i>	<i>8. 15</i>	<i>8. 16</i>
<i>8. 17</i>	<i>8. 18</i>	<i>8. 19</i>	<i>8. 20</i>	<i>8. 21</i>
<i>8. 22</i>	<i>8. 23</i>	<i>8. 24</i>	<i>8. 25</i>	<i>8. 26</i>
<i>8. 27</i>	<i>8. 28</i>	<i>8. 29</i>	<i>8. 30</i>	<i>8. 31</i>
<i>8. 32</i>	<i>8. 33</i>	<i>8. 34</i>	<i>8. 35</i>	<i>8. 36</i>
<i>8. 37</i>	<i>8. 38</i>	<i>8. 39</i>	<i>8. 40</i>	<i>8. 41</i>
<i>8. 42</i>	<i>8. 43</i>	<i>8. 44</i>	<i>8. 45</i>	<i>8. 46</i>
<i>8. 47</i>	<i>8. 48</i>	<i>8. 49</i>	<i>8. 50</i>	<i>8. 51</i>
<i>8. 52</i>	<i>8. 53</i>	<i>8. 54</i>	<i>8. 55</i>	<i>8. 56</i>
<i>8. 57</i>	<i>8. 58</i>	<i>8. 59</i>	<i>8. 60</i>	<i>8. 61</i>
<i>8. 62</i>	<i>8. 63</i>	<i>8. 64</i>	<i>8. 65</i>	<i>8. 66</i>
<i>8. 67</i>	<i>8. 68</i>	<i>8. 69</i>	<i>8. 70</i>	<i>8. 71</i>
<i>8. 72</i>	<i>8. 73</i>	<i>8. 74</i>	<i>8. 75</i>	<i>8. 76</i>
<i>8. 77</i>	<i>8. 78</i>	<i>8. 79</i>	<i>8. 80</i>	<i>8. 81</i>
<i>8. 82</i>	<i>8. 83</i>	<i>8. 84</i>	<i>8. 85</i>	<i>8. 86</i>
<i>8. 87</i>	<i>8. 88</i>	<i>8. 89</i>	<i>8. 90</i>	<i>8. 91</i>
<i>8. 92</i>	<i>8. 93</i>	<i>8. 94</i>	<i>8. 95</i>	<i>8. 96</i>
<i>8. 97</i>	<i>8. 98</i>	<i>8. 99</i>	<i>8. 100</i>	<i>8. 101</i>
<i>8. 102</i>	<i>8. 103</i>	<i>8. 104</i>	<i>8. 105</i>	<i>8. 106</i>
<i>8. 107</i>	<i>8. 108</i>	<i>8. 109</i>	<i>8. 110</i>	<i>8. 111</i>
<i>8. 112</i>	<i>8. 113</i>	<i>8. 114</i>	<i>8. 115</i>	<i>8. 116</i>

<i>8. 2</i>	<i>8. 3</i>	<i>8. 4</i>	<i>8. 5</i>	<i>8. 6</i>
<i>8. 117</i>	<i>8. 118</i>	<i>8. 119</i>	<i>8. 120</i>	<i>8. 121</i>
<i>8. 122</i>	<i>8. 123</i>	<i>8. 124</i>	<i>8. 125</i>	<i>8. 126</i>
<i>8. 127</i>	<i>8. 128</i>	<i>8. 129</i>	<i>8. 130</i>	<i>8. 131</i>
<i>8. 132</i>	<i>8. 133</i>	<i>8. 134</i>	<i>8. 135</i>	<i>8. 136 TRACEABILITY MATRIX 双向追溯矩阵</i>

软件需求 ID (SWE.1)

架构组件 ID (SWE.2)

本详细设计单元 ID (SWE.3)

物理源代码实体 (Code)

追溯关系及设计覆盖说明

IPCS_001

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Hal_Cmp、
Drv_Ipcs_Conf_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_UTIL、SWU_IPCS_CORE_QUEUE；章节 2.1 所列
OSAL/HAL 单元；Conf 见 章节 4.6

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c、ipc-util.c、ipc-types.h；ipcs/mcu/os/、ipcs/mcu/hw/
ipc-hw.c；ipcs/mpu/ 各实现文件

同核/异核点对点双向通信：实例、通道、队列、共享内存映射与核间通知

IPCS_002

Drv_Ipcs_Osal_Cmp

SWU_IPCS_OSAL_AUTOSAR

ipcs/mcu/os/autosar/ipc-os-autosar.c

AutoSAR OS 部署下 OSAL 集成边界；安全目标与证据见项目安全工件

IPCS_003

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Hal_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE、SWU_IPCS_HAL_MCU、

SWU_IPCS_HAL_LINUX

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c；ipcs/mcu/hw/ipc-hw.c；ipcs/mpu/hw/c1/ipc-hw.c

可移植逻辑在 Core/Queue；平台与字节序相关行为在 HAL

IPCS_005

Drv_Ipcs_Osal_Cmp

SWU_IPCS_OSAL_AUTOSAR

ipcs/mcu/os/autosar/ipc-os-autosar.c

AutoSAR CDD/OS 封装、调度与错误处理

IPCS_006

Drv_Ipcs_Osal_Cmp

SWU_IPCS_OSAL_THREADX

ipcs/mcu/os/threadx/ipc-os-threadx.c

ThreadX 任务、中断与同步原语映射

IPCS_010

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c

传输路径可观测计数/时间戳（可选编译）

IPCS_012

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Hal_Cmp

章节 2.1 Core/OSAL/HAL 单元

ipcs/ 目录下对应单元源文件

OSAL/HAL 接口可替换实现，便于单元测试

IPCS_014

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Conf_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE; Conf 见 章节 4.6

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c、ipc-types.h

多通信通道与共享内存布局

IPCS_015

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c

单通道内消息顺序保持

IPCS_016

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Conf_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE; Conf 见 章节 4.6

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c、ipc-types.h

每通道多缓冲池与队列管理

IPCS_017

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Conf_Cmp

SWU_IPCS_CORE_SHM; 章节 2.1 OSAL 单元; Conf 见 章节 4.6

ipcs/ipcs_cores/ipc-shm.c; 各 ipc-os-*.c

多 instance 与每核 OSAL 执行及中断亲和

IPCS_018

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp

SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE

ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c

零拷贝：BD/指针传递缓冲区

IPCS_019

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Osal_Cmp
SWU_IPCS_CORE_SHM; 章节 2.1 OSAL 单元
ipcs/ipcs_cores/ipc-shm.c; 各 OSAL 实现文件
基于通知的异步接收与回调分发

IPCS_020

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Conf_Cmp
SWU_IPCS_CORE_SHM; Conf 见 章节 4.6
ipcs/ipcs_cores/ipc-shm.c、ipc-types.h; 集成配置 ipcf_Ip_Cfg*.h
通信端内存隔离: 布局配置与 Core 边界检查

IPCS_021

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Conf_Cmp
SWU_IPCS_CORE_SHM; Conf 见 章节 4.6
ipcs/ipcs_cores/ipc-shm.c
非托管通道: ipcsShmUnmanagedAcquire/ipcsShmUnmanagedTx

IPCS_022

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Hal_Cmp
章节 2.1 所列相关单元
ipcs/ipcs_cores/; OSAL/HAL 源文件
虚拟中断与队列状态协调, 避免陈旧数据投递

IPCS_023

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp
SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE
ipcs/ipcs_cores/ipc-shm.c、ipc-queue.c
共享内存提交顺序保证; 可与 IPCS_039 轮询协同

IPCS_024

Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Conf_Cmp
SWU_IPCS_OSAL_AUTOSAR; Conf 见 章节 4.6
ipcs/mcu/os/autosar/ipc-os-autosar.c、ipc-types.h
与 AUTOSAR R4.4+ 接口及配置模型对齐

IPCS_025

Drv_Ipcs_Conf_Cmp
章节 4.6 类型定义 (无独立 SWU)
ipcs/ipcs_cores/ipc-types.h; ipcf_Ip_Cfg*.h
缓冲池数量与大小由集成配置提供

IPCS_028

Drv_Ipcs_Hal_Cmp、Drv_Ipcs_Osal_Cmp
SWU_IPCS_HAL_MCU、SWU_IPCS_HAL_LINUX; 章节 2.1 OSAL 单元
ipcs/mcu/hw/ipc-hw.c、ipcs/mpu/hw/c1/ipc-hw.c; 各 OSAL 文件
核间中断作为收发通知

IPCS_029

Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Osal_Cmp
SWU_IPCS_CORE_SHM、SWU_IPCS_CORE_QUEUE、SWU_IPCS_CORE_UTIL; OSAL 单元
ipcs/ipcs_cores/; ipcs/mcu/os/、ipcs/mpu/os_kernel/、ipcs/mpu/os_uio/、ipcs/mpu/os_cdev/
静态分配池与队列, 无动态堆
IPCS_031
Drv_Ipcs_Core_Cmp、Drv_Ipcs_Conf_Cmp
SWU_IPCS_CORE_SHM; Conf 见 章节 4.6
ipcs/ipcs_cores/ipc-shm.c、ipc-types.h
多核多 instance 独立 SHM/IRQ 配置
IPCS_034
Drv_Ipcs_Core_Cmp
SWU_IPCS_CORE_SHM
ipcs/ipcs_cores/ipc-shm.c、ipc-shm.h
公共 API 参数校验
IPCS_035
Drv_Ipcs_Core_Cmp
SWU_IPCS_CORE_SHM
ipcs/ipcs_cores/ipc-shm.c
managed/unmanaged 与 channel 类型及操作一致性
IPCS_036
Drv_Ipcs_Core_Cmp、Drv_Ipcs_Queue_Cmp、Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Hal_Cmp、
Drv_Ipcs_Linux_Adapt_Cmp、Drv_Ipcs_Conf_Cmp
章节 2.1 全部软件单元
ipcs/ 实现目录
按部署变体与编译选项裁剪 HW/OS/传输实现
IPCS_039
Drv_Ipcs_Osal_Cmp、Drv_Ipcs_Hal_Cmp
章节 2.1 OSAL/HAL 单元
ipcsShmPollChannels (ipc-shm.c) ; 各 OSAL/HAL 文件
IRQ_NONE 轮询接收路径